Understanding How Orthogonality of Parameters Improves Quantization of Neural Networks

Sukru Burc Eryilmaz and Aysegul Dundar

***Abstract*— We analyze why the orthogonality penalty improves quantization in deep neural networks. Using results from perturbation theory as well as through extensive experiments with Resnet50, Resnet101, and VGG19 models, we mathematically and experimentally show that improved quantization accuracy resulting from orthogonality constraint stems primarily from reduced condition numbers, which is the ratio of largest to smallest singular values of weight matrices, more so than reduced spectral norms, in contrast to the explanations in previous literature. We also show that the orthogonality penalty improves quantization even in the presence of a state-of-the-art quantized retraining method. Our results show that, when the orthogonality penalty is used with quantized retraining, ImageNet Top5 accuracy loss from 4- to 8-bit quantization is reduced by up to 7% for Resnet50, and up to 10% for Resnet101, compared to quantized retraining with no orthogonality penalty.**

***Index Terms*— Deep neural networks, orthogonality regularization, perturbation theory, quantization.**

I. Introduction

DEEP convolutional neural networks (CNNs) have achieved impressive results on a wide range of computer vision tasks from object classification, detection, segmentation, to image and video editing and interpolation. To achieve state-of-the-art results on these tasks, CNNs have become larger and deeper with increased complexity. On the one hand, the excellent accuracies coming from computationally heavy CNNs make them demanding to be used in various applications. On the other hand, it calls for computationally efficient implementations. There are several research directions to make CNNs run in a computationally efficient manner such as efficient implementations of CNNs on different hardware [1]–[6], novel convolutional neural network architectures that exploit memory and computation efficiency [7]–[11], knowledge distillation to decrease the number of parameters [12]–[15], pruning techniques to decrease the network size [16]–[18], and weight or activation quantization of CNNs from 32-bit floating point into lower bit-width representations [19]–[22].

Manuscript received 7 September 2021; revised 7 February 2022; accepted 27 April 2022. Date of publication 10 May 2022; date of current version 1 December 2023. *(Corresponding author: Aysegul Dundar.)*

Sukru Burc Eryilmaz is with NVIDIA Corporation, Santa Clara, CA 95051 USA (e-mail: seryilmaz@nvidia.com).

Aysegul Dundar is with the Department of Computer Science, Bilkent University, 06800 Ankara, Turkey (e-mail: adundar@cs.bilkent.edu.tr).

Color versions of one or more figures in this article are available at https://doi.org/10.1109/TNNLS.2022.3171297.

Digital Object Identifier 10.1109/TNNLS.2022.3171297

In this work, we are interested in network quantization to reduce the model size. Besides reducing the model size, low-precision quantization of weights and activations takes advantage of the low bit-width fixed-point arithmetic units on hardware [23], [24], resulting in power, area, and latency savings. These costs roughly go down linearly for addition operations with the number of bits and quadratically for multiplication operations. Model size, on the other hand, goes down linearly with the bit-width of quantized weights. Consequently, low-precision quantization is a valuable technique for enabling the highly efficient execution of CNNs. However, reduced precision commensurately reduces the number of representable values. This causes perturbations for each value of the input, weight, and output of every quantized layer at inference time, and these perturbations are carried on to subsequent layers, resulting in loss of accuracy, which can be significant for sufficiently low bit-widths (especially for less than 8 bits). Quantizing model weights and activations of a model trained in high precision (32-bit or 16-bit floating point) requires additional processing either during the initial training stage, a retraining stage, or post-training processing stage to mitigate the accuracy loss [24]–[26]. Recently, a retraining method for quantization was introduced [25], where a pretrained model is retrained with quantization thresholds included in the model parameters together with weights, where the quantization thresholds are trained simultaneously with gradient descent for optimizing the range of representable values after quantization. This method achieves state-of-the-art results for several networks and is called trained quantization thresholds (TQTs).

In a different research line, orthogonality in weights has proved to be beneficial for different purposes, such as making optimization more efficient [27], [28] or improving the generalization of the resulting models [28], and improving the quality of images synthesized by generative adversarial models [29]. Orthogonality constraint has also been used for improving the robustness of neural networks to adversarial examples [30], [31], with the latter work also studying robustness in the context of quantization, albeit with small datasets, such as CIFAR-10 or CIFAR-100. Alizadeh *et al.* [32] implements a second-order optimization method for imposing orthogonality constraints on weights, but this method increases training time by an order of magnitude, and it is only studied for small models such as Resnet18. Also, their work does not study whether their method is still beneficial when combined with trained quantization methods, or whether there are cases where trained quantization can

2162-237X © 2022 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Source: https://ieeexplore.ieee.org/document/9772057 (published print)

Google Scholar research | 2026-10-08 | Article 01 | Original p. 1 | Orange: discussion; blue: reference

replace the orthogonality regularization. Most importantly, both works [31], [32] attribute improved robustness or quantization from orthogonality to the ability of the orthogonality constraint to reduce the spectral norm (largest singular value) of weight matrices, without doing sufficient analysis. However, orthogonality imposes a stronger restriction, which is minimizing the condition number, or ratio of largest to smallest singular values, which determines the upper bound on the relative sensitivity of matrix multiplication. Previous studies have not studied in depth which of these effects contributes more to improved quantization and mostly cited the former to be the main contributor, in contrast to our observations.

In this article, through extensive analysis of condition numbers and spectral norms of weight matrices, we experimentally show that the ability of orthogonality constraint to minimize the condition number is what improves the accuracy of the quantized model, more so than its ability to lower spectral norm, in contrast to the claims in previous literature. Using existing results from perturbation theory, we support our experimental observations theoretically, namely how orthogonality constraint on weight parameters lowers the sharp upper bound on the relative sensitivity of a layer with respect to perturbations in inputs and weights (in this case, perturbations from quantization noise), by minimizing the condition number of weight parameters. We also perform our experiments on relatively large models such as Resnet50, Resnet101, and VGG19, using the ImageNet dataset. We further combine orthogonality with state-of-the-art TQTs quantized retraining method for quantization to study if the orthogonality is still beneficial when combined with existing quantization methods. Our method significantly improves the accuracy of the quantized model compared to the case where no orthogonality penalty is used in the model training stage. Contributions are summarized as follows.

1) We show through ablation experiments that, in contrast to existing literature, improved quantization due to orthogonality in neural networks is due to the reduction in condition number of weight parameters, and not necessarily due to the reduction in the spectral norm.
2) We use the existing literature in perturbation theory on the error bounds of perturbed matrices to theoretically explain the experimental results, that is, the observation that the reduced condition number leads to improved quantization.
3) By experimenting with a state-of-the-art quantized training method, we show that quantized training methods do not necessarily replace the orthogonality penalty in terms of improved quantization and that the orthogonality penalty still benefits quantization even when a quantized training method is employed.
4) While our aim is not to achieve the state-of-the-art quantization results, but rather to explain how orthogonality of parameters improves quantization of neural networks, we provide consistent improvements on three popular network architectures namely, ResNet50, ResNet101, and VGG19.

II. Related Work

A. Integer Quantization of Deep Neural Networks

Prior work on quantization includes employing ternary quantization [20], [33] or even as low as 1-bit quantization [21], [34]. However, these methods introduce higher accuracy loss than often desired, and more focus has been on intermediate bit-width quantization, with a bit-width of two or more [35]–[41]. For more practical deployment, recent work has emphasized uniform quantization [26], per-tensor quantization [25], and symmetric quantization [24]. Calibration of quantization thresholds using KL divergence or max-values of pretrained weights achieves good accuracy for CNNs for 8-bit quantization without a change in the training process [42]. Other works have explored quantization-aware training with different approaches [24], [43]–[45], where a quantization node is included in the training graph and it is made differentiable by using a straight-through estimator (STE) [46]. TQTs [25] report that a common problem with these approaches is that they do not balance well range and precision when training for quantization and implement a method where a pretrained model is retrained with quantization nodes that still use STE, but on top of that trains activation and weight thresholds in the log-domain. TQT achieves state-of-the-art results for some networks, such as MobileNetv2. As we show later, these quantized training methods do not fully replace the method of imposing the orthogonality penalty on parameters. Orthogonality penalty can be combined with quantized training methods to further improve the accuracy of the quantized model. In our experiments, we combine our method with TQT, and on top of that, we apply the orthogonality penalty during the initial training phase, as will be explained in Section V-A.

B. Orthogonality Regularization of Parameters in Deep Neural Networks

As will be explained in more depth in Section III-A, orthogonal matrices, where the columns are orthogonal to each other, minimize a sharp upper bound on the relative sensitivity of matrix multiplication operation with respect to perturbations on the matrix or the input, according to perturbation theory literature. Quantization is a form of perturbation on the inputs or weights to a convolution or fully-connected layer, hence it is expected that orthogonality of weight matrices reduces the relative sensitivity of convolutional or fully connected layers to perturbations in the inputs or weights, hence improving the accuracy of the quantized model, and we theoretically explain and experimentally show this phenomenon. Prior works have used orthogonality of parameters in different contexts. Various works [47]–[52] have studied orthogonality of parameters in the context of recurrent neural networks for improving training process and mitigating vanishing gradients problem. Others [53], [54] studied the benefits of orthogonal initialization for CNNs for more efficient optimization.

These methods, however, do not have a mechanism to maintain orthogonality throughout the training, and orthogonality breaks down without a proper regularizer during training [28], [53]. To achieve orthogonality of weights during training,

Source: https://ieeexplore.ieee.org/document/9772057 (published print)
Google Scholar research | 2026-10-08 | Article 01 | Original p. 2 | Orange: discussion; blue: reference

several works [55]–[57] use hard orthogonality constraints based on the Stiefel manifold. These works using hard constraints require repeatedly computing singular value decomposition during training, which could be expensive to compute. Furthermore, when the weight matrix is rank-deficient (for instance, when the number of rows is lower than the number of columns), hard orthogonality constraints require extra precautions [28]. Xie *et al.* [27] use a soft orthogonality constraint, where a penalty term proportional to the Frobenius norm of $W^TW - I$, given by $\lambda\|W^TW - I\|_F$, encouraging weights to be orthonormal during training. This study reports significant accuracy gains in certain CNN architectures, although most gains are reported by custom architectures and not necessarily for state-of-the-art convolutional residual models. It replaces a weight decay regularizer with an orthonormality penalty, but we found in our experiments that using a combination of two (albeit with a reduced factor of weight decay when combined with an orthonormal regularizer) results in better accuracy for larger models. Bansal *et al.* [28], in addition to soft orthogonality regularizer in [27], also experiment with some additional orthogonality penalty formulations. To handle the case of short and fat (rank-deficient) W, they propose "double soft orthogonality regularization," which is given by $\lambda(\|W^TW - I\|_F + \|WW^T - I\|_F)$. However, the results of this same study suggest that a regular orthogonal regularizer consistently outperforms a double orthogonality regularizer. Another orthogonality regularizer formulation in [28] is $\lambda\|W^TW - I\|_2$, where Frobenius norm is replaced by 2-norm, which is equal to the largest singular value of $W^TW - I$. To estimate the largest singular value, this method relies on an approximation based on power iteration. However, one problem with this is that power iteration approximation is only good when the largest singular value is sufficiently larger than the rest of the singular values, which may not hold as the training progresses and the largest singular value becomes progressively smaller.

There is limited work on studying orthogonality in the context of quantization. As mentioned in Section I, while [31] mainly studies improved robustness with orthogonality against adversarial examples, they also observed improved quantization with small datasets. As mentioned earlier, both [31] and [32] attribute improved robustness or quantization to the reduced spectral norm of weight matrices as a result of orthogonality constraint. However, orthogonality further minimizes condition number, and [31] and [32] do not study the impact of reduced condition number on quantization. As shown in this article, we found that reduced condition number results in a more pronounced effect on quantization than the spectral norm.

Even though not exactly a regularizer for orthogonality, one approach to reduce perturbation effects (not in the context of quantization, but rather in the context of adversarial attacks or improving accuracy, in general) was to minimize $\|W\|_2$, or the largest singular value of W [29], [58]. This forms an upper bound on the sensitivity of the matrix multiplication result with respect to perturbations on input. However, one problem with this approach is that it provides no bounds with respect to perturbations on W. Another problem is that the bound only consider the absolute values of the perturbations on the input and output and does not account for relative values. On the other hand, as explained in Section , orthonormality minimizes bounds on relative sensitivity to perturbations, both for input and W. Intuitively, relative values are more crucial in the context of quantization. As an example, a perturbation of 0.05 might be less important when the true value is 10.0, but it is more likely to cause issues if the true value is 0.10.

III. Method

We use the quantized retraining (fine-tuning) method [25] on the pretrained weights of various convolutional neural network architectures. In this study, we explore the effect of orthogonality constraints by quantizing pretrained weights that are trained with and without orthogonality constraints. We first describe the intuition behind the use of orthogonality for improved quantized models by visiting perturbation theory (see Section III-A), then we move to the quantizer design used in the retraining phase which we keep the same in all the experiments (see Section III-B), and lastly explain how we set the orthogonality regularization while training networks (see Section III-C).

A. Background on How Orthogonality Lowers the Error Bounds

In this section, we first explain how orthogonality can improve error bounds from perturbation and quantization. First, we note that both fully-connected and convolution layers can be formulated as matrix multiplication [28]. For fully-connected layers, this is trivial. For a convolution layer with weight tensor $C \in R^{C,H,W,K}$, where C, H, W, K denote the number of input channels, filter height, filter width, and the number of output channels, respectively, the computation can be reformulated as a matrix multiplication where the weight matrix is of size $m \times n$, with $m = C \times H \times W$ and $n = K$. In the expressions below, we can associate A with the weight matrix, x as input, and b as the output of a fully-connected layer or convolution layer. For simplicity, we assume the mini-batch size is 1, so x and b are vectors.

Let $Ax = b$ and $(A + \delta A)(x + \delta x) = b + \delta b$ hold, where A is an $n \times n$ non-singular matrix, $x \in R^n$, $b \in R^n$, and δA, δx denote perturbations in A and x, respectively. δb denotes the resulting perturbations in b. $\kappa(A)$ is the condition number of A and is defined as $\kappa(A) = \|A\|\|A^+\|$. $\|A^+\|$ refers to the pseudo-inverse of A and when A is non-singular, it is equal to $\|A^{-1}\|$. This implies

$$\delta b = (\delta A)x + (\delta A)(\delta x) + A(\delta x). \tag{1}$$

Assume $\delta A = 0$ for simplicity. Thus,

$$\begin{aligned} \delta b = A(\delta x) \implies \|\delta b\| \le \|A\|\|\delta x\| \\ \implies \frac{\|\delta b\|}{\|b\|} \le \frac{\|A\|\|\delta x\|}{\|b\|} = \frac{\|A\|\|x\|\|\delta x\|}{\|b\|\|x\|} \\ = \frac{\|A\|\|A^{-1}b\|\|\delta x\|}{\|b\|\|x\|} \le \frac{\|A\|\|A^{-1}\|\|\delta x\|\|b\|}{\|b\|\|x\|} \\ = \kappa(A)\,\frac{\|\delta x\|}{\|x\|} \implies \frac{\frac{\|\delta b\|}{\|b\|}}{\frac{\|\delta x\|}{\|x\|}} \le \kappa(A) \end{aligned} \tag{2}$$

Source: https://ieeexplore.ieee.org/document/9772057 (published print)

Google Scholar research | 2026-10-08 | Article 01 | Original p. 3 | Orange: discussion; blue: reference

where norms represent any p-norm on matrices or vectors. Above, we used the fact that $\|A\|\|B\| \geq \|AB\|$ and $\|A\|\|x\| \geq \|Ax\|$ for any two matrices A, B and vector x for p-norms; these inequalities follow from the definition of p-norm. For 2-norm, $\kappa(A) = \sigma_1/\sigma_n$, the ratio of the largest and smallest singular values of A. The last inequality implies that relative sensitivity of the result with respect to perturbations on x is bounded by $\kappa(A)$. These bounds are reasonably sharp. For instance, one can show that equality holds when $\delta b = c_1\sigma_n$, $\delta x = c_i\sigma_n u_n$, $b = c_2 v_1$, where $c_1, c_2 \neq 0$, $c_1, c_2 \in R$, u_n, v_n, and σ_n are the n'th left singular vector, n'th right singular vector, and n'th singular value of A^{-1}, respectively, and v_i is the first right singular vector of A^{-1}.

Now assume A is a full rank non-square matrix, with size $m \times n$, where $m \geq n$. $Ax = b$ and (1) above still hold, but $b = A^{-1}x$ does not hold since A is not invertible when it is not square. Instead, we have $A^T Ax = A^T b$, where $A^T A$ is a symmetric positive definite matrix, and hence invertible because A is full rank. Note that full-rankness is a reasonable assumption since weights are initialized randomly in neural networks. Thus, $x = (A^T A)^{-1}A^T b$, and following the inequality given in (2) from the second line:

$$\begin{aligned}\frac{\|\delta b\|}{\|b\|} &\leq \frac{\|A\|\|\delta x\|\|(A^T A)^{-1}A^T b\|}{\|b\|\|x\|} \\ &\leq \frac{\|A\|\|\delta x\|\|(A^T A)^{-1}\|\|A^T\|\|b\|}{\|b\|\|x\|} \\ &= (\kappa(A))^2 \frac{\|\delta x\|}{\|x\|} \implies \frac{\frac{\|\delta b\|}{\|b\|}}{\frac{\|\delta x\|}{\|x\|}} \leq (\kappa(A))^2 \end{aligned} \tag{3}$$

where we used the fact that $\|A\|\|A^T\|\|(A^T A)^{-1}\| = (\kappa(A))^2$ [59]. Thus, similar error bounds exist for non-square A that directly depend on $\kappa(A)$. Now assume $\delta x = 0$, $\delta A, \delta b \neq 0$.

For square, nonsingular A

$$\begin{aligned}\delta b = A(\delta x) &\implies \frac{\|\delta b\|}{\|b\|} = \frac{\|A(\delta x)\|}{\|b\|} \leq \frac{\|\delta A\|\|x\|}{\|b\|} \\ \frac{\|\delta A\|\|x\|}{\|b\|} &= \frac{\|\delta A\|\|A^{-1}b\|}{\|b\|} \leq \frac{\|\delta A\|\|A^{-1}\|\|b\|}{\|b\|} \\ &= \frac{\|\delta A\|}{\|A\|}\|A^{-1}\|\|A\| \\ \implies \frac{\frac{\|\delta b\|}{\|b\|}}{\frac{\|\delta A\|}{\|A\|}} &\leq \kappa(A). \end{aligned} \tag{4}$$

For non-square, full rank A

$$\begin{aligned}\frac{\|\delta A\|\|x\|}{\|b\|} &= \frac{\|\delta A\|\|(A^T A)^{-1}A^T b\|}{\|b\|} \\ &\leq \frac{\|\delta A\|\|(A^T A)^{-1}\|\|A^T\|\|A\|}{\|A\|} \\ \implies \frac{\frac{\|\delta b\|}{\|b\|}}{\frac{\|\delta A\|}{\|A\|}} &\leq (\kappa(A))^2. \end{aligned} \tag{5}$$

This shows that for the cases above, the relative sensitivity of matrix multiplication with respect to both A and x (where A can be associated with weights and x with inputs) has sharp bounds that are directly a function of $\kappa(A)$. $\kappa(A)$ cannot be smaller than 1 [60], and orthogonality results in $\kappa(A)$ achieving its smallest value 1, thus minimizing the sharp upper bound of the relative sensitivities for the cases above.

For $m < n$, where A is a short and fat matrix and cannot be full rank, no bounds exist for relative sensitivity, and A also cannot be exactly orthogonal. One can still show that the following bound on sensitivity (but not relative sensitivity as used earlier) with respect to perturbations in x exists:

$$\frac{\|A(x+\delta x) - Ax\|}{\|\delta x\|} = \frac{\|A\delta x\|}{\|\delta x\|} \leq \|A\| \tag{6}$$

where $\|A\| = \sigma_1$, for the 2-norm, largest singular value of A. As explained below, the orthogonality constraint we use on A also imposes the columns to be normal, which would imply $\sigma_1 = 1$, which prohibits the error bound above from growing also, benefiting this case as well.

B. Quantizer Design

Uniform quantization allows using integer arithmetic for the multiply-add operations and is the most commonly used design choice [26]. Symmetric quantization refers to the case where 0 in the quantized integer domain is mapped to 0 in the real domain. Furthermore, we use signed integers in quantized representations. As a result, the mapping between real and quantized domains can be described as $x = s * q$ where x is in the real domain and q is in the quantized domain, and s is the scaling factor for mapping. Another design choice is related to the granularity of the scale picked for quantization. To utilize an integer math pipeline, the granularity needs to be per-tensor or per-row for activation tensors and per-tensor or per-column (i.e., per-filter) for weight tensors [26]. Per-tensor quantization results in a good performance and is much cheaper to implement [25], [26], so we use per-tensor scaling both for activations and tensors. We use simulated quantization similar to previous literature [24]–[26], where 16-bit floating-point values are quantized and dequantized with the following steps.

1) Quantize: This step consists of scaling, rounding, and clipping. For a per-tensor threshold t, scaling occurs such that the largest value that can be represented in the quantized domain ($2^{(b-1)}$ for signed integers for a bit-width of b) maps to the threshold value chosen. Scaling factor s is determined by threshold t. Then, nearest integer rounding with round-half-to-even is performed. Finally, elements that exist with the largest representable value in the integer domain are clipped to the largest representable integer value.

2) De-Quantize: This step reverses the scaling step in quantizing phase above. This is used to emulate the effect of quantization while keeping the scale of the tensor in a high-precision data type. Finally, the process can be defined as

$$\text{quant}(x) = \text{clip}(\text{round}(x/s); -2^{(b-1)}, 2^{(b-1)} - 1) \times s \tag{7}$$

where $s = t/2^{(b-1)}$. We use a straight-through-estimator (STE) to backpropagate through quantization nodes for the retraining phase. We also train the threshold t in log-domain with gradient descent for the input and weight tensors, as described in [25], for each convolution and fully-connected layer.

Source: https://ieeexplore.ieee.org/document/9772057 (published print)
Google Scholar research | 2026-10-08 | Article 01 | Original p. 4 | Orange: discussion; blue: reference

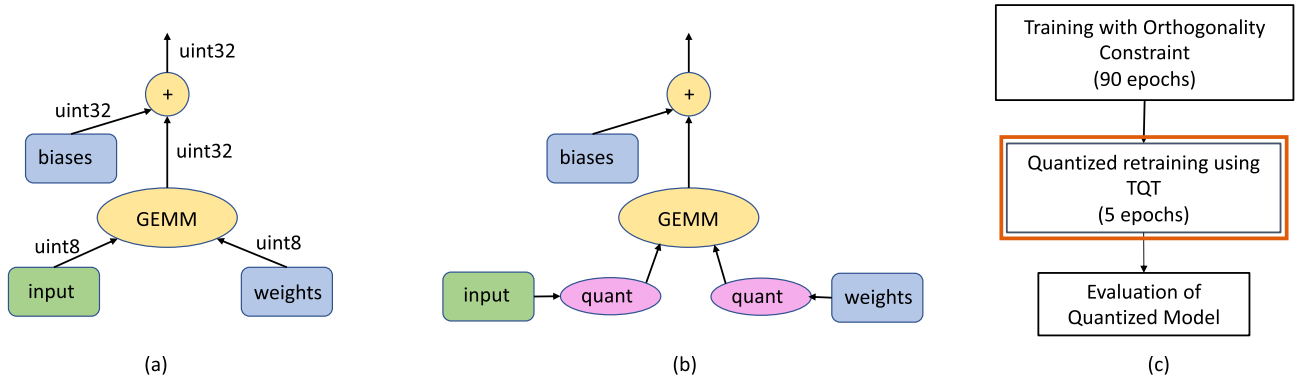


Fig. 1. (a) Integer-arithmetic inference, (b) simulated quantization for training diagrams, and (c) flowchart of the pipeline. GEMM refers to matrix-multiply operation, which includes both fully-connected layers and convolution layers. uint-b refers to unsigned b-bit integer quantization. For simulated quantization, all data is kept in the 32-bit floating points format, but quantization is simulated using quantize/de-quantize operations explained in Section III-B. Quantization is only performed for input and weight tensors of fully-connected and convolution layers, which implies that the batch-normalization layer is not quantized. This helps the study focus on the effect of weights trained with orthogonality penalty on quantization.

Fig. 1 describes the emulation of integer quantization during training. It is worth noting that we only use quantization nodes for the input tensors and weight tensors for convolution and fully-connected layers, which means batch-normalization layers are not quantized to keep the focus on the convolution and fully-connected layers for the purpose of understanding the effect of orthogonality of their weight parameters on quantization.

As mentioned, we use TQT [25] as our baseline retraining method. This means that we first train the model with and without orthogonality constraints from scratch and use the TQT retraining method on each of these trained models. The flowchart of the pipeline is given in Fig. 1(c).

C. Orthogonality Regularization

As mentioned in the related work section, different methods exist for orthogonality regularization. Results from hard constraints from the literature are not impressive, so we only consider soft orthogonality constraints. We use soft orthogonality regularization based on the Frobenius norm, which is given by

$$\lambda \| W^T W - I \|_F. \tag{8}$$

The reason why we choose this regularization is that it gives good performance in terms of accuracy [28], does not rely on iterative approximation as spectral-norm-based regularizations do, and has low-performance overhead during training (results in only a few percent reductions in training throughput). The gradient of this loss term can be expressed by $4\lambda W(W^T W - I)$, which can be seen as a different form of weight decay. One issue about this penalty term [28] is that for $W \in R^{m \times n}$, where $m < n$, $W^T W$ cannot be identity, since W is rank-deficient in that case. However, the double soft orthogonality penalty suggested in [28], which is an alternative that handles rank-deficient cases as explained earlier, performs worse than regular soft orthogonal regularization. Bansal *et al.* [28] suggests that orthogonality restriction for rank-deficient matrices can result in additional bias during training, but results from the same study suggests this bias can still be useful for accuracy. For this reason, we stick with soft orthogonal regularization based on the Frobenius norm. However, we believe there is value in exploring different orthogonality regularization formulations in the context of quantization, as explained later in the article with guidance from experimental results obtained in this study.

IV. Experiments

We run extensive experiments on three popular convolutional neural networks, namely ResNet50, ResNet101 [61], and VGG19 [62] with ImageNet dataset. For ease of direct reproducibility, we use the hyperparameters suggested in the PyTorch vision library, unless otherwise noted.[1] In the initial stage, we train both models with different weight decay (w_d) and orthogonality penalty parameter (λ) values. We performed a hyperparameter search for (w_d), where we start with the default value and decrease or increase by powers of 10. For each of these cases, we also search for (λ) within values of 0 as well as powers of 10, where (λ) being 0 means no orthogonality penalty is used. For fairness, each w_d/λ configuration is trained exactly once on Resnet50, Resnet101, and VGG19 networks for 90-epochs for the initial training phase. This makes sure that there is no cherry-picking involved in the experiments. Each training run takes roughly 20 h on a DGXV100-128 GB system.

After the initial 90-epoch training sessions with or without orthogonality penalty, we use the TQTs method on all trained models (for each w_d/λ configuration). We initialize the trainable threshold parameters for the quantization node using the threshold values obtained by running max calibration [42] both for activation and weights on the trained weights. We run the TQT retraining phase separately for three precision configurations: 8/8, 6/6, and 8/4, where A/W refers to bitwidths of input activation and weight matrix, respectively. For each case, TQT retraining is done for five epochs, as suggested in [25]. Each TQT run (five epochs) takes around 15 min on a DGXV100 system. There are some implementation differences in how

[1] https://github.com/pytorch/vision/tree/master/references

Source: https://ieeexplore.ieee.org/document/9772057 (published print)
Google Scholar research | 2026-10-08 | Article 01 | Original p. 5 | Orange: discussion; blue: reference

TABLE I

TOP1 AND TOP5 VALIDATION ACCURACIES OF DIFFERENT NETWORK ARCHITECTURES. QUANTIZATION BITWIDTH CONFIGURATION IS REPRESENTED AS *A*/*W*, WHERE *A* DENOTES ACTIVATION BIT-WIDTH AND *W* DENOTES THE BIT-WIDTH OF WEIGHT PARAMETERS. THE FIRST ROW FOR EACH *A*/*W* CASE SHOWS w_d THAT RESULTS IN THE HIGHEST FP32 ACCURACY WITHOUT ORTHOGONALITY REGULARIZATION. THE SECOND ROW SHOWS THE CASE WITH ORTHOGONALITY REGULARIZATION, WITH w_d/λ THAT RESULTS IN THE CLOSEST FP32 ACCURACY TO THE FIRST ROW. NOTE THAT DESPITE FP32 ACCURACIES BEING ALMOST IDENTICAL FOR EACH *A*/*W* CASE, QUANTIZED ACCURACY IS IMPROVED WITH ORTHOGONALITY PENALTY

ResNet50	w_d	λ	FP32 accuracy before TQT	Quantized accuracy after TQT
8/8 no Orthogonal regularizer	10^{-4}	-	75.37 / 92.58	75.09 / 92.39
8/8 with Orthogonal regularizer	10^{-5}	10^{-5}	75.88 / 92.89	**75.68 / 92.79**
6/6 no Orthogonal regularizer	10^{-4}	-	75.37 / 92.58	73.25 / 91.24
6/6 with Orthogonal regularizer	10^{-5}	10^{-6}	75.38 / 92.50	**74.27 / 91.77**
8/4 no Orthogonal regularizer	10^{-4}	-	75.37 / 92.58	69.53 / 89.11
8/4 with Orthogonal regularizer	10^{-5}	10^{-7}	74.64 / 91.74	**71.88 / 90.20**
ResNet101	w_d	λ	FP32 accuracy before TQT	Quantized accuracy after TQT
8/8 no Orthogonal regularizer	10^{-4}	-	77.09 / 93.57	76.77 / 93.35
8/8 with Orthogonal regularizer	10^{-4}	10^{-5}	77.57 / 93.85	**77.30 / 93.72**
6/6 no Orthogonal regularizer	10^{-4}	-	77.09 / 93.57	75.43 / 92.59
6/6 with Orthogonal regularizer	10^{-5}	10^{-6}	76.90 / 93.26	**76.12 / 92.77**
8/4 no Orthogonal regularizer	10^{-4}	-	77.09 / 93.57	71.78 / 90.50
8/4 with Orthogonal regularizer	10^{-5}	10^{-6}	76.90 / 93.26	**73.79 / 91.51**
VGG19	w_d	λ	FP32 accuracy before TQT	Quantized accuracy after TQT
8/8 no Orthogonal regularizer	10^{-4}	-	72.31 / 90.89	72.40 / 90.86
8/8 with Orthogonal regularizer	10^{-4}	10^{-5}	72.68 / 91.28	**72.80 / 91.26**
6/6 no Orthogonal regularizer	10^{-4}	-	72.31 / 90.89	71.62 / 90.46
6/6 with Orthogonal regularizer	10^{-4}	10^{-5}	72.80 / 91.26	**72.17 / 90.85**
8/4 no Orthogonal regularizer	10^{-4}	-	72.31 / 90.89	71.24 / 90.29
8/4 with Orthogonal regularizer	10^{-4}	10^{-5}	72.68 / 91.28	**71.79 / 90.74**

we apply the TQT method compared to [25]. Namely, they switch to Adam optimizer for the TQT phase and implement a sophisticated learning rate schedule during retraining. For simplicity, we use a stochastic gradient descent optimizer during quantized retraining similar to the initial model training phase and use the same hyperparameters as if the training was the continuation of the initial 90-epoch training phase. For quantization threshold, we still use a separate learning rate similar to [25], a learning rate of 10^{-4} for quantization thresholds during retraining. Another difference in our implementation is that we initialize quantization thresholds for input activations for retraining using the max-calibration (MAX) method explained in [25] as opposed to the KL-J method, where MAX uses 50 unlabeled images randomly selected from the validation set and computes maximum absolute value within the resulting input activation for initializing quantization thresholds for activation. This is done for simplicity and ease of reproducibility. Quantization thresholds for weights also use the MAX method, which is similar to [25]. We use per-tensor quantization both for activations and weights, instead of per-channel.

V. RESULTS

This section presents the results of various networks with (see Section V-A) and without TQT (see Section V-C). We investigate the training stability by reporting the training curves in Section V-B and provide an in-depth analysis in Section V-D.

A. Experiments With TQT

Results are summarized in Table I for ResNet50, ResNet101, and VGG19 models. In Table I, for each model and *A*/*W* configuration, the first two rows show the cases without and with orthogonality regularizer, respectively, where their pre-TQT FP32 accuracies are almost identical. Sometimes, one of them achieves a slightly better score than the other before quantization. With an orthogonality regularizer, quantized accuracies improve noticeably, despite FP32 accuracies being very close. For ResNet50, for 8-bit activation width, and 4 bit for weight parameters, FP32 accuracy for the network with no orthogonal regularizer achieves 75.37% Top 1 accuracy versus 74.64% Top 1 accuracy achieved by the network with the orthogonal regularizer. On the other hand, when these networks are quantized, the network trained with the orthogonal regularizer achieves 71.88% Top 1 accuracy, a significantly better result than the 69.53% achieved by the network which is trained without the orthogonal regularizer. The results are consistent for ResNet50, ResNet101, and VGG19 models for 8/8, 6/6, 8/4 bitwidth configurations as can be seen from Table I. Later, we analyze where this difference is coming from.

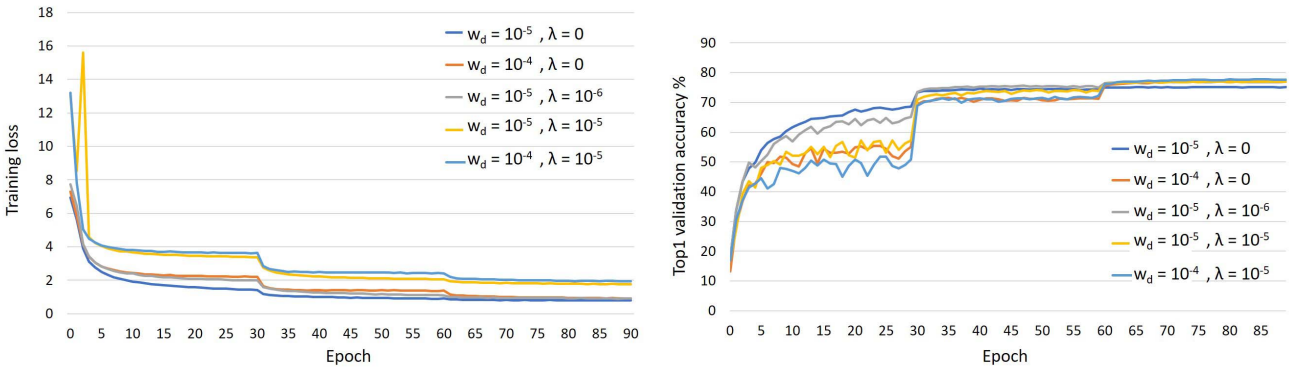
We also see that to achieve optimal results, the orthogonality penalty often requires reducing the weight decay penalty. In general, there is a balance between weight decay, which reduces spectral norm, and orthogonality penalty, which reduces the condition number, of weight matrices. This balance is further explored in Section . It is also important to note that more aggressive quantization gives better results with reduced regularization, both for weight decay and orthogonality penalties. This is expected as quantized training (TQT) also acts as a form of a regularizer by itself, and it may not favor additional regularizers being very strict.

B. Training Curves

Fig. 2 shows the evolution of Resnet101 training for different configurations of weight decay (w_d) and orthonormality

Source: https://ieeexplore.ieee.org/document/9772057 (published print)

Google Scholar research | 2026-10-08 | Article 01 | Original p. 6 | Orange: discussion; blue: reference



(a) (b)

Fig. 2. Evolution of training for Resnet101 for varying combinations of weight decay (w_d) and orthonormality penalties (λ). Training loss for higher λ is higher due to additional terms in the loss function. In the validation accuracy curve, the regularization effect of the orthogonality penalty is more clearly observed, and its effect on the evolution of validation accuracy seems similar to that of increasing weight decay. (a) Evolution of training loss. (b) Evolution of Top1 validation accuracy.

TABLE II

IN THIS EXPERIMENT, QUANTIZATION IS PERFORMED RIGHT AFTER THE INITIAL 90-EPOCH TRAINING PHASE, E.G., WITH NO ADDITIONAL QUANTIZED RETRAINING. QUANTIZATION BITWIDTH CONFIGURATION IS REPRESENTED AS A/W, WHERE A DENOTES ACTIVATION BIT-WIDTH AND W DENOTES THE BIT-WIDTH OF WEIGHT PARAMETERS. THE CORRESPONDING w_d IS SET TO 10^{-4} AND λ VALUES ARE SHOWN. EXPERIMENTS ARE REPEATED THREE TIMES AND MEDIAN VALUES AND MAXIMUM DEVIATION FROM THE MEDIAN VALUES ARE PRESENTED. THE IMPROVEMENTS COMING FROM THE ORTHOGONAL REGULARIZER ARE EVEN MORE SIGNIFICANT IN THIS SETTING. WE ALSO FIND THAT RESULTS FROM NETWORKS TRAINED WITH ORTHOGONAL REGULARIZER ARE MORE ROBOUST SHOWING SMALLER DEVIATION FROM THE MEDIAN VALUES

Resnet50v1.5	λ	FP32 accuracy before quantization	Quantized accuracy
8/8 no Orthogonal regularizer	0	76.29±0.28 / 93.06±0.13	76.12±0.24 / 92.98±0.10
8/8 with Orthogonal regularizer	10^{-5}	76.52±0.05 / 93.01±0.04	76.31±0.10 / 92.89±0.05
6/6 no Orthogonal regularizer	0	76.29±0.28 / 93.06±0.13	74.19±0.37 / 91.95±0.23
6/6 with Orthogonal regularizer	10^{-5}	76.52±0.05 / 93.01±0.04	74.93±0.14 / 92.21±0.15
5/5 no Orthogonal regularizer	0	76.29±0.28 / 93.06±0.13	65.36±0.51 / 86.78±0.33
5/5 with Orthogonal regularizer	10^{-5}	76.52±0.05 / 93.01±0.04	70.58±0.33 / 89.62±0.17

penalties (λ). Training loss for higher λ is higher due to additional term in the loss function. There is a spike observed for the setting of $w_d = 10^{-5}$ and $\lambda = 10^{-5}$ which suggests that one can potentially come up with a more carefully tailored orthogonality penalty schedule during training, as opposed to constant λ. However, we did not find this occasional spike that happens at the early stage of the training affecting the validation scores. As can be seen, training is very stable with the training loss decreasing at every epoch. The learning rate is halved at epochs 30 and 60. In the validation accuracy curve, the regularization effect of the orthogonality penalty is more clearly observed, and its effect on the evolution of validation accuracy seems similar to that of increasing weight decay. Even though the effect on the curves seems similar for these regularizers, their effect on quantized accuracy is different as will also be analyzed in Section V-D.

C. Experiments Without TQT

Table II shows results for a setup that is different from previously presented. We look at the impact of the orthogonalization penalty on quantized accuracy when no additional quantization-aware retraining is used. In other words, the model is directly quantized by post-training quantization (using MAX calibration as explained in the main text) after the 90-epoch training session. These results are obtained with Resnet50v1.5. For each configuration, three experiments are run and the median is shown, together with the maximum deviation from the median. In this scenario, introducing an orthogonality penalty improves quantization by a lot more than in the case where TQT is used since without TQT there is a lot more to gain from orthogonality. Namely, running TQT helps improve quantized accuracy by adjusting the ranges, and without TQT, the orthogonality penalty is the only mechanism that can adjust the weights so that the impact of perturbation due to quantization is reduced. Notice that FP32 accuracies are very similar with and without the orthogonality penalty, but the gap between quantized accuracies widens especially for more aggressive quantization. Specifically, for 5-bit precision, the quantized Top 1 accuracy is 65.36% for the model without orthogonality penalty, whereas it is 70.58% for the model with orthogonality penalty, showing a significant accuracy gain of the orthogonality.

In this setup, we also provide the maximum deviation from the median among the three runs of experiments. We find that models trained with an orthogonal regularizer output more robust results and deviate less compared to the networks trained without an orthogonal regularizer. This holds for both FP32 accuracy before quantization and quantized accuracy. We do not aggressively experiment with this setup and only

Source: https://ieeexplore.ieee.org/document/9772057 (published print)
Google Scholar research | 2026-10-08 | Article 01 | Original p. 7 | Orange: discussion; blue: reference

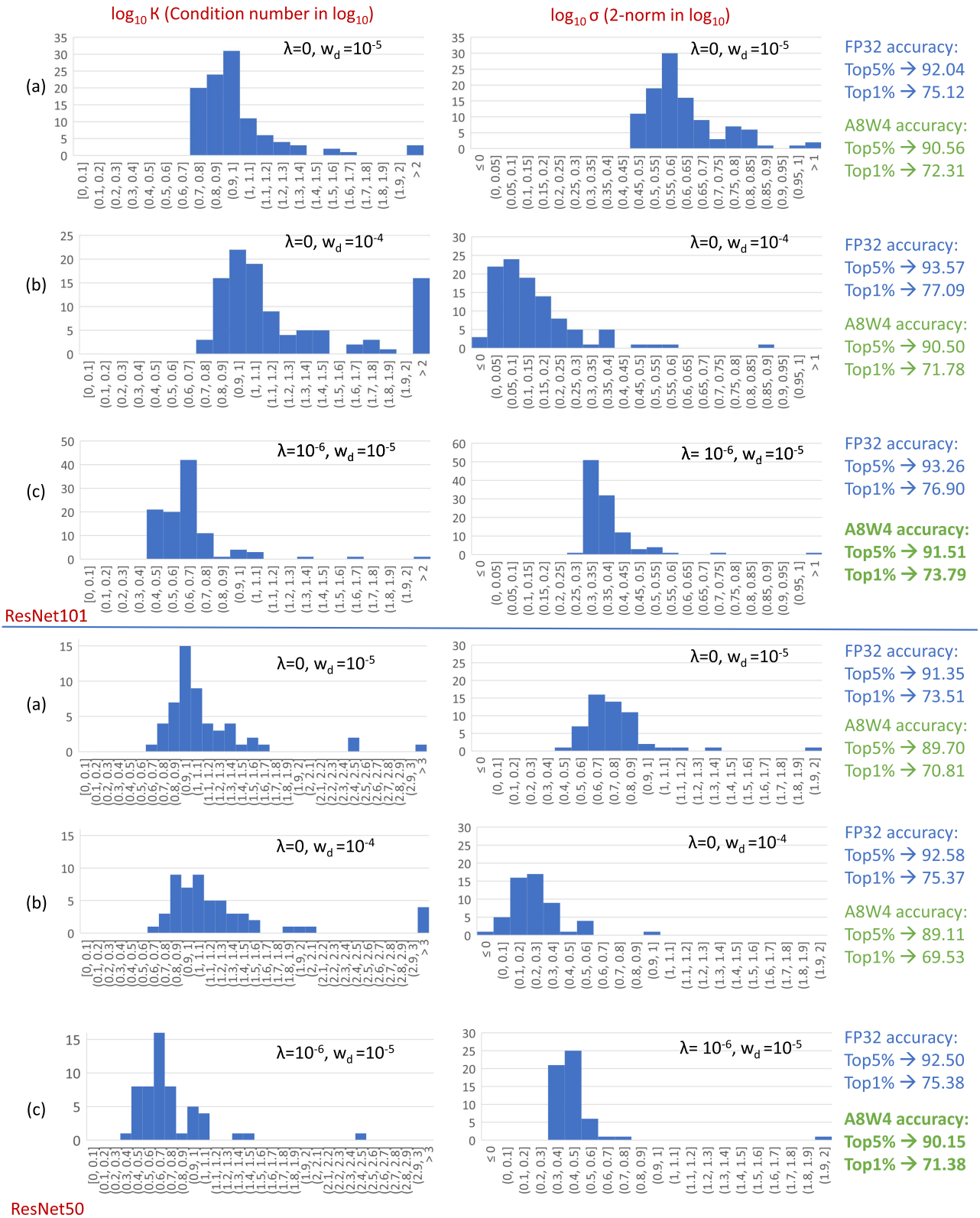


Fig. 3. Histogram of condition numbers and histogram of 2-norms of weight matrices for all convolution or fully connected layers for ResNet101 and ResNet50 after training and before TQT. From (a) to (b), the weight decay coefficient increases, while the orthonormality penalty stays 0, which reduces 2-norms (spectral norms) significantly but slightly increases the condition numbers (the distribution shifts to the right). This increased regularization increases FP32 validation accuracy (before quantization), but reduces quantized accuracy (obtained with TQT). On the other hand, adding the orthonormality penalty on top of (a) which is presented in (c) does not reduce 2-norms as much as (b), but substantially reduces condition numbers of weight matrices, also giving rise to higher quantized accuracies.

Source: https://ieeexplore.ieee.org/document/9772057 (published print)
Google Scholar research | 2026-10-08 | Article 01 | Original p. 8 | Orange: discussion; blue: reference

run with ResNet50, since this setup is less challenging to showcase improvements.

D. Analysis

Table I shows that despite FP32 accuracies in the non-quantized case can be very close, the orthogonality penalty improves quantized accuracy compared to the case with no orthogonality penalty. This shows that not all regularizers are created equally when it comes to quantization, and two regularizers that result in similar FP32 accuracies may not perform the same after quantization. We show further analysis below to explain where this difference stems from.

We analyze how the distribution of condition number and spectral norm of weight matrices vary after the initial 90-epoch training stage with different orthogonality regularizer (λ) and weight decay (w_d) configurations, for Resnet101 and Resnet50 in Fig. 3. We find that behavior is consistent between the two models under these two different regularizers. As we increase the weight decay coefficient from Fig. 3(a) and (b) (w_d increases to 10^{-4} from 10^{-5}) without the orthogonal regularizer, spectral norms reduce substantially, while condition numbers stay similar or slightly increase. With the original weight decay parameter but with orthogonality penalty introduced, from Fig. 3(a)–(c), condition numbers reduce substantially, resulting in optimal quantized accuracy among these three cases. This shows that the reduction in condition numbers is the cause for the improved quantization and not the spectral norm. This finding aligns with the theoretical background given in the Methods section on how a lower condition number reduces the bounds on the perturbation error, wherein our case quantization represents the perturbation.

This finding is important and an important contribution to this work as it shows for the first time, in contrast to existing literature, improved quantization due to orthogonality in neural networks is because of the reduction in condition number of weight parameters, and not due to the reduction in the spectral norm. This behavior is in line with the results presented in Table I. As shown in Table I, ImageNet Top5 accuracy loss for 4- to 8-bit quantization is reduced by up to 7% for Resnet50, and up to 10% for Resnet101, compared to trained quantization with no orthogonality penalty. This shows that quantized training does not quite have the same effect as the orthogonality penalty, and the orthogonality penalty still improves quantization when combined with a quantized training method.

VI. Conclusion

In this work, we theoretically explain and experimentally show that the ability of orthogonality constraint to reduce the condition number is the key feature that improves the accuracy of the quantized model, more so than the reduction in spectral norms of weights. To the best of our knowledge, this is shown for the first time and in contrast to existing literature. We experiment with Resnet50, Resnet101, and VGG19 and combine the orthogonality penalty with a state-of-the-art TQT quantized retraining method for quantization. Improvements are consistent for all networks for every bit of setting. This shows that quantized training does not capture all the benefits of the orthogonality penalty.

References

[1] S. Chetlur *et al.*, "CuDNN: Efficient primitives for deep learning," 2014, *arXiv:1410.0759*.

[2] V. Gokhale, J. Jin, A. Dundar, B. Martini, and E. Culurciello, "A 240 G-ops/s mobile coprocessor for deep neural networks," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. Workshops*, Jun. 2014, pp. 682–687.

[3] A. Dundar, J. Jin, V. Gokhale, B. Martini, and E. Culurciello, "Memory access optimized routing scheme for deep networks on a mobile coprocessor," in *Proc. IEEE High Perform. Extreme Comput. Conf. (HPEC)*, Sep. 2014, pp. 1–6.

[4] J. Jin, V. Gokhale, A. Dundar, B. Krishnamurthy, B. Martini, and E. Culurciello, "An efficient implementation of deep convolutional neural networks on a mobile coprocessor," in *Proc. IEEE 57th Int. Midwest Symp. Circuits Syst. (MWSCAS)*, Aug. 2014, pp. 133–136.

[5] A. Dundar, J. Jin, B. Martini, and E. Culurciello, "Embedded streaming deep neural networks accelerator with applications," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 28, no. 7, pp. 1572–1583, Jul. 2017.

[6] V. Sze, Y.-H. Chen, T.-J. Yang, and J. S. Emer, "Efficient processing of deep neural networks: A tutorial and survey," *Proc. IEEE*, vol. 105, no. 12, pp. 2295–2329, Dec. 2017.

[7] J. Jin, A. Dundar, and E. Culurciello, "Flattened convolutional neural networks for feedforward acceleration," 2014, *arXiv:1412.5474*.

[8] F. Chollet, "Xception: Deep learning with depthwise separable convolutions," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jul. 2017, pp. 1251–1258.

[9] A. G. Howard *et al.*, "MobileNets: Efficient convolutional neural networks for mobile vision applications," 2017, *arXiv:1704.04861*.

[10] X. Zhang, X. Zhou, M. Lin, and J. Sun, "ShuffleNet: An extremely efficient convolutional neural network for mobile devices," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2018, pp. 6848–6856.

[11] L.-C. Chen, Y. Zhu, G. Papandreou, F. Schroff, and H. Adam, "Encoder-decoder with atrous separable convolution for semantic image segmentation," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2018, pp. 801–818.

[12] G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," 2015, *arXiv:1503.02531*.

[13] A. Polino, R. Pascanu, and D. Alistarh, "Model compression via distillation and quantization," in *Proc. Int. Conf. Learn. Represent.*, 2018.

[14] A. Mishra and D. Marr, "Apprentice: Using knowledge distillation techniques to improve low-precision network accuracy," 2017, *arXiv:1711.05852*.

[15] H. Yin *et al.*, "Dreaming to distill: Data-free knowledge transfer via DeepInversion," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2020, pp. 8715–8724.

[16] J. M. Alvarez and M. Salzmann, "Learning the number of neurons in deep networks," in *Proc. Adv. Neural Inf. Process. Syst.*, 2016, pp. 2270–2278.

[17] Y. He, X. Zhang, and J. Sun, "Channel pruning for accelerating very deep neural networks," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, Oct. 2017, pp. 1389–1397.

[18] G. Huang, S. Liu, L. V. D. Maaten, and K. Q. Weinberger, "CondenseNet: An efficient DenseNet using learned group convolutions," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2018, pp. 2752–2761.

[19] I. Hubara, M. Courbariaux, D. Soudry, R. El-Yaniv, and Y. Bengio, "Binarized neural networks," in *Proc. Adv. Neural Inf. Process. Syst.*, 2016, pp. 4107–4115.

[20] F. Li, B. Zhang, and B. Liu, "Ternary weight networks," 2016, *arXiv:1605.04711*.

[21] M. Rastegari, V. Ordonez, J. Redmon, and A. Farhadi, "XNOR-Net: Imagenet classification using binary convolutional neural networks," in *Proc. Eur. Conf. Comput. Vis.* New York, NY, USA: Springer, 2016, pp. 525–542.

[22] T. Liang, J. Glossner, L. Wang, S. Shi, and X. Zhang, "Pruning and quantization for deep neural network acceleration: A survey," *Neurocomputing*, vol. 461, pp. 370–403, Oct. 2021.

[23] V. Vanhoucke, A. Senior, and M. Z. Mao, "Improving the speed of neural networks on CPUs," in *Proc. Deep Learn. Unsupervised Feature Learn. Workshop, Adv. Neural Inf. Process. Syst. (NIPS)*, 2011.

[24] B. Jacob *et al.*, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2018, pp. 2704–2713.

Source: https://ieeexplore.ieee.org/document/9772057 (published print)

Google Scholar research | 2026-10-08 | Article 01 | Original p. 9 | Orange: discussion; blue: reference

[25] S. R. Jain, A. Gural, M. Wu, and C. H. Dick, "Trained quantization thresholds for accurate and efficient fixed-point inference of deep neural networks," 2019, *arXiv:1903.08066*.
[26] H. Wu, P. Judd, X. Zhang, M. Isaev, and P. Micikevicius, "Integer quantization for deep learning inference: Principles and empirical evaluation," 2020, *arXiv:2004.09602*.
[27] D. Xie, J. Xiong, and S. Pu, "All you need is beyond a good init: Exploring better solution for training extremely deep convolutional neural networks with orthonormality and modulation," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jul. 2017, pp. 6176–6185.
[28] N. Bansal, X. Chen, and Z. Wang, "Can we gain more from orthogonality regularizations in training deep networks?" in *Proc. Adv. Neural Inf. Process. Syst.*, 2018, pp. 4261–4271.
[29] T. Miyato, T. Kataoka, M. Koyama, and Y. Yoshida, "Spectral normalization for generative adversarial networks," in *Proc. Int. Conf. Learn. Represent.*, 2018.
[30] M. Cisse, P. Bojanowski, E. Grave, Y. Dauphin, and N. Usunier, "Parseval networks: Improving robustness to adversarial examples," in *Proc. 34th Int. Conf. Mach. Learn.*, 2017, pp. 854–863.
[31] J. Lin, C. Gan, and S. Han, "Defensive quantization: When efficiency meets robustness," in *Proc. Int. Conf. Learn. Represent.*, 2019. [Online]. Available: https://openreview.net/forum?id=ryetZ20ctX
[32] M. Alizadeh, A. Behboodi, M. van Baalen, C. Louizos, T. Blankevoort, and M. Welling, "Gradient ℓ_1 regularization for quantization robustness," in *Proc. Int. Conf. Learn. Represent.*, 2020. [Online]. Available: https://openreview.net/forum?id=ryxK0JBtPr
[33] C. Zhu, S. Han, H. Mao, and W. J. Dally, "Trained ternary quantization," in *Proc. Int. Conf. Learn. Represent.*, 2016.
[34] M. Courbariaux, I. Hubara, D. Soudry, R. El-Yaniv, and Y. Bengio, "Binarized neural networks: Training deep neural networks with weights and activations constrained to +1 or -1," 2016, *arXiv:1602.02830*.
[35] S. Zhou, Y. Wu, Z. Ni, X. Zhou, H. Wen, and Y. Zou, "DoReFa-Net: Training low bitwidth convolutional neural networks with low bitwidth gradients," 2016, *arXiv:1606.06160*.
[36] D. Zhang, J. Yang, D. Ye, and G. Hua, "LQ-Nets: Learned quantization for highly accurate and compact deep neural networks," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2018, pp. 365–382.
[37] A. Mishra, E. Nurvitadhi, J. J. Cook, and D. Marr, "WRPN: Wide reduced-precision networks," in *Proc. Int. Conf. Learn. Represent.*, 2018.
[38] Z. Cai, X. He, J. Sun, and N. Vasconcelos, "Deep learning with low precision by half-wave Gaussian quantization," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jul. 2017, pp. 5918–5926.
[39] S. Jung *et al.*, "Learning to quantize deep networks by optimizing quantization intervals with task loss," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2019, pp. 4350–4359.
[40] M. Nagel, R. A. Amjad, M. Van Baalen, C. Louizos, and T. Blankevoort, "Up or down? Adaptive rounding for post-training quantization," in *Proc. 37th Int. Conf. Mach. Learn.*, 2020, pp. 7197–7206.
[41] Y. Li *et al.*, "BRECQ: Pushing the limit of post-training quantization by block reconstruction," in *Proc. Int. Conf. Learn. Represent.*, 2020.
[42] S. Migacz, "8-bit inference with tensorrt," in *Proc. GPU Technol. Conf.*, 2017, vol. 2, no. 4, p. 5.
[43] J. Choi, Z. Wang, S. Venkataramani, P. I.-Jen Chuang, V. Srinivasan, and K. Gopalakrishnan, "PACT: Parameterized clipping activation for quantized neural networks," 2018, *arXiv:1805.06085*.
[44] C. Baskin *et al.*, "NICE: Noise injection and clamping estimation for neural network quantization," 2018, *arXiv:1810.00162*.
[45] A. Goncharenko, A. Denisov, S. Alyamkin, and E. Terentev, "Fast adjustable threshold for uniform neural network quantization (winning solution of LPIRC-II)," *Int. J. Comput. Inf. Eng.*, vol. 13, no. 9, pp. 495–499, 2019.
[46] Y. Bengio, N. Léonard, and A. Courville, "Estimating or propagating gradients through stochastic neurons for conditional computation," 2013, *arXiv:1308.3432*.
[47] R. Pascanu, T. Mikolov, and Y. Bengio, "On the difficulty of training recurrent neural networks," in *Proc. Int. Conf. Mach. Learn.*, 2013, pp. 1310–1318.
[48] V. Dorobantu, P. Andre Stromhaug, and J. Renteria, "DizzyRNN: Reparameterizing recurrent neural networks for norm-preserving backpropagation," 2016, *arXiv:1612.04035*.
[49] M. Arjovsky, A. Shah, and Y. Bengio, "Unitary evolution recurrent neural networks," in *Proc. Int. Conf. Mach. Learn.*, 2016, pp. 1120–1128.
[50] Z. Mhammedi, A. Hellicar, A. Rahman, and J. Bailey, "Efficient orthogonal parametrisation of recurrent neural networks using householder reflections," in *Proc. Int. Conf. Mach. Learn.*, 2017, pp. 2401–2409.
[51] E. Vorontsov, C. Trabelsi, S. Kadoury, and C. Pal, "On orthogonality and learning recurrent networks with long term dependencies," in *Proc. Int. Conf. Mach. Learn.*, 2017, pp. 3570–3578.
[52] S. Wisdom, T. Powers, J. Hershey, J. Le Roux, and L. Atlas, "Full-capacity unitary recurrent neural networks," in *Proc. Adv. Neural Inf. Process. Syst.*, 2016, pp. 4880–4888.
[53] A. M. Saxe, J. L. McClelland, and S. Ganguli, "Exact solutions to the nonlinear dynamics of learning in deep linear neural networks," 2013, *arXiv:1312.6120*.
[54] D. Mishkin and J. Matas, "All you need is a good init," 2015, *arXiv:1511.06422*.
[55] K. Jia, D. Tao, S. Gao, and X. Xu, "Improving training of deep neural networks via singular value bounding," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jul. 2017, pp. 4344–4352.
[56] M. Harandi and B. Fernando, "Generalized BackPropagation, Étude de cas: Orthogonality," 2016, *arXiv:1611.05927*.
[57] M. Ozay and T. Okatani, "Optimization on submanifolds of convolution kernels in CNNs," 2016, *arXiv:1610.07008*.
[58] Y. Yoshida and T. Miyato, "Spectral norm regularization for improving the generalizability of deep learning," 2017, *arXiv:1705.10941*.
[59] E. Darve and M. Wootters, *Numerical Linear Algebra With Julia*, vol. 172. Philadelphia, PA, USA: SIAM, 2021.
[60] G. H. Golub and C. F. Van Loan, *Matrix Computations*. Baltimore, MD, USA: Johns Hopkins Univ. Press, 2013.
[61] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2016, pp. 770–778.
[62] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," in *Proc. Int. Conf. Learn. Represent.*, 2015.



Sukru Burc Eryilmaz received the B.Sc. degree in electrical and electronics engineering from Bilkent University, Ankara, Turkey, and the M.S. and Ph.D. degrees from Stanford University, Stanford, CA, USA, in 2011, 2013, and 2017 respectively.

He is currently a Senior Architect of deep learning architecture with NVIDIA Corporation, Santa Clara, CA, USA, where he works on optimizing deep learning hardware and software, both at single-chip scale and supercomputer scale. His research interests include performance analysis and optimization of machine learning workloads, distributed neural network training, AI systems HW/SW co-design, and numerical linear algebra.



Aysegul Dundar received the B.Sc. degree in electrical and electronics engineering from Boğaziçi University, İstanbul, Turkey, in 2011, and the Ph.D. degree from Purdue University, West Lafayette, IN, USA, in 2016, under the supervision of Prof. E. Culurciello.

She is currently an Assistant Professor of computer science with Bilkent University, Ankara, Turkey. In CVPR 2018, she won first place in the Domain Adaptation for Semantic Segmentation Competition in the Workshop on Autonomous Vehicle challenge.

Source: https://ieeexplore.ieee.org/document/9772057 (published print)
Google Scholar research | 2026-10-08 | Article 01 | Original p. 10 | Orange: discussion; blue: reference

MIXED PRECISION DNNS: ALL YOU NEED IS A GOOD PARAMETRIZATION

Stefan Uhlich[*], **Lukas Mauch**[*], **Fabien Cardinaux**[*], **Kazuki Yoshiyama**
Javier Alonso García, **Stephen Tiedemann**, **Thomas Kemp**
Sony Europe B.V., Germany
firstname.lastname@sony.com

Akira Nakamura
Sony Corporate, Japan
akira.b.nakamura@sony.com

ABSTRACT

Efficient deep neural network (DNN) inference on mobile or embedded devices typically involves quantization of the network parameters and activations. In particular, mixed precision networks achieve better performance than networks with homogeneous bitwidth for the same size constraint. Since choosing the optimal bitwidths is not straight forward, training methods, which can learn them, are desirable. Differentiable quantization with straight-through gradients allows to learn the quantizer's parameters using gradient methods. We show that a suited parametrization of the quantizer is the key to achieve a stable training and a good final performance. Specifically, we propose to parametrize the quantizer with the step size and dynamic range. The bitwidth can then be inferred from them. Other parametrizations, which explicitly use the bitwidth, consistently perform worse. We confirm our findings with experiments on CIFAR-10 and ImageNet and we obtain mixed precision DNNs with learned quantization parameters, achieving state-of-the-art performance.

1 INTRODUCTION

Quantized DNNs apply quantizers $Q : \mathbb{R} \to \{q_1, ..., q_I\}$ to discretize the weights and/or activations of a DNN (Han et al., 2015; Zhou et al., 2017; Li et al., 2016; Liu & Mattina, 2019; Cardinaux et al., 2018; Jain et al., 2019; Bai et al., 2018). They require considerably less memory and have a lower computational complexity, since discretized values $\{q_1, ..., q_I\}$ can be stored, multiplied and accumulated efficiently. This is particularly relevant for inference on mobile or embedded devices with limited computational power.

However, gradient based training of quantized DNNs is difficult, as the gradient of a quantization function vanishes almost everywhere, i.e., backpropagation through a quantized DNN almost always returns a zero gradient. Different solutions to this problem have been proposed in the literature:
A first possibility is to use DNNs with stochastic weights from a categorical distribution and to optimize the *evidence lower bound* (ELBO) to obtain an estimate of the posterior distribution of the weights. As proposed in (Jang et al., 2016; Maddison et al., 2016; Louizos et al., 2019), the categorical distribution can be relaxed to a concrete distribution – a smoothed approximation of the categorical distribution – such that the ELBO becomes differentiable under reparametrization.
A second possibility is to use the *straight through estimator* (STE) (Bengio et al., 2013). STE allows the gradients to be backpropagated through the quantizers and, thus, the network weights can be adapted with standard gradient descent (Hubara et al., 2016). Compared to STE based methods, stochastic methods suffer from large gradient variance, which makes training of large quantized DNNs difficult. Therefore, STE based methods are more popular in practice.

More recent research (Jain et al., 2019; Esser et al., 2019; Wang et al., 2018; Elthakeb et al., 2018) focuses on methods which can also learn the optimal quantization parameters, e.g., the stepsize, dynamic range and bitwidth, in parallel to the network weights. This is a promising approach as DNNs with learned quantization parameters almost always outperform DNNs with handcrafted ones.

[*]Equal contribution.
The source code can be found at https://github.com/sony/ai-research-code.

Source: https://arxiv.org/pdf/1905.11452
Google Scholar research | 2026-10-07 | Article 02 | Original p. 1 | Orange: discussion; blue: reference

Recently, and in parallel to our work, (Jain et al., 2019) explored the use of STE to define the gradient with respect to the quantizers's dynamic range. The authors applied a per-tensor quantization and used the dynamic range as an additional trainable parameter also learned with gradient descent. Similarly, (Esser et al., 2019) learned the stepsize using gradient descent. However, neither of them learned the optimal bitwidth of the quantizers.

One approach was proposed in (Wang et al., 2018; Elthakeb et al., 2018). They learn the bitwidth with reinforcement learning, i.e., they learn an optimal bitwidth assignment policy. Their experiments show that a DNN with a learned and heterogeneous bitwidth assignment outperforms quantized DNNs with a homogeneous bitwidth assignment. However, such methods have a high computational complexity as the bitwidth policy must be learned, which involves training many quantized DNNs.

In this paper, we will use the STE approach and show that the quantizer's parameters, including the bitwidth, can be learned with gradient methods if a good parametrization is chosen. Specifically, we show that directly learning the bitwidth is not optimal. Instead, we propose to learn the stepsize and dynamic range. The bitwidth can then be inferred from them. Compared to (Wang et al., 2018; Elthakeb et al., 2018), our method has the advantage that training quantized DNNs has nearly the same computational complexity as standard float32 training.

The contributions of this paper are:

1. We show that there are three different parametrizations for *uniform* and *power-of-two* quantization and that, in both cases, one of them has gradients particularly well suited to train quantized DNNs. The other parametrizations have the problem of yielding gradients with an unbounded gradient norm and coupled components.
2. Using this parametrization, we are able to learn all quantization parameters for DNNs with per-tensor quantization and global memory constraints. We formulate the training as a constrained optimization problem, where the quantized DNN is constrained not to exceed a given overall memory budget, and show how to solve it in a penalty framework.
3. We confirm our findings with experiments on CIFAR-10 and ImageNet. For example, we train a heterogeneously quantized MobileNetV2 on ImageNet requiring a total of only 1.65MB to store the weights and only 0.57MB to store its largest feature map. This is equivalent to a homogenous 4bit quantization of both weights and activations. However, our network learns to allocate the bitwidth heterogeneously in an optimal way. Our MobileNetV2 achieves an error of 30.26% compared to 29.82% for the floating point baseline. This is state-of-the-art for such a heavily quantized MobileNetV2.

We use the following notation throughout this paper: x, $\mathbf{x}$, $\mathbf{X}$ and $\boldsymbol{\mathcal{X}}$ denote a scalar, a (column) vector, a matrix and a tensor with three or four dimensions, respectively; $\lfloor . \rfloor$ and $\lceil . \rceil$ are the floor and ceiling operators. Finally, $\delta(.)$ denotes the Dirac delta function.

2 Choosing a quantization parametrization

Let $Q(x; \boldsymbol{\theta})$ be a quantizer with the parameters $\boldsymbol{\theta}$, which maps $x \in \mathbb{R}$ to discrete values $\{q_1, ..., q_I\}$. In this section, we compare different parametrizations of $Q(x; \boldsymbol{\theta})$ for *uniform quantization* and *power-of-two quantization* and analyze how well the corresponding straight-through gradient estimates $\partial_x Q(x; \boldsymbol{\theta})$ and $\boldsymbol{\nabla}_\theta Q(x; \boldsymbol{\theta})$ are suited to optimize the quantizer parameters $\boldsymbol{\theta}$. Our key result is, that the training of quantized DNNs which learns both, the optimal quantized weights and the optimal quantization parameters $\boldsymbol{\theta}$, is very sensitive to the choice of the parametrization of the quantizers. From an optimization point of view, it is best to parametrize the quantizer $Q(x; \boldsymbol{\theta})$ with the stepsize d and the dynamic range q_{max} as it leads to gradients with stable norms. Doing so, we can use standard gradient descent to learn the quantization parameters and do not need to use stochastic or reinforcement based algorithms, which are computationally expensive.

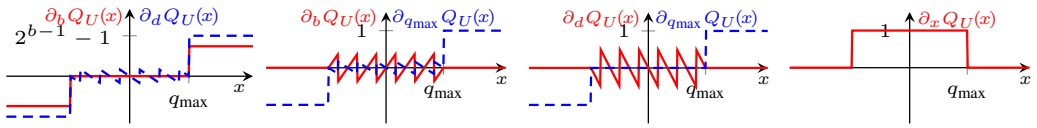
2.1 Parametrization and straight through gradient estimates

A symmetric uniform quantizer $Q_U(x; \boldsymbol{\theta})$ which maps a real value $x \in \mathbb{R}$ to one of $I = 2k + 1$ quantized values $q \in \{-kd, ..., 0, ..., kd\}$ computes

$$q = Q_U(x; \boldsymbol{\theta}) = \text{sign}(x) \begin{cases} d \left\lfloor \frac{|x|}{d} + \frac{1}{2} \right\rfloor & |x| \leq q_{\text{max}} \\ q_{\text{max}} & |x| > q_{\text{max}} \end{cases}, \tag{1}$$

Source: https://arxiv.org/pdf/1905.11452

Google Scholar research | 2026-10-07 | Article 02 | Original p. 2 | Orange: discussion; blue: reference



(a) Case U1 (b) Case U2 (c) Case U3 (d) Input derivative

Figure 2: Partial derivatives of $Q_U(x;\boldsymbol{\theta})$ with respect to the input and the quantization parameters $d, q_{\max}$ and b. Partial derivatives are coupled for "U1" and "U2" but are decoupled for "U3".

avoid divergence. However, parametrization "U3" does not suffer from such an unbounded gradient norm as both partial derivatives $\partial_d Q_U(x;\boldsymbol{\theta}) \in [-\frac{1}{2}, \frac{1}{2}]$ and $\partial_{q_{\max}} Q_U(x;\boldsymbol{\theta}) \in \{-1, 1\}$ are bounded.

Fig. 2 shows the gradients for the parametrization "U1" to "U3". For parametrization "U3", the partial derivatives in $\nabla_\theta Q_U(x;\boldsymbol{\theta})$ are *decoupled*, i.e., $\nabla_\theta Q_U(x;\boldsymbol{\theta})$ is a unit vector, which either points only in the direction of d if $|x| \leq q_{\max}$ or only in the direction of $q_{\max}$, if $|x| > q_{\max}$. We will show in Sec. 2.3 that this implies a diagonal Hessian, which results in a better convergence behavior of gradient descent. In contrast, both parametrizations "U1" and "U2" have partial derivatives that are coupled. In summary, this implies that parametrization "U3" is the best DQ parametrization.

Similar considerations can be made for the power-of-two quantization $Q_P(x;\boldsymbol{\theta})$, which maps a real-valued number $x \in \mathbb{R}$ to a quantized value $q \in \{\pm 2^k : k \in \mathbb{Z}\}$ by

$$q = Q_P(x;\boldsymbol{\theta}) = \text{sign}(x) \begin{cases} q_{\min} & |x| \leq q_{\min} \\ 2^{\lfloor 0.5 + \log_2 |x| \rfloor} & q_{\min} < |x| \leq q_{\max} \\ q_{\max} & |x| > q_{\max} \end{cases}, \tag{4}$$

where $q_{\min}$ and $q_{\max}$ are the minimum and maximum absolute values of the quantizer for a bitwidth of b bit. Power-of-two quantization is an especially interesting scheme for DNN quantization, since a multiplication of quantized values can be implemented as an addition of the exponents. Using the STE for the floor operation, the derivative $\partial_x Q_P(x;\boldsymbol{\theta})$ is given by

$$\partial_x Q_P(x) = \begin{cases} 0 & |x| \leq q_{\min} \\ \frac{2^{\lfloor 0.5 + \log_2 |x| \rfloor}}{|x|} & q_{\min} < |x| \leq q_{\max} \\ 0 & |x| > q_{\max} \end{cases}. \tag{5}$$

The power-of-two quantization has the three parameters $[b, q_{\min}, q_{\max}] =: \boldsymbol{\theta}$, which depend on each other with the relationship $q_{\max} = 2^{2^{b-1}-1} q_{\min}$. Therefore, we have again three different parametrizations with $\boldsymbol{\theta} = [b, q_{\min}]$, $\boldsymbol{\theta} = [b, q_{\max}]$ or $\boldsymbol{\theta} = [q_{\min}, q_{\max}]$, respectively. Similarly to the uniform case, one parametrization ($\boldsymbol{\theta} = [q_{\min}, q_{\max}]$) leads to a gradient of a very simple form

$$\nabla_\theta Q_P(x;\boldsymbol{\theta}) = \begin{bmatrix} \partial_{q_{\min}} Q_U(x;\boldsymbol{\theta}) \\ \partial_{q_{\max}} Q_U(x;\boldsymbol{\theta}) \end{bmatrix} = \begin{cases} [1,0]^T & |x| \leq q_{\min} \\ [0,0]^T & q_{\min} < |x| \leq q_{\max} \\ [0,1]^T & |x| > q_{\max} \end{cases}, \tag{6}$$

which has again a bounded gradient magnitude and independent components and is, hence, best suited for first order gradient based optimization.

2.2 Constraints on θ

In practice, for an efficient hardware implementation, we need to ensure that the quantization parameters only take specific discrete values: for uniform quantization, only integer values are allowed for the bitwidth b, and the stepsize d must be a power-of-two, see e.g. (Jain et al., 2019); for power-of-two quantization, the bitwidth must be an integer, and the minimum and maximum absolute values $q_{\min}$ and $q_{\max}$ must be powers-of-two.

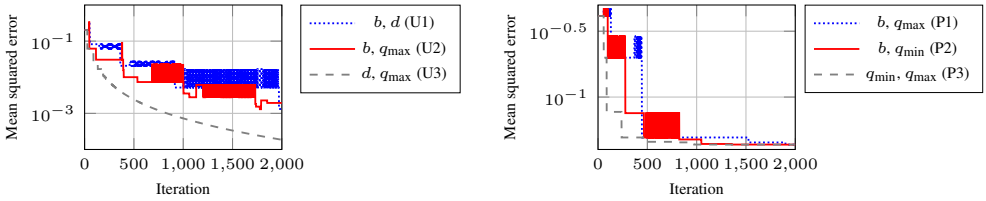
We fulfill these constraints by rounding the parameters in the forward pass to the closest integer or power-of-two value. In the backward pass we update the original float values, i.e., we used again the STE to propagate the gradients.

2.3 Experimental comparison of DQ parametrizations

In the following we compare the parametrizations using two experiments.

Source: https://arxiv.org/pdf/1905.11452

Google Scholar research | 2026-10-07 | Article 02 | Original p. 4 | Orange: discussion; blue: reference



(a) Uniform quantization

(b) Power-of-two quantization

Figure 3: MSE for quantizing Gaussian data $x \sim N(0,1)$ with uniform and power-of-two quantization. Parametrizations "U3" and "P3" converge to the lowest MSE without any oscillations.

1) Quantization of Gaussian data In our first experiment we use DQ to learn the optimal quantization parameters $\boldsymbol{\theta}^*$ which minimize the *mean squared error* (MSE) $\mathrm{E}\left[\frac{1}{2}(Q(x;\boldsymbol{\theta})-x)^2\right]$ with gradient descent and compare the convergence speed for three possible parametrizations of a uniform and power-of-two quantizer. We choose this example as the gradient $\boldsymbol{\nabla}_\theta Q(x;\boldsymbol{\theta}) = \mathrm{E}\left[(Q(x;\boldsymbol{\theta})-x)\boldsymbol{\nabla}_\theta Q(x;\boldsymbol{\theta})\right]$ is just a scaled version of $\boldsymbol{\nabla}_\theta Q(x;\boldsymbol{\theta})$, i.e., the gradient direction depends directly on the parametrization of $Q(x;\boldsymbol{\theta})$ and thus the effects of changing the parametrization can be observed.

It is interesting to study the Hessian $\mathbf{H} = \boldsymbol{\nabla}_\theta \boldsymbol{\nabla}_\theta^T \mathrm{E}\left[(Q(x;\boldsymbol{\theta})-x)^2\right] \in \mathbb{R}^{2\times 2}$ of the MSE:

$$\mathbf{H} = \mathrm{E}\left[\boldsymbol{\nabla}_\theta Q(x;\boldsymbol{\theta})\boldsymbol{\nabla}_\theta Q(x;\boldsymbol{\theta})^T + (Q(x;\boldsymbol{\theta})-x)\boldsymbol{\nabla}_\theta \boldsymbol{\nabla}_\theta^T Q(x;\boldsymbol{\theta})\right] \approx \mathrm{E}\left[\boldsymbol{\nabla}_\theta Q(x;\boldsymbol{\theta})\boldsymbol{\nabla}_\theta Q(x;\boldsymbol{\theta})^T\right].$$

Note that we use the outer-product approximation (Bishop, 2006) in order to simplify our considerations. From this equation it is apparent that the Hessian will be diagonal for the case U3 as $\boldsymbol{\nabla}_\theta Q(x;\boldsymbol{\theta})\boldsymbol{\nabla}_\theta Q(x;\boldsymbol{\theta})^T$ only contains an element in either $(1,1)$ or $(2,2)$ and, therefore, $\mathrm{E}\left[\boldsymbol{\nabla}_\theta Q(x;\boldsymbol{\theta})\boldsymbol{\nabla}_\theta Q(x;\boldsymbol{\theta})^T\right]$ is a diagonal matrix. From this, we can see that gradient descent with an individual learning rate for each parameter is equivalent to Newton's method and, therefore, efficient. In general this will not be the case for U1 and U2.

We conduct an experiment, using ADAM to optimize the mean squared quantization error on artificially generated data, which is generated by drawing 10^4 samples from $N(0,1)$. Please note that the same example was studied in (Jain et al., 2019). The results in Fig. 3 clearly show that the parametrizations "U3" and "P3" are best suited to optimize the uniform and power-of-two quantization parameters, respectively. Indeed, these quantizers converge without oscillation to the lowest MSE. It is interesting to see, that even adaptive gradient methods like ADAM can not solve the scaling issue described in Sec. 2.1. In the Appendix A.4 we give further empirical evidence to support this claim and compare the different parametrizations for the training of a quantized ResNet-20 on CIFAR-10 using ADAM. Note that all cases use the same learning rate. For the interested reader, a more detailed visualization of the error surfaces over the quantization parameters can be found in Appendix A.3.

2) CIFAR-10 In our second experiment we train a ResNet-20 (He et al., 2016) with quantized parameters and activations on CIFAR-10 (Krizhevsky & Hinton, 2009) using the same settings as proposed by (He et al., 2016). Fig. 4 shows the evolution of the training and validation error during training for the case of uniform quantization. The plots for power-of-two quantization can be found in the appendix (Fig. 10). We initialize this network from random parameters or from a pre-trained float network. The quantized DNNs are trained for 160 epochs, using SGD with momentum 0.9 and a learning rate schedule starting with 0.01 and reducing it by a factor of 10 after 80 and 120 epochs, respectively. We use random flips and crops for data augmentation. Each epoch takes about 2.5 min on a single GTX 1080 Ti.

In case of randomly initialized weights, we use an initial stepsize $d_l = 2^{-3}$ for the quantization of weights and activations. Otherwise, we initialize the weights using a pre-trained floating point network and the initial stepsize for a layer is chosen to be $d_l = 2^{\lfloor \log_2(\max|\boldsymbol{\mathcal{W}}_l|/(2^{b-1}-1))\rfloor}$. The remaining quantization parameters are chosen such that we start from an initial bitwidth of $b = 4$ bit. This is a reasonable upper limit for b, as in practice no performance degradation can be observed for $b > 4$bit. Even simple offline algorithms like min/max quantization result in networks with good accuracies. We define no memory constraints during training, i.e., the network can learn to use a large number of bits to quantize weights and activations of each layer. From Fig. 4, we again observe that the parametrization $\boldsymbol{\theta} = [d, q_{\max}]^T$ is best suited to train a uniformly quantized DNN as it converges

Source: https://arxiv.org/pdf/1905.11452

Google Scholar research | 2026-10-07 | Article 02 | Original p. 5 | Orange: discussion; blue: reference

Table 3: Homogeneous vs. heterogeneous quantization of ResNet-20 on CIFAR-10.

	Bitwidth Weight/Activ.	$q_{\max}$ Weight/Activ.	Size Weight/Activ.(max)/Activ.(sum)	**Uniform quant.** Validation error	**Power-of-two quant.** Validation error
Baseline	32bit/32bit	–	1048KB/64KB/736KB	7.29%	
Fixed	2bit/32bit	fixed/–	65.5KB/64KB/736KB	10.81%	8.99%
TQT (Jain et al., 2019)	2bit/32bit	learned/ –	65.5KB/64KB/736KB	9.47%	8.79%
Ours (w/ constr. (8a))	learned/32bit	learned/-	70KB/64KB/736KB	8.59%	8.53%
Fixed	2bit/4bit	fixed/fixed	65.5KB/8KB/92KB	11.30%	11.62%
TQT (Jain et al., 2019)	2bit/4bit	learned/learned	65.5KB/8KB/92KB	9.62%	11.29%
Ours (w/ constr. (8a) and (8b))	learned/learned	learned/learned	70KB/ – /92KB	9.38%	11.29%
Ours (w/ constr. (8a) and (8c))	learned/learned	learned/learned	70KB/8KB/ –	8.58%	11.23%

ImageNet, we train the quantized DNNs for 50 epochs, using SGD with momentum 0.9 and a learning rate schedule starting with 0.01 and reducing it by a factor of 10 after 16 and 32 epochs, respectively. Please note that we quantize *all* layers opposed to other papers which use a higher precision for the first and/or last layer.

In our experiments, we noticed that the performance of DQ is not sensitive to the choice of λ_j in (10). For the CIFAR-10 experiments, we use $\lambda = 0.1$ for both constraints (for sizes in kB). For the ImageNet experiments, we kept the same regularization level by scaling λ_j with the square of the size ratio between the ImageNet model and the CIFAR-10 model. We scale with the square-ratio as the constraints in (10) are squared penalty terms.

First, in Table 3/top, we train a ResNet-20 on CIFAR-10 with quantized weights and float32 activations. We start with the most restrictive quantization scheme with fixed $q_{\max}$ and $b = 2$bit ("Fixed"). Then, we allow the model to learn $q_{\max}$ while $b = 2$bit remains fixed as was done in (Jain et al., 2019) ("TQT"). Finally, we learn both $q_{\max}$ and b with the constraint that the weight size is at most 70KB ("Ours"), which is just 4.5kB larger that the previous 2Bit networks. This allows the model to allocate more than two bits to some layers. From Table 3/top, we observe that the error is smallest when we learn all quantization parameters.

In Table 3/bottom, weights and activations are quantized. For activation quantization, we consider two cases as discussed in Sec. 3. The first one constrains the total activation memory S^x while the second constrains the maximum activation memory $\hat{S}^x$ such that both have the same size as a homogeneously quantized model with 4bit activations. Again, we observe that the error is smallest when we learn all quantization parameters.

We also use DQ to train quantized ResNet-18 (He et al., 2016) and MobileNetV2 (Sandler et al., 2018) on ImageNet (Deng et al., 2009) with 4bit uniform weights and activations or equivalent-sized networks with learned quantization parameters. This is quite aggressive and, thus, a fixed quantization scheme loses more than 6% accuracy while our quantization scheme loses less than 0.5% compared to a float32 precision network.

Our results compare favorably to other recent quantization approaches. To our knowledge, the best result for a 4bit ResNet-18 was reported by (Esser et al., 2019) (29.91% error). This is very close to our performance (29.92% error). Importantly, (Esser et al., 2019) did not quantize the first and last layers, meaning that their network is much bigger. Specifically, compared to our quantized ResNet-18, their model with high precision input and output layers requires 37% more memory to store the weights. Moreover, (Esser et al., 2019) learns stepsizes which are not restricted to powers-of-two. As explained in Sec. 2.2, uniform quantization with power-of-two stepsize leads to more efficient inference, effectively allowing to efficiently compute any multiplication with an integer multiplication and bit-shift. To our knowledge only (Wang et al., 2018) reported results of MobileNetV2 quantized to 4bit. They keep the baseline performance constraining the network to the same size as the 4bit network. However, they do not quantize the activations in this case. In addition, DQ training is efficient since it is comparable to the training of unquantized network. Specifically, one epoch on ImageNet takes 37min for MobileNetV2 and 18min for ResNet-18 on four Nvidia Tesla V100.

Fig. 5 shows the weight bitwidth assignment over layers. We observe that small bitwidths are used for layers with many parameters, i.e., pointwise convolutions and fully connected layers. However, the resulting bitwidth assignments are complex, meaning that there is no simple heuristic. Therefore, it is important to learn the optimal bitwidth assignment.

Source: https://arxiv.org/pdf/1905.11452

Google Scholar research | 2026-10-07 | Article 02 | Original p. 8 | Orange: discussion; blue: reference

Table 4: Homogeneous vs. heterogeneous quantization of MobileNetV2 and ResNet-18 on ImageNet.

	Bitwidth Weight/Activ.	$q_{\max}$ Weight/Activ.	**MobileNetV2** Size Weight/Activ(max)	Validation Error	**ResNet-18** Size Weight/Activ(max)	Validation Error
Baseline	32bit/32bit	–	13.23MB/4.59MB	29.82%	44.56MB/3.04MB	29.72%
Fixed	4bit/4bit	fixed/fixed	1.65MB/0.57MB	36.27%	5.57MB/0.38MB	34.15%
TQT (Jain et al., 2019)	4bit/4bit	learned/learned	1.65MB/0.57MB	32.21%	5.57MB/0.38MB	30.49%
Ours (w/ constr. (8a) and (8c))	learned/learned	learned/learned	1.55MB/0.57MB	30.26%	5.40MB/0.38MB	29.92%
Ours (w/o constr.)	learned/learned	learned/learned	3.14MB/1.58MB	29.41%	10.50MB/1.05MB	29.34%

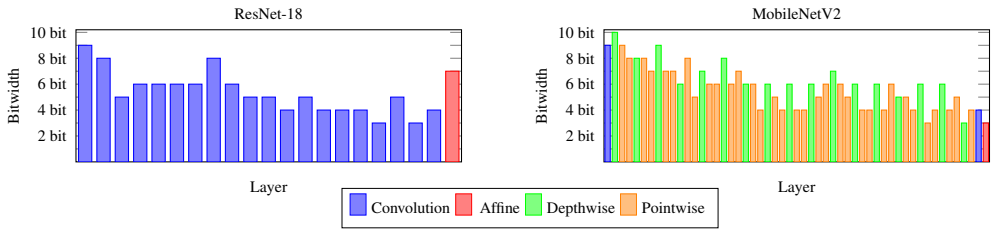


Figure 5: Weight bitwidth assignment over layers for ResNet-18 and MobileNetV2 on ImageNet with weights constrained to a maximum size of 5.57MB. Our method has learned a heterogeneous bitwidth distribution, which gives a better performance than a homogeneous one (see Table 4).

5 Conclusions

In this paper we discussed differentiable quantization and its application to the training of compact DNNs with memory constraints. In order to fulfill memory constraints, we introduced penalty functions during training and used stochastic gradient descent to find the optimal weights as well as the optimal quantization values in a joint fashion. We showed that there are several possible parametrizations of the quantization function. In particular, learning the bitwidth directly is not optimal; therefore, we proposed to parametrize the quantizer with the stepsize and dynamic range instead. The bitwidth can then be inferred from them. This approach is competitive to other recent quantization methods while it does not require to retrain the network multiple times in contrast to reinforcement learning approaches (Wang et al., 2018; Elthakeb et al., 2018).

Acknowledgements

We would like to thank Masato Ishii for many helpful comments during the preparation of this manuscript.

References

Yu Bai, Yu-Xiang Wang, and Edo Liberty. Proxquant: Quantized neural networks via proximal operators. *CoRR*, abs/1810.00861, 2018. URL `http://arxiv.org/abs/1810.00861`.

Yoshua Bengio, Nicholas Léonard, and Aaron Courville. Estimating or propagating gradients through stochastic neurons for conditional computation. *arXiv preprint arXiv:1308.3432*, 2013.

Dimitri P Bertsekas. *Constrained optimization and Lagrange multiplier methods*. Academic press, 2014.

Christopher M. Bishop. *Pattern Recognition and Machine Learning*. Springer, 2006.

Fabien Cardinaux, Stefan Uhlich, Kazuki Yoshiyama, Javier Alonso García, Stephen Tiedemann, Thomas Kemp, and Akira Nakamura. Iteratively training look-up tables for network quantization. *arXiv preprint arXiv:1811.05355*, 2018.

Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. Imagenet: A large-scale hierarchical image database. In *2009 IEEE conference on computer vision and pattern recognition*, pp. 248–255. Ieee, 2009.

Source: https://arxiv.org/pdf/1905.11452

Google Scholar research | 2026-10-07 | Article 02 | Original p. 9 | Orange: discussion; blue: reference

Ahmed T. Elthakeb, Prannoy Pilligundla, Amir Yazdanbakhsh, Sean Kinzer, and Hadi Esmaeilzadeh. Releq: A reinforcement learning approach for deep quantization of neural networks. *CoRR*, abs/1811.01704, 2018. URL http://arxiv.org/abs/1811.01704.

Steven K. Esser, Jeffrey L. McKinstry, Deepika Bablani, Rathinakumar Appuswamy, and Dharmendra S. Modha. Learned step size quantization. *CoRR*, abs/1902.08153, 2019. URL http://arxiv.org/abs/1902.08153.

Kaiyuan Guo, Shulin Zeng, Jincheng Yu, Yu Wang, and Huazhong Yang. A survey of fpga-based neural network accelerator. *arXiv preprint arXiv:1712.08934*, 2017.

Song Han, Huizi Mao, and William J. Dally. Deep compression: Compressing deep neural network with pruning, trained quantization and huffman coding. *CoRR*, abs/1510.00149, 2015. URL http://arxiv.org/abs/1510.00149.

Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 770–778, 2016.

Itay Hubara, Matthieu Courbariaux, Daniel Soudry, Ran El-Yaniv, and Yoshua Bengio. Quantized neural networks: Training neural networks with low precision weights and activations. *CoRR*, abs/1609.07061, 2016. URL http://arxiv.org/abs/1609.07061.

Sambhav R. Jain, Albert Gural, Michael Wu, and Chris Dick. Trained uniform quantization for accurate and efficient neural network inference on fixed-point hardware. *CoRR*, abs/1903.08066, 2019. URL http://arxiv.org/abs/1903.08066.

Eric Jang, Shixiang Gu, and Ben Poole. Categorical reparameterization with gumbel-softmax. *arXiv preprint arXiv:1611.01144*, 2016.

Alex Krizhevsky and Geoffrey Hinton. Learning multiple layers of features from tiny images. Technical report, Citeseer, 2009.

Fengfu Li, Bo Zhang, and Bin Liu. Ternary weight networks. *arXiv preprint arXiv:1605.04711*, 2016.

Zhi-Gang Liu and Matthew Mattina. Learning low-precision neural networks without straight-through estimator (ste). *arXiv preprint arXiv:1903.01061*, 2019.

Christos Louizos, Matthias Reisser, Tijmen Blankevoort, Efstratios Gavves, and Max Welling. Relaxed quantization for discretized neural networks. In *International Conference on Learning Representations*, 2019. URL https://openreview.net/forum?id=HkxjYoCqKX.

Chris J. Maddison, Andriy Mnih, and Yee Whye Teh. The concrete distribution: A continuous relaxation of discrete random variables. *CoRR*, abs/1611.00712, 2016. URL http://arxiv.org/abs/1611.00712.

Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov, and Liang-Chieh Chen. Mobilenetv2: Inverted residuals and linear bottlenecks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 4510–4520, 2018.

Sony. Neural Network Libraries (NNabla). https://github.com/sony/nnabla.

Kuan Wang, Zhijian Liu, Yujun Lin, Ji Lin, and Song Han. HAQ: hardware-aware automated quantization. *CoRR*, abs/1811.08886, 2018. URL http://arxiv.org/abs/1811.08886.

Penghang Yin, Jiancheng Lyu, Shuai Zhang, Stanley Osher, Yingyong Qi, and Jack Xin. Understanding straight-through estimator in training activation quantized neural nets. *arXiv preprint arXiv:1903.05662*, 2019.

Aojun Zhou, Anbang Yao, Yiwen Guo, Lin Xu, and Yurong Chen. Incremental network quantization: Towards lossless cnns with low-precision weights. *arXiv preprint arXiv:1702.03044*, 2017.

Source: https://arxiv.org/pdf/1905.11452

Google Scholar research | 2026-10-07 | Article 02 | Original p. 10 | Orange: discussion; blue: reference

A Reduced-Precision Network for Image Reconstruction

MANU MATHEW THOMAS, University of California, Santa Cruz
KARTHIK VAIDYANATHAN, Intel Corporation
GABOR LIKTOR, Intel Corporation
ANGUS G. FORBES, University of California, Santa Cruz



Fig. 1. A comparison of different reconstruction techniques on a frame from the INFILTRATOR scene with PSNR (top row) and SSIM (bottom row) metrics. TAA has the worst reconstruction quality with excessive blurring followed by direct prediction with U-Net. QW-Net produces the best results with quality comparable to brute force sampling with 256 samples per pixel.

Neural networks are often quantized to use reduced-precision arithmetic, as it greatly improves their storage and computational costs. This approach is commonly used in image classification and natural language processing applications. However, using a quantized network for the reconstruction of HDR images can lead to a significant loss in image quality. In this paper, we introduce *QW-Net*, a neural network for image reconstruction, in which close to 95% of the computations can be implemented with 4-bit integers. This is achieved using a combination of two U-shaped networks that are specialized for different tasks, a *feature extraction* network based on the U-Net architecture, coupled to a *filtering* network that reconstructs the output image. The feature extraction network has more computational complexity but is more resilient to quantization errors. The filtering network, on the other hand, has significantly fewer computations but requires higher precision. Our network recurrently warps and accumulates previous frames using motion vectors, producing temporally stable results with significantly better quality than TAA, a widely used technique in current games.

CCS Concepts: • **Computing methodologies** → **Neural networks**; **Rendering**.

Additional Key Words and Phrases: Kernel prediction, antialiasing

Authors' addresses: Manu Mathew Thomas, University of California, Santa Cruz, mthomas6@ucsc.edu; Karthik Vaidyanathan, Intel Corporation, karthik.vaidyanathan@intel.com; Gabor Liktor, Intel Corporation, gabor.liktor@intel.com; Angus G. Forbes, University of California, Santa Cruz, angus@ucsc.edu.

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).
© 2020 Copyright held by the owner/author(s).
0730-0301/2020/12-ART231
https://doi.org/10.1145/3414685.3417786

ACM Reference Format:
Manu Mathew Thomas, Karthik Vaidyanathan, Gabor Liktor, and Angus G. Forbes. 2020. A Reduced-Precision Network for Image Reconstruction. *ACM Trans. Graph.* 39, 6, Article 231 (December 2020), 12 pages. https://doi.org/10.1145/3414685.3417786

1 INTRODUCTION

Synthesizing computer-generated imagery involves two key functions: sampling the incident radiance on an image plane and applying a reconstruction filter to produce the final image [Cook et al. 1984]. If the incident radiance is undersampled, high-frequency components resulting from visibility discontinuities or specular lighting can appear as aliasing in the reconstructed image. To reduce aliasing, production renderers typically apply supersampling with a large number of samples per pixel. On the other hand, real-time applications rely on a combination of radiance prefiltering and specialized image reconstruction techniques. Temporal antialiasing (*TAA*) [Jimenez et al. 2012; Karis 2014; Yang et al. 2009] is an image reconstruction technique that is widely used in games today. TAA uses renderer-generated motion vectors to gather and accumulate samples from previous frames, effectively increasing the number of samples per pixel. However, TAA is susceptible to ghosting artifacts, loss of detail, and temporal instability [Yang et al. 2020].

Convolutional neural networks offer a promising alternative for image reconstruction, and have been successfully applied to closely related fields such as denoising [Bako et al. 2017; Chaitanya et al. 2017; Vogels et al. 2018]. The U-Net [Ronneberger et al. 2015] architecture is particularly well suited for such tasks as it processes features at multiple scales, achieving a large receptive field and more effective filtering with a relatively low computational cost.

Source: https://doi.org/10.1145/3414685.3417786
Google Scholar research | 2026-10-08 | Article 03 | Original p. 1 | Orange: discussion; blue: reference

2.2 Neural Networks for Reconstruction

In recent years there has been a growing interest in applying neural networks to problems in rendering. One area in particular that has seen rapid advances is *denoising* of Monte Carlo renderings, which aims to remove noise from simulated distributed effects like soft shadows, indirect lighting, etc. While neural networks have been previously explored for denoising natural images [Burger et al. 2012], Kalantari et al. [2015] were the first to apply it to Monte Carlo rendering, using a multi-layer perceptron to drive the parameters of feature-based denoising filters. Later, Bako et al. [2017] used a convolutional neural network to predict the filter kernel itself, achieving significantly better results.

Chaitanya et al. [2017] introduced an alternate approach, using a network based on *U-Net* [Ronneberger et al. 2015] to directly predict the denoised image. By introducing recurrent connections inside a U-Net network they were the first to demonstrate temporally stable results at interactive rates. U-Net also has the advantage of achieving a large receptive field using a multi-scale architecture, which is crucial for denoising sparsely sampled images. On the other hand, achieving a large receptive field with a single predicted kernel [Bako et al. 2017] is impractical, as the complexity of filtering and the last prediction layer grows significantly with the filter size [Gharbi et al. 2019]. This limitation was overcome by Vogels et al. [2018] by predicting filter kernels at different scales using multiple deep residual networks [He et al. 2016]. The filtered images were progressively upsampled and blended using predicted weights to produce a denoised result. They compared their approach against a variant of the direct prediction network of Chaitanya et al. observing over-blurring and color-shift artifacts with direct prediction. They also discussed the possibility of using a U-Net to efficiently predict kernels at different scale. The concurrent work of Hasselgren et al. [2020] uses such a network with a feedback loop to achieve temporally stable results. Along similar lines, we use a U-Net network to predict filter kernels for reconstruction but instead of applying independent filters at each scale, we use a filter network that resembles U-Net, progressively filtering and re-sampling the image. By incorporating predicted filters in between each downsampling and upsampling step, we can better reconstruct feature details at each scale.

Image reconstruction is a more general problem of producing an antialiased image from point samples. In real-time applications, the input to a reconstruction technique like TAA is denoised using filters, specialized for effects like shadows, reflections or ambient occlusion. Neural networks are also promising for image reconstruction as initially demonstrated by Marco Salvi [2017] using a U-Net with a warped feedback loop. Kaplanyan et al. [2019] also used a recurrent U-Net to reconstruct a peripheral image for foveated rendering from a very sparse set of samples. We compare our approach against a U-Net based direct prediction network, but with a simpler form of recurrence [Hasselgren et al. 2020; Sajjadi et al. 2018] using the warped output image instead of a hidden state.

Recently, a combined real-time image reconstruction technique called Deep Learning Super Sampling (*DLSS*) [Liu 2020] was introduced, but the details of the underlying network are unknown. Concurrent to our work, Xiao et al. [2020] introduced a reconstruction technique based on U-Net. Using an optimized inference implementation they reconstruct a 1080p image in 18 to 20 ms on a high-end GPU. In comparison, DLSS reconstructs a 4K image in under 2 ms. Both these approaches can reconstruct images at a higher resolution than the input render. In this paper we reconstruct a supersampled image at the same resolution, focusing on achieving high quality, temporally stable results with aggressive quantization.

2.3 Quantized Networks

A full-precision network can be quantized either post-training or by training the network with simulated quantization. With *post-training quantization*, a network is quantized using the distribution of trained weights and by measuring the distribution of activations on a sample dataset, a process known as *calibration*. For example, TensorRT [Migacz 2017] quantizes weights and activations by minimizing the Kullback-Leibler (KL) divergence between the quantized and un-quantized distributions. TensorFlow [Jacob et al. 2018] on the other hand maps the range of the weight tensor and the average range of the activations computed over several batches, to the range of the quantized format. Unfortunately post training quantization shows as significant degradation in accuracy with 4-bit integers (INT4) and lower precisions [Jacob et al. 2018; Krishnamoorthi 2018].

Jacob et al. [2018] proposed an alternate *quantization-aware training* approach that simulates quantization errors during training, achieving significantly better accuracy. Simulated quantization introduces quantization errors in the forward pass by quantizing the weights and activations and dequantizing them back to the original range. However, in the backward pass, weights and biases are updated with floating point precision without any loss in precision.

The choice of threshold values for quantizing weights and activations greatly impacts the accuracy of the network. While Jacob et al. [2018] derived the quantization thresholds based on a measurement of the tensor range, Jain et al.[2019] and Esser et al. [2019] showed that the quantization thresholds could be trained to further improve accuracy. They derived the gradient of the quantization function applying a *straight through estimator* [Bengio 2013] for the gradient of round/ceil operations but without approximating these operations with an identity function. This allowed the thresholds to grow (favoring larger dynamic range) or shrink (favoring higher precision) based on the gradients.

A majority of existing work on reduced precision networks use a uniform quantization scheme where the quantized values are evenly spaced [Lin et al. 2016; Nayak et al. 2019; Zhou et al. 2017, 2016]. A uniform quantizer can be further classified into *asymmetric* and *symmetric* quantizers. An asymmetric quantizer applies an affine transformation with a scale and a zero-point to map values in the floating-point range to the integer range. In a symmetric quantizer, the zero point is set to 0 reducing the affine mapping to a linear mapping. The symmetric quantizer avoids the overhead of handling zero-points making it computationally efficient. A large body of work explores binary and ternary neural networks, where the weights and/or activations are quantized to binary or ternary values [Courbariaux et al. 2015, 2016; Rastegari et al. 2016; Zhu et al. 2016]. These networks push quantization to its limits but also lose a significant amount of accuracy.

Source: https://doi.org/10.1145/3414685.3417786

Google Scholar research | 2026-10-08 | Article 03 | Original p. 3 | Orange: discussion; blue: reference

The mapping of quantized integer weights and activations u, v to their real values w, x is given by

$$w = s_w u, \tag{3}$$
$$x = s_x(v - z), \tag{4}$$

where s_w is a per-channel scale (step size) for the quantized weights, s_x is a per-layer scale for the quantized activations and z is the zero point. We can then represent the convolution for a single channel as:

$$y = \sum w_i x_i + b_i,$$

where b_i is the bias. Substituting with Equations 3 and 4 we get:

$$\begin{aligned} y &= s_w s_x \left(\sum u_i v_i - \sum u_i z \right) + b_i, \\ &= \alpha \sum u_i v_i + c_i, \end{aligned}$$

where $\alpha = s_w s_x$ and $c_i = b_i - \alpha \sum u_i z$, represents a single-precision floating point bias that can be precomputed.

Figure 4 shows the convolution and activation layers for a single channel along with their numeric formats. The convolutions constitute the majority of the computations but can be mapped to the tensor cores on Turing that can efficiently evaluate 4-bit multiplications and accumulate the result in a 32-bit integer. An additional floating-point MAC scales the convolution output by α and introduces the bias c, following which an ELU activation is computed with single-precision floating point. A final MAC operation then maps the floating-point activation back to a 4-bit integer.

The activations are quantized using a function $\mathrm{Q}(x, l, u)$ that maps an activation x in the range $[l, u]$ to a quantized integer x_q in the range $[M_l, M_u]$. For a b-bit integer, $M_l = -2^{b-1}$, $M_u = 2^{b-1} - 1$ and $M = M_l + M_u$ is the number of quantization levels. Following Jacob et al. [2018], we adjust the quantization range such that a zero value gets quantized without error, preserving zero-padded data. The adjusted range $[l_a, u_a]$ is derived as follows:

$$l_a = s \left\lfloor \frac{l}{s} \right\rceil, \tag{5}$$
$$u_a = l_a + sM, \tag{6}$$

where $s = \frac{u-l}{M}$ is the quantization step size and a common term for both s_w and s_x. The quantization function Q is then given by

$$x_q = \begin{cases} \left\lfloor \frac{x}{s} \right\rceil + z & \text{if } l_a \le x \le u_a \\ M_l & \text{if } x < l_a \\ M_u & \text{if } x > u_a, \end{cases} \tag{7}$$

where $z = M_l - \frac{l_a}{s}$ is the zero point introduced earlier in Equation 4.

5.1 Trained Quantization

We initially train our network with full precision and then quantize the weights and activations by fine tuning the network with simulated quantization [Jacob et al. 2018]. Training with simulated quantization achieves significant improvements over post-training quantization, especially with 4-bit precision [Krishnamoorthi 2018]. Simulated quantization applies a quantization followed by a dequantization in the forward pass, introducing quantization errors while maintaining and updating the weights as un-quantized variables.

The simulated quantization function $\mathrm{Q}'(x, l, u)$ can be derived from Equations 4 and 7:

$$\mathrm{Q}'(x, l, u) = s\,(\mathrm{Q}(x, l, u) - z) \tag{8}$$

While Jacob et al. [2018] use the moving averages of the minimum and maximum activations in a batch to derive the quantization thresholds l and u, we adopt a more recent approach [Esser et al. 2019; Jain et al. 2019] where the quantization thresholds are derived from trained variables.

Following Jain et al. [2019], we train the thresholds in log space using variables t_l and t_u, such that $l = -e^{t_l}$ and $u = e^{t_u}$ to improve stability with quantized training. We derive the gradient of Q' for backpropagation, using a straight through estimator [Bengio 2013] for rounding operations i.e. $\frac{\mathrm{d}}{\mathrm{d}x} \lfloor x \rceil = 1$. For reference, we provide the equations for the gradients in Appendix A. We initialize $t_l = \ln(1)$ and $t_u = \ln(3)$, where $l = -1$ corresponds to the lower limit of the ELU activation and $u = 3$ is a value that we have chosen based on experimenting with $u = 6$ [Krizhevsky 2010] and $u = 3$.

We use the same quantization functions Q and Q' for the weights but enforce a symmetric range by setting $l = -u$. Unlike the activations we did not derive the quantization threshold from a trained variable. Instead we set u to the maximum absolute value of the weights corresponding to a channel.

5.2 Batch Normalization

It is common practice to eliminate the batch normalization layer in the inference network by folding the layer parameters into the weights and bias of the preceding convolution layer. The folded weights and bias are given by:

$$w_{\text{inference}} = \frac{\gamma}{\sigma} w,$$
$$b_{\text{inference}} = \beta - \frac{\gamma}{\sigma} \mu,$$

where μ and σ are the moving average of the batch mean and standard deviation and γ and β are the trained scale and bias respectively. While scaling the weights by $\lambda = \frac{\gamma}{\sigma}$ we also scale the quantization threshold by the same amount. This has no impact on quantization error as $\mathrm{Q}(\lambda w, \lambda l, \lambda l) = \mathrm{Q}(w, l, l)$. During trained quantization, we freeze the values of γ and β and use the moving average of the batch mean and variance computed during the initial training.

5.3 Filtering Network

As discussed in Section 3.2, the filter network includes a 1×1 convolution layer to predict the filter kernels. We quantize the weights of this layer to 8-bit using trained quantization but we do not quantize the activations (filter kernel) as all filtering operations are computed with single-precision floating-point.

5.4 Computation Costs

Although the focus of our paper is high-quality reconstruction with quantization, we provide some data to support the feasibility of high performance 4-bit inference. Table 1 lists the number of MAC operations per pixel for the QW-Net network and the corresponding numeric formats.

Source: https://doi.org/10.1145/3414685.3417786

Google Scholar research | 2026-10-08 | Article 03 | Original p. 6 | Orange: discussion; blue: reference

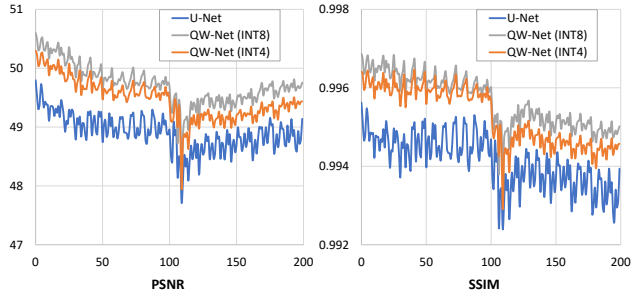


Fig. 11. Per-frame quality metrics on a continuous frame sequence from the KITE dataset. QW-Net consistently outperforms U-Net, with a slight degradation in the case of INT4.



Fig. 12. The loss of history samples due to disoccluded regions results in a short-term reduction in image quality. Before occlusion (left), at disocclusion (middle), 10th frame after disocclusion.

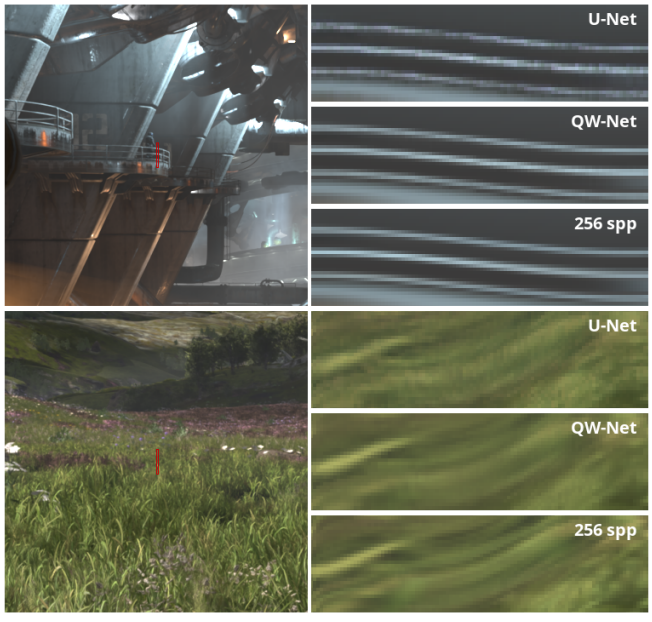


Fig. 13. Temporal Profiles from the INFILTRATOR (top) and KITE (bottom). QW-Net has a significantly smoother profile than U-Net, indicating better temporal coherence.

Yoshua Bengio. 2013. Estimating or Propagating Gradients Through Stochastic Neurons. *arXiv preprint arXiv:1305.2982* (2013).

H. C. Burger, C. J. Schuler, and S. Harmeling. 2012. Image denoising: Can plain neural networks compete with BM3D?. In *Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2392–2399.

Jose Caballero, Christian Ledig, Andrew Aitken, Alejandro Acosta, Johannes Totz, Zehan Wang, and Wenzhe Shi. 2017. Real-Time Video Super-Resolution with Spatio-Temporal Networks and Motion Compensation. In *Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 4778–4787.

Chakravarty R. Alla Chaitanya, Anton S. Kaplanyan, Christoph Schied, Marco Salvi, Aaron Lefohn, Derek Nowrouzezahrai, and Timo Aila. 2017. Interactive Reconstruction of Monte Carlo Image Sequences Using a Recurrent Denoising Autoencoder. *ACM Transactions on Graphics* 36, 4, Article 98 (July 2017), 12 pages.

Jungwook Choi, Zhuo Wang, Swagath Venkataramani, Pierce I-Jen Chuang, Vijayalakshmi Srinivasan, and Kailash Gopalakrishnan. 2018. PACT: Parameterized Clipping Activation for Quantized Neural Networks. *arXiv preprint arXiv:1805.06085* (2018).

Ö. Çiçek, A. Abdulkadir, S.S. Lienkamp, T. Brox, and O. Ronneberger. 2016. 3D U-Net: Learning Dense Volumetric Segmentation from Sparse Annotation. In *Medical Image Computing and Computer-Assisted Intervention (MICCAI)*, Vol. 9901. Springer, 424–432.

Djork-Arné Clevert, Thomas Unterthiner, and Sepp Hochreiter. 2015. Fast and accurate deep network learning by exponential linear units (ELUs). *arXiv preprint arXiv:1511.07289* (2015).

Robert L. Cook, Thomas Porter, and Loren Carpenter. 1984. Distributed Ray Tracing. In *Computer Graphics (Proc. ACM SIGGRAPH)*, Vol. 18. 137–145.

Matthieu Courbariaux, Yoshua Bengio, and Jean-Pierre David. 2015. BinaryConnect: Training deep neural networks with binary weights during propagations. *arXiv preprint arXiv:1511.00363* (2015).

Matthieu Courbariaux, Itay Hubara, Daniel Soudry, Ran El-Yaniv, and Yoshua Bengio. 2016. Binarized neural networks: Training deep neural networks with weights and activations constrained to +1 or -1. *arXiv preprint arXiv:1602.02830* (2016).

Steven K Esser, Jeffrey L McKinstry, Deepika Bablani, Rathinakumar Appuswamy, and Dharmendra S Modha. 2019. Learned step size quantization. *arXiv preprint arXiv:1902.08153* (2019).

Epic Games. 2019. Unreal Engine 4.24 on GitHub. https://github.com/EpicGames/UnrealEngine/tree/4.24

Michaël Gharbi, Tzu-Mao Li, Miika Aittala, Jaakko Lehtinen, and Frédo Durand. 2019. Sample-Based Monte Carlo Denoising Using a Kernel-Splatting Network. *ACM Transactions on Graphics* 38, 4, Article 125 (July 2019), 12 pages.

J. Hasselgren, J. Munkberg, M. Salvi, A. Patney, and A. Lefohn. 2020. Neural Temporal Adaptive Sampling and Denoising. *Computer Graphics Forum* 39, 2 (2020), 147–155.

Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2015. Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification. *Proc. IEEE International Conference on Computer Vision (ICCV)* (Dec 2015), 1026–1034.

Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2016. Deep Residual Learning for Image Recognition. *Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR)* (Jun 2016), 770–778.

Sergey Ioffe and Christian Szegedy. 2015. Batch normalization: Accelerating deep network training by reducing internal covariate shift. *arXiv preprint arXiv:1502.03167* (2015).

Benoit Jacob, Skirmantas Kligys, Bo Chen, Menglong Zhu, Matthew Tang, Andrew Howard, Hartwig Adam, and Dmitry Kalenichenko. 2018. Quantization and training of neural networks for efficient integer-arithmetic-only inference. In *Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2704–2713.

Sambhav R Jain, Albert Gural, Michael Wu, and Chris H Dick. 2019. Trained quantization thresholds for accurate and efficient fixed-point inference of deep neural networks. *arXiv preprint arXiv:1903.08066* (2019).

Jorge Jimenez, Jose I. Echevarria, Tiago Sousa, and Diego Gutierrez. 2012. SMAA: Enhanced Morphological Antialiasing. *Computer Graphics Forum* 31, 2 (2012), 355–364.

Florian Kainz, Rod Bogart, and Drew Hess. 2004. The OpenEXR image file format. In *GPU Gems*, Randima Fernando (Ed.). Addison-Wesley Reading, Chapter 26.

Nima Khademi Kalantari, Steve Bako, and Pradeep Sen. 2015. A Machine Learning Approach for Filtering Monte Carlo Noise. *ACM Transactions on Graphics*, 34, 4, Article 122 (2015), 12 pages.

A. S. Kaplanyan, S. Hill, A. Patney, and A. Lefohn. 2016. Filtering Distributions of Normals for Shading Antialiasing. In *Proc. High-Performance Graphics (HPG)*. 151–162.

Anton S. Kaplanyan, Anton Sochenov, Thomas Leimkühler, Mikhail Okunev, Todd Goodall, and Gizem Rufo. 2019. DeepFovea: Neural Reconstruction for Foveated Rendering and Video Compression Using Learned Statistics of Natural Videos. *ACM Transactions on Graphics* 38, 6, Article 212 (Nov. 2019), 13 pages.

Brian Karis. 2014. High Quality Temporal Supersampling. SIGGRAPH 2014 Advances in Real-Time Rendering in Games course. http://advances.realtimerendering.com/s2014/

Raghuraman Krishnamoorthi. 2018. Quantizing deep convolutional networks for efficient inference: A whitepaper. *arXiv preprint arXiv:1806.08342* (2018).

Alex Krizhevsky. 2010. Convolutional deep belief networks on CIFAR-10. https://www.cs.toronto.edu/~kriz/conv-cifar10-aug2010.pdf

Source: https://doi.org/10.1145/3414685.3417786

Google Scholar research | 2026-10-08 | Article 03 | Original p. 11 | Orange: discussion; blue: reference

Optimizing the Deployment of Tiny Transformers on Low-Power MCUs

Victor Jean-Baptiste Jung, *Graduate Student Member, IEEE*, Alessio Burrello, Moritz Scherer, *Member, IEEE*, Francesco Conti, *Member, IEEE*, and Luca Benini, *Fellow, IEEE*

***Abstract*—Transformer networks are rapidly becoming State of the Art (SotA) in many fields, such as Natural Language Processing (NLP) and Computer Vision (CV). Similarly to Convolutional Neural Networks (CNNs), there is a strong push for deploying Transformer models at the extreme edge, ultimately fitting the tiny power budget and memory footprint of Micro-Controller Units (MCUs). However, the early approaches in this direction are mostly ad-hoc, platform, and model-specific. This work aims to enable and optimize the flexible, multi-platform deployment of encoder Tiny Transformers on commercial MCUs. We propose a complete framework to perform end-to-end deployment of Transformer models onto single and multi-core MCUs. Our framework provides an optimized library of kernels to maximize data reuse and avoid unnecessary data marshaling operations into the crucial attention block. A novel Multi-Head Self-Attention (MHSA) inference schedule, named Fused-Weight Self-Attention (FWSA), is introduced, fusing the linear projection weights offline to further reduce the number of operations and parameters. Furthermore, to mitigate the memory peak reached by the computation of the attention map, we present a Depth-First Tiling (DFT) scheme for MHSA tailored for cache-less MCU devices that allows splitting the computation of the attention map into successive steps, never materializing the whole matrix in memory. We evaluate our framework on three different MCU classes exploiting ARM and RISC-V Instruction Set Architecture (ISA), namely the STM32H7 (ARM Cortex M7), the STM32L4 (ARM Cortex M4), and GAP9 (RV32IMC-XpulpV2). We reach an average of 4.79 × and 2.0 × lower latency compared to SotA libraries CMSIS-NN (ARM) and PULP-NN (RISC-V), respectively. Moreover, we show that our MHSA depth-first tiling scheme reduces the memory peak by up to 6.19 ×, while the fused-weight attention can reduce the runtime by 1.53 ×, and number of parameters by 25%. Leveraging the optimizations proposed in this work, we run end-to-end inference of three SotA Tiny Transformers for three applications characterized by different input dimensions and network hyperparameters. We report significant improvements across the networks: for instance, when executing a transformer block for the task of radar-based hand-gesture recognition on GAP9, we achieve a latency of 0.14 ms and energy consumption of 4.92 µJ, 2.32 × lower than the SotA PULP-NN library on the same platform.**

***Index Terms*—Deep neural networks, transformers, micro-controller units, edge computing, DNN acceleration.**

I. Introduction

In the last few years, there has been a significant trend towards moving computing workload from centralized facilities towards the extreme edge of the Internet of Things (IoT), improving privacy, efficiency, and ensuring a predictable and constant latency, independent from the network coverage and congestion [1]. As a consequence, modern IoT endpoints have been changing from simple Micro-Controller Units (MCUs) equipped with the sensor infrastructure for data collection and transmission towards more complex heterogeneous Systems-on-Chip (SoCs) characterized by dedicated accelerators and enhanced Instruction Set Architectures (ISAs) to optimize efficient on-board computations – while still meeting to tight power, performance, and cost constraints. For this reason, extensive research has been conducted on how to squeeze complex Machine Learning (ML) models into such constrained devices with a strong emphasis on Deep Neural Networks (DNNs) due to their success in solving many challenging tasks. Standard hardware-agnostic optimization techniques include topological changes [2], data quantization [3], [4], and pruning [5]. These efforts are often collectively labeled as *TinyML* and are applied to State of the Art (SotA) Convolutional Neural Networks (CNNs), such as ResNets [6] or MobileNets [7], that dominates most of the tasks of the IoT domain, including image recognition, object detection, or time-series analysis.

At the same time, increasing efforts have been dedicated to finding a successor architecture to CNNs for IoT processing to increase tasks' accuracy further. A strong candidate is the Transformer architecture [8], which first emerged as SotA model in Natural Language Processing (NLP) and is now also SotA in other domains such as Computer Vision (CV) and

Received 28 March 2024; revised 19 September 2024; accepted 25 October 2024. Date of publication 18 November 2024; date of current version 20 January 2025. This work was supported in part by the Chips Joint Undertaking (Chips-JU) TRISTAN Project under Grant Agreement 101095947, and in part by the Convolve Project under Grant Agreement 101070374. The JU and CONVOLVE receive support from the European Union's Horizon Europe Research and Innovation Program. Recommended for acceptance by R. Bunescu. *(Corresponding author: Victor Jean-Baptiste Jung.)*

Victor Jean-Baptiste Jung and Moritz Scherer are with the Integrated Systems Laboratory (IIS), ETH Zürich, ETZ, 8092 Zürich, Switzerland (e-mail: jungvi@iis.ee.ethz.ch; scheremo@iis.ee.ethz.ch).

Alessio Burrello is with the Department of Electrical, Electronic and Information Engineering, University of Bologna, 40136 Bologna, Italy, and also with the Interuniversity Department of Regional and Urban Studies and Planning, Politecnico di Torino, 10129 Turin, Italy (e-mail: alessio.burrello@polito.it).

Francesco Conti is with the Department of Electrical, Electronic and Information Engineering, University of Bologna, 40136 Bologna, Italy (e-mail: f.conti@unibo.it).

Luca Benini is with the Integrated Systems Laboratory (IIS), ETH Zürich, ETZ, 8092 Zürich, Switzerland, and also with the Department of Electrical, Electronic and Information Engineering, University of Bologna, 40136 Bologna, Italy (e-mail: lbenini@iis.ee.ethz.ch; luca.benini@unibo.it).

Digital Object Identifier 10.1109/TC.2024.3500360

0018-9340 © 2024 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Source: https://ieeexplore.ieee.org/document/10755971 (published print)

Google Scholar research | 2026-10-08 | Article 04 | Original p. 1 | Orange: discussion; blue: reference

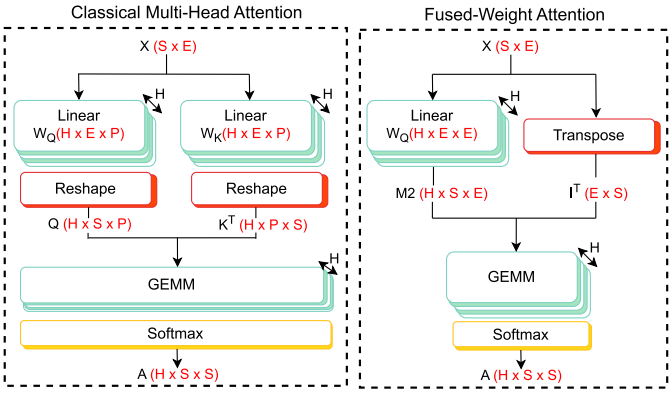


Fig. 3. Diagram of the Classical Attention used in the MHSA and the proposed Fused-Weight Attention. The dimension of each tensor is specified in red.

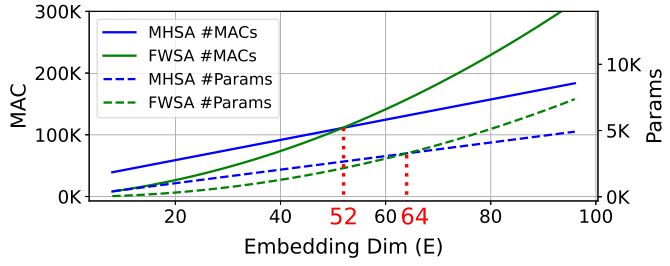


Fig. 4. Number of parameters and Multiply-Accumulate (MAC) as a function of the embedding dimension E and for $S = 32$, $P = 32$ and $H = 8$. The intersection points happen at $E = 52$ and $E = 64$ for the number of MAC and the number of parameters, respectively.

$$A = \phi(X \cdot W_Q \cdot (X \cdot W_K)^T) \tag{4}$$
$$A = \phi(X \cdot W_Q \cdot W_K^T \cdot X^T) \tag{5}$$
$$A = \phi(X \cdot W^* \cdot X^T) \tag{6}$$

In the equations above, we reorder the operations using the associativity of the matrix product and the transposition's reversal order of the product. The fused weight matrix W^* is computed offline and considered as fixed parameters during the inference. In detail, we reduce the number of matrix multiplications to perform online from three to two. Fig. 3 shows the computational graph of the classical Multi-Head Attention (left) and the FWSA (right), we indicate the dimension of the tensors in red.

To evaluate when this transformation is beneficial, we compare the number of operations O and parameters θ between the FWSA and the MHSA in Eq. 7 & 8 below:

$$\begin{cases} O_{MHSA} = 2HSPE + HS^2P \\ O_{FWSA} = HSE^2 + HS^2E \end{cases} \tag{7}$$

$$\begin{cases} \theta_{MHSA} = 2HPE \\ \theta_{FWSA} = HE^2 \end{cases} \tag{8}$$

Looking at Eq. 7 & 8, we notice that using the FWSA is indeed beneficial for some conditions of S, P, and E values. More specifically, in Eq. 9, we provide the inequalities that need to be satisfied to benefit from the FWSA in terms of number of operations (left) and parameters (right).

$$E < P - \frac{S}{2} + \frac{\sqrt{4P^2 + S^2}}{2} \qquad E < 2P \tag{9}$$

In Fig. 4, we show the relationship between E and the number of MAC/Parameters for the MHSA and FWSA. For $E < 52$, FWSA always shows a lower number of MAC and parameters. When considering the intermediate condition of $52 \leq E < 64$, the FWSA reduces the number of parameters but increases the number of MACs. Finally, in the third case ($E \geq 64$), the MHSA is more advantageous than the FWSA.

C. Transformers Quantization With QuantLib

To quantize Transformers, we employ and extend the open-source quantization library QuantLib. QuantLib transforms the ONNX graph representing the floating-point model of a DNN to an integerized ONNX graph. This transformation from a floating-point to an integerized graph is done in two steps.

All operation nodes in the graph, such as matrix multiplication, convolution, or softmax, are replaced by their integer equivalent. As the softmax, GELU, and layer normalization present in Transformers do not have an integer equivalent, we extend QuantLib to support the integer version of these operands from I-BERT.

Then, we add a re-quantization step after operations whose output is not represented in 8 bit. For instance, the output type of an integer GEMM is *int32*; thus, to convert it to 8 bit, we have to re-quantize it. We use the uniform symmetric quantizer from the SotA Trained Quantization Threshold (TQT) method [3]. Below, we describe the functional behavior of the TQT quantizer denoted ψ:

$$\psi(x) = \max(2^{b-1}, \min((x \cdot \epsilon_{mul}) >> \epsilon_{div}, 2^{b-1} - 1)) \tag{10}$$

The number of bits of the quantized values is b, while ϵ_{mul} and ϵ_{div} are the re-quantization parameters. This quantizer constrains the scale factor to a powers-of-2 (i.e., $2^{\epsilon_{div}}$) and uses per-tensor scaling of activations and weights, making it hardware-friendly as we can use the bit-shift operation to apply ϵ_{div}. Additionally, we use Quantization Aware Training (QAT) to adjust weights to reduce the accuracy loss induced by quantization. Note that we could use Post-Training Quantization (PTQ) instead of QAT, we choose to use QAT because it mitigates even further the quantization accuracy loss, compared to PTQ.

After applying the necessary transformations and obtaining an integerized ONNX graph, we integrate into QuantLib a new export module to format the graph such that it can be ingested by the deployment framework (DORY or DumpO).

D. Deployment Framework Integration

For ARM platforms, we use DumpO to compile the integerized ONNX graph into C code. DumpO first transforms the graph so that each node can be executed by one kernel from

Source: https://ieeexplore.ieee.org/document/10755971 (published print)
Google Scholar research | 2026-10-08 | Article 04 | Original p. 6 | Orange: discussion; blue: reference

VI. Conclusion and Future Work

In this work, we proposed an end-to-end flow to enable efficient deployment of small Transformer models onto commercial MCUs. Our kernel library, tailored for MHSA, together with our optimized schedule and tiling strategy, allows us to speed up the execution of the attention block by a factor of $2.94\times$ on average on RISC-V and ARM platforms. Furthermore, we demonstrate the efficiency of our flow by deploying three Tiny Transformers onto the GAP9 MCU, reaching an average energy consumption and latency of $202\,\mu J$ and $5.92\,ms$, respectively. Our work is open-source at https://github.com/pulp-platform/pulp-transformer. Future research direction would extend our framework to support emerging transformers such as encoder-decoder and decoder-only.

Acknowledgment

The authors thank Marco Fariselli from GreenWaves Technologies for the insightful discussions, particularly on the FWSA strategy.

References

[1] C. R. Banbury et al., "Benchmarking TinyML systems: Challenges and direction," 2020, *arXiv:2003.04821*.
[2] J. Lin et al., "MCUNet: Tiny deep learning on IoT devices," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 33, 2020, pp. 11711–11722.
[3] S. Jain, A. Gural, M. Wu, and C. Dick, "Trained quantization thresholds for accurate and efficient fixed-point inference of deep neural networks," *Proc. Mach. Learn. Syst.*, vol. 2, 2020, pp. 112–128.
[4] C. Gong, Y. Chen, Y. Lu, T. Li, C. Hao, and D. Chen, "VecQ: Minimal loss DNN model compression with vectorized weight quantization," *IEEE Trans. Comput.*, vol. 70, no. 5, pp. 696–710, May 2021.
[5] S. Han, H. Mao, and W. J. Dally, "Deep compression: Compressing deep neural network with pruning, trained quantization and Huffman coding," in *Proc. 4th Int. Conf. Learn. Representations (ICLR)*, San Juan, Puerto Rico, 2016.
[6] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2016, pp. 770–778.
[7] A. G. Howard et al., "MobileNets: Efficient convolutional neural networks for mobile vision applications," 2017, *arXiv:1704.04861*.
[8] A. Vaswani et al., "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 30, 2017, pp. 6000–6010.
[9] A. Burrello, M. Scherer, M. Zanghieri, F. Conti, and L. Benini, "A microcontroller is all you need: Enabling transformer execution on low-power IoT endnodes," in *Proc. IEEE Int. Conf. Omni-Layer Intell. Syst. (COINS)*, 2021, pp. 1–6.
[10] A. Burrello, A. Garofalo, N. Bruschi, G. Tagliavini, D. Rossi, and F. Conti, "DORY: Automatic end-to-end deployment of real-world DNNs on low-cost IoT MCUs," *IEEE Trans. Comput.*, vol. 70, no. 8, pp. 1253–1268, Aug. 2021.
[11] D. Hendrycks and K. Gimpel, "Gaussian error linear units (GELUs)," 2016, *arXiv:1606.08415*.
[12] A. Garofalo, M. Rusci, F. Conti, D. Rossi, and L. Benini, "PULP-NN: Accelerating quantized neural networks on parallel ultra-low-power RISC-V processors," *Philos. Trans. Roy. Soc. A*, vol. 378, no. 2164, 2020, Art. no. 20190155.
[13] A. Katharopoulos, A. Vyas, N. Pappas, and F. Fleuret, "Transformers are RNNs: Fast autoregressive transformers with linear attention," in *Proc. Int. Conf. Mach. Learn.*, PMLR, 2020, pp. 5156–5165.
[14] K. M. Choromanski et al., "Rethinking attention with performers," in *Proc. 9th Int. Conf. Learn. Representations (ICLR), Virtual Event, Austria,* OpenReview.net, 2021, pp. 63–101.
[15] D. Bolya, C.-Y. Fu, X. Dai, P. Zhang, and J. Hoffman, "Hydra attention: Efficient attention with many heads," in *Proc. Eur. Conf. Comput. Vis.*, Switzerland: Springer Nature, 2022, pp. 35–49.
[16] B. Peng et al., "RWKV: Reinventing RNNs for the transformer era," in *Findings Assoc. Comput. Linguistics: EMNLP*, H. Bouamor, J. Pino, and K. Bali, Eds., Singapore: Association for Computational Linguistics, Dec. 2023, pp. 14048–14077.
[17] S. Wang, B. Z. Li, M. Khabsa, H. Fang, and H. Ma, "Linformer: Self-attention with linear complexity," 2020, *arXiv:2006.04768*.
[18] Z. Qin et al., "The devil in linear transformer," in *Proc. Conf. Empirical Methods Natural Lang. Process.*, Abu Dhabi, United Arab Emirates: Association for Computational Linguistics, 2022, pp. 7025–7041.
[19] N. Shazeer, "Fast transformer decoding: One write-head is all you need," 2019, *arXiv:1911.02150*.
[20] J. Ainslie, J. Lee-Thorp, M. de Jong, Y. Zemlyanskiy, F. Lebron, and S. Sanghai, "GQA: Training generalized multi-query transformer models from multi-head checkpoints," in *Proc. Conf. Empirical Methods Natural Lang. Process.*, H. Bouamor, J. Pino, and K. Bali, Eds., Singapore: Association for Computational Linguistics, Dec. 2023, pp. 4895–4901.
[21] H. Touvron et al., "Llama 2: Open foundation and fine-tuned chat models," 2023, *arXiv:2307.09288*.
[22] T. Dao, "FlashAttention-2: Faster attention with better parallelism and work partitioning," in *Proc. 12th Int. Conf. Learn. Representations*, 2024, pp. 210–224.
[23] W. Kwon et al., "Efficient memory management for large language model serving with PagedAttention," in *Proc. 29th Symp. Operating Syst. Princ. (SOSP '23)*, New York, NY, USA: ACM, 2023, pp. 611–626.
[24] R. Pope , "Efficiently scaling transformer inference," *Proc. Mach. Learn. Syst.*, vol. 5, 2023, pp. 606–624.
[25] M. Milakov and N. Gimelshein, "Online normalizer calculation for softmax," 2018, *arXiv:1805.02867*.
[26] L. Liu, Z. Qu, Z. Chen, F. Tu, Y. Ding, and Y. Xie, "Dynamic sparse attention for scalable transformer acceleration," *IEEE Trans. Comput.*, vol. 71, no. 12, pp. 3165–3178, Dec. 2022.
[27] T. J. Ham et al., "ELSA: Hardware-software co-design for efficient, lightweight self-attention mechanism in neural networks," in *Proc. ACM/IEEE 48th Annu. Int. Symp. Comput. Archit. (ISCA)*, 2021, pp. 692–705.
[28] L. Lai, N. Suda, and V. Chandra, "CMSIS-NN: Efficient neural network kernels for ARM cortex-M CPUs," 2018, *arXiv:1801.06601*.
[29] S. G. Bhaskaracharya, J. Demouth, and V. Grover, "Automatic kernel generation for Volta tensor cores," 2020, *arXiv:2006.12645*.
[30] C. L. Lawson, R. J. Hanson, D. R. Kincaid, and F. T. Krogh, "Basic linear algebra subprograms for Fortran usage," *ACM Trans. Math. Softw.*, vol. 5, no. 3, pp. 308–323, Sep. 1979.
[31] M. Abadi et al., "TensorFlow: A system for large-scale machine learning," in *Proc. 12th USENIX Conf. Operating Syst. Des. Implementation (OSDI)*, USA: USENIX Association, 2016, pp. 265–283.
[32] A. Ivanov, N. Dryden, T. Ben-Nun, S. Li, and T. Hoefler, "Data movement is all you need: A case study on optimizing transformers," in *Proc. Mach. Learn. Syst.*, vol. 3, 2021, pp. 711–732.
[33] W. Liu, D. Anguelov, D. Erhan, C. Szegedy, S. Reed, C.-Y. Fu, and A. C. Berg, "SSD: Single shot MultiBox detector," in *Lecture Notes in Computer Science*. Springer International Publishing, 2016, pp. 21–37.
[34] L. Mei, P. Houshmand, V. Jain, S. Giraldo, and M. Verhelst, "ZigZag: Enlarging joint architecture-mapping design space exploration for DNN accelerators," *IEEE Trans. Comput.*, vol. 70, no. 8, pp. 1160–1174, Aug. 2021.
[35] L. Mei, K. Goetschalckx, A. Symons, and M. Verhelst, "DeFiNES: Enabling fast exploration of the depth-first scheduling space for DNN accelerators through analytical modeling," in *Proc. IEEE Int. Symp. High-Perform. Comput. Archit. (HPCA)*, Piscataway, NJ, USA: IEEE Press, 2023, pp. 570–583.
[36] M. Alwani, H. Chen, M. Ferdman, and P. Milder, "Fused-layer CNN accelerators," in *Proc. 49th Annu. IEEE/ACM Int. Symp. Microarchit. (MICRO)*, 2016, pp. 1–12.
[37] P. Busia et al., "Reducing false alarms in wearable seizure detection with EEGformer: A compact transformer model for MCUs," *IEEE Trans. Biomed. Circuits Syst.*, vol. 18, no. 3, pp. 608–621, Jun. 2024.
[38] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh, "OPTQ: Accurate quantization for generative pre-trained transformers," in *Proc. 11th Int. Conf. Learn. Representations*, 2023, pp. 153–169.
[39] A. Dosovitskiy et al., "An image is worth 16x16 words: Transformers for image recognition at scale," in *Proc. 9th Int. Conf. Learning Representations (ICLR), Virtual Event, Austria,* OpenReview.net, 2021, pp. 307–329.

Source: https://ieeexplore.ieee.org/document/10755971 (published print)
Google Scholar research | 2026-10-08 | Article 04 | Original p. 15 | Orange: discussion; blue: reference

Understanding and Overcoming the Challenges of Efficient Transformer Quantization

Yelysei Bondarenko, Markus Nagel, Tijmen Blankevoort
Qualcomm AI Research*
{ybond, markusn, tijmen}@qti.qualcomm.com

Abstract

Transformer-based architectures have become the de-facto standard models for a wide range of Natural Language Processing tasks. However, their memory footprint and high latency are prohibitive for efficient deployment and inference on resource-limited devices. In this work, we explore quantization for transformers. We show that transformers have unique quantization challenges – namely, high dynamic activation ranges that are difficult to represent with a low bit fixed-point format. We establish that these activations contain structured outliers in the residual connections that encourage specific attention patterns, such as attending to the special separator token. To combat these challenges, we present three solutions based on post-training quantization and quantization-aware training, each with a different set of compromises for accuracy, model size, and ease of use. In particular, we introduce a novel quantization scheme – per-embedding-group quantization. We demonstrate the effectiveness of our methods on the GLUE benchmark using BERT, establishing state-of-the-art results for post-training quantization. Finally, we show that transformer weights and embeddings can be quantized to ultra-low bit-widths, leading to significant memory savings with a minimum accuracy loss. Our source code is available at https://github.com/qualcomm-ai-research/transformer-quantization.

1 Introduction

Recently, transformer architectures have shown remarkable improvement in many Natural Language Processing (NLP) tasks and beyond. Based on the original Transformer (Vaswani et al., 2017), language models pre-trained from large corpora of unlabeled text, such as BERT (Devlin et al., 2019), RoBERTa (Liu et al., 2019), XLNet (Yang et al., 2019), Transformer-XL (Dai et al., 2019), GPT family (Radford et al., 2018, 2019; Brown et al., 2020), have become an indispensable building block in modern NLP pipelines. They are also increasingly adopted in other areas, including vision (Carion et al., 2020; Dosovitskiy et al., 2020; Chen et al., 2020; Girdhar et al., 2019) and audio (Dong et al., 2018; Child et al., 2019).

Despite cutting edge results in many applications, pre-trained transformer-based models are extremely large, sometimes exceeding billions of parameters. Hence, efficient deployment of these models on resource-constrained embedded systems, and even sometimes in data centers, has become an important problem due to high latency and prohibitively large memory footprint and energy consumption.

One effective method to tackle this problem is neural network quantization. Quantization reduces memory consumption by using low-bit precision for weight and activation tensors. Is also reduces inference time, and improves energy efficiency by employing low-bit fixed-point arithmetic instead of floating-point arithmetic (Horowitz, 2014).

Quantization, however, is not free. It introduces additional noise in the network that can lead to a drop in the model's performance. While prior work has demonstrated the feasibility of integer-only inference for computer vision models (Lin et al., 2016; Jacob et al., 2018; Krishnamoorthi, 2018; Zhang et al., 2018; Choukroun et al., 2019; Dong et al., 2019; Esser et al., 2019; Nagel et al., 2019, 2020), there is relatively little work done on quantizing NLP models (Wang et al., 2018b; Xu et al., 2018), and specifically on transformer models.

Understanding the challenges of transformer quantization and designing a robust and easy-to-use quantization pipeline for them constitute the primary goal of this paper. The contributions of our work include:

* Qualcomm AI Research is an initiative of Qualcomm Technologies, Inc.

Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing, pages 7947–7969
November 7–11, 2021. ©2021 Association for Computational Linguistics

Source: https://aclanthology.org/2021.emnlp-main.627.pdf
Google Scholar research | 2026-10-07 | Article 05 | Original p. 1 | Orange: discussion; blue: reference

Quantizing a real-valued tensor $\mathbf{x}$ is performed by first mapping it to an unsigned integer grid:

$$\mathbf{x}^{(\mathbb{Z})} = \text{clip}\left(\left\lfloor \frac{\mathbf{x}}{s} \right\rceil + z; 0, 2^b - 1\right), \quad (1)$$

It is possible to approximately recover the real-valued input $\mathbf{x}$ through an operation that is often referred to as *de-quantization*:

$$\widehat{\mathbf{x}} := q\left(\mathbf{x};\, s, z, b\right) = s\left(\mathbf{x}^{(\mathbb{Z})} - z\right) \approx \mathbf{x}. \quad (2)$$

In the case of symmetric quantization, we restrict the quantization grid to be symmetric around z.

It is common to have a single set of quantization parameters per tensor, known as *per-tensor quantization*. One could also increase the *quantization granularity* by defining separate quantizers for individual segments of a tensor. This will improve the accuracy of a network, but at the cost of an additional compute and memory overhead.

An important class of quantization methods is *post-training quantization* (PTQ) algorithms, which take a pre-trained FP32 network and convert it directly into a fixed-point network without the need for the original training pipeline (Krishnamoorthi, 2018). A vital step in the PTQ process is finding good quantization ranges for each quantizer. One way of doing this is *static range estimation*, which determines quantization parameters for the network by passing a few batches of calibration data through the model before inference. It yields more efficient inference since all the quantization parameters are known in advance and fixed. Several of the most common range estimators include:

current min-max or simply *min-max*, uses the full dynamic range of the tensor (Zhou et al., 2016; Wu et al., 2018b; Zhu et al., 2020);

running min-max uses exponential moving average of the min and max over multiple batches (Krishnamoorthi, 2018);

MSE finds quantization parameters that minimize mean squared error between quantized and floating-point tensors (Choukroun et al., 2019; Banner et al., 2018).

An alternative to PTQ is to train a neural network with the simulated quantization operations in the network, known as *quantization-aware training* (QAT, Jacob et al. 2018; Gupta et al. 2015; Krishnamoorthi 2018). It allows the model to better adapt to the introduced quantization noise compared to PTQ, at the cost of longer training times, the need for labeled data and doing a hyper-parameter search. Gradients through the non-differentiable quantization step are usually approximated using the *straight-through* estimator (Bengio et al., 2013). Ranges for both weights and activations can be set using PTQ range estimators or learned jointly with the weights during training, as in Esser et al. (2019); Jain et al. (2019).

Finally, it is possible to assign different bit-widths for different layers or parts of the network, a technique known as *mixed precision* (Lin et al., 2016; Wu et al., 2018a; Zhou et al., 2018; Dong et al., 2019; Wang et al., 2019; van Baalen et al., 2020).

Transformer quantization Junczys-Dowmunt et al. (2018) applied knowledge distillation and 8-bit post-training quantization to speed up transformer models for neural machine translation. Bhandare et al. (2019) also applied 8-bit post-training quantization to the transformer model for machine translation and demonstrated how to utilize specialized hardware to accelerate the inference process.

Zafrir et al. (2019) proposed an 8-bit quantization scheme for BERT-like models and achieves compression of up to 25% of the original model size. Shen et al. (2020) applies mixed-precision quantization on BERT, where they assign a different precision to different layers according to their sensitivity defined by Hessian information. Kim et al. (2021) proposed a fully integer-only arithmetic inference scheme based on second-order polynomial approximations for GELU, Softmax, and LayerNorm non-linearities. Some examples of transformer-based model quantization with alternative quantization schemes include Fan et al. (2020); Darvish Rouhani et al. (2020).

Note that all the mentioned approaches for BERT-like transformer quantization employ some form of QAT and either do not discuss PTQ alternatives or only use them as weak baselines.

3 Problem investigation

First, we investigate what happens when we apply standard 8-bit post-training quantization to the BERT model and evaluate it on eight downstream tasks from the GLUE benchmark (Wang et al., 2018a). To quantize fine-tuned models, we use uniform affine quantization with static range

Source: https://aclanthology.org/2021.emnlp-main.627.pdf
Google Scholar research | 2026-10-07 | Article 05 | Original p. 3 | Orange: discussion; blue: reference

PEG quantization with permutation can still be simulated on hardware that only supports per-tensor operations. First, we can share the same permutation for FFN's input, output and sum since we expect the outliers in the output dominate the ones from the input. Second, we use the permutation-equivariant properties of LayerNorm and linear layers (weights are permuted accordingly before inference). We first permute the output of the first LayerNorm, proceed as described above, and then apply inverse permutation before the next LayerNorm.

Note that the PEG quantization has a negligible memory overhead, introducing only $d + 2 \cdot 3 \cdot K$ extra parameters per attention layer (permutation indices and scale & zero points per group for FFN's input, output, and sum), which is less than 0.04% of the total size of BERT-base model.

Quantization-aware training Finally, we consider a variant of QAT with learnable ranges for both weights and activations by adapting the procedure from Esser et al. (2019); Jain et al. (2019) for BERT-like transformer models. Simulating the quantization process during fine-tuning allows the model to adapt to quantization noise and often significantly increases performance compared to post-training quantization.

Criterion	MP-PTQ	PEG-PTQ	QAT
Post-training	✓	✓	✗
Per-tensor	✓	✗	✓
Same bit-width	✗	✓	✓

Table 3: Comparison between proposed techniques (MP = mixed precision, PEG = per-embedding-group).

Comparison of methods We summarize different trade-offs for the proposed techniques in Table 3. As discussed in Section 2, usually PTQ methods are preferred over QAT algorithms since they are faster and require either no data or only a small calibration dataset. Additionally, they typically require almost no hyperparameter tuning, enabling easy and computationally efficient quantization. Allocating a higher bit-width to certain parts of the network will reduce the efficiency gain from quantization, because higher bit-width layers are more computationally expensive. It is also not supported by all target hardware. Per-embedding-group quantization has a smaller granularity compared to per-tensor quantization. It leads to a minor amount of extra compute (and potential latency) due to the additional summation and re-quantization that occurs and might not be supported natively on every fixed-point platform. Meanwhile, we have shown a way to simulate this scheme on a hardware that only support per-tensor quantization operations.

5 Experiments

In this section, we evaluate the proposed quantization techniques for the BERT model on GLUE downstream tasks.

Experimental setup In all experiments, we use uniform affine quantization – symmetric weights, asymmetric activations – with the static activation range setting, as discussed in Section 2. We quantize all layer's weights and activations. For 8-bit weight quantization, we use the best range settings found in the experiment from Section 3, which can be found in Appendix B.2. However, for low (<8) bit weight and token embedding quantization, we always use the MSE range estimator, as recommended by Choukroun et al. (2019); Banner et al. (2018). We set activation ranges based on min and max from a single input sequence. For PTQ experiments, we report the median score over five runs with different random seeds.

For QAT experiments, we initialize all quantization parameters from the PTQ setup described above. Similarly to full-precision fine-tuning, we use Adam (Kingma and Ba, 2014) and a maximum sequence length of 128, with padding using a special `[PAD]` token for shorter sequences. We use a typical learning rate schedule from the transformer literature (Devlin et al., 2019; Liu et al., 2019; Lan et al., 2019) – a linear warmup for the first 10% of training steps followed by a linear decay to zero. We perform a hyper-parameter search over the maximum learning rate, batch size, number of epochs, and the self-attention dropout rate for every task and report the best median score over three runs with different random seeds. For reproducibility, we included more details on the search space and selected hyper-parameters in Appendix B.3.

Mixed precision PTQ First, we present the results for mixed precision post-training quantization (MP-PTQ) in Table 4, where we start from 8-bit activations and progressively keep more and more operations in 16-bit precision. We see that for classification tasks (MNLI, QNLI, RTE), it is sufficient to keep a few of the most problematic parts in 16-bit

Source: https://aclanthology.org/2021.emnlp-main.627.pdf
Google Scholar research | 2026-10-07 | Article 05 | Original p. 7 | Orange: discussion; blue: reference

Suyog Gupta, Ankur Agrawal, Kailash Gopalakrishnan, and Pritish Narayanan. 2015. Deep learning with limited numerical precision. In *International conference on machine learning*, pages 1737–1746. PMLR.

M. Horowitz. 2014. 1.1 computing's energy problem (and what we can do about it). In *2014 IEEE International Solid-State Circuits Conference Digest of Technical Papers (ISSCC)*, pages 10–14.

Neil Houlsby, Andrei Giurgiu, Stanislaw Jastrzebski, Bruna Morrone, Quentin De Laroussilhe, Andrea Gesmundo, Mona Attariyan, and Sylvain Gelly. 2019. Parameter-efficient transfer learning for nlp. In *International Conference on Machine Learning*, pages 2790–2799. PMLR.

Itay Hubara, Matthieu Courbariaux, Daniel Soudry, Ran El-Yaniv, and Yoshua Bengio. 2017. Quantized neural networks: Training neural networks with low precision weights and activations. *The Journal of Machine Learning Research*, 18(1):6869–6898.

Forrest Iandola, Albert Shaw, Ravi Krishna, and Kurt Keutzer. 2020. SqueezeBERT: What can computer vision teach NLP about efficient neural networks? In *Proceedings of SustaiNLP: Workshop on Simple and Efficient Natural Language Processing*, pages 124–135, Online. Association for Computational Linguistics.

Benoit Jacob, Skirmantas Kligys, Bo Chen, Menglong Zhu, Matthew Tang, Andrew Howard, Hartwig Adam, and Dmitry Kalenichenko. 2018. Quantization and training of neural networks for efficient integer-arithmetic-only inference. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 2704–2713.

Sambhav R Jain, Albert Gural, Michael Wu, and Chris Dick. 2019. Trained uniform quantization for accurate and efficient neural network inference on fixed-point hardware. *arXiv preprint arXiv:1903.08066*, 6.

Xiaoqi Jiao, Yichun Yin, Lifeng Shang, Xin Jiang, Xiao Chen, Linlin Li, Fang Wang, and Qun Liu. 2020. TinyBERT: Distilling BERT for natural language understanding. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, pages 4163–4174, Online. Association for Computational Linguistics.

Marcin Junczys-Dowmunt, Kenneth Heafield, Hieu Hoang, Roman Grundkiewicz, and Anthony Aue. 2018. Marian: Cost-effective high-quality neural machine translation in c++. *arXiv preprint arXiv:1805.12096*.

Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and François Fleuret. 2020. Transformers are rnns: Fast autoregressive transformers with linear attention. In *International Conference on Machine Learning*, pages 5156–5165. PMLR.

Sehoon Kim, Amir Gholami, Zhewei Yao, Michael W Mahoney, and Kurt Keutzer. 2021. I-bert: Integer-only bert quantization. *arXiv preprint arXiv:2101.01321*.

Diederik P Kingma and Jimmy Ba. 2014. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*.

Nikita Kitaev, Łukasz Kaiser, and Anselm Levskaya. 2020. Reformer: The efficient transformer. *arXiv preprint arXiv:2001.04451*.

Raghuraman Krishnamoorthi. 2018. Quantizing deep convolutional networks for efficient inference: A whitepaper. *arXiv preprint arXiv:1806.08342*.

Zhenzhong Lan, Mingda Chen, Sebastian Goodman, Kevin Gimpel, Piyush Sharma, and Radu Soricut. 2019. Albert: A lite bert for self-supervised learning of language representations. *arXiv preprint arXiv:1909.11942*.

Hector Levesque, Ernest Davis, and Leora Morgenstern. 2012. The winograd schema challenge. In *Thirteenth International Conference on the Principles of Knowledge Representation and Reasoning*. Citeseer.

Darryl Lin, Sachin Talathi, and Sreekanth Annapureddy. 2016. Fixed point quantization of deep convolutional networks. In *International conference on machine learning*, pages 2849–2858. PMLR.

Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. 2019. Roberta: A robustly optimized bert pretraining approach. *arXiv preprint arXiv:1907.11692*.

Markus Nagel, Rana Ali Amjad, Mart Van Baalen, Christos Louizos, and Tijmen Blankevoort. 2020. Up or down? adaptive rounding for post-training quantization. In *International Conference on Machine Learning*, pages 7197–7206. PMLR.

Markus Nagel, Mart van Baalen, Tijmen Blankevoort, and Max Welling. 2019. Data-free quantization through weight equalization and bias correction. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 1325–1334.

Markus Nagel, Marios Fournarakis, Rana Ali Amjad, Yelysei Bondarenko, Mart van Baalen, and Blankevoort Tijmen. 2021. A white paper on neural network quantization. *arXiv preprint arXiv:2106.08295*.

Sai Prasanna, Anna Rogers, and Anna Rumshisky. 2020. When bert plays the lottery, all tickets are winning. *arXiv preprint arXiv:2005.00561*.

Alec Radford, Karthik Narasimhan, Tim Salimans, and Ilya Sutskever. 2018. Improving language understanding by generative pre-training.

Source: https://aclanthology.org/2021.emnlp-main.627.pdf

Google Scholar research | 2026-10-07 | Article 05 | Original p. 11 | Orange: discussion; blue: reference

Future Generation Computer Systems 149 (2023) 717–731

Contents lists available at ScienceDirect

Future Generation Computer Systems

journal homepage: www.elsevier.com/locate/fgcs

7 μJ/inference end-to-end gesture recognition from dynamic vision sensor data using ternarized hybrid convolutional neural networks

Georg Rutishauser [a,*], Moritz Scherer [a], Tim Fischer [a], Luca Benini [a,b]

[a] ETH Zürich, Gloriastrasse 35, Zürich, 8092, Switzerland
[b] Università di Bologna, Via Zamboni 33, Bologna, 40126, Italy

ARTICLE INFO

Article history:
Received 8 December 2022
Received in revised form 23 May 2023
Accepted 14 July 2023
Available online 29 July 2023

Keywords:
Dynamic vision sensors
Gesture recognition
Ternary neural networks
Edge computing
Low-power systems

ABSTRACT

Dynamic vision sensor (DVS) cameras enable energy-activity proportional visual sensing by only propagating events produced by changes in the observed scene. Furthermore, by generating these events asynchronously, they offer μs-scale latency while eliminating the redundant data transmission inherent to classical, frame-based cameras. However, the potential of DVS to improve the energy efficiency of IoT sensor nodes can only be fully realized with efficient and flexible systems that tightly integrate sensing, processing, and actuation capabilities. In this paper, we propose a complete end-to-end pipeline for DVS event data classification implemented on the Kraken parallel ultra-low power (PULP) system-on-chip and apply it to gesture recognition. A dedicated on-chip peripheral interface for DVS cameras aggregates the received events into ternary *event frames*. We process these video frames with a fully ternarized two-stage temporal convolutional network (TCN). The neural network can be executed either on Kraken's PULP cluster of general-purpose RISC-V cores or on CUTIE, the on-chip ternary neural network accelerator. We perform extensive ablations on network structure, training, and data generation parameters. We achieve a validation accuracy of 97.7 % on the DVS128 11-class gesture dataset, a new record for embedded implementations. With in-silicon power and energy measurements, we demonstrate a classification energy of 7 μJ at a latency of 0.9 ms when running the TCN on CUTIE, a reduction of inference energy by 67× when compared to the state of the art in embedded gesture recognition. The processing system consumes as little as 4.7 mW in continuous inference, enabling always-on gesture recognition and closing the gap between the efficiency potential of DVS cameras and application scenarios.

© 2023 The Author(s). Published by Elsevier B.V. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

1. Introduction

The exponential proliferation of small-scale sensing systems such as Internet of Things (IoT) sensor nodes and wearable devices has coincided with a revolution in data processing, where machine learning (ML) algorithms such as CNNs and transformers have dramatically improved the state of the art in fields such as computer vision, natural language processing or biomedical signal processing [1–4]. As many applications involving the collection of data demand for their interpretation and reaction to the result, these developments are naturally symbiotic. However, the approach of transmitting raw data to cloud servers to be processed with powerful and compute-intensive ML model does not scale to the enormous amounts of data collected and conflicts with the constraints of battery-powered, sensor-driven applications requiring real-time performance: wireless communication is power-intensive and incurs communication latency, and the transmission of raw sensor data raises privacy and security concerns.

This conflict is addressed by the emerging research field of edge AI, which aims to enable efficient and accurate ML-based processing of sensor data on edge sensor nodes. These systems, typically built around Microcontrollers (MCUs), operate under stringent power, memory, and compute resource constraints. A key algorithmic technique in edge AI is *quantization* of neural networks, where parameters and intermediate activations are represented in low-bitwidth data formats, reducing a model's memory and storage footprints. With hardware support for low-precision arithmetic through specialized extensions to general-purpose instruction set architectures (ISAs) [5] or dedicated accelerators, quantization also increases computational throughput and efficiency, leading to lower energy consumption per inference.

However, to optimize a system's power consumption and performance, the entire pipeline from the sensor to the reaction to

* Corresponding author.
E-mail addresses: georgr@iis.ee.ethz.ch (G. Rutishauser), scheremo@iis.ee.ethz.ch (M. Scherer), fischeti@iis.ee.ethz.ch (T. Fischer), lbenini@iis.ee.ethz.ch (L. Benini).

https://doi.org/10.1016/j.future.2023.07.017
0167-739X/© 2023 The Author(s). Published by Elsevier B.V. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

Source: https://doi.org/10.1016/j.future.2023.07.017
Google Scholar research | 2026-10-08 | Article 06 | Original p. 1 | Orange: discussion; blue: reference

their respective ε. We provide more details on the process of integerization in Section 4.3 and a detailed mathematical derivation can be found in [60].

Methods to convert a full-precision network to a FQ one can be divided into two categories: Post-Training Quantization (PTQ) and QAT.

PTQ methods [61–63] generally start from a pre-trained full-precision model and do not optimize the model parameters any further. They determine suitable clipping bounds for each tensor to be quantized by calibration with real [62,64] or synthetic data [63,65].

In contrast, QAT methods train a network's parameters (or fine-tune them from a converged full-precision checkpoint) while incorporating quantization. Most such methods [66–69] are variations of the Straight-Through-Estimator (STE). Introduced in [70], the STE is implemented by applying a (non-differentiable) uniform quantizer as in Eq. (1) in the forward pass of the training procedure and defining its derivative as

$$\frac{dQ(x, a, b, n_{lvl})}{dx} = \mathbb{1}_{a \leq x \leq b},$$

i.e., treating it as a clipped identity function during the backward pass. Different QAT algorithms provide various ways of finding the clipping bounds $\{a, b\}$. The Learned Step Size Quantization (LSQ) [68] and TQT [67] methods apply the STE to the quantizer (1) to derive a gradient with respect to the quantization step size ε, which is used to optimize ε directly (in [68]) or the logarithm of the clipping interval $\log_2(b - a)$ (in [67]).

In contrast to the previously introduced methods, INQ [71] quantizes weights without employing the STE. This is achieved by iteratively quantizing and freezing a fraction of all weights and retraining the remaining full-precision weights until all weights are quantized and frozen. As this procedure does not apply to activations, they must still be quantized using an STE-based technique.

3.2.1. Extreme quantization - Binarized and ternarized networks

The techniques introduced in Section 3.2 can be applied to achieve quantization to any desired precision, with lower-precision quantization generally leading to more significant degradation of task accuracy. The most extreme forms of QNNs are BNNs [72,73] and TNNs [74,75]. In BNNs, all weights and activations are quantized to values in $\{-1, 1\}$. TNNs can be considered an extension of BNNs, with tensor elements taking values in the set $\{-1, 0, 1\}$. Consequently, arithmetic operations on binary and ternary values can be implemented with simple hardware: Computing the dot product of two N-element binary vectors can be achieved by N XNOR gates and an N-bit popcount unit [72]. The dot product of two N-element ternary vectors can be computed using N ternary multipliers and 2 N-element popcount units. This property makes BNNs and TNNs highly attractive targets for acceleration. While binary and ternary quantization schemes allow for the design of simple arithmetic units which improves inference energy efficiency considerably [11], statistical inference accuracy is typically degraded, even when employing QAT. Significant research has been conducted to combat this negative impact on accuracy and significant advances have been reported [76,77]; however, low-bitwidth quantization typically still carries a non-negligible penalty on statistical accuracy. As such, binary and ternary neural networks are best suited for applications where the penalty on accuracy is acceptable, such as in low-complexity tasks or always-on sensing or wake-up trigger applications. State-of-the-art digital BNN and TNN accelerators achieve high efficiencies of 145-392 TOp/J [11,78,79].

3.2.2. The CUTIE TNN accelerator

The Completely Unrolled Ternary Inference Engine (CUTIE) is an accelerator for TNNs, introduced in [11]. In contrast to most neural network accelerator architectures, CUTIE uses a compute architecture that is fully parallel in the computation of each output pixel by processing elements termed output channel compute unit (OCU). Each OCU computes the sum of dot products corresponding to a single filter in a fully unrolled manner and in a single cycle. To parallelize the computation of each pixel, CUTIE uses one OCU per output channel. Each OCU has a dedicated latch-based weight buffer that holds all filter weights. In order to reuse input kernel windows optimally, CUTIE features a 3-line buffer, which dispatches full 3×3 kernel windows.

CUTIE's maximally parallel design not only increases arithmetic optimization by forming large adder trees over the sum of dot products in each OCU, but also minimizes data movement since every pixel in the input feature map and every weight in every filter is only loaded once per layer. These characteristics make CUTIE highly efficient for neural network inference at the edge while enabling throughput at rates exceeding 1000 inf./s. Kraken features an updated version of CUTIE, described and evaluated in detail in [80]. The main improvements from the original version are support for causal, dilated 1D convolutions and the addition of an intermediate buffer for a window of up to 24 CNN output vectors, enabling the execution of ternarized TCNs. The CUTIE accelerator in Kraken features 96 OCUs, computing one output pixel with up to 96 input/output channels per cycle and resulting in a peak throughput of 56 TOp/s when clocked at 54 MHz for maximum efficiency in a 22 nm implementation.

4. Event frame processing pipeline

4.1. Overview

The class of cameras we target produce events consisting of three values each: Two spatial coordinates (x_e, y_e) describing the location of the event on the sensor grid, and a polarity $p_e \in \{-1, 1\}$ to indicate a decrease or increase in brightness, respectively. The first stage of our proposed pipeline performs data preparation, aggregating events detected by the DVS camera into 2-dimensional frames. The resulting frames are natively ternary: Pixels where at least one event occurred during a frame interval take the value of the most recent event's polarity, while pixels with no activity take the value 0. The prepared data is then processed by a two-stage hybrid CNN inspired by [20]. In the first processing stage, C_{in} sequential frames are fed into a fully ternarized 2D CNN. By processing multiple frames per inference, the 2D CNN can analyze short-term temporal dependencies, which are encoded into ternary feature vectors by the last layer.

In the third and final stage, a fully ternarized TCN analyzes the longer-term temporal dependencies by performing inference on a sliding window covering N_{TCN} feature vectors. Its output is a vector of class scores $V_p \in \mathbb{Z}^{N_c}$, where N_c is the number of classes. For the DVS128 dataset, $N_c = 11$. Finally, the class label of the sequence is given as $Cls = \arg\max_i \left(V_p\right)$.

Fig. 1 shows a diagram of the proposed processing pipeline. The frame extraction process is shown in Fig. 2 and described in more detail in Section 4.5.

4.2. Network design

The ternarized hybrid CNN/TCN adopts a simple feed-forward architecture, i.e., there are no residual branches in the network. All layers but the first have N_{ch} output channels. The two networks' topologies are listed in Table 2. The final, fully ternarized network consists only of convolutional layers with no bias,

Source: https://doi.org/10.1016/j.future.2023.07.017
Google Scholar research | 2026-10-08 | Article 06 | Original p. 5 | Orange: discussion; blue: reference

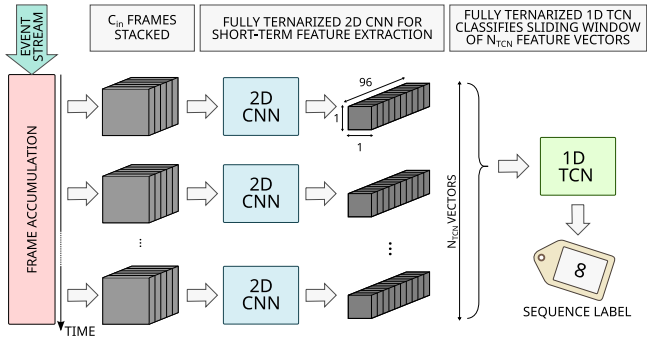


Fig. 1. Overview of the proposed processing pipeline.

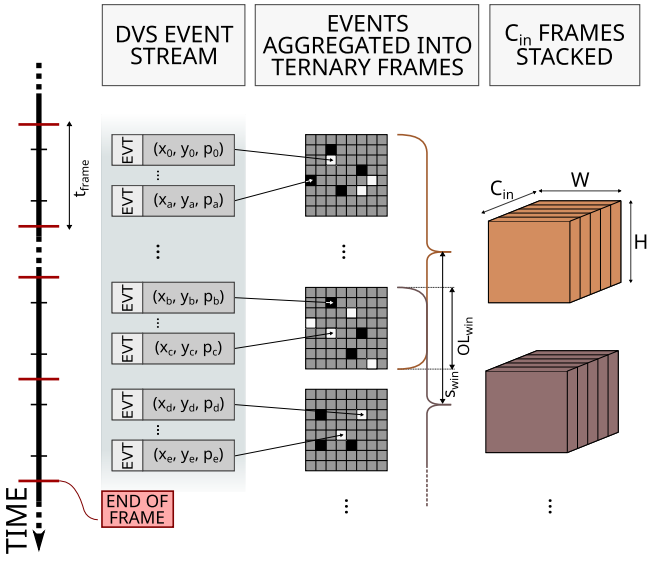


Fig. 2. Frame aggregation for processing by the 2D CNN. All events occurring during a time window of length t_{frame} are assigned to the same frame. The 2D CNN takes C_{in} frames as input, and two successive stacks of C_{in} frames overlap by OL_{win} frames.

ternary activations (3) and pooling layers. Consider a layer stack consisting of a convolutional layer with N_i/N_o input/output channels respectively and an optional pooling layer and denote the ternary values as $\mathcal{T} \triangleq \{-1, 0, 1\}$. The layer stack takes as input a tensor $\mathbf{X} \in \mathcal{T}^{N_i \times H_X \times W_X}$, where H_X/W_X are the spatial dimensions. $\mathbf{X}$ is convolved with the convolutional weights $\mathbf{W} \in \mathcal{T}^{N_o \times N_i \times k \times k}$ and pooled, to yield pre-activations $\mathbf{Z} \in \mathbb{Z}^{N_o \times H_Y \times W_Y}$, shown in Eq. (2). $\mathbf{Z}$ is then mapped to ternary activations $\mathbf{Y} \in \mathcal{T}^{N_o \times H_Y \times W_X}$ by channel-wise thresholding as shown in Eq. (3) with $\mathbf{t}^{lo}$ and $\mathbf{t}^{hi}$, $\mathbf{t}^{lo}, \mathbf{t}^{hi} \in \mathbb{Z}^{N_o}$.

$$\mathbf{Z} = pool\,(\mathbf{X} * \mathbf{W}) \tag{2}$$

$$y_{i,x,y} = \begin{cases} -1, & z_{i,x,y} < t_i^{lo} \\ 0, & t_i^{lo} \le z_{i,x,y} < t_i^{hi} \\ 1, & z_{i,x,y} \ge t_i^{hi} \end{cases} \tag{3}$$

4.3. Quantization-aware training

To train the ternarized networks we evaluate, we perform QAT using two algorithms: INQ [71] and TQT [67]. To determine the impact of the activation function used to train the network, we compare Hard Hyperbolic Tangent (HtanH)-activated networks to ReLU-based ones. Note that both types of FQ networks are ultimately converted to fully ternarized networks as described in Section 4.2. QAT is performed in 4 steps (3 in the case of HtanH activations):

1. Training full-precision network to convergence,
2. quantization of activations,
3. (incremental) weight quantization, and, if ReLU activations were used,
4. fine-tuning to compensate for non-zero padding.

For all training algorithm and activation function combinations, we train networks with Batch Normalization (BN) layers inserted after the convolutional layers to allow for channel-wise scaling. After full-precision training has converged, HtanH and ReLU activations of layer l are replaced with symmetrical and asymmetrical 3-level step functions (4) and (5), respectively.

$$A^l_{symm}(x) = \begin{cases} \varepsilon^l_A, & x \ge \varepsilon^l_A/2 \\ 0, & -\varepsilon^l_A/2 \le x < \varepsilon^l_A/2 \\ -\varepsilon^l_A, & x < -\varepsilon^l_A/2 \end{cases} \tag{4}$$

$$\begin{aligned} A^l_{asymm}(x) &= \begin{cases} 2\varepsilon^l_A, & x \ge 3/2\varepsilon^l_A \\ \varepsilon^l_A, & \varepsilon^l_A/2 \le x < 3/2\varepsilon^l_A \\ 0, & x < \varepsilon^l_A/2 \end{cases} \\ &= A^l_{symm}(x - \varepsilon^l_A) + \varepsilon^l_A \end{aligned} \tag{5}$$

The network is retrained for a few epochs before weight quantization is applied. When using INQ, weight and activation quantization step sizes are kept constant at 1, i.e., $\varepsilon_W = \varepsilon_A = 1$. Weights are frozen to the nearest values in $\mathcal{T}$ incrementally in order of decreasing magnitude, and the unmodified STE is used to propagate gradients through quantized activations. After each freezing step, the network's remaining learnable parameters (free weights and BN parameters) are retrained to convergence. When using TQT, both weight and activation clipping bounds are learned, with weight clipping bounds of each layer learned individually for each output channel. When quantizing weights with TQT, fake-quantization is applied to all weights simultaneously. The detailed training parameters are listed in Section 6.1.

4.4. Network ternarization

To be deployed on CUTIE, FQ networks must be mapped to fully ternarized models containing only ternary parameters and

Table 2
Topology of the proposed hybrid CNN architecture, consisting of a 2D CNN for short-term feature extraction and a 1D TCN to capture longer-term temporal dependencies. **C{1,2}D**: 1/2-dimensional convolution, **MP**: Max-Pooling. **(C)S/V** indicate (**c**ausal) **s**ame/**v**alid padding, respectively.

	Layer	Out Ch.	Out Res.	Pad	Dilation
2D CNN	Input	C_{in}	64×64	N/A	N/A
	C2D(3×3)	32	64×64	S	1
	MP(2×2)	32	32×32	V	1
	C2D(3×3)	N_{ch}	32×32	S	1
	MP(2×2)	N_{ch}	16×16	V	1
	C2D(3×3)	N_{ch}	16×16	S	1
	MP(2×2)	N_{ch}	8×8	V	1
	C2D(3×3)	N_{ch}	8×8	S	1
	MP(2×2)	N_{ch}	4×4	V	1
	C2D(3×3)	N_{ch}	2×2	V	1
	MP(2×2)	N_{ch}	1×1	V	1
1D TCN	Input	N_{ch}	N_{TCN}	N/A	N/A
	C1D(2)	N_{ch}	N_{TCN}	CS	1
	C1D(2)	N_{ch}	N_{TCN}	CS	2
	C1D(2)	N_{ch}	N_{TCN}	CS	4
	C1D(N_{TCN})	11	1	V	1

Source: https://doi.org/10.1016/j.future.2023.07.017

Google Scholar research | 2026-10-08 | Article 06 | Original p. 6 | Orange: discussion; blue: reference

Table 3
Hyperparameters for QAT with TQT.

Parameter	Value
Epochs	50
Batch size	128
Opt.	Adam
LR_0	0.01
LR decr.[a]	Cosine annealing [86]
E_{Q_Wt}/E_{Q_Act}[b]	0/8
Act. clip init.[c]	Const. 6.0

[a]We perform one cycle of cosine annealing over 50 epochs.
[b]Activations and weights are quantized starting from the specified epoch.
[c]Const. x: clipping bounds initialized to x.

or other intervention by the FC core. After CUTIE has finished running the TNN on the input data, it raises an interrupt line and the FC can process and interpret the network's output.

5.2. Mapping TNNs on Kraken

The two processing domains we target for the deployment of our networks are the PULP cluster and CUTIE. In both cases, networks are executed sequentially in a layer-by-layer fashion. To efficiently use the cluster's computational resources, the data on which the cores operate must be stored in the high-bandwidth L1 memory. However, Kraken's L1 memory is too small to hold the inputs, outputs, and weights, necessitating *tiled* execution of layers: the complete input, weight, and output buffers for a layer are held in L2 memory and the layer is divided into smaller execution units by tiling the input, output, and weight tensors along the input channel, output channel or spatial dimensions. For the execution of each tile, the corresponding inputs and weights are transferred by direct memory access (DMA) from L2 to L1 memory and the cluster computes the partial output, which is then transferred back to L2. Double buffering is used to hide the latency of the DMA transfers.

In contrast, CUTIE specializes in efficient processing of TNNs by implementing dedicated activation and weight memories, which offer sufficient memory to avoid tiling of the network. In order to maximize energy efficiency of the system, inferences are executed on the accelerator in a self-contained fashion. To prepare for network execution, weights are written to CUTIE's internal weight memory and parameters for each layer (e.g., kernel size, input size, stride, etc.) are configured, both via CUTIE's external data interface. A ternary input tensor can then be written to CUTIE's activation memory. Inference is triggered by writing to a configuration register or if CUTIE's start interrupt line is raised. CUTIE executes networks autonomously on its internal memories and raises an interrupt when inference has concluded, allowing the host system to read and interpret the results.

6. Results

In this section, we present the results of our algorithmic evaluations as well as power measurements conducted on the Kraken system. First, we show the impact of data preparation, architectural, and training parameters on the network's classification accuracy. Then, we show power consumption and latency measurements for networks mapped both to CUTIE and to Kraken's PULP cluster.

6.1. Network design, data preparation and QAT algorithms

We evaluate the impact of network design, training, and data preparation parameters on statistical accuracy from multiple aspects. The hyperparameters for QAT with TQT are shown in Table 3, the training schedule for QAT with INQ is shown in Fig. 4.

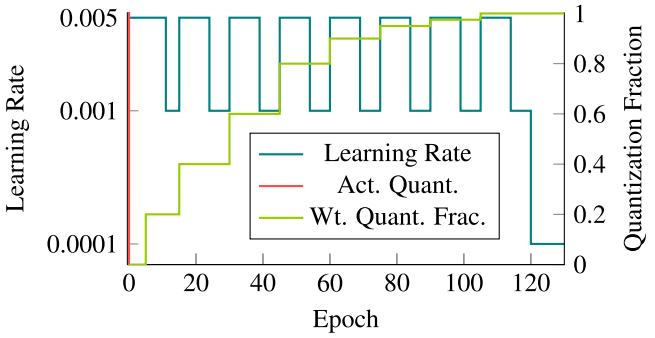


Fig. 4. Quantization and learning rate schedules used to ternarize full-precision networks with the INQ algorithm. The left y-axis measures the learning rate, the right y-axis measures the fraction of weights quantized. Hyperparameters not shown in this plot are identical to those used for QAT with TQT.

6.1.1. Receptive time interval t_p

The receptive time interval t_p is tied to the network's reaction time to changes in the input: For larger t_p, it will take longer for the new inputs to propagate through the network and saturate the TCN's input window. To minimize the system's reaction time, it is thus desirable to keep t_p as short as possible. However, a very short t_p limits the temporal context of the network's input, which may negatively impact classification accuracy. In this context, we consider a configuration *Pareto-optimal* if there is no network that achieves higher accuracy with a shorter t_p, and the set of Pareto-optimal networks forms a *Pareto front*. As described in Section 4.5, t_p is influenced both by network design (C_{in}, N_{TCN}) and data preparation parameters (s_{win}, FPS). Setting $N_{ch} = 96$, we trained 47 networks with parameter combinations resulting in t_p ranging from 42 ms to 1433 ms. We trained full-precision networks on the dataset split used by most previous works (users 1–23 in the training set and users 24–29 as the validation set). Of the full-precision networks forming the Pareto front, we also trained ternarized versions using the TQT algorithm and ReLU activations using 4-fold cross-validation (CV). The results are shown in Fig. 5 for both the standard dataset split and with 4-fold CV, selecting different users as the validation set. From Fig. 5, we can observe multiple points of interest. First, at $t_p = 300$ ms, a ternarized network already reaches 96.86% validation accuracy on the standard dataset split, only 0.74 percentage points below the maximum observed accuracy of 97.7%. The accuracy drop from ternarization is below 1.5 percentage points in all evaluated networks, with an average drop of 0.6 percentage points. Quantization even results in increased accuracy in some cases, confirming that the problem is well-suited to be solved by TNNs. Lastly, the average statistical accuracy when using 4-fold CV is lower by 1.55 percentage points than with the standard dataset split, indicating that the standard split is easier, with the training set representing the validation set accurately.

6.1.2. Network design, dataset generation and QAT algorithms

As a fully exhaustive network design, training, and dataset generation parameter search would be infeasible, we present ablation results over those parameters and design choices we found to have the largest impact on statistical accuracy. Fig. 6 shows the impact of varying FPS while maintaining $t_p = 900$ ms by changing C_{in}. We observed that increasing framerates significantly improves classification accuracy only up to 60 FPS, after which it stagnates between 97.3% and 97.9%. Next, we sought to determine the impact of the network's TCN stage on statistical accuracy by training two of the Pareto-optimal networks shown in Fig. 5 ($t_p = 300$ ms and $t_p = 900$ ms) without the TCN. Instead, the N_{TCN} outputs of the CNN are fed directly into a linear

Source: https://doi.org/10.1016/j.future.2023.07.017
Google Scholar research | 2026-10-08 | Article 06 | Original p. 9 | Orange: discussion; blue: reference

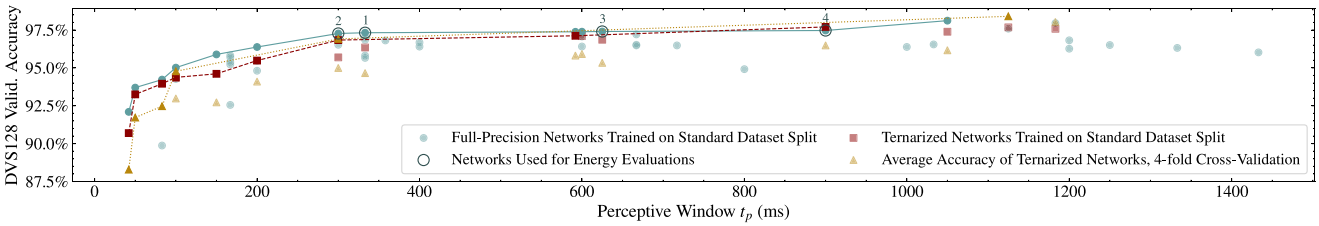


Fig. 5. Validation accuracy vs. t_p for various network architecture and dataset generation parametrizations. Data points connected by lines form the Pareto front of the respective experiments.

Table 4
Impact of QAT algorithm, quantization levels and the TCN stage on classification accuracy. Classification accuracy is specified for ternarized networks trained on the standard dataset split with full-precision accuracy in parentheses. n_{lvls} denotes the number of quantization levels, with 2 yielding a BNN and 3 a TNN. For the full parametrization of the networks, see Table 6.

Net (t_p)	n_{lvls}	Act.	QAT	TCN?	Acc. Quant. (FP)
Net 1 (300 ms)	3	ReLU	TQT	✗	95.3% (96.9%)
	3	ReLU	TQT	✓	**96.8**% (97.3%)
	3	ReLU	INQ	✓	93.9% (97.3%)
	3	HtanH	TQT	✓	95.5% (95.5%)
	3	HtanH	INQ	✓	93.8% (95.5%)
	2	ReLU	TQT	✓	95.8% (97.3%)
Net 4 (900 ms)	3	ReLU	TQT	✗	96.3% (96.9%)
	3	ReLU	TQT	✓	**97.7**% (97.5%)
	3	ReLU	INQ	✓	97.2% (97.5%)
	3	HtanH	TQT	✓	96.4% (96.1%)
	3	HtanH	INQ	✓	95.5% (96.1%)
	2	ReLU	TQT	✓	97.0% (97.5%)

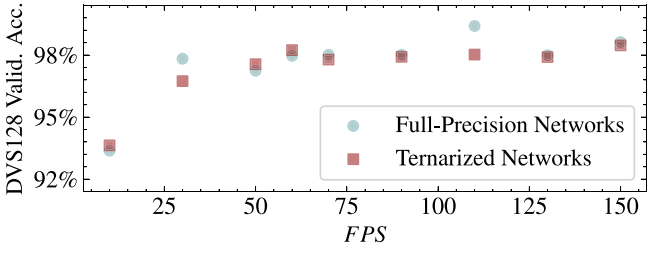


Fig. 6. Validation accuracy vs. *FPS*. Apart from *FPS* and C_{in}, the network's parametrization corresponds to network 4 from Table 6 with C_{in} chosen so that $t_p = 900$ ms, i.e., $C_{in} = FPS/10$. The highest accuracy is achieved at 150*FPS* with 97.9%.

classifier layer. Table 4 shows that the inclusion of the TCN stage improves classification accuracy by 1.5 and 1.4 percentage points respectively. Finally, we evaluated the impact of the choice of QAT algorithm and activation function on accuracy. As Table 4 shows, the combination of ReLU activations and TQT yields the highest classification accuracy. Furthermore, ReLU activations and TQT outperform HtanH and INQ individually. To determine the impact of the number of quantization levels on accuracy, we also trained BNNs with equivalent architectures using ReLU activations and the TQT algorithm. Since our DVS event frame representation is ternary, the first layer of each BNN processes ternary inputs but uses binary weights. As Table 4 shows, these networks still perform well but achieve lower statistical accuracies by 1.0 and 0.7 percentage points than their ternary counterparts.

6.2. End-to-end gesture recognition on the Kraken SoC

We evaluated the real-world energy consumption of our processing pipeline on the Kraken SoC with the four network parametrizations shown in Table 6. On both CUTIE and the PULP cluster, we run inference under realistic operating conditions by matching the inference rates to the evaluated networks' training and accounting for the power consumption of idle domains. Our

Table 5
Operating conditions for power measurements and power consumption of the different power domains on the Kraken SoC.

	FC	Cluster	CUTIE
f_{clk} (MHz)	40	115	15
V_{DD} (V)	0.55	0.55	0.5
P_{idle} (mW)	2.0	1.6	2.7
P_{inf} (mW)	2.0	21.1	5.3
t_{inf} (ms)	N/A	17.8	0.9

results show that the CUTIE-based implementation of our TCN network requires only 7 µJ, 58.5 × less than a comparable implementation on the state-of-the-art RISC-V cluster using efficient, specialized ISA extensions.

6.2.1. Experimental setup

To map the network to the PULP cluster, we use DORY [87], an open-source DNN mapping utility targeting PULP systems, to generate test applications. The network layers are mapped to 2-bit kernels from the PULP-NN library [88] which take advantage of the XpulpNN ISA extension. As the DVSI's operation does not result in a measurable increase in power consumption, our test applications store input samples in L2 memory. An on-chip timer generates interrupts to the FC at the rate appropriate for each tested network, upon which the FC triggers inference either on the cluster or on CUTIE. As CUTIE operates on data from its internal activation memory, the FC transfers input activations before triggering the computation, which is slower than direct transfer by the DVSI and results in pessimistic latency and energy estimations. Table 5 details the operating conditions for our experiments, the power consumption of each domain during each phase of inference, and the inference latency t_{inf}. While the tested networks differ in the values of both N_{TCN} and C_{in}, this has no measurable impact on inference latency. Due to CUTIE's fully unrolled architecture, the number of input channels does not influence latency. For cluster networks, the number of input channels must be padded to a multiple of 16 to comply with the

Source: https://doi.org/10.1016/j.future.2023.07.017
Google Scholar research | 2026-10-08 | Article 06 | Original p. 10 | Orange: discussion; blue: reference

[56] K. Wang, Z. Liu, Y. Lin, J. Lin, S. Han, HAQ: Hardware-aware automated quantization with mixed precision, in: 2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), Vol. 2019-June, IEEE, 2019, pp. 8604–8612, http://dx.doi.org/10.1109/CVPR.2019.00881, arXiv:1811.08886.

[57] Z. Yao, Z. Dong, Z. Zheng, A. Gholami, J. Yu, E. Tan, L. Wang, Q. Huang, Y. Wang, M.W. Mahoney, K. Keutzer, HAWQV3: Dyadic neural network quantization, in: M. Meila, T. Zhang (Eds.), Proceedings of the 38th International Conference on Machine Learning, in: Proceedings of Machine Learning Research, vol. 139, PMLR, 2021, pp. 11875–11886, arXiv:2011.10680.

[58] Y. Umuroglu, L. Rasnayake, M. Sjalander, BISMO: A scalable bit-serial matrix multiplication overlay for reconfigurable computing, in: 2018 28th International Conference on Field Programmable Logic and Applications, FPL, IEEE, 2018, pp. 307–3077, http://dx.doi.org/10.1109/FPL.2018.00059, arXiv:arXiv:1806.08862v1.

[59] H. Sharma, J. Park, N. Suda, L. Lai, B. Chau, V. Chandra, H. Esmaeilzadeh, Bit fusion: Bit-level dynamically composable architecture for accelerating deep neural network, in: 2018 ACM/IEEE 45th Annual International Symposium on Computer Architecture, ISCA, IEEE, 2018, pp. 764–775, http://dx.doi.org/10.1109/ISCA.2018.00069, arXiv:1712.01507.

[60] F. Conti, Technical report: NEMO DNN quantization for deployment model, 2020, pp. 1–12, CoRR abs/2004.0. arXiv:2004.05930.

[61] R. Banner, Y. Nahshan, D. Soudry, Post-training 4-bit quantization of convolutional networks for rapid-deployment, in: H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché Buc, E. Fox, R. Garnett (Eds.), Advances in Neural Information Processing Systems, Curran Associates, Inc., Vancouver, 2019, pp. 1–9, http://dx.doi.org/10.5555/3454287.3455001.

[62] Y. Choukroun, E. Kravchik, F. Yang, P. Kisilev, Low-bit quantization of neural networks for efficient inference, in: 2019 IEEE/CVF International Conference on Computer Vision Workshop, ICCVW, IEEE, 2019, pp. 3009–3018, http://dx.doi.org/10.1109/ICCVW.2019.00363, arXiv:1902.06822.

[63] Y. Cai, Z. Yao, Z. Dong, A. Gholami, M.W. Mahoney, K. Keutzer, Zeroq: A novel zero shot quantization framework, in: 2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR, IEEE, 2020, pp. 13166–13175, http://dx.doi.org/10.1109/CVPR42600.2020.01318, arXiv:2001.00281.

[64] M. Eldad, F. Alexander, A. Uri, G. Mark, Same, same but different: Recovering neural network quantization error through weight factorization, in: 36th International Conference on Machine Learning, ICML 2019, Vol. 2019-June, 2019, pp. 7886–7896, http://dx.doi.org/10.48550/arXiv.1902.01917, arXiv:1902.01917.

[65] M. Nagel, M.V. Baalen, T. Blankevoort, M. Welling, Data-free quantization through weight equalization and bias correction, in: Proceedings of the IEEE International Conference on Computer Vision 2019-Octob, 2019, pp. 1325–1334, http://dx.doi.org/10.1109/ICCV.2019.00141, arXiv:1906.04721.

[66] J. Choi, S. Venkataramani, V.V. Srinivasan, K. Gopalakrishnan, Z. Wang, P. Chuang, Accurate and efficient 2-bit quantized neural networks, in: A. Talwalkar, V. Smith, M. Zaharia (Eds.), Proceedings of Machine Learning and Systems, Vol. 1, 2019, pp. 348–359.

[67] S.R. Jain, A. Gural, M. Wu, C.H. Dick, Trained quantization thresholds for accurate and efficient fixed-point inference of deep neural networks, in: Proceedings of Machine Learning and Systems, 2020, pp. 112–128, arXiv:1903.08066.

[68] S.K. Esser, J.L. McKinstry, D. Bablani, R. Appuswamy, D.S. Modha, Learned step size quantization, in: International Conference on Learning Representations, 2020, pp. 1–12, arXiv:1902.08153.

[69] I. Hubara, M. Courbariaux, D. Soudry, R. El-Yaniv, Y. Bengio, Quantized neural networks: Training neural networks with low precision weights and activations, J. Mach. Learn. Res. 18 (1) (2017) 6869–6898, http://dx.doi.org/10.5555/3122009.3242044, arXiv:1609.07061.

[70] Y. Bengio, N. Léonard, A. Courville, Estimating or propagating gradients through stochastic neurons for conditional computation, 2013, pp. 1–12, CoRR abs/1308.3. arXiv:1308.3432.

[71] A. Zhou, A. Yao, Y. Guo, L. Xu, Y. Chen, Incremental network quantization: Towards lossless CNNs with low-precision weights, in: International Conference on Learning Representations, 2017, arXiv:1702.03044.

[72] M. Rastegari, V. Ordonez, J. Redmon, A. Farhadi, XNOR-net: ImageNet classification using binary convolutional neural networks, in: Lecture Notes in Computer Science (Including Subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics), in: LNCS, vol. 9908, 2016, pp. 525–542, http://dx.doi.org/10.1007/978-3-319-46493-0_32, arXiv:arXiv:1603.05279v4.

[73] M. Courbariaux, I. Hubara, D. Soudry, R. El-Yaniv, Y. Bengio, Binarized neural networks: Training deep neural networks with weights and activations constrained to 1 or -1, 2016, arXiv:1602.02830.

[74] L. Deng, P. Jiao, J. Pei, Z. Wu, G. Li, GXNOR-net: Training deep neural networks with ternary weights and activations without full-precision memory under a unified discretization framework, Neural Netw. 100 (2018) 49–58, http://dx.doi.org/10.1016/j.neunet.2018.01.010, arXiv:1705.09283.

[75] H. Alemdar, V. Leroy, A. Prost-Boucle, F. Petrot, Ternary neural networks for resource-efficient AI applications, in: 2017 International Joint Conference on Neural Networks, IJCNN, IEEE, 2017, pp. 2547–2554, http://dx.doi.org/10.1109/IJCNN.2017.7966166.

[76] A. Gholami, S. Kim, Z. Dong, Z. Yao, M.W. Mahoney, K. Keutzer, A survey of quantization methods for efficient neural network inference, 2021, arXiv preprint arXiv:2103.13630.

[77] L. Deng, G. Li, S. Han, L. Shi, Y. Xie, Model compression and hardware acceleration for neural networks: A comprehensive survey, Proc. IEEE 108 (4) (2020) 485–532, http://dx.doi.org/10.1109/JPROC.2020.2976475.

[78] B. Moons, D. Bankman, L. Yang, B. Murmann, M. Verhelst, BinarEye: An always-on energy-accuracy-scalable binary CNN processor with all memory on chip in 28 nm CMOS, in: 2018 IEEE Custom Integrated Circuits Conference, CICC, IEEE, 2018, pp. 1–4, http://dx.doi.org/10.1109/CICC.2018.8357071, arXiv:1804.05554.

[79] R. Andri, G. Karunaratne, L. Cavigelli, L. Benini, ChewBaccaNN: A flexible 223 TOPS/W BNN accelerator, in: 2021 IEEE International Symposium on Circuits and Systems, ISCAS, IEEE, 2021, pp. 1–5, http://dx.doi.org/10.1109/ISCAS51556.2021.9401214.

[80] M. Scherer, A.D. Mauro, T. Fischer, G. Rutishauser, L. Benini, TCN-CUTIE: A 1, 036-TOp/s/W, 2.72-μ j/inference, 12.2-mW all-digital ternary accelerator in 22-nm FDX technology, IEEE Micro 43 (1) (2023) 42–48, http://dx.doi.org/10.1109/MM.2022.3226630, arXiv:arXiv:2212.00688v1.

[81] M. Rusci, A. Capotondi, L. Benini, Memory-driven mixed low precision quantization for enabling deep network inference on microcontrollers, in: I. Dhillon, D. Papailiopoulos, V. Sze (Eds.), Proceedings of Machine Learning and Systems, Vol. 2, 2020, pp. 326–335, arXiv:1905.13082.

[82] A. Di Mauro, M. Scherer, D. Rossi, L. Benini, Kraken: A direct event/frame-based multi-sensor fusion SoC for ultra-efficient visual processing in nano-UAVs, in: 2022 IEEE Hot Chips 34 Symposium, HCS, IEEE, 2022, pp. 1–19, http://dx.doi.org/10.1109/HCS55958.2022.9895621.

[83] M. Gautschi, P.D. Schiavone, A. Traber, I. Loi, A. Pullini, D. Rossi, E. Flamand, F.K. Gurkaynak, L. Benini, Near-threshold RISC-V core with DSP extensions for scalable IoT endpoint devices, IEEE Trans. Very Large Scale Integr. (VLSI) Syst. 25 (10) (2017) 2700–2713, http://dx.doi.org/10.1109/TVLSI.2017.2654506.

[84] A. Pullini, D. Rossi, G. Haugou, L. Benini, μDMA: An autonomous I/O subsystem for IoT end-nodes, in: 2017 27th International Symposium on Power and Timing Modeling, Optimization and Simulation (PATMOS), Vol. 1, IEEE, 2017, pp. 1–8, http://dx.doi.org/10.1109/PATMOS.2017.8106971.

[85] A. Di Mauro, A.S. Prasad, Z. Huang, M. Spallanzani, F. Conti, L. Benini, SNE: an energy-proportional digital accelerator for sparse event-based convolutions, in: 2022 Design, Automation & Test in Europe Conference & Exhibition, DATE, IEEE, 2022, pp. 825–830, http://dx.doi.org/10.23919/DATE54114.2022.9774552.

[86] I. Loshchilov, F. Hutter, SGDR: Stochastic gradient descent with warm restarts, in: International Conference on Learning Representations, 2017, pp. 1–16, arXiv:1608.03983.

[87] A. Burrello, A. Garofalo, N. Bruschi, G. Tagliavini, D. Rossi, F. Conti, DORY: Automatic end-to-end deployment of real-world DNNs on low-cost IoT MCUs, IEEE Trans. Comput. 70 (8) (2021) 1253–1268, http://dx.doi.org/10.1109/TC.2021.3066883, arXiv:2008.07127.

[88] A. Garofalo, M. Rusci, F. Conti, D. Rossi, L. Benini, Pulp-NN: Accelerating quantized neural networks on parallel ultra-low-power RISC-V processors, Phil. Trans. R. Soc. A 378 (2164) (2020) http://dx.doi.org/10.1098/rsta.2019.0155, arXiv:1908.11263.



Georg Rutishauser received the B.Sc. and M.Sc. degrees in electrical engineering and information technology from ETH Zürich in 2015 and 2018, respectively, where he is currently pursuing the Ph.D. degree with the Integrated Systems Laboratory. His research interests include algorithms and hardware for reduced-precision deep learning, and their application in computer vision and embedded systems. Contact him at georgr@iis.ee.ethz.ch.



Moritz Scherer received the B.Sc. and M.Sc. degree in electrical engineering and information technology from ETH Zürich in 2018 and 2020, respectively, where he is currently pursuing a Ph.D. degree at the Integrated Systems Laboratory. His current research interests include the design of ultra-low power and energy-efficient circuits and accelerators as well as system-level and embedded design for machine learning and edge computing applications. Contact him at scheremo@iis.ee.ethz.ch.

Source: https://doi.org/10.1016/j.future.2023.07.017

Google Scholar research | 2026-10-08 | Article 06 | Original p. 14 | Orange: discussion; blue: reference

electronics

Article

Integer-Only CNNs with 4 Bit Weights and Bit-Shift Quantization Scales at Full-Precision Accuracy

Maarten Vandersteegen *, Kristof Van Beeck and Toon Goedemé

KU Leuven, EAVISE-Jan Pieter De Nayerlaan 5, 2860 Sint-Katelijne-Waver, Belgium; kristof.vanbeeck@kuleuven.be (K.V.B.); toon.goedeme@kuleuven.be (T.G.)
* Correspondence: maarten.vandersteegen@kuleuven.be; Tel.: +32-15-31-69-44

Abstract: Quantization of neural networks has been one of the most popular techniques to compress models for embedded (IoT) hardware platforms with highly constrained latency, storage, memory-bandwidth, and energy specifications. Limiting the number of bits per weight and activation has been the main focus in the literature. To avoid major degradation of accuracy, common quantization methods introduce additional scale factors to adapt the quantized values to the diverse data ranges, present in full-precision (floating-point) neural networks. These scales are usually kept in high precision, requiring the target compute engine to support a few high-precision multiplications, which is not desirable due to the larger hardware cost. Little effort has yet been invested in trying to avoid high-precision multipliers altogether, especially in combination with 4 bit weights. This work proposes a new quantization scheme, based on power-of-two quantization scales, that works on-par compared to uniform per-channel quantization with full-precision 32 bit quantization scales when using only 4 bit weights. This is done through the addition of a low-precision lookup-table that translates stored 4 bit weights into nonuniformly distributed 8 bit weights for internal computation. All our quantized ImageNet CNNs achieved or even exceeded the Top-1 accuracy of their full-precision counterparts, with ResNet18 exceeding its full-precision model by 0.35%. Our MobileNetV2 model achieved state-of-the-art performance with only a slight drop in accuracy of 0.51%.

Keywords: quantization; neural networks; nonuniform; power-of-two scales; low-cost hardware

check for updates

Citation: Vandersteegen, M.; Van Beeck, K.; Goedemé, T. Integer-Only CNNs with 4 Bit Weights and Bit-Shift Quantization Scales at Full-Precision Accuracy. *Electronics* **2021**, *10*, 2823. https://doi.org/10.3390/electronics10222823

Academic Editor:Luis Gomes

Received: 31 October 2021
Accepted: 15 November 2021
Published: 17 November 2021

Publisher's Note: MDPI stays neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Copyright: © 2021 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (https://creativecommons.org/licenses/by/4.0/).

1. Introduction

Quantization of neural networks dates back to the 1990s [1,2], where the discretization of models was a necessity to make their implementation feasible on the available hardware. More recently, neural networks became popular again because of the ImageNet challenge [3] and the availability of powerful GPU hardware. This breakthrough started a new area of research with hundreds of new potential applications. Today, neural networks are found in various electronic devices such as smartphones, wearables, robots, self-driving cars, smart sensors, and many others. The embedded electronic processors found in these applications are often very limited in capabilities to make them affordable, compact, and energy efficient in the case of battery-powered applications. Most neural networks are too large and require too many computations to be implemented directly on such processors, and therefore need to be compressed first. One of the most effective ways to reduce latency, storage cost, memory-bandwidth, energy efficiency, and silicon area among popular compression techniques such as model pruning [4] and network architecture search [5] is model quantization [6]. The quantization of neural networks is a frequently visited research topic with numerous publications that mostly focus on reducing the number of bits per weight or activation as much as possible in order to achieve high compression rates [7–11].

The commonly accepted method to achieve low-precision quantization introduces high-precision scale factors, and in some cases zero-points, to adapt the quantized values to the diverse weight and activation ranges present in full-precision neural networks. This

Source: https://lirias.kuleuven.be/retrieve/02b3b220-64bc-4e7a-833e-6e319c5dfa77
Google Scholar research | 2026-10-07 | Article 07 | Original p. 1 | Orange: discussion; blue: reference

is done to avoid major degradation in accuracy. The downside of this approach is that these high-precision quantization scales have to be computed at runtime as well in order to make the math work. Because these high-precision multipliers handle the minority of computations, their involvement is not considered a problem by most works. Their required presence in hardware, however, increases silicon area, energy consumption, and cost. In addition, neural network compute engines implemented on FPGAs may require more expensive FPGAs, due to the limited number or lack of high-precision multipliers on less expensive platforms. This is especially problematic when multiple compute cores are used to increase parallelism.

Jain et al. [12] already proved that 8 bit models with per-layer quantization and power-of-two scales can achieve full-precision accuracy or even better. Their experiments with 4 bit weights and 8 bit activations, however, resulted in a significant degradation in accuracy and even a complete failure for MobileNets. Using 4 bit weights, 8 bit activations, and per-layer quantization with power-of-two scales, we prove that we can achieve full-precision accuracy or even better for all models and near full-precision accuracy for MobileNetV2. Moreover, we prove that our method performs on-par or even better compared to uniformly per-channel quantized models that use full-precision quantization scales. Our proposed compute engine is depicted in Figure 1b, next to a typical compute engine [6,13,14] with high-precision scaling capability in Figure 1a. We propose to use a lookup-table to translate 4 bit weight addresses into 8 bit weights for internal computation. Since the LUT can hold any number representable through 8 bit, different value combinations can be chosen to match the underlying weight distribution within a specific convolution layer, compensating the limited capabilities of the bit-shift scaling. A different set of LUT values is used for every layer to best match the layer-specific data distribution. Note that a single LUT can be shared among multiple parallel compute engines, allowing additional hardware simplification.

Reducing the number of bits per weight from eight to four is a wanted feature in many cases because it reduces the storage cost by 50% and decreases the load-time of the weights from external memory significantly, which results in faster computation. With our method, 32 bit (integer) multipliers can be avoided, transforming the neural network compute engine to simple, yet very effective hardware.

Our contributions can be summarized as follows:

- We present an extensive literature overview of uniform and nonuniform quantization for fixed-point inference;
- A novel modification to a neural network compute engine is introduced to improve the accuracy of models with 4 bit weights and 8 bit activations, in conjunction with bit-shift-based scaling, through the aid of a lookup-table;
- A quantization-aware training method is proposed to optimize the models that need to run on our proposed compute engine;
- We are the first to make a fair empirical comparison between the performance of (uniform) quantized models with full-precision and power-of-two scales with either per-layer or per-channel quantization using 4 bit weights;
- Our source code has been made publicly available https://gitlab.com/EAVISE/lut-model-quantization (accessed on 16 November 2021).

The remainder of this paper is organized as follows: Section 2 presents an extensive literature overview of quantization in greater detail, organized into different topics for convenience. For each topic, we also highlight the choices we made for our own approach. Our proposed method is explained in Section 3; our results are presented in Section 4; conclusions are made in final Section 5.

Source: https://lirias.kuleuven.be/retrieve/02b3b220-64bc-4e7a-833e-6e319c5dfa77

Google Scholar research | 2026-10-07 | Article 07 | Original p. 2 | Orange: discussion; blue: reference

zation scales. Lookup-table approaches seemed the most interesting, given that they can represent any value, compared to the limited number of steps representable through (sums of) power-of-two values.

2.3.4. Mixed Precision

Most works consider *homogeneous* quantization, which means that all weight or all activations across a model share the same bit-widths. Several recent works [28–31] explored the quantization of heterogeneous models, where each weight tensor in a layer can have its own bit-width, also commonly referred to as mixed-precision models. Assigning low bit-widths to layers that are less sensitive to quantization can increase compression and bandwidth efficiency even further. Moreover, a fine-grained trade-off can be selected between model accuracy and model compression by choosing the right configuration. The major challenge for mixed-precision quantization algorithms is to determine the bit-width configuration for an entire model. Several works [28–30] designed a sensitivity metric that represents the sensitivity to quantization for a given layer. A bit-width, proportional to this metric, is then assigned to that layer. Cai et al. [31] solved the selection of the right mixed-precision model through differential NAS.

Although mixed precision has many great advantages, it requires specialized hardware to support it. In this work, we did not consider using mixed precision; however, our method is suited to be used in combination with mixed precision, which can be explored in future work.

2.4. Quantizer Parameters

This section describes the different data types that are used in practice to represent quantizer parameters (Section 2.4.1) and how these parameter are estimated in the PTQ and QAT algorithms (Section 2.4.2).

2.4.1. Quantizer Parameter Data Types

The (simulated) data type of quantizer parameters such as scale factors and zero-points matters, since they influence the requirements of the target hardware. If, for example, s would be a full-precision floating-point number during simulation, the accelerator would also need to support scaling operations in full-precision. Since floating-point operations in integer-based accelerators are usually not preferred, other solutions have been proposed in the literature for scale factors: dynamic fixed-point integers [13], which are 32 bit or 16 bit fixed-point integers plus a shift operation to enlarge their dynamic range, and powers-of-two values [12]. Zero-points are less critical and in practice represented through integers with the same bit-width as their integer data $\mathbf{q}$. Dynamic fixed-point integers are mostly used for scale factors since they closely resemble the dynamics and precision of floating-point numbers, which allows simulators to just represent them in floating-points. Power-of-two (POT) scale factors, according to Jain et al. [13], can be simulated with:

$$s_{POT} = 2^{\text{ceil}(\log_2(s))} \tag{23}$$

s is a full-precision scale factor; s_{POT} is the power-of-two scale factor; ceil is a ceiling operator that rounds upwards. In this work, we preferred to use these power-of-two scale factors because they can be executed with a single bit-shift operation.

2.4.2. Quantizer Parameter Estimation

We found different techniques to estimate the quantizer parameters in both the PTQ and QAT frameworks. More traditional works estimate these parameters using simple data statistics, such as observed minimum and maximum data values [13,14,19], and use them as lower and upper bounds of the quantization grid, respectively. This approach is however sensitive to outliers and tends to overestimate the range, resulting in large quantization errors, especially for bit-widths smaller than eight. To overcome this issue,

Source: https://lirias.kuleuven.be/retrieve/02b3b220-64bc-4e7a-833e-6e319c5dfa77
Google Scholar research | 2026-10-07 | Article 07 | Original p. 11 | Orange: discussion; blue: reference

many PTQ algorithms minimize some distance function D between the full-precision data and the quantized data:

$$\arg\min_{s,z} \quad D\Big(\mathbf{x}, Q_f(\mathbf{x}, s, z)\Big) \tag{24}$$

Different metrics for D include the Kullback–Leibler divergence between both distributions [12,32] and the mean-squared error [18,33]. Modern QAT methods make quantizer parameters trainable and optimize them together with the weights [6,9,12,21,24,34,35]. Using the chain rule and the STE, the gradient w.r.t. the scale factor can be calculated. For the uniform symmetric quantizer, this will be:

$$\frac{\partial}{\partial s}\left[s\,\text{clamp}\Big(\text{round}\Big(\frac{\mathbf{x}}{s}\Big), n, p\Big)\right] = \begin{cases} \text{round}\big(\frac{\mathbf{x}}{s}\big) - \frac{\mathbf{x}}{s} & \text{if} \quad n \leq \text{round}\big(\frac{\mathbf{x}}{s}\big) \leq p \\ n & \text{if} \quad \text{round}\big(\frac{\mathbf{x}}{s}\big) < n \\ p & \text{if} \quad \text{round}\big(\frac{\mathbf{x}}{s}\big) > p \end{cases} \tag{25}$$

$n = -2^{b-1}$ and $p = 2^{b-1} - 1$ for signed and $n = 0$ and $p = 2^b - 1$ for unsigned data.

Some works define $s = 1$ and $z = 0$ [11,36,37], which completely eliminates scaling on the target device. The consequence, however, is that all layers have to learn quantized values that lie more or less within the same range, which works since these authors trained their models from scratch. Training from scratch with quantizers is, however, less preferable since it takes much longer to train and sometimes results in less accurate models.

As mentioned in Section 2.4.1, we preferred power-of-two scales. We optimized the scales through backpropagation as proposed by Jain et al. [12]. In the forward pass, the power-of-two scale is calculated with Equation (23). In the backward pass, the STE is used for the ceil function so $\frac{\partial \,\text{ceil}(x)}{\partial x} = 1$, and $s_{log} = \log_2 s$ is treated as a training parameter rather than s itself. The latter enables training in the $\log_2$ domain, which increases the numerical stability and provides better convergence. The gradient w.r.t. s_{log} of Equation (23) is:

$$\frac{\partial}{\partial s_{log}}\Big[2^{\text{ceil}(s_{log})}\Big] = 2^{\text{ceil}(s_{log})} \ln 2 \tag{26}$$

2.5. Quantizing Other Layers

Sections 2.1 and 2.2 discuss the internals of a fixed-point compute engine and how to simulate it, respectively. In this section, we discuss how other common layers, such as elementwise adders or concatenation layers, need to be handled in hardware in order to correctly deal with scale factors and zero-points:

Max pooling: This layer does not need special handling because it only has a single input branch and does not alter these input values numerically;

Average pooling: The average of multiple integers is not necessarily an integer, which requires inserting a *fake* quantization node at the output. In practice, average pooling is often executed through a depthwise convolution layer without bias and with weights $\mathbf{w} = 1/K^2\mathbf{J}$ where K is the kernel size of the average pool layer and $\mathbf{J}$ is an all-ones tensor. This way, the convolution compute engine can be reused for average pool operations, and quantization simulation happens in the same way as for the convolution engine;

Elementwise add: If a scale or zero-point of both input branches of the adder do not match exactly, the quantization grids will be different and the addition will be incorrect. Krishnamoorthi et al. [14] placed a *fake* quantizer node at both input branches and one at the output. Their hardware implementation used a scale operation at both input branches and a round operator at the output. Jain et al. [12] had a slightly different approach. They forced the *fake* quantizer nodes at both input branches to share the same quantizer parameters, as depicted in Figure 5a by the dashed line. This way, the hardware implementation, shown in Figure 5b, only needs a requantization step at the output. In case a ReLU activation function directly follows the elementwise adder, which is for instance the case with ResNet, this function can be fused into the elementwise add operation through the clipping function from the requantization step;

Source: https://lirias.kuleuven.be/retrieve/02b3b220-64bc-4e7a-833e-6e319c5dfa77
Google Scholar research | 2026-10-07 | Article 07 | Original p. 12 | Orange: discussion; blue: reference

Concatenation: The input branches of a concatenation function likely do not share the same scale or zero-point. Just as elementwise addition, the quantization parameters of the input branches need to be shared. This can be done in simulation by inserting *fake* quantization nodes on every input branch with shared parameters or by using a single *fake* quantization node at the output, which has the same effect (Figure 5c). The hardware implementation is shown in Figure 5d.

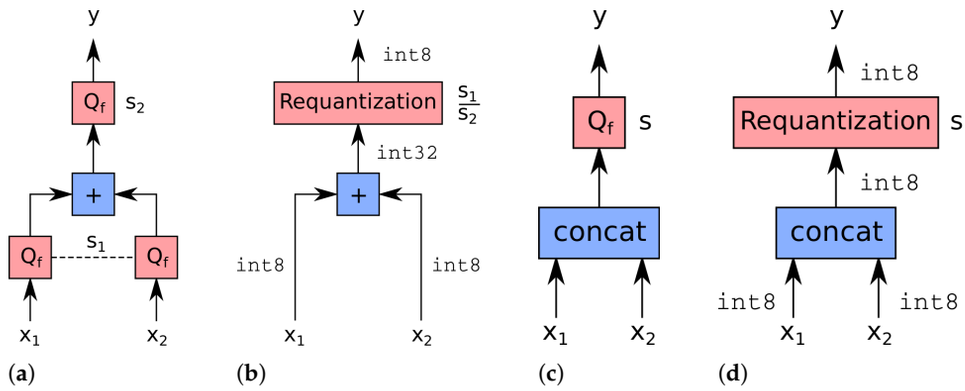


Figure 5. *Fake* quantized elementwise add (**a**), its implementation in hardware (**b**), *fake* quantized concat (**c**), and its implementation in hardware (**d**). The figures assume a uniform symmetric quantization scheme (only scale factors) for the sake of simplicity.

2.6. QAT Initialization

The superior performance of QAT over PTQ is significantly influenced by proper initialization of the QAT process. It is known that initializing QAT with pretrained floating-point weights gives better results compared to training a quantized model from scratch [12–14]. The proper initialization of the quantizer parameters is however equally important, which is why QAT algorithms first perform PTQ to give the quantizer parameters a meaningful starting point. Nagel et al. [6] experimented with different initialization techniques for uniform QAT. They concluded that the mean-squared-error-based estimation of the quantization parameters is favorable compared to min-max-based estimation, though the advantage becomes small towards the end of the training. Other techniques to nudge the weights or biases prior to training such as AdaRound [38] or bias-correction [22] seem not to be beneficial, with the exception of Cross-Layer Equalization (CLE) [22] which, according to their experiments, is required in order to successfully train MobileNets with 4 bit weights.

In addition to PTQ initialization, QAT-trained models with higher bit-widths are also used to initialize QAT trainings with lower bit-widths, in which case the bit-width is progressively reduced in multiple training stages. Commonly known as *progressive* quantization, this strategy has proven to work well and gives a small boost to the final accuracy. Progressive quantization can also mean that parts of the network are quantized progressively rather that the whole network at once. Zhuang et al. [7] proposed to first quantize the weights and leave the activations in full-precision, followed by quantizing both the weights and activations in a second stage. Park et al. [9] exploited this idea even further and proposed PROFIT, a technique to identify layers that cause more instability during QAT than others in MobileNets. By freezing the most unstable layers (mainly depthwise convolutions) in a consecutive training stage, that stage would be trained more smoothly, resulting in better performance. Progressive freezing, by Park et al. [9], was performed in three stages: (1) all layers were trained; (2) 2/3 of the layers were trained; finally, (3) 1/3 of the layers were trained. Each training stage was initialized with the weights from the previous training stage.

We applied CLE for all our MobileNetV2 experiments and used PROFIT in some experiments to further close the gap between quantized and full-precision models.

Source: https://lirias.kuleuven.be/retrieve/02b3b220-64bc-4e7a-833e-6e319c5dfa77
Google Scholar research | 2026-10-07 | Article 07 | Original p. 13 | Orange: discussion; blue: reference

3. Materials and Methods

As discussed in our related work, most methods rely on high-precision scaling to cope with the different dynamic data ranges, present in full-precision neural networks. High-precision scaling requires additional high-precision multipliers, which require larger silicon area and more energy and, in case FPGAs are used, leads to more expensive platforms. We propose a novel method that avoids high-precision scales in fixed-point compute engines by using power-of-two scales, which can be applied through a single bit-shift. Jain et al. [12] already proved that quantized models with 8 bit weights and power-of-two quantization scales can achieve full-precision accuracy or even better. Although much harder, we prove that we can achieve full-precision accuracy with 4 bit weights and power-of-two scales. Our method was inspired by the superior performance of nonuniform quantizers over uniform quantizers, but does not require complex hardware.

The difference between a typical compute engine and our proposed engine is depicted in Figure 1a,b, respectively. Our compute engine relies on bit-shift-based scaling with a single scale per layer, and we propose to use a lookup-table to translate 4 bit weights into 8 bit weights for internal computation. The lookup-table allows the compute engine to have nonuniform properties, boosting its expressiveness significantly. Since the LUT can hold any number representable through eight bits, different value combinations can be chosen to match the underlying weight distribution within a specific convolution layer. A different set of LUT values is used for every layer to best match the layer-specific data distribution. Note that a single LUT can be shared among multiple parallel compute engines, allowing an additional hardware simplification.

Section 3.1 presents our quantization scheme, the design changes to the compute engine, and the *fake* quantizer that we propose to simulate our design. Section 3.2 discusses the proposed optimization algorithm to estimate the 8 bit lookup-table values for each layer, and Section 3.3 describes the initialization method of both lookup-table values and power-of-two scales, used prior to training.

3.1. Lookup-Table Quantizer

Our quantization scheme can be expressed by Equation (27), where a floating-point vector $\mathbf{x}$ is approximated by a power-of-two scale factor s, multiplied by the output of LUT function L. L simply returns the integer values from LUT content $\mathbf{v} \in [-2^{b-1} ... 2^{b-1} - 1]^K$, which correspond to quantized values $\mathbf{q}$ (the lookup addresses). Here, K is the number of values in the LUT and b is the number of bits to represent them.

$$\mathbf{x} \approx sL(\mathbf{q}, \mathbf{v}) \qquad s.t. \quad s = \frac{2^l}{2^{b-1}} \tag{27}$$

If, for example, $\mathbf{q}$ contains 4 bit addresses, the LUT will contain $K = 16$ values, each with b bits (typical $b = 8$). Because $\mathbf{v}$ is a set of discrete b bit values rather than floating-point values, we used the power-of-two scale factor s to increase the dynamic range of the scheme, where $l \in \mathbb{Z}$ is the amount of left-shifting. Figure 6 illustrates this scheme graphically.

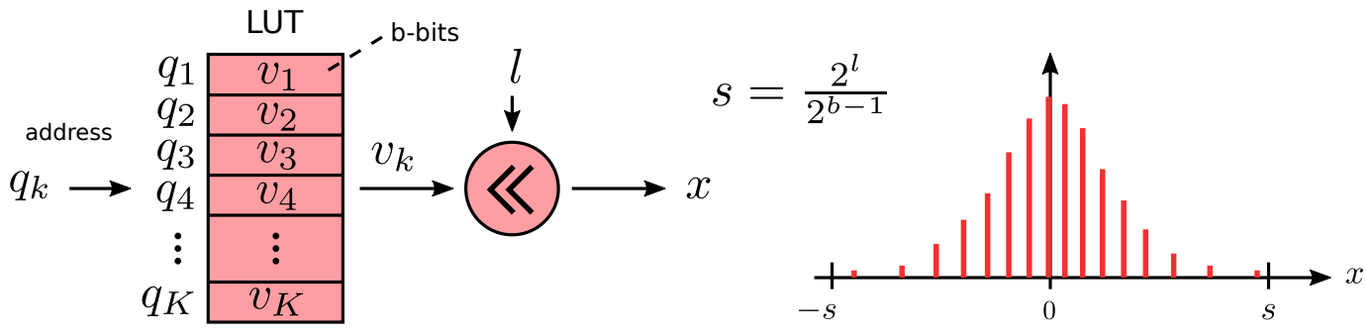


Figure 6. Our quantization scheme, which calculates a value x by translating a lookup-address q_k into a b bit value v_k and bit-shifts the results by l places.

Source: https://lirias.kuleuven.be/retrieve/02b3b220-64bc-4e7a-833e-6e319c5dfa77
Google Scholar research | 2026-10-07 | Article 07 | Original p. 14 | Orange: discussion; blue: reference

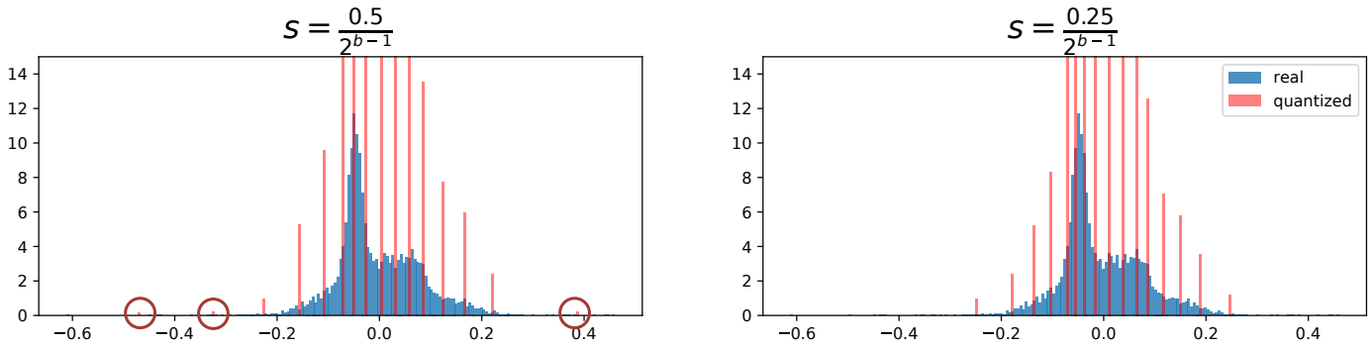


Figure 7. Real and quantized (16 bins) distributions of a weight tensor from MobileNetV2, generated using Algorithm 1. On the left, a few quantization bins are assigned to outliers due to the larger scale factor s. On the right, a smaller s results in a more compact quantized distribution because of the clamp operator.

3.2.1. Stopping Optimization of the LUT Values

Each weight layer has its own LUT vector $\mathbf{v}$ within a model. During training, when a LUT vector $\mathbf{v}$ is stabilized, it is rounded and its optimization procedure is stopped, which we call *freezing* (setting *enable_optimization* to false). We considered a vector stabilized when the following criteria are met:

$$\text{round}(\mathbf{v}) = \text{round}(\mathbf{v}_{smooth}) \tag{33}$$

where $\mathbf{v}_{smooth}$ is a smoothed version of $\mathbf{v}$ calculated by an EMA filter with a high decay (0.999). We start checking all LUT vectors after the first 1000 training iterations every 50 cycles. We only froze one LUT vector at a time. In case the criteria are met for multiple $\mathbf{v}$, the vector with the smallest round error $\|\mathbf{v} - \text{round}(\mathbf{v})\|_2$ is preferred.

3.3. Initialization

Both scale factor s and LUT values $\mathbf{v}$ are initialized prior to training using our PTQ as the initialization method. Our initialization method, summarized in Algorithm 3, starts with uniformly distributed LUT values $\mathbf{v}$ and a power-of-two scale factor determined from the maximum value of $\mathbf{x}$. It then optimizes $\mathbf{v}$ through Algorithm 2 for that specific scale factor followed by calculating the mean-squared error ϵ between quantized data $\mathbf{q}$ and $\mathbf{x}$. This is repeated for five smaller power-of-two values of s. Optimal scale factor $\hat{s}$ and LUT vector $\hat{\mathbf{v}}$ that produced the lowest ϵ are selected as the final initialization values for training. Note that our PTQ does not yet round the values in $\mathbf{v}$.

Algorithm 3 is applied for all weight tensors. Batch normalization layers are folded prior to quantization as discussed in Section 2.2.2. We did not quantize bias parameters in our experiments since we assumed 32 bit integer bias parameters and accumulators.

3.4. Quantizing Feature-Map Data

For quantizing feature-map data or activations, 8 bit quantization with power-of-two quantization scales was used. This was implemented through the symmetric uniform quantizer from Jain et al. [12] with learnable power-of-two scales. Their hyperparameter settings and scale freezing technique for optimizing the scales were adopted as well. Note that we deliberately did not choose to use asymmetric quantizers for feature-map data. Although this is possible, it introduces additional zero-points that need optimization, which complicates the training process. In addition, we empirically discovered that the added value is virtually nothing at 8 bit compared to a mix of signed and unsigned symmetric quantizers at the right locations. Throughout the model, quantizers are inserted at the inputs, activation functions, and other nonparametric layers such as elementwise adders, concatenations and average pooling layers. This is done to achieve correct integer-only inference simulation, as discussed in Section 2.5.

Source: https://lirias.kuleuven.be/retrieve/02b3b220-64bc-4e7a-833e-6e319c5dfa77
Google Scholar research | 2026-10-07 | Article 07 | Original p. 17 | Orange: discussion; blue: reference

Algorithm 3: Initialization of LUT values $\mathbf{v}$ and scale factor s.

Input: Floating-point data $\mathbf{x} = [x_1, x_2, \dots x_N]$
Output: Optimal scale factor $\hat{s}$ and LUT values $\hat{\mathbf{v}} = [v_1, v_2, \dots v_K]$
Procedure:
$\hat{\epsilon} \leftarrow \infty$
$\hat{s} \leftarrow s \leftarrow \frac{2^l}{2^{b-1}} \quad s.t. \quad l = \text{ceil}(\log_2 \max |\mathbf{x}|)$
for $i = 1$ *to* 5 **do**
  $\mathbf{v} = [v_1, v_2, \dots v_K]$ with uniformly distributed values in range $[-2^{b-1}, 2^{b-1} - 1]$
  for $j = 1$ *to* 100 **do**
    $\mathbf{q}, \mathbf{v} \leftarrow$ Algorithm 2$(\mathbf{x}, s, \mathbf{v})$ `// Optimization enabled`
  end
  $\epsilon \leftarrow \|\mathbf{x} - \mathbf{q}\|_2^2$
  if $\epsilon < \hat{\epsilon}$ **then**
    $\hat{\epsilon} \leftarrow \epsilon \quad \hat{s} \leftarrow s \quad \hat{\mathbf{v}} \leftarrow \mathbf{v}$
  end
  $s \leftarrow s/2$
end

4. Results

To evaluate the performance of our LUT-based method, we conducted several experiments on CNN-based image classifier models and an object detection model. Section 4.1 compares the PTQ and QAT performance of our proposed method against other uniform quantizers with both full-precision and power-of-two quantization scales in both per-layer and per-channel mode. Section 4.2 compares our results against results from other state-of-the-art works for five different image classifier models. Section 4.3 presents the results of our method, tested on an object detector model. Section 4.4 presents a small study on the added value of optimizing the LUT values during QAT. Section 4.5 discusses additional tools to further improve the accuracy of MobileNetV2. Finally, in Section 4.6, we present the simulated inference speed of a few CNN models with both 8 and 4 bit weights on an FPGA platform.

4.1. Comparison to Different Quantizers

In this first set of experiments, we compared our proposed nonuniform weight quantization method against uniform quantization methods with the POT scale and float scale in both the per-layer and per-channel configurations. Table 1 lists the Top-1 ImageNet accuracies of quantized ResNet18 [39] and MobileNetV2 [40] with 4 bit weights and 8 bit activations. We included both QAT's and PTQ's results, where the PTQ models were used to initialize the QAT models. Note that all these results were produced using the same framework for a fair comparison.

Table 1 shows that our method achieved the highest scores for both models, even slightly outperforming per-channel quantized models with full-precision scales. The advantage of the lookup-table clearly made the difference: our method can be used as a worthy alternative for uniform quantizers with both per-layer and per-channel quantization with floating-point scales. We also noticed that uniform quantization with POT scales in both per-layer and per-channel quantization did not completely recover the baseline accuracy of ResNet18, while our method did with a safe margin of +0.35%, which proves the superiority of our method for POT scales.

It is known that the accuracy of QAT models can exceed the accuracy of their full-precision counterparts [6,12,14], which can also be seen with our method. This phenomenon can be explained by the additional training cycles of QAT and the regularization effect of the quantization noise during QAT.

Source: https://lirias.kuleuven.be/retrieve/02b3b220-64bc-4e7a-833e-6e319c5dfa77
Google Scholar research | 2026-10-07 | Article 07 | Original p. 18 | Orange: discussion; blue: reference

Table 1. ImageNet Top-1 results using different quantizers with 4 bit weights and 8 bit activations. The first number represents the relative accuracy or difference between the quantized model and the full-precision model, while the second number in parenthesis represents the absolute accuracy.

Model (FP32 Accuracy)	ResNet18 (69.76)		MobileNetV2 (71.88)	
	PTQ	QAT	PTQ	QAT
Float scale per-layer	−50.18 (19.58)	+0.08 (69.84)	−67.11 (4.77)	−1.51 (70.37)
Float scale per-channel	−15.85 (53.91)	+0.23 (69.99)	−64.96 (6.92)	−0.97 (70.91)
POT scale per-layer	−60.14 (9.62)	−0.27 (69.49)	−71.05 (0.83)	−1.73 (70.15)
POT scale per-channel	−31.75 (38.01)	−0.08 (69.68)	−69.87 (6.34)	−1.66 (70.22)
POT scale per-layer + LUT (ours)	−12.51 (57.25)	**+0.35** (**70.11**)	−55.95 (15.93)	−**0.92** (**70.96**)

Although per-channel quantization with POT-scales was never attempted by Jain et al. [12], their per-layer results were consistent with ours. Per-channel quantization with POT scales has, to the best of our knowledge, never been tried in the literature before. Our procedure for uniform per-channel quantization with POT scales adopted the gradual freezing method of Jain et al. [12] to freeze the POT scales once stabilized, but applied it on a per-channel level. Instead of allowing a single POT scale to be frozen every 50 training iterations, we allowed up to 400 per-channel scales to be frozen, using the same stabilization criteria from Jain et al. [12].

For MobileNetV2, however, no quantizer succeeded in recovering the full-precision accuracy, because it is very sensitive to quantization due to the depthwise separable convolutions. Although CLE [22] was applied prior to quantization to improve MobileNetV2's QAT accuracy in all our experiments, as suggested by Nagel et al. [6], a drop in accuracy of approximately one percent still existed. In Section 4.5, we further analyze the effect of CLE on POT-scale-based quantizers in more detail, and we discuss what can be done in addition to further improve the results of MobileNetV2.

Both models were trained for 20 epochs on ImageNet, with standard preprocessing, data augmentation, and batch sizes of 256 and 128 for ResNet18 and MobileNetV2, respectively. We used the Adam optimizer to optimize both weights and quantization scales with its default parameters. For the weights, learning rates of 1×10^{-5} and 3×10^{-5} were used for ResNet18 and MobileNetV2, respectively, with a cosine learning rate decay that reduced the learning rate to zero at the end of the training. The quantization scales had a learning rate of 1×10^{-2} that quickly decayed in a step-based fashion, as suggested by Jain et al. [12] for their POT scale quantizer. We discovered that this learning rate schedule worked better for both POT-scale and float-scale models compared to the same learning rate schedule as used for the weights. Note that other best practices to train our POT-scale models, including gradual freezing of the scales once stabilized, were also adopted from Jain et al. [12], for our experiments that optimize POT scales.

The PTQ methods for the uniform models used mean-squared error estimation to initialize both weight and activation scales, as suggested by Nagel et al. [6]. Note that all experiments used symmetric quantizers for the activations.

4.2. Comparison to the State-of-the-Art

In this section, we compare our method against results from other state-of-the-art methods for a number of different models. The selection criteria used to select works to compare to were twofold: first, mainly QAT methods and, second, works that reported results with 4 bit weights and 8 bit activations. Table 2 presents the Top-1 ImageNet accuracies of ResNet18 [39], ResNet50 [39], MobileNetV2 [40], VGG16 [41] without batch normalization, and InceptionV3 [42]. The accuracies of the full-precision models among the related works slightly differed and are therefore listed separately (32/32) in addition to the accuracies of the quantized models with 4 bit weights and 8 bit activations (4/8). In addition to the results of our method (*Ours LUT*), we also present our own results of uniform quantization with POT scales (*Ours uniform*), to highlight the added value of our LUT.

Source: https://lirias.kuleuven.be/retrieve/02b3b220-64bc-4e7a-833e-6e319c5dfa77
Google Scholar research | 2026-10-07 | Article 07 | Original p. 19 | Orange: discussion; blue: reference

Results from the other works were directly taken from their papers. AB [18], QAT1 [14], and QAT2 [6] are all QAT methods that use per-channel quantization with full-precision quantization scales, where AB and QAT2 also experimented with per-layer quantization. TQT [12] is a QAT method that uses POT scales with per-layer quantization, and PWLQ [43] is a PTQ method that uses multiple uniform quantizers to obtain nonuniform quantization.

We can conclude that all models using our method achieved and even exceeded full-precision model accuracy with a safe margin, with the exception of MobileNetV2. Although the latter model was not fully recovered, we still achieved the highest accuracy compared to other methods. Nagel et al. [6] achieved a slightly higher accuracy on ResNet50 and InceptionV3 using per-channel quantization with float scales, but comparable to ours.

For ResNet50, VGG16, and InceptionV3, we used a learning rate 3×10^{-6} and their standard respective preprocessing and data augmentation. All other hyperparameters were as described in the previous Section 4.1. InceptionV3 was fine-tuned without auxiliary loss, as suggested by Jaint et al. [12].

In Table 3, we also present the ImageNet results of uniform quantized models with power-of-two scales and 8 bit weights instead of 4 bit weights, to further put our 4/8 results into perspective. Overall, these results were slightly higher compared to our results from Table 2, which was however to be expected. Similar training conditions were used as with our 4/8 experiments.

Table 2. Top-1 ImageNet accuracies of different works. The first value represents the relative accuracy or difference between the quantized model and the full-precision model, while the second number in parenthesis represents the absolute accuracy. Columns Per-Ch and POT scale indicate whether per-channel quantization or POT scales were used, respectively.

Method	w/a	Per-Ch	POT Scale	ResNet18	ResNet50	MobileNetV2	VGG16	InceptionV3
AB [18]	32/32			/	75.20	/	/	/
	4/8	no	no	/	−1.40 (73.80)	/	/	/
	4/8	yes	no	/	−0.90 (74.30)	/	/	/
PWLQ [43]	32/32			/	76.13	71.88	/	77.49
	4/8	yes	no	/	−0.51 (75.62)	−2.68 (69.22)	/	−1.04 (76.45)
TQT [12]	32/32			/	75.20	/	70.90	78.00
	4/8	no	yes	/	−0.80 (74.40)	/	**+0.60** (71.50)	−1.60 (76.4)
QAT1 [14]	32/32			/	75.20	71.90	/	78.00
	4/8	yes	no	/	−2.00 (73.2)	−9.90 (62.00)	/	−2.00 (76.00)
QAT2 [6]	32/32			69.76	76.07	71.72	/	77.40
	4/8	no	no	+0.08 (69.76)	−0.18 (75.89)	−1.55 (70.17)	/	+0.44 (77.84)
	4/8	yes	no	+0.33 (70.01)	**+0.45** (**76.52**)	−1.24 (70.48)	/	**+0.72** (**78.12**)
Ours uniform	32/32			69.76	76.13	71.88	71.59	77.34
	4/8	no	yes	−0.27 (69.49)	−0.11 (76.02)	−1.73 (70.15)	+0.09 (71.68)	+0.23 (77.57)
Ours LUT	4/8	no	yes	**+0.35** (**70.11**)	+0.18 (76.31)	−**0.92** (**70.96**)	+0.28 (**71.87**)	+0.39 (77.73)

Table 3. Top-1 ImageNet accuracies of uniform quantized models with 8 bit weights and 8 bit activations using POT scales.

w/a	ResNet18	ResNet50	MobileNetV2	VGG16	InceptionV3
32/32	69.76	76.13	71.88	71.59	77.34
8/8	+0.74 (70.50)	+0.54 (76.67)	+0.00 (71.88)	+0.37 (71.96)	+0.99 (78.33)

4.3. Object Detection

We also tested our method on the Tiny YOLOV2 [44] object detector. Table 4 presents the mean-average-precision results of Tiny YOLOV2 on the MS COCO dataset with 4 bit weights and 8 bit activations. The model was quantized with per-layer POT scales, both with and without our LUT method. We can conclude that also for this object detector, our method improved the final QAT result compared to uniform quantization. We trained the

Source: https://lirias.kuleuven.be/retrieve/02b3b220-64bc-4e7a-833e-6e319c5dfa77
Google Scholar research | 2026-10-07 | Article 07 | Original p. 20 | Orange: discussion; blue: reference

In future work, our method can be used easily in mixed-precision computation, where each weight tensor in a layer can have its own bit-width, to achieve a more fine-grained trade-off between model compression and model accuracy. Models with a mix of 2 bit, 4 bit, and 8 bit weights could be constructed and executed on our proposed compute engine without any modification. A 2 bit layer could be configured by simply using only four entries of the LUT, while an 8 bit layer could be configured through a simple bypass of the LUT.

Author Contributions: Methodology, M.V.; software, M.V.; supervision, T.G.; validation, K.V.B.; writing—original draft, M.V.; writing—review and editing, M.V., K.V.B., and T.G. All authors have read and agreed to the published version of the manuscript.

Funding: This work was funded by VLAIO and EASICS https://www.easics.com (accessed on 16 November 2021) through the Start to Deep Learn TETRA project and FWO via the OmniDrone SBO project.

Data Availability Statement: The following publicly available datasets were used in this research: the ImageNet dataset can be found here https://www.image-net.org/download.php and the MS COCO dataset can be found here https://cocodataset.org/#download (both accessed on 16 November 2021). The code used to conduct this research can be found here https://gitlab.com/EAVISE/lut-model-quantization (accessed on 16 November 2021).

Conflicts of Interest: The authors declare no conflict of interest.

References

1. Fiesler, E.; Choudry, A.; Caulfield, H.J. Weight discretization paradigm for optical neural networks. Optical interconnections and networks. *Int. Soc. Opt. Photonics* **1990**, *1281*, 164–173.
2. Balzer, W.; Takahashi, M.; Ohta, J.; Kyuma, K. Weight quantization in Boltzmann machines. *Neural Netw.* **1991**, *4*, 405–409. [CrossRef]
3. Deng, J.; Dong, W.; Socher, R.; Li, L.J.; Li, K.; Fei-Fei, L. Imagenet: A large-scale hierarchical image database. In Proceedings of the 2009 IEEE Conference on Computer Vision and Pattern Recognition, Miami, FL, USA, 20–25 June 2009; pp. 248–255.
4. Molchanov, P.; Tyree, S.; Karras, T.; Aila, T.; Kautz, J. Pruning convolutional neural networks for resource efficient transfer learning. *arXiv* **2016**, arXiv:1611.06440.
5. Banbury, C.; Zhou, C.; Fedorov, I.; Matas, R.; Thakker, U.; Gope, D.; Janapa Reddi, V.; Mattina, M.; Whatmough, P. Micronets: Neural network architectures for deploying tinyml applications on commodity microcontrollers. *Proc. Mach. Learn. Syst.* **2021**, *3*, 517–532.
6. Nagel, M.; Fournarakis, M.; Amjad, R.A.; Bondarenko, Y.; van Baalen, M.; Blankevoort, T. A White Paper on Neural Network Quantization. *arXiv* **2021**, arXiv:2106.08295.
7. Zhuang, B.; Shen, C.; Tan, M.; Liu, L.; Reid, I. Towards effective low-bitwidth convolutional neural networks. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, Salt Lake City, UT, USA, 18–23 June 2018; pp. 7920–7928.
8. Rastegari, M.; Ordonez, V.; Redmon, J.; Farhadi, A. Xnor-net: Imagenet classification using binary convolutional neural networks. In *European Conference on Computer Vision*; Springer: Berlin/Heidelberg, Germany, 2016; pp. 525–542.
9. Park, E.; Yoo, S. Profit: A novel training method for sub-4 bit mobilenet models. In *European Conference on Computer Vision*; Springer: Berlin/Heidelberg, Germany, 2020; pp. 430–446.
10. Zhang, D.; Yang, J.; Ye, D.; Hua, G. Lq-nets: Learned quantization for highly accurate and compact deep neural networks. In Proceedings of the European Conference on Computer Vision (ECCV), Munich, Germany, 8–14 September 2018; pp. 365–382.
11. Zhou, S.; Wu, Y.; Ni, Z.; Zhou, X.; Wen, H.; Zou, Y. Dorefa-net: Training low bitwidth convolutional neural networks with low bitwidth gradients. *arXiv* **2016**, arXiv:1606.06160.
12. Jain, S.; Gural, A.; Wu, M.; Dick, C. Trained Quantization Thresholds for Accurate and Efficient Fixed-Point Inference of Deep Neural Networks. *Proc. Mach. Learn. Syst.* **2020**, *2*, 112–128.
13. Jacob, B.; Kligys, S.; Chen, B.; Zhu, M.; Tang, M.; Howard, A.; Adam, H.; Kalenichenko, D. Quantization and training of neural networks for efficient integer-arithmetic-only inference. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, Salt Lake City, UT, USA, 18–23 June 2018; pp. 2704–2713.
14. Krishnamoorthi, R. Quantizing deep convolutional networks for efficient inference: A whitepaper. *arXiv* **2018**, arXiv:1806.08342.
15. Ioffe, S.; Szegedy, C. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In Proceedings of the International Conference on Machine Learning, Lille, France, 6–11 July 2015; pp. 448–456.
16. Bengio, Y.; Léonard, N.; Courville, A. Estimating or propagating gradients through stochastic neurons for conditional computation. *arXiv* **2013**, arXiv:1308.3432.
17. Yang, J.; Shen, X.; Xing, J.; Tian, X.; Li, H.; Deng, B.; Huang, J.; Hua, X.S. Quantization networks. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, Long Beach, CA, USA, 16–17 June 2019; pp. 7308–7316.

Source: https://lirias.kuleuven.be/retrieve/02b3b220-64bc-4e7a-833e-6e319c5dfa77

Google Scholar research | 2026-10-07 | Article 07 | Original p. 23 | Orange: discussion; blue: reference

IEEE Access
Multidisciplinary | Rapid Review | Open Access Journal

Received July 1, 2021, accepted July 18, 2021, date of publication July 26, 2021, date of current version July 30, 2021.

Digital Object Identifier 10.1109/ACCESS.2021.3099075

A Resource Efficient Integer-Arithmetic-Only FPGA-Based CNN Accelerator for Real-Time Facial Emotion Recognition

JAEMYUNG KIM[1], JIN-KU KANG[1], (Senior Member, IEEE), AND YONGWOO KIM[2], (Member, IEEE)

[1]Department of Electrical and Computer Engineering, Inha University, Incheon 22212, South Korea
[2]Department of System Semiconductor Engineering, Sangmyung University, Cheonan 31066, South Korea

Corresponding authors: Yongwoo Kim (yongwoo.kim@smu.ac.kr) and Jin-Ku Kang (jkang@inha.ac.kr)

ABSTRACT Recently, many researches have been conducted on recognition of facial emotion using convolutional neural networks (CNNs), which show excellent performance in computer vision. To obtain a high classification accuracy, a CNN architecture with many parameters and high computational complexity is required. However, this is not suitable for embedded systems where hardware resources are limited. In this paper, we present a lightweight CNN architecture optimized for embedded systems. The proposed CNN architecture has a small memory footprint and low computational complexity. Furthermore, a novel hardware-friendly quantization method that uses only integer-arithmetic is proposed. The proposed hardware-friendly quantization method maps the scale factors to power-of-two terms and replaces multiplication and division operations using scale factors with shift operations. To improve the generalization and classification performance of the CNN, we create the FERPlus-A dataset. This is a new training dataset created using a variety of image processing algorithms. After training with FERPlus-A, quantization has been performed. The size of a quantized CNN parameter is about 0.39 MB, and the number of operations is about 28 M integer operations (IOPs). By evaluating the performance of the quantized CNN that uses only integer-arithmetic on the FERPlus test dataset, the classification accuracy is approximately 86.58%. It achieved higher accuracy than other lightweight CNNs in prior studies. The proposed CNN architecture that uses only integer-arithmetic is implemented on the Xilinx ZC706 SoC platform for real-time facial emotion recognition by applying parallelism strategies and efficient data caching strategies. The FPGA-based CNN accelerator implemented for real-time facial emotion recognition achieves about 10 frame per second (FPS) at 250 MHz and consumes 2.3 W.

INDEX TERMS Emotion recognition, convolutional neural network, quantization, accelerator, FPGA.

I. INTRODUCTION

Today, computers play a central role in industry and society, and are rapidly becoming a part of everyday life. Accordingly, the need for research on the interaction between humans and computers is increasing. For smooth interaction between them, a computer must be able to analyze human intention and respond accordingly. Emotions that appear in facial expressions are a universal and effective way of expressing human intentions. Analyzing human intentions through the emotions revealed in faces is called facial emotion recognition technology, and is used in diverse fields such as automobiles and robot industries. To understand accurately the emotions displayed on human faces, a computer must recognize the face and automatically classify the emotions according to specific emotion groups.

Ekman *et al.* [1] defined seven basic emotions: happiness, anger, fear, surprise, disgust, sadness, and neutral. Moreover, they found that basic emotions are perceived in the same way regardless of human culture. In the field of computer vision, the Facial Action Coding System (FACS), a facial expression analysis method proposed in [2], was used as a typical facial emotion recognition model. FACS defines the basic movement of the facial muscles as Action

The associate editor coordinating the review of this manuscript and approving it for publication was Gulistan Raja.

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

Source: https://doi.org/10.1109/ACCESS.2021.3099075
Google Scholar research | 2026-10-08 | Article 08 | Original p. 1 | Orange: discussion; blue: reference

constructed the FERPlus dataset, which addressed the problem of the FER2013 [24] dataset using the crowd-sourcing and label distribution methods, and four different training schemes using a lighten VGG [25] were proposed. Li *et al.* [15] extracted identity features and emotion features through DeepID2 [26] and ResNet [27], respectively, and then classified them by combining features from the output of each CNN. Wikanningrum *et al.* [16] proposed a transfer learning and ensemble method using pre-trained parameters with a dataset frequently used in image classification. Pandey *et al.* [17], [18] suggested an approach to train a MobileNetv2 [28] with a dataset incorporating a Sobel gradient and Laplacian of the original image dataset. Based on the method proposed in [18], Pandey *et al.* [19] used a method of training by adding center loss [29] to global loss to improve the discrimination power of the CNN.

Saurav *et al.* [20] constructed two custom CNNs using gradient weighted class activation mapping (Grad-CAM). They also proposed a dual-integrated CNN, which combines the features extracted from two CNNs and uses them for facial emotion recognition. Miao *et al.* [21] proposed SHCNN, which can alleviate the over-fitting problem for a relatively small dataset and recognize both static and micro-expressions simultaneously. Lian *et al.* [22] introduced a method to quantify the contribution of various facial areas using class activation mapping (CAM) from features extracted through a DenseNet [30]. Zhao *et al.* [23] created a new dataset integrating two similar emotion groups into one. They proposed a method to train the DenseNet by applying face detection and face alignment methods.

However, the previous works [14], [21] occupied a large amount of memory space and had a problem of high computational complexity. In earlier works [15], [20], the CNN was used with smaller parameters, but because two CNNs were used for inference, they still occupied a large amount of memory space and remain unsuitable for real-time processing. In other cases [16]–[19], CNN was used with small parameters and low computational complexity, but their facial emotion recognition accuracy is very poor. Even in works using a small number of parameters [22], [23], they were not suitable for real-time processing because they require too much memory space due to dense connectivity. This is a characteristic of the DenseNet. To address these problems, further research is needed to design a lightweight CNN architecture by optimizing memory footprint and computational complexity while maintaining the accuracy of facial emotion recognition.

B. QUANTIZATION

Quantization is a method that converts parameters trained with a 32-bit floating-point (FP) into a low-bit fixed-point (FX) or an integer (INT). Applying quantization makes it possible to optimize memory usage and to reduce the number of calculations using simpler hardware. Quantization is divided into two methods. The post-training quantization (PTQ) method [31]–[34] minimizes quantization errors by applying calibration when inferencing using pre-trained parameters. The quantization-aware training (QAT) method [35]–[39] retrains parameters considering the effects of quantization.

Nagel *et al.* [31] proposed bias correction and cross-layer equalization methods to solve biased weight errors and imbalances during the quantization process. Choukroun *et al.* [32] introduced a method of minimizing the quantization error using the least mean squared error between the floating-point value and the quantized. Banner *et al.* [33] minimized quantization errors by calculating the optimal clipping value using analytical clipping for integer quantization (ACIQ). Cai *et al.* [34] suggested a distilled dataset using the mean and variance of the batch normalization (BN) layer to obtain the optimal quantization range.

Because the PTQ method does not require a retraining process, it saves computing resources and optimization time and enables rapid deployment. However, existing researches [31], [32] need fine-tuned values to minimize loss of accuracy. In [33] and [34], different numbers of bits should be assigned for each channel to minimize the drop in accuracy. In other works [32], [33] should obtain separate scale factors for each channel to maintain accuracy. Moreover, because all PTQ methods require floating-point parameters, dedicated operators are required when implementing the hardware.

Choi *et al.* [35] proposed a method to minimize quantization errors by defining a parameterized clipping activation (PACT) function to find appropriate scale factors. Jacob *et al.* [36] defined the operation of the quantizer to perform matrix multiplication using only integer-arithmetic and presented a layer fusion method. Jung *et al.* [37] suggested a method that minimizes task loss using parameterized quantization intervals. Esser *et al.* [38] proposed a method of learning optimal quantization mapping by making scale factors learnable parameters. Jain *et al.* [39] proposed a method of mapping scale factors to power-of-two terms using scale factor conversion equation.

The QAT method can obtain higher accuracy than the PTQ method by training quantization parameters with other parameters. However, some works [37], [38] did not quantize the BN layer. In [35]–[38], the parameters required for quantization are floating-point, and dedicated operators are necessary when implementing hardware. In another approach [39], calibration of the activation scale factor is required before starting QAT. To compensate for these problems, a hardware-friendly quantization method that does not require pre-calibration step is needed.

C. FPGA-BASED CNN ACCELERATOR FOR FACIAL EMOTION RECOGNITION

Performing facial emotion recognition using a GPU provides the best performance in throughput and speed. However, it is difficult to use a GPU in embedded systems. These systems require solutions that consume less energy and use fewer hardware resources. A field programmable gate array (FPGA) has flexibility that allows designers to adapt designs

Source: https://doi.org/10.1109/ACCESS.2021.3099075

Google Scholar research | 2026-10-08 | Article 08 | Original p. 3 | Orange: discussion; blue: reference

to fit an application. It is suitable for real-time processing in embedded systems because it can be programmed to enable optimal operation speed and reasonable power consumption.

Due to these advantages, several FPGA-based CNN accelerators for facial emotion recognition have been proposed [40]-[44]. Phan-Xuan *et al.* [40] implemented a CNN accelerator on a Xilinx Zynq-XC7Z020 FPGA using high-level-synthesis (HLS). They designed the CNN accelerator that stores the results and inputs of the layer in DRAM to reduce block RAM (BRAM) usage, and used VDMA for fast DRAM access. Phuc *et al.* [41] designed a CNN accelerator by applying an efficient structure and data pre-processing method for facial emotion recognition and implemented it on Altera DE-10 FPGA using Verilog HDL. In Vinhe *et al.* [42], the accelerator configured the processing engine core on Altera DE-10 FPGA to accelerate the convolution operation. Ding *et al.* [43] proposed the accelerator on Altera Cyclone-V FPGA using Verilog HDL with different parallel processing for different convolution layers constituting the CNN. In Ding *et al.* [44], the CNN accelerator implemented on Altera Cyclone-V FPGA using Verilog HDL. They designed a configurable convolution computing array to maximize DSP usage on the FPGA.

In previous works [40]–[42], a simple CNN architecture was implemented involving fully-connected, convolution, and pooling layers. Its number of parameters and computational complexity was low, but so was its accuracy, making it unsuitable for applications requiring highly accurate facial emotion recognition. In [43] and [44], DeepID architecture is implemented on an FPGA that requires a great deal of hardware resources, making it unusable in embedded systems. Therefore, it is necessary to design a low-power, low-cost FPGA-based CNN accelerator for real-time facial emotion recognition and that retains its high facial emotion recognition performance.

III. PROPOSED METHODS

A. PROPOSED DESIGN FLOW

In this paper, a novel lightweight CNN architecture is proposed that is suitable for embedded systems. This is achieved by optimizing the memory footprint and computational complexity while maintaining the accuracy of facial emotion recognition. In addition, log level threshold quantization (LLTQ) is proposed, which addresses the disadvantages of the previous works [38], [39]. Moreover, a low-power, low-cost FPGA-based CNN accelerator is proposed for real-time facial emotion recognition.

Fig. 1 shows the overall process for implementing the proposed real-time facial emotion recognition application. First, a new training dataset was generated combining various image processing algorithms to enhance CNN facial emotion recognition performance. Next, a novel lightweight CNN optimized for embedded systems as floating-point was trained. Then, LLTQ was performed, which is a hardware-friendly quantization method that uses only integer-arithmetic. Finally, a CNN accelerator IP was designed using an HLS compiler and a device driver was created using a standard C/C++ compiler. The newly developed IP and device driver were implemented in the PL and PS areas of the heterogeneous SoC platform for a real-time facial emotion recognition application.

B. PROPOSED LIGHTWEIGHT CNN ARCHITECTURE FOR EMBEDDED SYSTEMS

In this section, a lightweight CNN architecture is introduced that provides optimized memory usage and computational complexity on embedded devices. Fig. 2 shows the proposed lightweight CNN architecture consisting of 21 convolution layers and one global average pooling layer. The computation block includes two basic computation modules, FireA and FireB. The number of filters in the squeeze layer (s) and the number of filters in the expand layer (e), which are parameters of the computation block, were increased by two times in the following computation block. The proposed CNN was designed by applying the following techniques.

1) BASIC COMPUTATION MODULE

The newly designed lightweight CNN was inspired by the fire module proposed on SqueezeNet. The fire module consists of two types of layers. The squeeze layer (SQ) can reduce the number of channels of the feature map, reducing the number of operations in the expand layer that follows. The expand layer serves to expand the channel again and comprises two types of layers: a layer with a kernel size of 3 (EX3) and a layer with a kernel size of 1 (EX1). The output feature map of the SQ is divided into two paths and enters to EX1 and EX3, respectively. Furthermore, the output feature maps of EX1 and EX3 are merged through channel-wise concatenation and then enter the next layer.

The proposed lightweight CNN uses a modified fire module as a basic computation module and consists of two types (FireA and FireB). The configuration of the expand layer of FireA and FireB is the same. However, the kernel size in the SQ is set to 3 in FireA and to 1 in FireB. The reason for setting the kernel size to 3 in FireA is to minimize the accuracy drop by maintaining an appropriate reception field.

2) ALL CONVOLUTIONAL AND REGULAR NETWORK

The proposed CNN uses only the convolutional layer with kernel size of 3 or 1, so no consideration of the acceleration methods is needed for other kernel sizes when implementing in hardware. The max-pooling layer halves the resolution of the feature map and compresses the information. It can become a bottleneck when implemented in hardware because it must find the maximum value of the feature map. The proposed CNN sets the stride of FireA to 2; this halves the resolution of the feature map and replaces the max-pooling layer.

The fully-connected (FC) layer, which classifies the extracted features at the very end of the CNN, involves many parameters. The number of parameters can be reduced by

Source: https://doi.org/10.1109/ACCESS.2021.3099075

Google Scholar research | 2026-10-08 | Article 08 | Original p. 4 | Orange: discussion; blue: reference

number of operations and parameters generated by the BN layer can be ignored when implementing in hardware.

3) POWER-OF-2 RESOLUTION AND MULTIPLE-OF-4 OUTPUT CHANNELS

By making both the width and height of the feature map power-of-two terms, the multiplication and division operations of the part that calculates the address of the feature map can be replaced with a shift operation. In addition, the output channels of the convolution layer were all set to multiples of four for efficient parallel multiply-accumulate (MAC) hardware operations. A detailed description of the parallel MAC operations is presented in Section III-D.

C. PROPOSED QUANTIZATION METHOD

In this section, log level threshold quantization (LLTQ) is described, which is a novel hardware-friendly quantization method. Furthermore, an integer scale conversion method is described, which converts quantization-affected floating-point values to actual integer values and reconstructs CNN to use only integer-arithmetic.

1) LOG LEVEL THRESHOLD QUANTIZATION (LLTQ)

The current state-of-the-art quantization methods (learned step size quantization (LSQ) method [38] and trained quantization threshold (TQT) method [39]) use uniform quantization. A CNN with uniform quantizers inserted, performs quantization-aware training (QAT) using pre-trained floating-point parameters. Fig. 3 shows the structure of the convolution layer with a uniform quantizer inserted and the four-step operation of the quantizer. In the first step, the scale process, the quantizer maps a real value range to an integer value range of specified bit. The second step, called the round process, converts all real values mapped to an integer value range into integers. The third step is the clamp process. This process removes elements that exceed the quantization range to suit the quantization level. When the quantization bit is n, the values after the clamp process are located inside *Bound* as shown in (1):

$$Bound = \begin{cases} \left[-2^{n-1}, 2^{n-1}-1\right], & signed\ data \\ \left[0, 2^{n}-1\right], & unsigned\ data. \end{cases} \tag{1}$$

The fourth step, the de-quant process, converts values mapped to an integer range into floating-point values affected by quantization. The reason for converting the value quantized to an integer back to floating-point is that most existing deep learning frameworks using GPUs are optimized for floating-point training. The overall equation for the forward path can be described as follows:

$$x_d = clamp\left(round\left(\frac{x_f}{s}\right), Bound\right) \cdot s \tag{2}$$

where x_f is a floating-point value that is an input of the quantizer and x_d is the output value of the quantizer that passed de-quant process. The *clamp(x, Bound)* function is the second step of the uniform quantizer and cuts the rounded value (x) to fit inside the *Bound*. The *round*($\cdot$) function is the third step of the uniform quantizer. It rounds the real data to the nearest integer. A straight-through estimator (STE) [45] was used to solve the discontinuity occurring in the rounding process.

The LSQ [38] method sets the trainable parameter as a scale factor (s) and trains the scale factor along with other parameters. Therefore, the LSQ does not require scale factor conversion in the quantization process. Although it is well learned for use with the lightweight CNN described in Section III-B, there is the disadvantage that dedicated operators are needed when implementing it in hardware. This is because this is a floating-point scale factor. The TQT [39] method sets the trainable parameter as a log-threshold ($log_2 t$) and maps a scale factor to power-of-two terms as follows:

$$s = \frac{2^{ceil(log_2 t)}}{q_level} \tag{3}$$

where the *q_level* is the value of adding 1 to the maximum value of the *Bound* in (1).

Because the TQT maps the scale factor to power-of-two terms, there is an advantage in that multiplication and division using the scale factor can be replaced with a shift operation. However, before starting retraining, calibration of the activation scale factor is required. Therefore, in this paper, a novel hardware-friendly quantization method that fixes the problems in earlier works [38], [39] is proposed. The proposed LLTQ can directly map the trainable parameter to the power-of-two scale factor. In addition, unlike TQT [39], the LLTQ method enables initializing the activation scale factor with the activation data statistics value of the first batch of the first epoch; there is no need for the pre-calibration process. Moreover, by mapping the scale factor to the power-of-two terms, the multiplication and division operations that occur in the scale and the de-quant process could all be replaced by shift operations. The performance of the LLTQ method is evaluated using only integer-arithmetic through the integer scale conversion method, which will be explained in Section III-C-2. LLTQ sets the trainable parameter as a level-threshold (*level_th*) and maps the scale factor to the power-of-two terms using the following equation:

$$s = \frac{2^{ceil(log_2(abs(level_th * q_level)))}}{q_level} \tag{4}$$

where the *q_level* is the same as in (3). A detailed description of the LLTQ equation and its trainable parameter is presented in Appendix A of the supplementary material document. Many operations are required to calculate the scale factor (s). However, when performing inference with a CNN reconfigured to use only integer-arithmetic, only shift amounts of the scale factor mapped to the power-of-two terms are needed.

Additionally, several hardware-friendly quantization schemes were adapted for LLTQ. By using symmetric quantization, the computational overhead for zero-point that occurs during uniform quantization is eliminated. By introducing per-layer scaling, quantization can be performed with one

Source: https://doi.org/10.1109/ACCESS.2021.3099075

Google Scholar research | 2026-10-08 | Article 08 | Original p. 6 | Orange: discussion; blue: reference

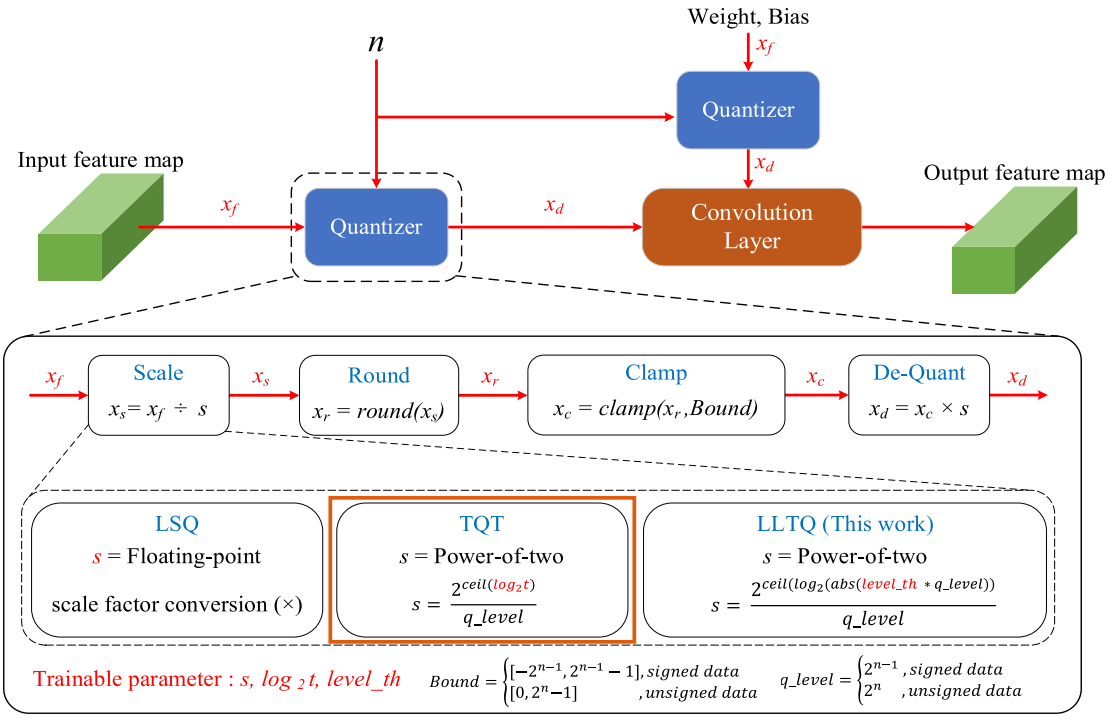


FIGURE 3. The process of a uniform quantizer using the LSQ [38], the TQT [39] method, and the proposed quantization method, LLTQ.

scale factor for all elements of a given parameter or input tensor. The BN layer fusion, which combines the parameters of the convolution layer and of the BN layer, was applied. This scheme does not need to proceed with quantization on the BN layer. This can reduce the number of operations and inference time by eliminating the multiplication and division operations required for the BN layer. The fire module is a structure in which the output of a squeeze layer enters the input of two expand layers. In floating-point, the two expand layer inputs are the same. However, in QAT, each convolution layer requires a different scale factor. Therefore, the squeeze layer has to calculate the output for each expand layer separately. This can form a bottleneck when performing CNN inference in hardware. To solve this problem, QAT was performed by making the quantizers of two expand layers into a single shared quantizer.

2) INTEGER SCALE CONVERSION

In several papers researching quantization, performance was evaluated using quantization-affected floating-point values rather than actual integer values. Using the parameters made by the proposed quantization method, the CNN was reconstructed using only integer-arithmetic through the integer scale conversion method proposed in this section. As shown in Fig. 4 (b), (d), the red line indicates the flow of floating-point data, and the black line indicates the flow of integer data. The superscript of each letter represents the type of data: f denotes floating-point data, i denotes an integer, and the subscripts k, $k+1$, $k+2$, $k+3$ below, indicate the number of each layer. The term Q_a is the integer-arithmetic-only (IAO) activation quantizer, Q_b is the IAO bias quantizer, and Q_w is the IAO weight quantizer. The terms $S_{w,k}$, $S_{w,k+1}$, and $S_{w,k+2}$ are weight scale factors, $S_{b,k}$, $S_{b,k+1}$, and $S_{b,k+2}$ are bias scale factors, and $S_{a,k}$, $S_{a,k+1}$, $S_{a,k+2}$, and $S_{a,k+3}$ are the activation scale factors. Here, *A, W, B,* and *C* marked on the dataflow arrow mean layer input, weight, bias, and activation function output, respectively. In Fig. 4, the part marked *Offline* is the process of converting quantization-affected floating-point parameters into integers. This process is done before inference and thus does not affect inference time. The process of converting quantization-affected floating-point parameters into integer parameters is represented as:

$$W_k^i = clamp(round(W_k^f \ll S_{w,k}, Bound)) \quad (5)$$

$$B_k^i = clamp(round(B_k^f \ll S_{b,k}, Bound)) \ll (S_{a,k} + S_{w,k} - S_{b,k}). \quad (6)$$

It was assumed that the input of the convolution layer is an integer quantized with the bit-precision specified at QAT. As shown in Fig. 4 (a) and (b), the general convolution layer performs MAC operation on integer scale weights and integer scale activations and adds the integer scale bias to the MAC result. As shown in (6), the bias value should then be converted to the same scale as the MAC operation result. This method is called bias up-scaling, and without this process, an error occurs because the bias and the MAC operation result have different bit-precision. After adding bias, the data passes the activation function ReLU, and the result of the ReLU can

Source: https://doi.org/10.1109/ACCESS.2021.3099075

Google Scholar research | 2026-10-08 | Article 08 | Original p. 7 | Orange: discussion; blue: reference

together and called a feature map caching module (FMCM). To maximize parameter data reuse, an on-chip memory was created that can store all parameters of the layer in which the operation is performed, which memory is called the parameter caching module (PCM). In addition, the PCM for each processing module was divided to use the inter-feature map parallelism strategy. A memory was created to accumulate the MAC operation results for the channel of the output feature map during MAC operation, which memory is called output channel accumulation module (OCAM).

4) POST-PROCESSING MODULE

All the remaining operations except convolution operations are performed in the PPM. The PPM consists of the shift operations, add operations, and sign-bit comparison operations. The PPM receives the final output value of the OCAM as an input. Moreover, like the parallelized processing module, the inter-feature map parallelism strategy is applied with the unroll factor set to 4. The final output of the PPM is writeback to the DRAM to enter the input of the next layer.

5) TASK-SCHEDULING

Fig. 6 shows task-scheduling that optimizes the dataflow of each of the parallelized modules constituting the CNN accelerator. First, the CPU calculates the size of the data caching module and the address of the layer and data region using the pre-calculated network constraints before starting the CNN operations. The computed addresses and layer configurations are transferred from the PS to the PL configuration registers through the AXI_LITE bus. Data caching modules (FMCM, PCM) prefetch the data from DRAM through the AXI_MEM bus using the pre-transferred address and layer configurations. The prefetched data is loaded into the pixel buffer and parameter buffer, and the MAC operation is performed in the parallelized processing module. The OCAM stores the intermediate results of the MAC operation and performs accumulation. After the accumulation is completed, the PPM executes the remaining operations. This dataflow lasts until all processes in the layer are terminated.

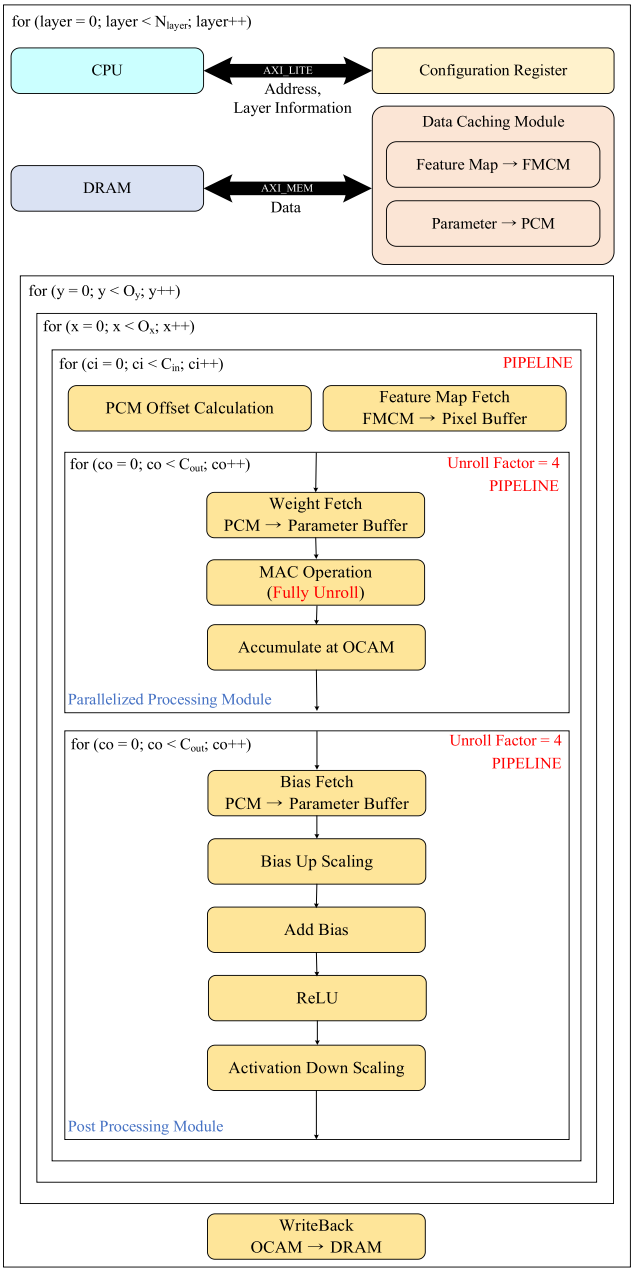


FIGURE 6. Overall task-scheduling for the proposed CNN accelerator.

IV. EXPERIMENTAL RESULTS

In this section, the experimental environment and results are described that were set to evaluate the performance of the proposed CNN optimized for embedded systems and the proposed hardware-friendly quantization method. All evaluations and measurements were performed on a workstation equipped with Intel®Xeon®Gold 512 CPU @ 2.2 GHz, 180 GB RAM, and an NVIDIA Tesla V100 (SXM2) GPU. A new training dataset was created using the OpenCV library. The CUDA library and Pytorch framework were used for floating-point training, quantization-aware training, and integer-arithmetic-only inference. The number of operations was measured using the framework of [47]. The proposed CNN accelerator for real-time facial emotion recognition was implemented using the Xilinx Zynq-7000 SoC ZC706 Evaluation Kit. It consisted of a dual-core ARM Cortex-A9 processor (PS) and XC7Z045 FPGA (PL).

During floating-point training, the batch size was 64, and the number of epochs was 200. The Adam method was chosen as the optimization algorithm with a learning rate of 1e-2 and weight decay of 1e-5. The quantization-aware training methods used in the experiment herein included LSQ [38], TQT [39] and the proposed LLTQ. The hyperparameters of the three quantization methods were set as follows: the batch size was 32 and the number of epochs was 100. The initial learning rate was set to 1e-5 for weights and

Source: https://doi.org/10.1109/ACCESS.2021.3099075

Google Scholar research | 2026-10-08 | Article 08 | Original p. 10 | Orange: discussion; blue: reference



FIGURE 7. Some samples for the FERPlus-A dataset: (a) Original image. (b) Bilateral filter, CLAHE applied to (a). (c) Sharpening filter, CLAHE applied to (a). (d) Bilateral filter, CLAHE, sharpening filter applied to (a).

biases, and 1e-3 for scale factors, log-thresholds, and level-thresholds.

A. RESULTS FOR THE NEW TRAINING DATASET

In real-world situations, it cannot be assumed that only images of good quality become CNN input. Therefore, data were added considering various conditions to assure high performance of the CNN in real-world situations. A bilateral filter [11] was applied to add noise-removed and border-preserved face images. CLAHE [12] was used to add high-contrast images and appropriate contrast images. By applying a sharpening filter, images were added that emphasize the boundary features of the face. In this way, FERPlus-A was constructed. This is a dataset that can build a more robust CNN in real-world situations. The FERPlus-A training dataset includes 143,545 images that were added to the 28,709 FERPlus training dataset, resulting in 172,254 images. Fig. 7 shows some samples from the FERPlus-A training dataset. The CNN proposed in Section III-B was used to evaluate whether the FERPlus-A dataset showed improved performance. The CNN was trained using the FERPlus and FERPlus-A training dataset and their performance was evaluated using the same FERPlus test dataset. When trained using the FERPlus training dataset, the facial emotion recognition accuracy was approximately 71.85%, and when trained with the FERPlus-A training dataset, the accuracy was approximately 86.58%. A performance improvement of approximately 14.73% was confirmed when using the FERPlus-A training dataset.

B. COMPARISON OF OTHER QUANTIZATION WORKS

Eight-bit quantization-aware training was performed for all three quantization methods. Table 1 shows the experimental results. Compared to the floating-point, the method of [38] improved accuracy by approximately 0.13%, but because the scale factor was still a floating-point type, the method was not hardware-friendly. This is because it still required a dedicated operator. The method of [39] is hardware-friendly, but it reduced accuracy by approximately 0.45% compared to the floating-point result. After quantization-aware training using LLTQ, CNN inference was conducted using only integer-arithmetic. As a result of applying LLTQ, it was confirmed that there was no accuracy drop relative to the floating-point method. Additional experiments of the LLTQ are provided in Appendix B of the supplementary material document.

C. COMPARISON WITH OTHER LIGHTWEIGHT CNNs

The performance of the proposed lightweight CNN optimized for embedded devices was compared with other facial emotion recognition studies using a lightweight CNN. Comparison of facial emotion recognition accuracy, the number of operations, and parameter size are shown in Table 2. The proposed CNN parameter size is 0.39 MB, the number of operations is 28 M IOPs, and the accuracy is approximately 86.58%. The accuracy of facial emotion recognition by the proposed CNN achieved the highest facial emotion recognition accuracy among the other methods considered [14]–[23]. As shown in Table 2, the new method outperforms the methods of [14]–[16], [18]–[21], [23]-2, [23]-3 in terms of accuracy, the number of operations, and parameter size. In a comparison with the method of [22], it was not possible to measure the number of operations and parameters, but the proposed method exhibits 5.07% higher accuracy. Compared to [17], the new method is 6.33 times smaller parameter size and 3.28% more accurate. Although its number of operations is 1.4 times larger, it is more efficient because it uses only integer-arithmetic. Compared with the method of [23]-1, the proposed method is 1.05 times larger parameter size but 2.52% more accurate. Compared with the method of [23]-2, the new method is 2.15 times smaller and 1% more accurate. Regarding the method of [23]-3, the new approach is 1.67 times smaller, uses 6.07 times fewer operations, and has 0.91% greater accuracy. These results indicate that the proposed CNN achieved the best trade-off between memory usage, number of operations, and accuracy.

D. HARDWARE IMPLEMENTATION ON FPGA

HLS is a way of designing hardware using a high-level language. Implementation details such as state machine and interface are handled by the HLS compiler, so an abstract design can be possible, code can be shared during design and simulation, and rapid deployment is possible. Taking note

Source: https://doi.org/10.1109/ACCESS.2021.3099075

Google Scholar research | 2026-10-08 | Article 08 | Original p. 11 | Orange: discussion; blue: reference

TABLE 1. Comparison of three different quantization-aware training methods.

Method	Bit-Width (W/A/B)	Scale factor	Accuracy (%)	vs Baseline	# of Operations	Remark
Baseline	32/32/32	-	86.58	-	28M FLOPs	Floating-point
TQT [39]	8/8/8	Power-of-two	86.13	-0.45	28M FLOPs	Need de-quant, Need pre-calibration
LSQ [38]	8/8/8	Floating-point	86.71	+0.13	28M FLOPs	Need de-quant
LLTQ (This work)*	8/8/8	Power-of-two	86.58	-	28M IOPs	Integer-arithmetic-only

* Inference result using integer-arithmetic after quantization-aware training

TABLE 2. Comparison with other facial emotion recognition studies using a lightweight CNN.

Method	Precision	Input Resolution	Accuracy (%)	Parameter Size (MB)	FLOPs (M)
[21]	FP32	224	86.54	33.23	204
[23]-3	FP32	48	85.67	0.65	170
[23]-2	FP32	48	85.58	0.84	-
[20]	FP32	48	85.29	5.40	394
[14]	FP32	64	84.89	33.42	1,940
[19]	FP32	64	84.80	1.37	70
[18]	FP32	64	84.47	1.37	70
[16]	FP32	32	84.30	3.62	288
[15]	FP32	48	84.30	1.22	190
[23]-1	FP32	48	84.06	0.37	-
[17]	FP32	64	83.30	2.47	20
[22]	FP32	64	81.51	-	-
This work	INT8	64	86.58**	0.39	28*

* Integer Operations (IOPs)
** The best two results are highlighted in red and blue colors, respectively.

TABLE 3. Resource utilization and power consumption of the proposed CNN accelerators.

Resource / Power		Type-1 (FP32)	Type-2 (INT8)	Area / Power Reduction Ratio (Type-1 vs Type-2)
LUT (Avail. 218,600)		16,453	5,090	×3.23 ↓
FF (Avail. 437,200)		33,326	7,588	×4.39 ↓
BRAM (Avail. 1,090)		120	61	×1.97 ↓
DSP (Avail. 900)		205	49	×4.18 ↓
Power (W)	PS	1.670	1.670	-
	PL	2.398	0.415	×5.78 ↓
	Others	0.244	0.225	×1.08 ↓
	Total	4.312	2.310	×1.87 ↓

of these advantages, HLS was chosen for the design. The CNN accelerator IP was designed using Vivado HLS 2019.2. The target clock frequency was 250 MHz. In Vivado 2019.2, the designed IP was connected to the PS and then synthesis and implementation were carried out. Petalinux 2019.2 was used to create the boot image and Linux kernel. A device driver was developed, and real-time facial emotion recognition application was built using Vitis 2019.2. The operation of the CNN accelerator was verified at the terminal connected to the UART.

Here, two types of FPGA-based CNN accelerators are shown in Table 3: Type-1 using 32-bit floating-point parameters (FP32) and Type-2 using 8-bit integer parameters (INT8) that quantized. Table 3 shows the resource utilization and power consumption of the proposed CNN accelerators. When comparing the area reduction ratio of Type-2 to that of Type-1, it was possible to reduce LUTs by 3.23 times, FFs by 4.39 times, BRAMs by 1.97 times, and DSPs by 4.18 times. Furthermore, the power was measured using the Vivado power estimator. The three factors that cause power consumption are the power generated by the CPU in the PS area, the power generated by the CNN accelerator IP in the PL area, and other power. The power consumption of the PS area that occurs in Type-1 and Type-2 is the same. However, the power of the PL area decreased by 5.78 times and the other power decreased by 1.08 times; thus, the total power decreased by 1.87 times. As shown in Table 4, The proposed CNN accelerator was compared to previous implementations. Compared with the method of [40], the new Type-2 method was found to be more accurate, to use fewer resources, and to have frame rate approximately 25 times faster. Compared to the work in [41], the new Type-2 approach is more accurate, uses 2.2 times fewer DSPs, and has a frame rate about 20 times faster. Compared to the accelerator in [42], the new Type-2 CNN accelerator is more accurate, but has a frame rate of about 5 FPS slower. Even so, the new accelerator

Source: https://doi.org/10.1109/ACCESS.2021.3099075
Google Scholar research | 2026-10-08 | Article 08 | Original p. 12 | Orange: discussion; blue: reference

[39] S. R. Jain, A. Gural, M. Wu, and C. H. Dick, "Trained quantization thresholds for accurate and efficient fixed-point inference of deep neural networks," in *Proc. Mach. Learn. Syst.*, Mar. 2020, pp. 112–128.
[40] H. Phan-Xuan, T. Le-Tien, and S. Nguyen-Tan, "FPGA platform applied for facial expression recognition system using convolutional neural networks," *Procedia Comput. Sci.*, vol. 151, pp. 651–658, Jan. 2019.
[41] T. N. D. Phuc, N. N. Nhut, N. Trinh, and L. Tran, "Facial expression recognition system using FPGA-based convolutional neural network," in *Research in Intelligent and Computing in Engineering*, vol. 1254. Singapore: Springer, Jan. 2021, pp. 341–351.
[42] P. T. Vinh and T. Q. Vinh, "Facial expression recognition system on SoC FPGA," in *Proc. Int. Symp. Electr. Electron. Eng. (ISEE)*, Ho Chi Minh City, Vietnam, Oct. 2019, pp. 1–4.
[43] R. Ding, G. Su, G. Bai, W. Xu, N. Su, and X. Wu, "A FPGA-based accelerator of convolutional neural network for face feature extraction," in *Proc. IEEE Int. Conf. Electron Devices Solid-State Circuits (EDSSC)*, Xi'an, China, Jun. 2019, pp. 1–3.
[44] R. Ding, X. Tian, G. Bai, G. Su, and X. Wu, "Hardware implementation of convolutional neural network for face feature extraction," in *Proc. IEEE 13th Int. Conf. ASIC (ASICON)*, Chongqing, China, Oct. 2019, pp. 1–4.
[45] Y. Bengio, N. Léonard, and A. Courville, "Estimating or propagating gradients through stochastic neurons for conditional computation," 2013, *arXiv:1308.3432*. [Online]. Available: https://arxiv.org/abs/1308.3432
[46] K. Abdelouahab, M. Pelcat, J. Serot, and F. Berry, "Accelerating CNN inference on FPGAs: A survey," 2018, *arXiv:1806.01683*. [Online]. Available: https://arxiv.org/abs/1806.01683
[47] L. Zhu. *THOP: PyTorch-OpCounter*. Accessed: Apr. 26, 2021. [Online]. Available: https://github.com/Lyken17/pytorch-OpCounter



JAEMYUNG KIM received the B.S. degree in electronics engineering from Inha University. He is currently pursuing the M.S. degree in electrical and computer engineering. His research interests include system-on-chip design, field-programmable gate array-based design, and convolutional neural network optimization and acceleration.



JIN-KU KANG (Senior Member, IEEE) received the B.S. degree from Seoul National University, Seoul, South Korea, in 1983, the M.S. degree in electrical engineering from New Jersey Institute of Technology, Newark, NJ, USA, in 1990, and the Ph.D. degree in electrical and computer engineering from North Carolina State University, Raleigh, NC, USA, in 1996. From 1983 to 1988, he was with Samsung Electronics, Inc., South Korea, where he was involved in memory design. In 1988, he was with Texas Instruments, South Korea. From 1996 to 1997, he was with Intel Corporation, Portland, OR, USA, as a Senior Design Engineer, where he was involved in high-speed I/O and timing circuits for processors. Since 1997, he has been with Inha University, Incheon, South Korea, where he is currently a Professor and leads the System IC Design Laboratory, Department of Electronics Engineering. His research interest includes high-speed/low-power mixed-mode circuit design for high-speed serial interfaces.



YONGWOO KIM (Member, IEEE) received the B.S. and M.S. degrees from Inha University, Incheon, South Korea, in 2007 and 2009, respectively, and the Ph.D. degree in electrical engineering from Korea Advanced Institute of Science and Technology, Daejeon, South Korea, in 2019. From 2009 to 2017, he was a Senior Engineer with Silicon Works Company Ltd., Daejeon. From 2019 to 2020, he was a Senior Researcher with the Artificial Intelligence Research Division, Korea Aerospace Research Institute, Daejeon. Since 2020, he has been with Sangmyung University, Cheonan, South Korea, where he is currently an Assistant Professor with the Department of System Semiconductor Engineering. His current research interests include image/video processing algorithm, super-resolution, and deep learning hardware architecture for vision processing.

Source: https://doi.org/10.1109/ACCESS.2021.3099075
Google Scholar research | 2026-10-08 | Article 08 | Original p. 15 | Orange: discussion; blue: reference

2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW) | 978-1-6654-8739-9/22/$31.00 ©2022 IEEE | DOI: 10.1109/CVPRW56347.2022.00302

A Low Memory Footprint Quantized Neural Network for Depth Completion of Very Sparse Time-of-Flight Depth Maps

Xiaowen Jiang[1] Valerio Cambareri[2*] Gianluca Agresti[3]
Cynthia Ifeyinwa Ugwu[4] Adriano Simonetto[4] Fabien Cardinaux[3] Pietro Zanuttigh[4]
[1] EPFL, Switzerland [2] Sony Depthsensing Solutions NV, Belgium
[3] Sony Europe B.V., R&D Center, Stuttgart Laboratory 1, Germany [4] University of Padova, Italy

Abstract

Sparse active illumination enables precise time-of-flight depth sensing as it maximizes signal-to-noise ratio for low power budgets. However, depth completion is required to produce dense depth maps for 3D perception. We address this task with realistic illumination and sensor resolution constraints by simulating ToF datasets for indoor 3D perception with challenging sparsity levels. We propose a quantized convolutional encoder-decoder network for this task. Our model achieves optimal depth map quality by means of input pre-processing and carefully tuned training with a geometry-preserving loss function. We also achieve low memory footprint for weights and activations by means of mixed precision quantization-at-training techniques. The resulting quantized models are comparable to the state of the art in terms of quality, but they require very low GPU times and achieve up to 14*-fold memory size reduction for the weights w.r.t. their floating point counterpart with minimal impact on quality metrics.*

1. Introduction

Time-of-flight (ToF) sensors are active depth sensing devices [53] with the potential to provide more reliable scene understanding by true 3D perception. Due to their low power consumption and accuracy at real-time frame rates, ToF sensors were recently integrated in mobile consumer devices [33]. However, ToF relies on active illumination [53] which accounts for a relevant part of its power consumption. To use the limited power budget of a mobile device more efficiently, the scene can be illuminated with a dot pattern light source so that its radiant intensity concentrates onto a small number of regions (*dots*). A low-power ToF sensor for indoors 3D perception typically captures $500 \sim 1500$ dots per frame. Because of this sparsity level, sensor fusion techniques are necessary to obtain dense depth maps. Hereafter, we tackle this *depth completion* task by using a single RGB image frame together with a sparse depth map warped to the RGB camera view (at small baseline of a few mm to minimize occlusion issues w.r.t. the ToF sensor). We dub this the *sparse ToF* setting.

To obtain dense depth from a sparse depth map, deep learning-based approaches have gained significant attention, with several contributions showing competitive performances especially on autonomous driving datasets [17, 44, 47]. We instead focus on efficient indoor 3D perception with ToF. In order to attain overall low power budget for both ToF sensing and depth completion, neural networks must be designed and trained accordingly for efficient inference. In this paper we introduce a carefully designed encoder-decoder CNN for depth completion. We achieve a low memory footprint for weights and activations by means of mixed-precision quantization-aware training and show that the resulting quantized models achieve conspicuous memory size savings for weights and activations, while remaining competitive w.r.t. quality metrics. Our contributions are as follows:

- We propose an efficient depth completion model suitable for low-power sparse ToF sensing. We evaluate our model and other state-of-the-art models on two datasets: the public-domain dataset NYU-Depth v2 [43], and SDS-ST1k [6], a contributed dataset providing accurate sparse ToF and RGB camera simulations.
- We analyze the impact of input pre-processing providing fast initialization; a normals estimation block enabling supervision with ground truth normals; different loss functions and network configurations. Each of these elements contributes to achieving optimal depth map quality for the resulting model.
- We evaluate quantization-aware training strategies to quantize weights and activations of the previous model. To the best of our knowledge, mixed-precision quantization [46] has been used successfully to classification tasks. In this paper we present the first ap-

*Corresponding author: valerio.cambareri@sony.com. XJ, CU, and AS thank Sony Europe B.V. for supporting their internships.

978-1-6654-8739-9/22/$31.00 ©2022 IEEE
DOI 10.1109/CVPRW56347.2022.00302

Source: https://ieeexplore.ieee.org/document/9857469 (published print)
Google Scholar research | 2026-10-08 | Article 09 | Original p. 1 | Orange: discussion; blue: reference

plication of mixed precision quantization to the depth completion task. This technique allows us to minimize the weights and activations memory requirements with very limited quality degradation.

2. Related Works

2.1. Depth Completion

The main task we explore in this paper is popular in computer vision since the introduction of depth sensors [3, 11] and has been addressed by neural networks with convolutional encoder-decoder architectures [9,24,34,35,37,39,52]. Many other works define the state of the art in depth completion [12, 13, 20, 22, 28, 31, 40, 45]. We will compare our results against CNN solutions [8, 9, 34, 37] selected on the basis of their similar complexity (single encoder-decoder). These will be retrained on our sparse ToF datasets. There exist also higher-complexity networks leveraging, *e.g.*, multiple decoder branches [39, 51] or separate encoder branches [16, 42, 48, 52] but in this work we aimed for a more compact and lightweight model. We will only focus on *single-frame* depth completion. Many contributions extend depth completion by either self-supervision or by providing as input multiple RGB-D frames with known or inferred poses [16,34,38,49,50]. By tackling only the single-frame case, we grant temporally-independent behavior for both dynamic and static cameras and remove additional computational effort required by pose estimation via neural networks (*e.g.*, [25]) or traditional odometry. We now discuss the main works we compare against.

Sparse-to-Dense (S2D [34]) is a popular approach using a single encoder-decoder network. Its core contribution is self-supervision by pose estimation (a feature we do not leverage) as well as its long-standing role in the KITTI leaderboard compared to its low complexity. The network architecture is similar to ours. As shown in [34, Sec. 6.4] and confirmed in our experiments, its robustness to sparsity is limited. We reproduced and retrained their model.

$\mathbf{D^3}$ [8] is developed using DenseNet [21] layers as building blocks. Its input pre-processing is an efficient way to initialize the depth map and attain high quality while maintaining low complexity. NYU-Depth v2 results are provided, but the authors' code is not public and we could not reproduce their quality in our implementation and retraining.

Convolutional Spatial Propagation Network (CSPN [9]) was among the first works [9,37,51] to propose a two-stage network architecture using *spatial propagation* layers. This entails: (i) generation of a "blurred" depth map and affinity matrix; (ii) iterative refinement using the learned affinity matrix as weights for anisotropic diffusion. We retrained the authors' model via their code.



(a) 1216×352, $K \approx 4.4\%$



(b) 304×224, $K \approx 1.4\%$



(c) 640×480, $K \approx 0.4\%$

Figure 1. RGB-annotation overlay of (a) KITTI (LiDAR; range: $[0, 85]m$), (b) NYU-Depth v2 (processed; range: $[0, 10]m$), (c) SDS-ST1k (sparse ToF; range: $[0, 15]m$). Annotations (red) at available sparse depth coordinates. Figure best viewed in color.

Non-Local Spatial Propagation Network (NLSPN [37]) improves upon [9], its main novelty being the use of non-local neighbors in the diffusion process, which is implemented by deformable convolutions. The authors show it achieves superior performances w.r.t. S2D and CSPN on NYU-Depth v2. In our experiments, we confirm this is indeed the most competitive approach at its network complexity. We retrained the authors' model via their code.

2.2. Neural Networks Quantization

Quantized Neural Networks (QNN) [18, 29, 54] are DNNs for which a smaller number of bits $b \ll 32$ is used to represent the weights and activations. QNNs are key to achieving efficient and low-power inference: they require considerably less memory and have a lower computational complexity, since quantized values can be stored, multiplied and accumulated efficiently.

Quantization-Aware Training (QAT) refers to a set of special algorithms to train QNNs, leading to superior results than post-training quantization. Recent QAT methods [2, 7, 10, 23, 30] proved effective on classification tasks.

These methods often use uniform quantization and the same bit width in each network layer. To improve QAT, Uhlich *et al.* [46] and Nikolić *et al.* [36] proposed to *learn* the optimal bit width for each layer while achieving a target size budget. On classification tasks, these *mixed precision* approaches show superior results since the bit widths can be allocated optimally across the network. Much less evidence of successful QAT exists for regression tasks such as depth completion.

3. Sparse ToF Datasets

We target datasets that are critically challenging in terms of sparsity. We consider indoor scenes (range: $[0, 15]m$) with rich structure and detailed objects with diverse cam-

Source: https://ieeexplore.ieee.org/document/9857469 (published print)

Google Scholar research | 2026-10-08 | Article 09 | Original p. 2 | Orange: discussion; blue: reference

n_f	n_s	MParams	RMSE (mm)	MNS
64	5	50.3	98.7	0.783
32	5	12.6	105.0	0.775
16	5	3.2	181.1	0.751
64	4	12.6	219.2	0.779
64	3	3.1	248.4	0.761
32	4	3.1	274.8	0.767

Table 1. Network tuning on SDS-ST1k (resolution 640×480). We confirm a steady decrease of quality metrics as the feature maps n_f relative to the highest scale and the scales n_s are reduced.

sufficient to retrieve scale. Thus, we minimize

$$\mathcal{L}(\mathcal{D}) := \mathcal{L}_{\ell_p}(\mathcal{D}) + \lambda_{\mathrm{n}}\mathcal{L}_{\mathrm{n}}(\mathcal{D}) \tag{4}$$

which balances between scale-dependent and scale-independent terms using $\lambda_{\mathrm{n}} \in [0,1]$. This weight depends on the dataset and must be tuned empirically. To do so, we run preliminary tests of our approach (Fig. 2a) on SDS-ST1k under different[6] $\mathcal{L} := \{\mathcal{L}_{\ell_2}, \mathcal{L}_{\ell_1}, \mathcal{L}_{\ell_1} + \lambda_{\mathrm{n}}\mathcal{L}_{\mathrm{n}}\}$ and $\lambda_{\mathrm{n}} := 10^{-q}, q = 0,\dots,4$, in the same settings described in Sec. 6.1. Indeed, defining the conventional RMSE and Mean Normals Similarity

$$\mathrm{MNS} := \frac{1}{Jn_hn_v}\sum_{j=1}^{J}\sum_{i=1}^{n_hn_v}\langle(N_{\mathrm{GT},j})_i, (\hat{N}_j)_i\rangle \tag{5}$$

as quality metrics we see that large values of λ_{n} degrade the RMSE, and *vice versa* for MNS. We find the best trade-off at $\lambda_{\mathrm{n}}^{\mathrm{opt}} = 10^{-3}$. As confirmation of this, in Fig. 3b we report one sample error map $\hat{D} - D_{\mathrm{GT}}$ under different $\mathcal{L}$. As we move from $\mathcal{L}_{\ell_2}$ to $\mathcal{L}_{\ell_1}$ the observed error map pattern artefact disappears. Remaining errors at depth map discontinuities are reduced by using the normals loss $\mathcal{L}_{\mathrm{n}}$.

4.5. Network Tuning

We investigated simple strategies to reduce the size of the initial model, such as tuning the $\mathrm{UNet}_{\theta,n_f,n_s}$ module by its number of feature maps n_f relative to the highest scale, and the number of scales n_s. Our preliminary tests, in the same setting as Sec. 6.1 on SDS-ST1k, actually confirm (Tab. 1) that any pair smaller than $(n_f, n_s) = (64, 5)$ leads to a rapid degradation of RMSE and MNS, with the reduction of n_s (shallower network) being generally worse than that of n_f. Thus, we choose to achieve memory size reduction by quantization of the full model with $(n_f, n_s) = (64, 5)$ which, as we will show, allows for a more compact model with limited impact on quality metrics.

5. Quantization-Aware Training

Let $Q(x;\boldsymbol{\phi})$ be an arbitrary scalar quantizer with parameters $\boldsymbol{\phi}$, which maps $x \in \mathbb{R}$ to discrete values $\{q_1, ..., q_I\}$ represented by b bits, $b : I \leq 2^b$. This quantizer may be applied to both weights and activations to reduce their memory size requirements. This can be done by either post-training quantization or by Quantization-Aware Training (QAT), which generally achieves better accuracy than the former. QAT consists in training the quantized version of a DNN (in this case, CNN) while applying $Q(x;\boldsymbol{\phi})$ to its weights and/or activations. However, this is challenging as the gradient through the quantizer is zero almost everywhere; a solution to this is the Straight-Through Estimator (STE) [4]. A *symmetric uniform* quantizer $Q_U(x;\boldsymbol{\phi})$ then maps $x \in \mathbb{R}$ to uniformly quantized values by

$$q = Q_U(x;\boldsymbol{\phi}) := \mathrm{sign}(x)\begin{cases} d\left\lfloor \frac{|x|}{d} + \frac{1}{2}\right\rfloor & |x| \leq q_{\max} \\ q_{\max} & |x| > q_{\max}\end{cases}, \tag{6}$$

where the parameter vector $\boldsymbol{\phi} := [d, q_{\max}, b]^T$ with step size $d \in \mathbb{R}^+$, maximum value $q_{\max} \in \mathbb{R}^+$, and number of bits or *bit width* $b \in \mathbb{N} : b \geq 2$ of the quantized value q.

5.1. Uniform Precision Models

We start by considering *uniform precision* (UP) QAT, *i.e.*, every weight (or activation) of every layer has the same predefined bit width b and QAT optimizes over the remaining parameters $d, q_{\max}$ of $\boldsymbol{\phi}$. We conducted extensive experiments to preserve depth completion accuracy while quantizing with (6) the weights and activations in our model. The most effective UP QAT training procedure was obtained by initializing from the pretrained $\mathrm{float32}$ model and using learnable $q_{\max}$ as in the Trained Quantization Threshold (TQT) approach [23]; to stabilize the QAT, we used cosine learning rate decay scheduling [32], and we first trained with RMSprop until convergence (32 epochs in our case), followed by Adam [26] for 20 more epochs.

5.2. Mixed Precision Models

Recent results [36, 46] show that, for a given memory size budget, better accuracy than UP QAT can be achieved using *mixed precision* (MP) QAT, *i.e.*, each layer uses its own bit width *learned at training* to fit a target total network size. Following [46] we parametrize $Q_U(x;\boldsymbol{\phi})$ with their range and step size d, from which the bit width $b := \lceil 1 + \log_2(q_{\max}/d + 1)\rceil$ is then inferred. With this parametrization and STE [4] gradient, we can use standard stochastic gradient descent methods to learn per-layer optimal bit widths jointly with network parameters. To achieve target network sizes we add to (4) *size constraints* on the weights and activations as penalty terms [5], minimizing

$$\mathcal{L}_{\mathrm{MP}} = \mathcal{L}(\mathcal{D}) + \lambda_{\mathrm{W}}\max(0, S_{\mathrm{W}})^2 + \lambda_{\mathrm{A}}\max(0, S_{\mathrm{A}})^2 \tag{7}$$

$$S_x = (\textstyle\sum_{l=1}^{L} S_x^l) - S_x^o,\ S_x^l = N_{x,l}b_{x,l},\ S_x^o = \bar{b}_x\textstyle\sum_{l=1}^{L} N_{x,l} \tag{8}$$

[6] We verified in preliminary tests that $\mathcal{L}_{\ell_2} + \lambda_{\mathrm{n}}\mathcal{L}_{\mathrm{n}}$ is systematically worse than the ℓ_1 case and excluded it accordingly.

Source: https://ieeexplore.ieee.org/document/9857469 (published print)

Google Scholar research | 2026-10-08 | Article 09 | Original p. 5 | Orange: discussion; blue: reference

References

[1] Gianluca Agresti, Henrik Schafer, Piergiorgio Sartor, Yalcin Incesu, and Pietro Zanuttigh. Unsupervised Domain Adaptation of Deep Networks for ToF Depth Refinement. *IEEE Trans. Pattern Anal. Mach. Intell.*, 2021.

[2] Yu Bai, Yu-Xiang Wang, and Edo Liberty. ProxQuant: Quantized Neural Networks via Proximal Operators. In *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2019.

[3] Jonathan T. Barron and Ben Poole. The Fast Bilateral Solver. In *Proc. Eur. Conf. Comput. Vis. (ECCV)*, pages 617–632. Springer International Publishing, 2016.

[4] Yoshua Bengio, Nicholas Léonard, and Aaron Courville. Estimating or propagating gradients through stochastic neurons for conditional computation. *arXiv preprint arXiv:1308.3432*, 2013.

[5] Dimitri P. Bertsekas. *Constrained optimization and Lagrange multiplier methods*. Academic Press, 2014.

[6] Valerio Cambareri. SDS-ST1K Dataset. Technical report, Sony Depthsensing Solutions NV, May 2021.

[7] Fabien Cardinaux, Stefan Uhlich, Kazuki Yoshiyama, Javier Alonso García, Lukas Mauch, Stephen Tiedemann, Thomas Kemp, and Akira Nakamura. Iteratively Training Look-Up Tables for Network Quantization. *IEEE J. Sel. Top. Signal Process.*, 14(4):860–870, 2020.

[8] Zhao Chen, Vijay Badrinarayanan, Gilad Drozdov, and Andrew Rabinovich. Estimating depth from rgb and sparse sensing. In *Proc. Eur. Conf. Comput. Vis. (ECCV)*, pages 167–182, 2018.

[9] Xinjing Cheng, Peng Wang, and Ruigang Yang. Learning depth with convolutional spatial propagation network. *IEEE Trans. Pattern Anal. Mach. Intell.*, 42(10):2361–2379, 2019.

[10] Jungwook Choi, Zhuo Wang, Swagath Venkataramani, Pierce I.-Jen Chuang, Vijayalakshmi Srinivasan, and Kailash Gopalakrishnan. PACT: Parameterized clipping activation for quantized neural networks. *arXiv preprint arXiv:1805.06085*, 2018.

[11] James Diebel and Sebastian Thrun. An Application of Markov Random Fields to Range Sensing. In Y. Weiss, B. Schölkopf, and J. Platt, editors, *Proc. Advances Neural Inf. Process. Syst. (NeurIPS)*, volume 18. MIT Press, 2005.

[12] Abdelrahman Eldesokey, Michael Felsberg, Karl Holmquist, and Michael Persson. Uncertainty-Aware CNNs for Depth Completion: Uncertainty from Beginning to End. In *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2020.

[13] A. Eldesokey, M. Felsberg, and F. Khan. Confidence Propagation through CNNs for Guided Sparse Depth Regression. *IEEE Trans. Pattern Anal. Mach. Intell.*, 42(10):2423–2436, Oct. 2020. Place: Los Alamitos, CA, USA Publisher: IEEE Computer Society.

[14] Epic Games. Unreal Engine (v4.25).

[15] Pedro F. Felzenszwalb and Daniel P. Huttenlocher. Distance Transforms of Sampled Functions. *Theory of Computing*, 8(19):415–428, 2012.

[16] Vitor Guizilini, Rares Ambrus, Wolfram Burgard, and Adrien Gaidon. Sparse auxiliary networks for unified monocular depth prediction and completion. In *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, pages 11078–11088, 2021.

[17] Vitor Guizilini, Rares Ambrus, Sudeep Pillai, Allan Raventos, and Adrien Gaidon. 3D Packing for Self-Supervised Monocular Depth Estimation. In *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2020.

[18] Song Han, Huizi Mao, and William J. Dally. Deep Compression: Compressing Deep Neural Networks with Pruning, Trained Quantization and Huffman Coding. In *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2016.

[19] Akio Hayakawa, Masato Ishii, Yoshiyuki Kobayashi, Akira Nakamura, Takuya Narihira, Yukio Obuchi, Andrew Shin, Takuya Yashima, and Kazuki Yoshiyama. Neural Network Libraries: A Deep Learning Framework Designed from Engineers' Perspectives, 2021.

[20] Mu Hu, Shuling Wang, Bin Li, Shiyu Ning, Li Fan, and Xiaojin Gong. PENet: Towards Precise and Efficient Image Guided Depth Completion. In *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, pages 13656–13662, 2021.

[21] Gao Huang, Zhuang Liu, Laurens van der Maaten, and Kilian Q. Weinberger. Densely Connected Convolutional Networks. In *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2017.

[22] Saif Imran, Yunfei Long, Xiaoming Liu, and Daniel Morris. Depth Coefficients for Depth Completion. In *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2019.

[23] Sambhav Jain, Albert Gural, Michael Wu, and Chris Dick. Trained Quantization Thresholds for Accurate and Efficient Fixed-Point Inference of Deep Neural Networks. In I. Dhillon, D. Papailiopoulos, and V. Sze, editors, *Proceedings of Machine Learning and Systems*, volume 2, pages 112–128, 2020.

[24] M. Jaritz, R. Charette, E. Wirbel, X. Perrotton, and F. Nashashibi. Sparse and Dense Data with CNNs: Depth Completion and Semantic Segmentation. In *Proc. IEEE Int. Conf. 3D Vis. (3DV)*, pages 52–60, 2018.

[25] Alex Kendall, Matthew Grimes, and Roberto Cipolla. PoseNet: A Convolutional Network for Real-Time 6-DOF Camera Relocalization. In *Proc. Int. Conf. Comput. Vis.(ICCV)*, pages 2938–2946, 2015.

[26] Diederick P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2015.

[27] Jason Ku, Ali Harakeh, and Steven L. Waslander. In Defense of Classical Image Processing: Fast Depth Completion on the CPU. In *2018 15th Conference on Computer and Robot Vision (CRV)*, pages 16–22. IEEE, 2018.

[28] A. Li, Z. Yuan, Y. Ling, W. Chi, S. Zhang, and C. Zhang. A Multi-Scale Guided Cascade Hourglass Network for Depth Completion. In *Proc. IEEE Winter Conf. Appl. Comput. Vis. (WACV)*, pages 32–40, 2020.

[29] Fengfu Li, Bo Zhang, and Bin Liu. Ternary weight networks. *arXiv preprint arXiv:1605.04711*, 2016.

[30] Zhi-Gang Liu and Matthew Mattina. Learning Low-precision Neural Networks without Straight-Through Estimator (STE). In *Proc. Int. Joint Conf. Artif. Intell. (IJCAI)*, pages 3066–3072, 2019.

Source: https://ieeexplore.ieee.org/document/9857469 (published print)

Google Scholar research | 2026-10-08 | Article 09 | Original p. 9 | Orange: discussion; blue: reference

arXiv:1911.12491v1 [cs.CV] 28 Nov 2019

QKD: Quantization-aware Knowledge Distillation

Jangho Kim*†
Seoul National University
kjh91@snu.ac.kr

Yash Bhalgat*
Qualcomm AI Research‡
Qualcomm Technologies, Inc.
ybhalgat@qti.qualcomm.com

Jinwon Lee
Qualcomm AI Research‡
Qualcomm Technologies, Inc.
jinwonl@qti.qualcomm.com

Chirag Patel
Qualcomm AI Research‡
Qualcomm Technologies, Inc.
cpatel@qti.qualcomm.com

Nojun Kwak§
Seoul National University
nojunk@snu.ac.kr

Abstract

Quantization and Knowledge distillation (KD) methods are widely used to reduce memory and power consumption of deep neural networks (DNNs), especially for resource-constrained edge devices. Although their combination is quite promising to meet these requirements, it may not work as desired. It is mainly because the regularization effect of KD further diminishes the already reduced representation power of a quantized model. To address this shortcoming, we propose Quantization-aware Knowledge Distillation (QKD) wherein quantization and KD are carefully coordinated in three phases. First, Self-studying (SS) phase fine-tunes a quantized low-precision student network without KD to obtain a good initialization. Second, Co-studying (CS) phase tries to train a teacher to make it more quantizaion-friendly and powerful than a fixed teacher. Finally, Tutoring (TU) phase transfers knowledge from the trained teacher to the student. We extensively evaluate our method on ImageNet and CIFAR-10/100 datasets and show an ablation study on networks with both standard and depthwise-separable convolutions. The proposed QKD outperformed existing state-of-the-art methods (e.g., 1.3% improvement on ResNet-18 with W4A4, 2.6% on MobileNetV2 with W4A4). Additionally, QKD could recover the full-precision accuracy at as low as W3A3 quantization on ResNet and W6A6 quantization on MobilenetV2. (See Fig. 1)

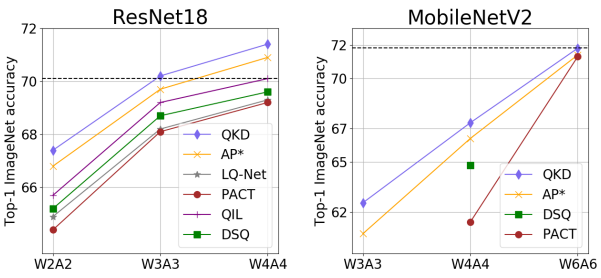


Figure 1: Top-1 accuracy on ImageNet with ResNet-18 and MobileNetV2. Our QKD is compared with various methods such as AP* [30], LQ-Net [50], PACT [6], QIL [18] and DSQ [10]. Dotted line indicates the full-Precision accuracy. More details are in Table 3 and 4.

1. Introduction

Deploying complex DNNs on resource-constrained edge devices such as smartphones or IoT devices is still challenging due to their tight memory and computation requirements. To address this, several research works have been proposed which can be roughly categorized into three folds: weight pruning [12, 11, 27, 9], quantization, [23, 50, 26, 47, 5] and knowledge distillation [15, 20, 48].

Today, a majority of edge devices require fixed-point inference for compute- and power-efficiency. Hence, quantizing the weights and activations of deep networks to a certain bit-width becomes a necessity to make them compatible to edge devices. Recently, hardware accelerators like NPUs (Neural Processing Units), NVIDIA's Tensor Core units and even CIM (Compute-in-memory) devices have targeted support for sub-byte level processing such as 4-bit×4-bit or 2-bit×2-bit matrix multiply-and-accumulate operations

*Equal contribution
†Work done while interning at Qualcomm AI Research.
‡Qualcomm AI Research is an initiative of Qualcomm Technologies, Inc.
§Currently a Visiting Researcher at Qualcomm Technologies, Inc.

Source: https://arxiv.org/pdf/1911.12491
Google Scholar research | 2026-10-07 | Article 10 | Original p. 1 | Orange: discussion; blue: reference

[28, 33, 40, 42]. These are much more power- and compute-efficient than conventional 8-bit processing. Thus, there are increased demands for lower-bit quantization.

The accuracy of very low-bit quantized networks such as using 4-, 3- or 2-bits inevitably decreases because of their low representation power compared to typical 8-bit quantization. Some works [30, 34] used knowledge distillation (KD) to improve the accuracy of quantized networks by transferring knowledge from a full-precision teacher network to a quantized student network. However, even though KD is one of the widely used techniques to enhance a DNN's accuracy [15], directly applying KD to very low-bit quantized networks incurs several challenges:

1. Due to its limited representation power, quantized networks generally show lower training and test accuracies compared to the networks with full precision. On the other hand, because KD uses not only the ground truth labels but also the teacher's estimated class probability distributions, it acts as a heavy regularizer [15]. Combining these contradictory characteristics of quantization and KD could sometimes result in further degradation of performance from that of quantization-only.
2. In general, it has been shown that a powerful teacher with high accuracy can teach a student better than a weaker one [54]. Also, [20, 29] show that if there is a large gap from the capacity or inherent differences between the teacher and student, it can hinder the knowledge transfer because this knowledge is unadaptable to the student. Previous works of quantization with KD [30, 34] directly applied conventional KD to quantization along with fixed teacher network, and thus they suffer from above mentioned limitations.

In this paper, we propose Quantization-aware Knowledge Distillation (QKD) to address the above mentioned challenges, especially for very low-precision quantization. QKD consists of three phases. In the first **'self-studying'** phase, instead of directly applying knowledge distillation to the quantized student network from the beginning, we first try to find a good initialization. In the second **'co-studying'** phase, we improve the teacher by using the knowledge of the student network. This phase makes the teacher more powerful and quantization-friendly (See Section 4.5), which is essential to improve accuracy. In final **'tutoring'** phase, we freeze the teacher network and transfer its knowledge to the student. This phase saves unnecessary training time and memory of the teacher network which tends to have already saturated in the co-studying phase. The overal process of QKD is depicted in the Figure 2. Our key contributions are the following:

- We propose Quantization-aware Knowledge Distillation which can be effectively applied to very low-bit (2-,3-,4-bit) quantized networks. Considering the characteristics of low bit networks and distillation methods, we design a combination of three phases for QKD that overcome the shortcomings posed by conventional quantization + KD methods as described above.
- We empirically verify that QKD works well on depth-wise convolution networks which are known to be difficult to quantize and our work is the first to apply KD to train low-bit depth-wise convolution networks (viz. MobileNetV2 and EfficientNet)
- We show that our QKD obtains state-of-the-art accuracies on CIFAR and ImageNet datasets compared to other existing methods. (See Figure 1.)

2. Related work

Quantization Reducing the precision of neural networks [7, 25, 53, 32, 43] has been studied extensively due to its computational and storage benefits. Although binary $\{-1, +1\}$[7] or tenary $\{-1, 0, +1\}$[25] quantizations are typically used, their methods only consider the quantization of weights. To fully utilize bit-wise operations, activation maps also should be quantized in the same way as weights. Some researches consider quantizing both weights and activation maps [16, 41, 35, 52, 46]. Binary neural networks [16] quantize both weights and activation maps as binary and compute gradients with binary values. XNOR-Net [35] also quantizes its weights and activation maps with scaling factors obtained by constrained optimization. Furthermore, HAQ [46] adaptively changes bit-width per layer by leveraging reinforcement learning and HW-awareness.

Recent works have tried to improve quantization further by learning the range of weights and activation maps [6, 18, 8, 45]. These approaches can easily outperform previous methods which do not train quantization-related parameters. PACT [6] proposes a clipping activation function using trainable a parameter which limits the maximum value of activation. LSQ [8] and TQT [17] introduce uniform quantization using trainable interval values. QIL [18] uses a trainable quantizer which performs both pruning and clipping. Our QKD leverages these trainable approaches in the baseline quantization implementation. On top of this quantization-only method, our elaborated knowledge distillation process boosts the accuracy, thereby achieving the state of art accuracy on low-bit quantization.

Knowledge Distillation KD is one of the most popular methods in model compression. It is widely used in many computer vision tasks. This framework transfers the knowledge of a teacher network to a smaller-sized student network in two ways: Offline and Online. First, offline KD uses a fixed pre-trained teacher networks and the knowledge transfer can happen in different ways. In [15], they

Source: https://arxiv.org/pdf/1911.12491

Google Scholar research | 2026-10-07 | Article 10 | Original p. 2 | Orange: discussion; blue: reference

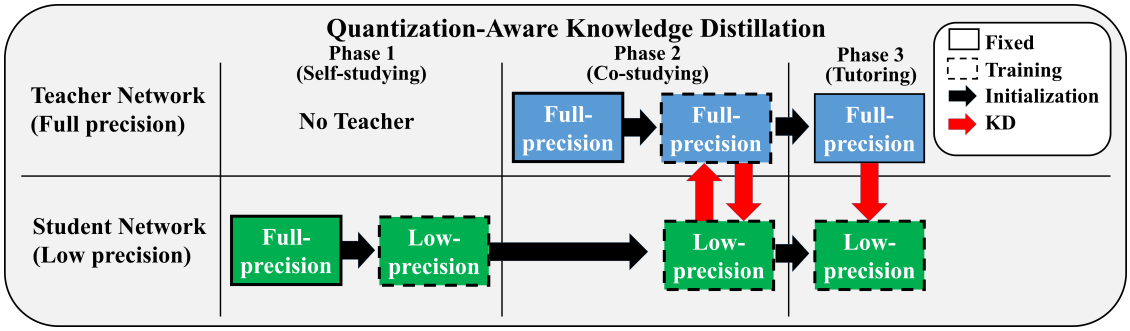


Figure 2: The overall process of QKD. Self-studying (SS) phase give a good starting point to alleviate low representative power and regularization effect of KD . Co-studying (CS) phase makes a teacher adaptable to a student and powerful than the fixed teacher. In tutoring (TU) phase, the teacher transfers its adaptable and powerful knowledge to the student.

encourage the student network to mimic the softened distribution of the teacher network. Other works [49, 14, 20, 37] transfer the information using different forms of activation maps from the teacher network. Second, online KD methods [54, 19, 51] train both the teacher and the student networks simultaneously without pre-trained teacher models. Deep Mutual Learning (DML) [51] have tried to transfer each knowledge of independent networks using KL loss, although it is not studied in the context of quantization. Our QKD uses both online and offline KD methods sequentially.

Quantization + Knowledge distillation Some researches have tried to use distillation methods to train low precision network [30, 34]. They use a full precision network as the teacher and the low bit network as a student. In Apprentice (AP) [30], the teacher and student networks are initialized with the corresponding pre-trained full precision networks and the student is then fine-tuned using distillation. Due to AP's initialization of the student, AP tends to get stuck in a local minimum in the case of very low-bit quantized student networks. The self-study phase of QKD directly mitigates this issue. Also, using a fixed teacher, as in [30, 34], can limit the knowledge transfer due to the inherent differences between the distributions of the full-precision teacher and low-precision student network. We tackle this problem via online co-studying (CS) and offline tutoring (TU).

3. Proposed Method

In this section, we first explain the quantization method used in QKD and then describe the proposed QKD method.

3.1. Quantization

In QKD, we use a trainable uniform quantization scheme for our baseline quantization implementation. We choose uniform quantization because of its hardware-friendly characteristics. In this work, we quantize both weights and activations. So, we introduce two trainable parameters for the interval values of each layer's weight parameters and input activations similar to [8, 17, 45]. These parameters can be trained with either the task loss or a combination of the task and distillation losses. Considering k-bit quantization for weights and activations, the weight and activation quantizers are defined as follows.

Weight Quantizer: Since weights can take positive as well as negative values, we quantize each weight to integers in the range $[-2^{k-1}, 2^{k-1}-1]$. Before rounding, the weights are first constrained to this range using a clamping function $F_W(w) \triangleq F(w, -2^{k-1}, 2^{k-1}-1)$ where

$$F(x, min, max) = \begin{cases} max & \text{if} \quad x > max \\ min & \text{if} \quad x < min \\ x & \text{elsewise.} \end{cases} \tag{1}$$

We use a trainable parameter I_W for the interval value of weight quantization. It is trained along with the weights of the network. We can calculate the quantization level of the input weight w using a rounding operation on F_W. The overall quantization-dequantization scheme for the weights of the network is defined as follows:

$$\hat{w} = Q_W(w) = \left\lfloor F_W(\frac{w}{I_W}) + \frac{1}{2} \right\rfloor \times I_W \tag{2}$$

where $\lfloor \cdot \rfloor$ is the flooring operation. Note that the *dequantization* step (multiplication by I_W) just brings the quantized values ($\hat{w}$) back to their original range and is commonly used when emulating the effect of quantization [3, 17, 36].

Activation Quantizer: Since most of the networks today use ReLU as the activation function, the activation values are unsigned[1]. Hence, to quantize the activations, we use an quantization function with the range $[0, 2^k - 1]$. The activation quantizer Q_X can be obtained as

$$\hat{x} = Q_X(x) = \left\lfloor F_X(\frac{x}{I_X}) + \frac{1}{2} \right\rfloor \times I_X \tag{3}$$

[1]EfficientNet uses Swish extensively which introduces a small proportion of negative activation values, but we can save one bit per activation by clamping these negative values at 0 using unsigned quantization

Source: https://arxiv.org/pdf/1911.12491

Google Scholar research | 2026-10-07 | Article 10 | Original p. 3 | Orange: discussion; blue: reference

Table 3: ImageNet performance comparison on architectures with standard convolutions

Bitwidth	Method	ResNet18		ResNet34		ResNet50	
		top-1	top-5	top-1	top-5	top-1	top-5
W32A32	-	70.1	89.5	73.5	91.7	76.3	93.0
W4A4	PACT [6]	69.2	89.0	-	-	76.5	93.2
	LQ-Nets [50]	69.3	88.8	-	-	75.1	92.4
	DSQ [10]	69.6	-	72.8	-	-	-
	QIL [18]	70.1	-	73.7	-	-	-
	BL	70.7	89.4	73.8	91.5	76.6	93.2
	SS + BL	70.8	89.4	74.9	91.5	76.6	93.2
	AP*	70.9	89.5	74.0	91.7	76.7	93.2
	SS + AP*	71.1	89.6	74.2	91.8	76.9	93.3
	CS + TU	71.1	90.1	74.4	91.9	77.0	93.5
	QKD	**71.4**	**90.3**	**74.6**	**92.1**	**77.3**	**93.6**
W3A3	PACT [6]	68.1	88.2	-	-	75.3	92.6
	LQ-Nets [50]	68.2	87.9	71.9	90.2	74.2	91.6
	DSQ [10]	68.7	-	72.5	-	-	-
	QIL [18]	69.2	-	73.1	-	-	-
	BL	69.4	88.5	73.3	90.5	75.3	92.6
	SS + BL	69.5	88.5	73.3	90.6	75.4	92.7
	AP*	69.7	88.7	73.5	90.7	76.0	92.7
	SS + AP*	69.9	88.7	73.7	90.8	76.1	92.8
	CS + TU	70.0	89.4	73.7	91.2	76.2	92.9
	QKD	**70.2**	**89.9**	**73.9**	**91.4**	**76.4**	**93.2**
W2A2	PACT [6]	64.4	85.6	-	-	72.2	90.5
	LQ-Nets [50]	64.9	85.9	69.8	89.1	71.5	90.3
	DSQ [10]	65.2	-	70.0	-	-	-
	QIL [18]	65.7	-	70.6	-	-	-
	BL	66.7	86.7	71.0	89.8	73.3	91.0
	SS + BL	66.8	86.9	71.0	89.9	73.3	91.0
	AP*	66.8	86.9	71.1	89.9	73.4	91.0
	SS + AP*	67.2	87.3	71.3	90.2	73.7	91.2
	CS + TU	67.1	87.2	71.3	90.1	73.6	91.2
	QKD	**67.4**	**87.5**	**71.6**	**90.3**	**73.9**	**91.6**

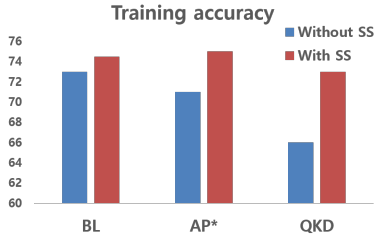


Figure 3: Training accuracy of student network using BL, AP* and QKD at W2A2 on CIFAR-100. Red and blue represent the accuracy with and without Self-studying (SS)

With W2A2, we observe a drastic drop in top-1 accuracy. This was expected since depth-wise convolution layers are known to be highly sensitive to quantization [46, 17]. So, we ran another set of experiments where the weights and input activations of the depth-wise separable layers are quantized to W8A8 whereas the rest of the layers are quantized to W2A2, W3A3 or W4A4. We include these results in the supplementary details.

Table 4: ImageNet performance comparison on architectures with depthwise separable convolutions

Bitwidth	Method	MobileNetV2		EfficientNet-B0	
		top-1	top-5	top-1	top-5
W32A32	-	71.8	90.3	76.3	93.2
W6A6	PACT [6]	71.3	90.0	-	-
	BL	71.2	89.9	74.5	92.3
	SS + BL	71.3	90.0	74.6	92.3
	AP*	71.4	90.0	74.8	92.4
	SS + AP*	71.4	90.0	75.0	92.4
	CS + TU	71.5	90.1	75.1	92.6
	QKD	**71.8**	**90.2**	**75.4**	**92.6**
W4A4	PACT [6]	61.4	83.7	-	-
	DSQ [10]	64.8	-	-	-
	BL	66.1	86.2	71.9	90.4
	SS + BL	66.3	86.3	72.0	90.4
	AP*	66.4	86.4	72.3	90.6
	SS + AP*	66.6	86.5	72.6	90.7
	CS + TU	66.7	86.7	72.8	91.0
	QKD	**67.4**	**87.0**	**73.1**	**91.2**
W3A3	BL	60.1	83.0	67.5	85.8
	SS + BL	61.5	83.4	67.7	85.9
	AP*	60.7	83.2	68.4	86.1
	SS + AP*	62.0	83.5	68.7	86.2
	CS + TU	62.0	83.5	68.8	86.4
	QKD	**62.6**	**84.0**	**69.2**	**86.9**
W2A2	BL	37.7	64.1	43.5	67.5
	SS+BL	42.7	66.2	44.9	67.6
	AP*	39.8	65.3	46.4	68.2
	SS+AP*	43.9	67.3	47.7	68.5
	CS + TU	43.6	66.9	48.0	68.7
	QKD	**45.7**	**68.1**	**50.0**	**69.3**

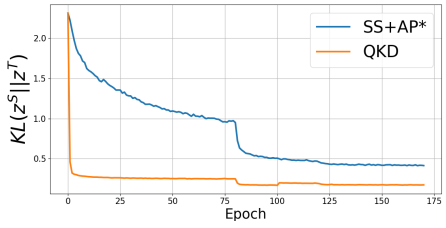


Figure 4: KL-divergence between teacher & student distributions on CIFAR100 with QKD and SS+AP* (SS part is not shown in both because it doesn't involve the teacher network).

4.5. Discussion

Effectiveness of Self-studying Figure 3 shows the training accuracy of the student in the final epoch with and without the Self-Studying phase on the CIFAR-100 dataset. At very low-bit quantization, the model has low representative power and without SS, it gets stuck in a local minima as is reflected from the lower training accuracy. The SS phase helps the student start from a better initialization point and KD then guides it to a better local minima, hence increasing the training accuracy. Note that although training accuracy

Source: https://arxiv.org/pdf/1911.12491
Google Scholar research | 2026-10-07 | Article 10 | Original p. 7 | Orange: discussion; blue: reference

each of these methods on the quantized model accuracy. We show how self-studying is important in alleviating the regularization effect imposed by KD and how co-studying enhances the quantization performance by making the teacher more adaptable and more powerful in terms of accuracy. Overall, with an extensive set of experiments, we show that QKD gives significant performance boost over our baseline quantization-only method and outperforms the existing state-of-the-art approaches. We demonstrate QKD's results on networks with both standard and depthwise separable convolutions and show that we can recover the full-precision accuracy at as low as W3A3 quantization of the ResNet architectures and W6A6 quantization of MobileNetV2.

References

[1] Martín Abadi, Ashish Agarwal, Paul Barham, Eugene Brevdo, Zhifeng Chen, Craig Citro, Greg S. Corrado, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Ian Goodfellow, Andrew Harp, Geoffrey Irving, Michael Isard, Yangqing Jia, Rafal Jozefowicz, Lukasz Kaiser, Manjunath Kudlur, Josh Levenberg, Dan Mané, Rajat Monga, Sherry Moore, Derek Murray, Chris Olah, Mike Schuster, Jonathon Shlens, Benoit Steiner, Ilya Sutskever, Kunal Talwar, Paul Tucker, Vincent Vanhoucke, Vijay Vasudevan, Fernanda Viégas, Oriol Vinyals, Pete Warden, Martin Wattenberg, Martin Wicke, Yuan Yu, and Xiaoqiang Zheng. TensorFlow: Large-scale machine learning on heterogeneous systems, 2015. Software available from tensorflow.org. 4

[2] Yoshua Bengio, Nicholas Léonard, and Aaron Courville. Estimating or propagating gradients through stochastic neurons for conditional computation. *arXiv preprint arXiv:1308.3432*, 2013. 4

[3] Aishwarya Bhandare, Vamsi Sripathi, Deepthi Karkada, Vivek Menon, Sun Choi, Kushal Datta, and Vikram Saletore. Efficient 8-bit quantization of transformer neural machine language translation model. *arXiv preprint arXiv:1906.00532*, 2019. 3

[4] Léon Bottou. Large-scale machine learning with stochastic gradient descent. In *Proceedings of COMPSTAT'2010*, pages 177–186. Springer, 2010. 5

[5] Zhaowei Cai, Xiaodong He, Jian Sun, and Nuno Vasconcelos. Deep learning with low precision by half-wave gaussian quantization. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 5918–5926, 2017. 1

[6] Jungwook Choi, Zhuo Wang, Swagath Venkataramani, Pierce I-Jen Chuang, Vijayalakshmi Srinivasan, and Kailash Gopalakrishnan. Pact: Parameterized clipping activation for quantized neural networks. *arXiv preprint arXiv:1805.06085*, 2018. 1, 2, 6, 7

[7] Matthieu Courbariaux, Yoshua Bengio, and Jean-Pierre David. Binaryconnect: Training deep neural networks with binary weights during propagations. In *Advances in neural information processing systems*, pages 3123–3131, 2015. 2

[8] Steven K. Esser, Jeffrey L. McKinstry, Deepika Bablani, Rathinakumar Appuswamy, and Dharmendra S. Modha. Learned step size quantization, 2019. 2, 3

[9] Jonathan Frankle and Michael Carbin. The lottery ticket hypothesis: Finding sparse, trainable neural networks. *arXiv preprint arXiv:1803.03635*, 2018. 1

[10] Ruihao Gong, Xianglong Liu, Shenghu Jiang, Tianxiang Li, Peng Hu, Jiazhen Lin, Fengwei Yu, and Junjie Yan. Differentiable soft quantization: Bridging full-precision and low-bit neural networks, 2019. 1, 6, 7

[11] Song Han, Huizi Mao, and William J Dally. Deep compression: Compressing deep neural networks with pruning, trained quantization and huffman coding. *arXiv preprint arXiv:1510.00149*, 2015. 1

[12] Song Han, Jeff Pool, John Tran, and William Dally. Learning both weights and connections for efficient neural network. In *Advances in neural information processing systems*, pages 1135–1143, 2015. 1

[13] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778, 2016. 5, 6

[14] Byeongho Heo, Jeesoo Kim, Sangdoo Yun, Hyojin Park, Nojun Kwak, and Jin Young Choi. A comprehensive overhaul of feature distillation. *arXiv preprint arXiv:1904.01866*, 2019. 3, 8

[15] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*, 2015. 1, 2

[16] Itay Hubara, Matthieu Courbariaux, Daniel Soudry, Ran El-Yaniv, and Yoshua Bengio. Binarized neural networks. In *Advances in neural information processing systems*, pages 4107–4115, 2016. 2

[17] Sambhav R Jain, Albert Gural, Michael Wu, and Chris H Dick. Trained quantization thresholds for accurate and efficient fixed-point inference of deep neural networks. *arXiv preprint arXiv:1903.08066*, 2019. 2, 3, 7

[18] Sangil Jung, Changyong Son, Seohyung Lee, Jinwoo Son, Jae-Joon Han, Youngjun Kwak, Sung Ju Hwang, and Changkyu Choi. Learning to quantize deep networks by optimizing quantization intervals with task loss. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 4350–4359, 2019. 1, 2, 5, 6, 7

[19] Jangho Kim, Minsung Hyun, Inseop Chung, and Nojun Kwak. Feature fusion for online mutual knowledge distillation. *arXiv preprint arXiv:1904.09058*, 2019. 3

[20] Jangho Kim, SeongUk Park, and Nojun Kwak. Paraphrasing complex network: Network compression via factor transfer. In *Advances in Neural Information Processing Systems*, pages 2760–2769, 2018. 1, 2, 3, 8

[21] Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014. 5

[22] Guillaume Klein, Yoon Kim, Yuntian Deng, Jean Senellart, and Alexander M. Rush. OpenNMT: Open-source toolkit for neural machine translation. In *Proc. ACL*, 2017. 5

Source: https://arxiv.org/pdf/1911.12491

Google Scholar research | 2026-10-07 | Article 10 | Original p. 9 | Orange: discussion; blue: reference

2021 IEEE/CVF International Conference on Computer Vision (ICCV) | 978-1-6654-2812-5/21/$31.00 ©2021 IEEE | DOI: 10.1109/ICCV48922.2021.00521

Improving Low-Precision Network Quantization via Bin Regularization

Tiantian Han Dong Li Ji Liu Lu Tian Yi Shan

Xilinx Inc., Beijing, China

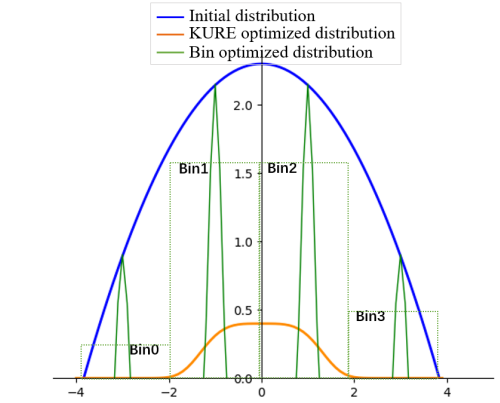
{hantian, dongl, jiliu1, lutian, yishan}@xilinx.com

Abstract

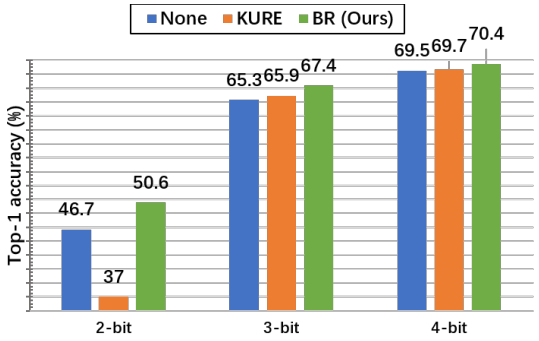
Model quantization is an important mechanism for energy-efficient deployment of deep neural networks on resource-constrained devices by reducing the bit precision of weights and activations. However, it remains challenging to maintain high accuracy as bit precision decreases, especially for low-precision networks (e.g., 2-bit MobileNetV2). Existing methods have been explored to address this problem by minimizing the quantization error or mimicking the data distribution of full-precision networks. In this work, we propose a novel weight regularization algorithm for improving low-precision network quantization. Instead of constraining the overall data distribution, we separably optimize all elements in each quantization bin to be as close to the target quantized value as possible. Such bin regularization (BR) mechanism encourages the weight distribution of each quantization bin to be sharp and approximate to a Dirac delta distribution ideally. Experiments demonstrate that our method achieves consistent improvements over the state-of-the-art quantization-aware training methods for different low-precision networks. Particularly, our bin regularization improves LSQ for 2-bit MobileNetV2 and MobileNetV3-Small by 3.9% and 4.9% top-1 accuracy on ImageNet, respectively.

1. Introduction

Deep Convolutional Neural Networks (CNNs) have achieved remarkable progress in a wide range of applications including computer vision, natural language processing, speech recognition, etc. With the popularity of CNNs, there is an increasing demand for techniques to run networks efficiently on resource-constrained devices (e.g., mobile phones or FPGA). These techniques include network quantization, pruning, manual design of efficient architecture, or neural architecture search (NAS). We focus on network quantization by quantizing the networks to low bit precision in this work. It allows reduced memory footprint, faster inference time and lower power consumption when deploying CNN models on the edge devices.



(a) Target distribution



(b) Top-1 accuracy of low-bit MobileNetV2

Figure 1: (a) Illustration of target distributions by different regularization methods. KURE [31] encourages the overall distribution to be uniform, while our bin regularization (BR) encourages each quantization bin distribution to be sharp. (b) Top-1 accuracy of low-bit MobileNetV2 with different regularization methods.

Usually, the network accuracy decreases and the hardware performance increases as bit precision decreases. Recent network quantization methods have shown promising performance for 8-bit quantization by post-training quanti-

978-1-6654-2812-5/21/$31.00 ©2021 IEEE
DOI 10.1109/ICCV48922.2021.00521

Source: https://ieeexplore.ieee.org/document/9711384 (published print)

Google Scholar research | 2026-10-08 | Article 11 | Original p. 1 | Orange: discussion; blue: reference

zation (PTQ). But PTQ methods tend to suffer from significant performance drop when applying for lower-bit quantization. Quantization-aware training (QAT) has become a common practice to learn a low-bit network to reduce the performance degradation from full-precision to quantized models. Prior methods attempt to minimize the quantization error or mimic the data distribution of full-precision networks. However, the final performance for the target task is still not satisfactory.

In this work, we propose a novel regularization algorithm for improving low-precision network quantization. When quantizing a full-precision network to be an n-bit network, all the floating elements of each convolution layer will be discretized into $m = 2^n$ specific values. In other words, these floating elements are grouped into m quantization bins and all the elements in each bin will be approximately represented by the same target value. We hypothesize that the quantization error will be approaching zero if all the floating elements in each bin are close enough to the target quantized value. Figure 1 (a) illustrates our idea of bin regularization. Compared to constraining the overall distribution of a certain layer to be uniform [31], our bin regularization can be viewed as a fine-grained constrain to encourage the bin distribution to be as sharp as possible. Compared to directly optimizing the overall quantization error (e.g., by L2 loss or KL loss) [8, 15, 22, 24, 35, 37], our bin regularization is expected to reduce the quantization error at the bin level. Experimental results validate the effectiveness of our method and show improved performance over the state-of-the-art methods for low-bit network quantization. Figure 1 (b) shows that our method can yield higher performance than the LSQ baseline and KURE [31] regularization method for quantized MobileNetV2 with different bit widths.

To summarize our contributions, we propose a novel bin regularization algorithm for low-precision network quantization in this work. Different from prior work on constraining the overall data distribution to be uniform, our bin regularization encourages sharp distribution for each quantization bin. The proposed algorithm is easy-to-implement and compatible with the common quantization-aware training paradigm. Experiments demonstrate that our bin regularization achieves consistent performance improvements over the state-of-the-art methods for low-precision network quantization, e.g., surpassing the LSQ baseline for 2-bit MobileNetV2 and MobileNetV3-Small by 3.9% and 4.9% top-1 accuracy on ImageNet, respectively.

2. Related Work

An overview of techniques for quantizing CNNs for efficient inference has been presented in [21]. [21] introduces different quantizer designs and generally classifies quantization methods into post-training quantization (PTQ) and quantization-aware training (QAT) methods.

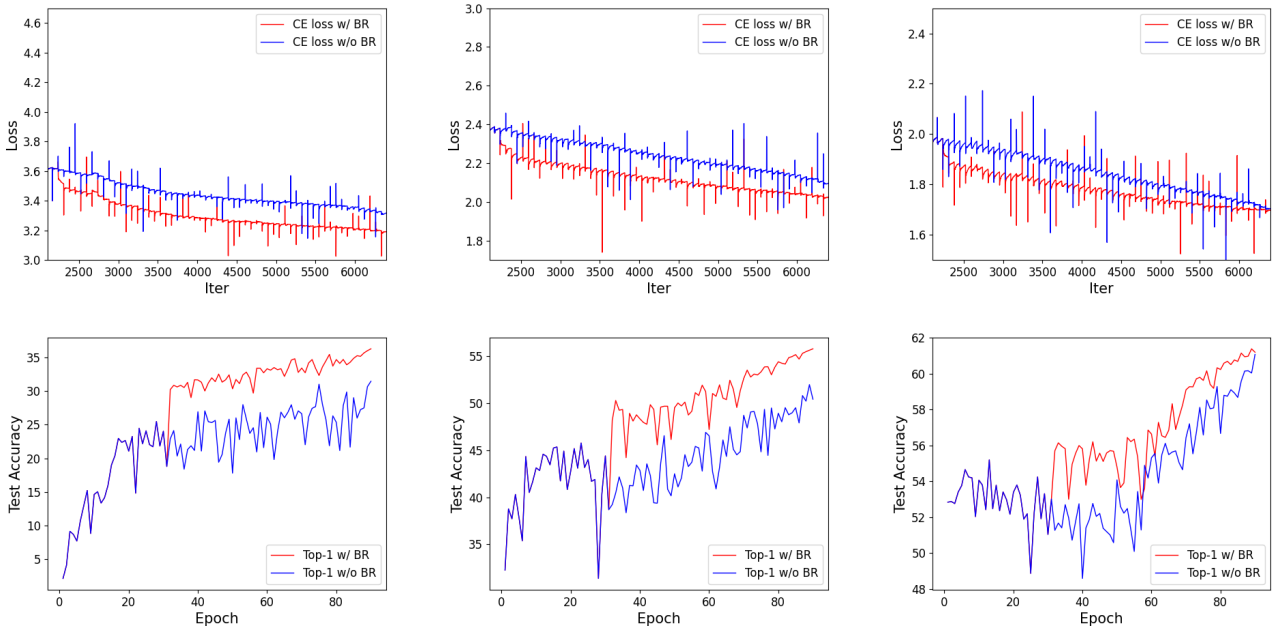
Post-Training Quantization. PTQ methods usually quantize a network without full training and full data [2, 9, 13, 18, 27, 38]. Some methods require a small amount of training data for calibration or fine-tuning to optimize the network parameters [18]. Other methods explore data-free quantization without access to original training data by weight equalization [27] or generative schemes [6, 36]. Although these PTQ methods have shown impressive performance for typical 8-bit quantization, they are not capable of maintaining high accuracy for very low-precision (e.g., 2-bit) networks.

Quantization-Aware Training. QAT methods generally can provide higher accuracy than PTQ methods for low-precision networks with sufficient training on original data. The basic principle is inserting quantization operations into the neural network computational graph and calculating the forward and backward propagation with simulated weights and activations. Prior methods have been explored to optimize the quantization parameters (e.g., clipping value, quantization step size) and approximate the gradients during QAT [5, 7, 12, 14, 19, 20, 41]. Another line of related work has attempted to quantize the networks for improved performance by knowledge distillation [12, 26, 29], progressive quantization [3, 20, 40, 43], learning optimal bit-widths for each layer [11, 15, 32, 34, 42] and adjusting the network architecture to adapt the quantization [24, 25]. Our method is built upon the uniform symmetric quantization framework [21] and orthogonal to those additional quantization schemes.

Regularization for Quantization. Regularization is one of the important optimization techniques to reduce the generalization error in neural network training. Recent work attempts to employ weight regularization for improving network quantization performance [1] or robustness [31]. [8] proposes mean squared quantization error (MSQE) regularization to alleviate the mismatch between high-precision backward and low-precision forward passes. [1] improves the robustness to quantization by penalizing the L1 norm of gradients. [31] proposes Kurtosis regularization (KURE) to learn uniformly distributed weight tensors for improving the robustness to quantization across different bit-widths and parameters of step size. The goal of our work is to improve the final performance of low-precision networks through QAT. Instead of controlling the overall weight distribution to be uniform-like or other shapes, we optimize each quantization bin to be as sharp as possible and approximate to a Dirac delta distribution ideally. We find such a regularization scheme is crucial for maintaining the high accuracy of low-bit quantized networks.

Source: https://ieeexplore.ieee.org/document/9711384 (published print)

Google Scholar research | 2026-10-08 | Article 11 | Original p. 2 | Orange: discussion; blue: reference



(a) 2-bit (b) 3-bit (c) 4-bit

Figure 5: Loss curves and validation accuracies with QAT for different bit widths on MobileNetV3-Small.

Method	W/A	Acc@1 (%)	Acc@5 (%)
Full-precision	32/32	71.8	90.2
LSQ [12]	4/4	69.5	89.2
LSQ [12] + BR (Ours)	4/4	**70.4**	**89.4**
TQT [19]	4/4	63.8	85.2
TQT [19]+L2	4/4	65.8	86.7
TQT [19]+BR(Ours)	4/4	**67.5**	**87.7**

Table 9: Performance comparisons with other QAT methods on MobileNetV2.

4.7. Training Convergence

Figure 5 presents the loss and accuracy curves during training the low-precision MobileNetV2. With our bin regularization method, the task loss (i.e., CE loss) can be further decreased and the final accuracy can be improved.

4.8. Hardware-Friendly Implementation

Most of the existing quantization methods adopt some relaxations in order to maintain the high accuracy of quantized networks (e.g., the first and last layers are not quantized or quantized to 8 bits). We follow [19, 21] to re-implement a hardware-friendly quantization baseline by power-of-2 step size, quantizing the first and last layer with the same low bit-widths, quantizing both inputs and outputs of element-wise addition modules, quantizing bias to 8 bits, merging BN to the previous linear layer during training. Table 9 shows that our bin regularization still can improve significantly over the hardware-friendly quantization baseline (e.g., +3.7% over TQT for 4-bit MobileNetV2). We believe that our work can promote the applications of network quantization for different hardware platforms and accelerators in practical industrial scenarios.

5. Conclusion

In this work, we address the neural network quantization problem, especially for low-precision networks. Instead of constraining the overall data distribution, we propose to encourage each quantization bin to be as sharp as possible. By constraining the data falling into each bin to be close to the target quantized value, we can reduce the quantization error at a fine-grained level. We evaluate our method on multiple popular network architectures and achieve the state-of-the-art accuracy on ImageNet classification for 4-bit, 3-bit, 2-bit quantization. We believe that the proposed idea can inspire new insights for the network quantization problem and promote efficient model deployment on resource-limited devices in practical applications.

References

[1] Milad Alizadeh, Arash Behboodi, Mart van Baalen, Christos Louizos, Tijmen Blankevoort, and Max Welling. Gradient _1

Source: https://ieeexplore.ieee.org/document/9711384 (published print)
Google Scholar research | 2026-10-08 | Article 11 | Original p. 8 | Orange: discussion; blue: reference

regularization for quantization robustness. In *ICLR*, 2020. 2

[2] Ron Banner, Yury Nahshan, Elad Hoffer, and Daniel Soudry. Post-training 4-bit quantization of convolution networks for rapid-deployment. In *NeurIPS*, 2019. 2

[3] Chaim Baskin, Natan Liss, Yoav Chai, Evgenii Zheltonozhskii, Eli Schwartz, Raja Giryes, Avi Mendelson, and Alexander M Bronstein. Nice: Noise injection and clamping estimation for neural network quantization. In *ICLR*, 2019. 2

[4] Yoshua Bengio, Nicholas Léonard, and Aaron Courville. Estimating or propagating gradients through stochastic neurons for conditional computation. *arXiv preprint arXiv:1308.3432*, 2013. 3

[5] Yash Bhalgat, Jinwon Lee, Markus Nagel, Tijmen Blankevoort, and Nojun Kwak. Lsq+: Improving low-bit quantization through learnable offsets and better initialization. In *CVPR Workshops*, 2020. 2

[6] Yaohui Cai, Zhewei Yao, Zhen Dong, Amir Gholami, Michael W Mahoney, and Kurt Keutzer. Zeroq: A novel zero shot quantization framework. In *CVPR*, 2020. 2

[7] Jungwook Choi, Zhuo Wang, Swagath Venkataramani, Pierce I-Jen Chuang, Vijayalakshmi Srinivasan, and Kailash Gopalakrishnan. Pact: Parameterized clipping activation for quantized neural networks. *arXiv preprint arXiv:1805.06085*, 2018. 2, 4, 5

[8] Yoojin Choi, Mostafa El-Khamy, and Jungwon Lee. Learning sparse low-precision neural networks with learnable regularization. *IEEE Access*, 8:96963–96974, 2020. 2

[9] Yoni Choukroun, Eli Kravchik, Fan Yang, and Pavel Kisilev. Low-bit quantization of neural networks for efficient inference. In *ICCV Workshops*, 2019. 2

[10] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. Imagenet: A large-scale hierarchical image database. In *CVPR*, 2009. 4

[11] Ahmed Elthakeb, Prannoy Pilligundla, FatemehSadat Mireshghallah, Amir Yazdanbakhsh, Sicuan Gao, and Hadi Esmaeilzadeh. Releq: An automatic reinforcement learning approach for deep quantization of neural networks. In *NeurIPS Workshop*, 2019. 2

[12] Steven K Esser, Jeffrey L McKinstry, Deepika Bablani, Rathinakumar Appuswamy, and Dharmendra S Modha. Learned step size quantization. In *ICLR*, 2020. 2, 3, 4, 5, 6, 8

[13] Alexander Finkelstein, Uri Almog, and Mark Grobman. Fighting quantization bias with bias. In *CVPR Workshop*, 2019. 2

[14] Ruihao Gong, Xianglong Liu, Shenghu Jiang, Tianxiang Li, Peng Hu, Jiazhen Lin, Fengwei Yu, and Junjie Yan. Differentiable soft quantization: Bridging full-precision and low-bit neural networks. In *ICCV*, 2019. 2, 4, 5

[15] Hai Victor Habi, Roy H Jennings, and Arnon Netzer. Hmq: Hardware friendly mixed precision quantization block for cnns. In *ECCV*, 2020. 2

[16] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *CVPR*, 2016. 4

[17] Andrew Howard, Mark Sandler, Grace Chu, Liang-Chieh Chen, Bo Chen, Mingxing Tan, Weijun Wang, Yukun Zhu, Ruoming Pang, Vijay Vasudevan, et al. Searching for mobilenetv3. In *ICCV*, 2019. 4

[18] Itay Hubara, Yury Nahshan, Yair Hanani, Ron Banner, and Daniel Soudry. Improving post training neural quantization: Layer-wise calibration and integer programming. *arXiv preprint arXiv:2006.10518*, 2020. 2, 4, 5

[19] Sambhav R Jain, Albert Gural, Michael Wu, and Chris H Dick. Trained quantization thresholds for accurate and efficient fixed-point inference of deep neural networks. In *MLSys*, 2020. 2, 8

[20] Sangil Jung, Changyong Son, Seohyung Lee, Jinwoo Son, Jae-Joon Han, Youngjun Kwak, Sung Ju Hwang, and Changkyu Choi. Learning to quantize deep networks by optimizing quantization intervals with task loss. In *CVPR*, 2019. 2, 4

[21] Raghuraman Krishnamoorthi. Quantizing deep convolutional networks for efficient inference: A whitepaper. *arXiv preprint arXiv:1806.08342*, 2018. 2, 8

[22] Yuhang Li, Xin Dong, and Wei Wang. Additive powers-of-two quantization: An efficient non-uniform discretization for neural networks. In *ICLR*, 2020. 2, 4

[23] Yuhang Li, Ruihao Gong, Xu Tan, Yang Yang, Peng Hu, Qi Zhang, Fengwei Yu, Wei Wang, and Shi Gu. Brecq: Pushing the limit of post-training quantization by block reconstruction. In *ICLR*, 2021. 4, 5

[24] Zechun Liu, Zhiqiang Shen, Marios Savvides, and Kwang-Ting Cheng. Reactnet: Towards precise binary neural network with generalized activation functions. In *ECCV*, 2020. 2

[25] Zechun Liu, Baoyuan Wu, Wenhan Luo, Xin Yang, Wei Liu, and Kwang-Ting Cheng. Bi-real net: Enhancing the performance of 1-bit cnns with improved representational capability and advanced training algorithm. In *ECCV*, 2018. 2

[26] Asit Mishra and Debbie Marr. Apprentice: Using knowledge distillation techniques to improve low-precision network accuracy. In *ICLR*, 2018. 2

[27] Markus Nagel, Mart van Baalen, Tijmen Blankevoort, and Max Welling. Data-free quantization through weight equalization and bias correction. In *ICCV*, 2019. 2

[28] Yury Nahshan, Brian Chmiel, Chaim Baskin, Evgenii Zheltonozhskii, Ron Banner, Alex M Bronstein, and Avi Mendelson. Loss aware post-training quantization. *arXiv preprint arXiv:1911.07190*, 2019. 4, 5

[29] Antonio Polino, Razvan Pascanu, and Dan Alistarh. Model compression via distillation and quantization. In *ICLR*, 2018. 2

[30] Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov, and Liang-Chieh Chen. Mobilenetv2: Inverted residuals and linear bottlenecks. In *CVPR*, 2018. 4

[31] Moran Shkolnik, Brian Chmiel, Ron Banner, Gil Shomron, Yuri Nahshan, Alex Bronstein, and Uri Weiser. Robust quantization: One model to rule them all. In *NeurIPS*, 2020. 1, 2, 5

[32] Mart van Baalen, Christos Louizos, Markus Nagel, Rana Ali Amjad, Ying Wang, Tijmen Blankevoort, and Max Welling. Bayesian bits: Unifying quantization and pruning. In *NeurIPS*, 2020. 2

Source: https://ieeexplore.ieee.org/document/9711384 (published print)

Google Scholar research | 2026-10-08 | Article 11 | Original p. 9 | Orange: discussion; blue: reference

Bayesian Bits: Unifying Quantization and Pruning

Mart van Baalen[*], **Christos Louizos**[*], **Markus Nagel, Rana Ali Amjad,**
Ying Wang, Tijmen Blankevoort, Max Welling
Qualcomm AI Research[†]
{mart,clouizos,markusn,ramjad,yinwan,tijmen,mwelling}@qti.qualcomm.com

Abstract

We introduce Bayesian Bits, a practical method for joint mixed precision quantization and pruning through gradient based optimization. Bayesian Bits employs a novel decomposition of the quantization operation, which sequentially considers doubling the bit width. At each new bit width, the residual error between the full precision value and the previously rounded value is quantized. We then decide whether or not to add this quantized residual error for a higher effective bit width and lower quantization noise. By starting with a power-of-two bit width, this decomposition will always produce hardware-friendly configurations, and through an additional 0-bit option, serves as a unified view of pruning and quantization. Bayesian Bits then introduces learnable stochastic gates, which collectively control the bit width of the given tensor. As a result, we can obtain low bit solutions by performing approximate inference over the gates, with prior distributions that encourage most of them to be switched off. We experimentally validate our proposed method on several benchmark datasets and show that we can learn pruned, mixed precision networks that provide a better trade-off between accuracy and efficiency than their static bit width equivalents.

1 Introduction

To reduce the computational cost of neural network inference, quantization and compression techniques are often applied before deploying a model in real life. The former reduces the bit width of weight and activation tensors by quantizing floating-point values onto a regular grid, allowing the use of cheap integer arithmetic, while the latter aims to reduce the total number of multiply-accumulate (MAC) operations required. We refer the reader to [18] and [19] for overviews of hardware-friendly quantization and compression techniques, respectively.

In quantization, the default assumption is that all layers should be quantized to the same bit width. While it has long been understood that low bit width quantization can be achieved by keeping the first and last layers of a network in higher precision [34; 5], recent work [7; 35; 36] has shown that carefully selecting the bit width of each tensor can yield a better trade-off between accuracy and complexity. Since the choice of quantization bit width for one tensor may affect the quantization sensitivity of all other tensors, the choice of bit width cannot be made without regarding the rest of the network.

The number of possible bit width configurations for a neural network is exponential in the number of layers in the network. Therefore, we cannot exhaustively search all possible configurations and pick the best one. Several approaches to learning the quantization bit widths from data have been proposed, either during training [35; 24], or on pre-trained networks [36; 7; 6]. However, these works do not consider the fact that commercially available hardware typically only supports efficient computation

[*]Equal contribution
[†]Qualcomm AI Research is an initiative of Qualcomm Technologies, Inc.

34th Conference on Neural Information Processing Systems (NeurIPS 2020), Vancouver, Canada.

Source: https://proceedings.neurips.cc/paper/2020/file/3f13cf4ddf6fc50c0d39a1d5aeb57dd8-Paper.pdf
Google Scholar research | 2026-10-07 | Article 12 | Original p. 1 | Orange: discussion; blue: reference

The drawback, similarly to [35], of such an approach is that there is no inherent control over the resulting bit widths.

[38] frames the mixed precision search problem as an architecture search. For each layer in their network, the authors maintain a separate weight tensor for each bit width under consideration. A stochastic version of DARTS [22] is then used to learn the optimal bit width setting jointly with the network's weights. [36] model the assignment of bit widths as a reinforcement learning problem. Their agent's observation consists of properties of the current layer, and its action space is the possible bit widths for a layer. The agent receives the validation set accuracy after a short period of fine-tuning as a reward. Besides the reward, the agent receives direct hardware feedback from a target device. This feedback allows the agent to adapt to specific hardware directly, instead of relying on proxy measures.

Learning the scale along with the model parameters for a fixed bit width network was independently introduced by [8] and [15]. Both papers redefine the quantization operation to expose the scale parameter to the learning process, which is then optimized jointly with the network's parameters. Similarly, [5] reformulate the clipping operation such that the range of activations in a network can be learned from data, leading to activation ranges that are more amenable to quantization.

The recursive decomposition introduced in this paper shares similarities with previous work on residual vector quantization [4], in which the residual error of vectors quantized using K-means is itself (recursively) quantized. [9] apply this method to neural network weight compression: the size of a network can be significantly reduced by only storing the centroids of K-means quantized vectors. Our decomposition also shares similarites with [21]. A crucial difference is that Bayesian bits produces valid fixed-point tensors by construction whereas for [21] this is not the case. Concurrent work [40] takes a similar approach to ours. The authors restrict themselves to what is essentially one step of our decomposition (without handling the scales), and to conditional gating during inference on activation tensors. The decomposition is not extended to multiple bit widths.

4 Experiments

To evaluate our proposed method we conduct experiments on image classification tasks. In every model, we quantized all of the weights and activations (besides the output logits) using per-tensor quantization, and handled the batch norm layers as discussed in [18]. We initialized the parameters of the gates to a large value so that the model initially uses its full 32-bit capacity without pruning.

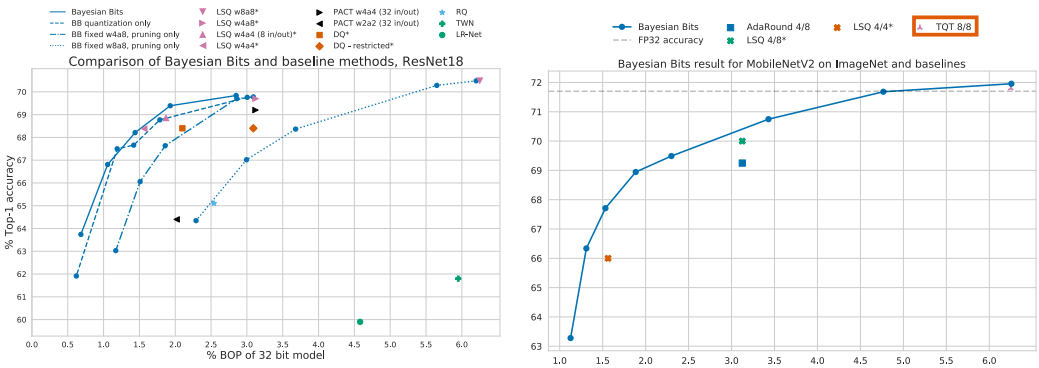
We evaluate our method on two axes: test set classification accuracy, and bit operations (BOPs), as a hardware-agnostic proxy to model complexity. Intuitively the BOP count measures the number of multiplication operations multiplied by the bit widths of the operands. To compute the BOP count we use the formula introduced by [1], but ignore the terms corresponding to addition in the accumulator since its bit width is commonly fixed regardless of operand bit width. We refer the reader to the Appendix for details. We include pruning by performing group sparsity on the output channels of the weight tensors only, as pruning an output channel of the weight tensor corresponds to pruning that specific activation. Output channel group sparsity can often be exploited by hardware [11].

Finally, we set the prior probability $p(z_{jk} = 1 \mid z_{(j/2)k} = 1) = e^{-\mu\lambda_{jk}}$, where λ_{jk} is proportional to the contribution of gate z_{jk} to the total model BOPs, which is a function of both the tensor k and the bit width j, and μ is a (positive) global regularization parameter. See the Appendix for details. It is worth noting that improvements in BOP count may not directly correspond to reduced latency on specific hardware. Instead, these results should be interpreted as an indication that our method can optimize towards a hardware-like target. One could alternatively encourage low memory networks by e.g. using the regularizer from [35] or even allow for hardware aware pruning and quantization by using e.g. latency timings from a hardware simulator, similar to [36].

We compare the results of our proposed approach to literature that considers both static as well as mixed precision architectures. If BOP counts for a specific model are not provided by the original papers, we perform our own BOP computations, and in some cases we run our own baselines to allow for apples-to-apples comparison to alternative methods (details in Appendix). All tensors (weight and activation) in our networks are quantized, and the bit widths of all quantizers in our network are learned, contrary to common practice in literature to keep the first and last layers of the networks in a higher bit width (e.g. [5; 38]).

Source: https://proceedings.neurips.cc/paper/2020/file/3f13cf4ddf6fc50c0d39a1d5aeb57dd8-Paper.pdf

Google Scholar research | 2026-10-07 | Article 12 | Original p. 6 | Orange: discussion; blue: reference



(a) ResNet18 Imagenet results

(b) MobileNet V2 ImageNet results

Figure 2: **Imagenet Results.** (a) Bayesian Bits Imagenet validation accuracy on ResNet18. Bayesian Bits an BB Quantization only use $\mu \in \{0.03, 0.05, 0.07, 0.2\}$. BB pruning only uses $\mu \in \{0.05, 0.2, 0.5, 0.7, 1\}$ The Bayesian Bits results show the mean over 3 training runs. The quantization only and pruning only results show the mean over 2 training results. The BOP count per model is presented in the Appendix. The notation 'wXaY' indicates a fixed bit width architecture with X bit weights and Y bit activations. 'Z in/out' indicates that the weights of the first layer as well as the inputs and weights of the last layer are kept in Z bits. In this plot we additionally compare to PACT [5]. Note that PACT uses 32 bit input and output layers, which negatively affects their BOP count. In the Appendix we compare against a hypothetical setting in which PACT with 8 bit input and output layers yields the same results. * results run by the authors. (b) Bayesian Bits results on MobileNet V2, compared to AdaRound [27], LSQ [8], and TQT [15] * results run by the authors.

parameter μ. Due to differences in experimental setup, we ran our own baseline experiments to obtain results for LSQ [8]. Full details of the differences between the published experiments and ours, as well as experimental setup for baseline experiments can be found in the Appendix.

Besides experiments with combined pruning and quantization, we ran two sets of ablation experiments in which Bayesian Bits was used for pruning a fixed bit width model, and for mixed precision quantization only, without pruning. This was achieved through learning only the 4 bit and higher gates for the quantization only experiment, and only the zero bit gates for the pruning only experiment. In 2a we see that combining pruning with quantization yields superior results.

We provide the tables of the results in the Appendix along with visualizations of the learned architectures. Overall, we observe that Bayesian Bits provides better trade-offs between accuracy and efficiency compared to the baselines. NB: we cannot directly compare our results to those of [36], [7; 6] and [38], for reasons outlined in the Appendix, and therefore omit these results in this paper.

A note on computational cost Bayesian Bits requires the computation of several residual error tensors for each weight and activation tensor in a network. While the computational overhead of these operations is very small compared to the computational overhead of the convolutions and matrix multiplications in a network, we effectively need to store N copies of the model for each forward pass, for N possible quantization levels. To alleviate the resulting memory pressure and allow training with reasonable batch sizes, we use gradient checkpointing [3]. Gradient checkpointing itself incurs extra computational overhead. The resulting total runtime for one ResNet18 experiment, consisting of 30 epochs of training with Bayesian Bits and 10 epochs of fixed-gate fine-tuning, is approximately 70 hours on a single Nvidia TeslaV100. This is a slowdown of approximately 2X compared to 40 epochs of quantization aware training.

4.2.1 Post-training mixed precision

In this experiment, we evaluate the ability of our method to find sensible mixed precision settings by running two sets of experiments on a pre-trained ResNet18 model and a small version of ImageNet. In the first experiment only the values of the gates are learned, while in the second experiment both the values of the gates and the quantization ranges are learned. In both experiments the weights are not updated. We compare this method to an iterative baseline, in which weights and activation

Source: https://proceedings.neurips.cc/paper/2020/file/3f13cf4ddf6fc50c0d39a1d5aeb57dd8-Paper.pdf

Google Scholar research | 2026-10-07 | Article 12 | Original p. 8 | Orange: discussion; blue: reference

neural networks. *arXiv preprint arXiv:1804.10969*, 2018.

[2] Yoshua Bengio, Nicholas Léonard, and Aaron Courville. Estimating or propagating gradients through stochastic neurons for conditional computation. *arXiv preprint arXiv:1308.3432*, 2013.

[3] Tianqi Chen, Bing Xu, Chiyuan Zhang, and Carlos Guestrin. Training deep nets with sublinear memory cost. *arXiv preprint arXiv:1604.06174*, 2016.

[4] Yongjian Chen, Tao Guan, and Cheng Wang. Approximate nearest neighbor search by residual vector quantization. *Sensors*, 10(12):11259–11273, 2010.

[5] Jungwook Choi, Zhuo Wang, Swagath Venkataramani, Pierce I-Jen Chuang, Vijayalakshmi Srinivasan, and Kailash Gopalakrishnan. Pact: Parameterized clipping activation for quantized neural networks. *arXiv preprint arXiv:1805.06085*, 2018.

[6] Zhen Dong, Zhewei Yao, Yaohui Cai, Daiyaan Arfeen, Amir Gholami, Michael W Mahoney, and Kurt Keutzer. Hawq-v2: Hessian aware trace-weighted quantization of neural networks. *arXiv preprint arXiv:1911.03852*, 2019.

[7] Zhen Dong, Zhewei Yao, Amir Gholami, Michael W. Mahoney, and Kurt Keutzer. HAWQ: hessian aware quantization of neural networks with mixed-precision. *International Conference on Computer Vision (ICCV)*, 2019.

[8] Steven K. Esser, Jeffrey L. McKinstry, Bablani Deepika, Rathinakumar Appuswamy, and Dharmendra S. Modha. Learned step size quantization. *International Conference on Learning Representations (ICLR)*, 2020.

[9] Yunchao Gong, Liu Liu, Ming Yang, and Lubomir Bourdev. Compressing deep convolutional networks using vector quantization. *International Conference on Learning Representations (ICLR)*, 2015.

[10] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. *Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.

[11] Yihui He, Xiangyu Zhang, and Jian Sun. Channel pruning for accelerating very deep neural networks. In *Proceedings of the IEEE International Conference on Computer Vision*, pages 1389–1397, 2017.

[12] Geoffrey E Hinton and Drew Van Camp. Keeping the neural networks simple by minimizing the description length of the weights. In *Conference on Computational learning theory (COLT)*, 1993.

[13] Andrey Ignatov, Radu Timofte, Andrei Kulik, Seungsoo Yang, Ke Wang, Felix Baum, Max Wu, Lirong Xu, and Luc Van Gool. Ai benchmark: All about deep learning on smartphones in 2019. *International Conference on Computer Vision (ICCV) Workshops*, 2019.

[14] Benoit Jacob, Skirmantas Kligys, Bo Chen, Menglong Zhu, Matthew Tang, Andrew Howard, Hartwig Adam, and Dmitry Kalenichenko. Quantization and training of neural networks for efficient integer-arithmetic-only inference. *Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018.

[15] Sambhav R. Jain, Albert Gural, Michael Wu, and Chris Dick. Trained uniform quantization for accurate and efficient neural network inference on fixed-point hardware. *arxiv preprint arxiv:1903.08066*, 2019.

[16] Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *International Conference on Learning Representations (ICLR)*, 2015.

[17] Diederik P Kingma and Max Welling. Auto-encoding variational bayes. *International Conference on Learning Representations (ICLR)*, 2014.

[18] Raghuraman Krishnamoorthi. Quantizing deep convolutional networks for efficient inference: A whitepaper. *arXiv preprint arXiv:1806.08342*, 2018.

Source: https://proceedings.neurips.cc/paper/2020/file/3f13cf4ddf6fc50c0d39a1d5aeb57dd8-Paper.pdf

Google Scholar research | 2026-10-07 | Article 12 | Original p. 10 | Orange: discussion; blue: reference

F8NET: FIXED-POINT 8-BIT ONLY MULTIPLICATION FOR NETWORK QUANTIZATION

Qing Jin[1,2*] **Jian Ren**[1] **Richard Zhuang**[1] **Sumant Hanumante**[1] **Zhengang Li**[2]
Zhiyu Chen[3] **Yanzhi Wang**[2] **Kaiyuan Yang**[3] **Sergey Tulyakov**[1]
[1]Snap Inc. [2]Northeastern University, USA [3] Rice University, USA

arXiv:2202.05239v1 [cs.CV] 10 Feb 2022

ABSTRACT

Neural network quantization is a promising compression technique to reduce memory footprint and save energy consumption, potentially leading to real-time inference. However, there is a performance gap between quantized and full-precision models. To reduce it, existing quantization approaches require high-precision INT32 or full-precision multiplication during inference for scaling or dequantization. This introduces a noticeable cost in terms of memory, speed, and required energy. To tackle these issues, we present F8Net, a novel quantization framework consisting of only fixed-point 8-bit multiplication. To derive our method, we first discuss the advantages of fixed-point multiplication with different formats of fixed-point numbers and study the statistical behavior of the associated fixed-point numbers. Second, based on the statistical and algorithmic analysis, we apply different fixed-point formats for weights and activations of different layers. We introduce a novel algorithm to automatically determine the right format for each layer during training. Third, we analyze a previous quantization algorithm—parameterized clipping activation (PACT)—and reformulate it using fixed-point arithmetic. Finally, we unify the recently proposed method for quantization fine-tuning and our fixed-point approach to show the potential of our method. We verify F8Net on ImageNet for MobileNet V1/V2 and ResNet18/50. Our approach achieves comparable and better performance, when compared not only to existing quantization techniques with INT32 multiplication or floating-point arithmetic, but also to the full-precision counterparts, achieving state-of-the-art performance.

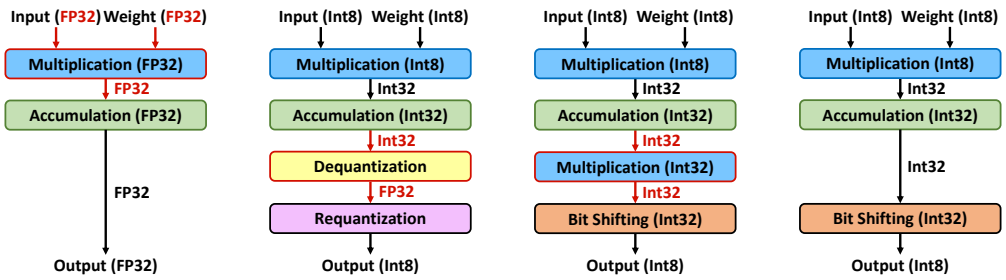
1 INTRODUCTION

Real-time inference on resource-constrained and efficiency-demanding platforms has long been desired and extensively studied in the last decades, resulting in significant improvement on the trade-off between efficiency and accuracy (Han et al., 2015; Liu et al., 2018; Mei et al., 2019; Tanaka et al., 2020; Ma et al., 2020; Mishra et al., 2020; Liang et al., 2021; Jin et al., 2021; Liu et al., 2021). As a model compression technique, quantization is promising compared to other methods, such as network pruning (Tanaka et al., 2020; Li et al., 2021; Ma et al., 2020; 2021a; Yuan et al., 2021) and slimming (Liu et al., 2017; 2018), as it achieves a large compression ratio (Krishnamoorthi, 2018; Nagel et al., 2021) and is computationally beneficial for integer-only hardware. The latter one is especially important because many hardwares (e.g., most brands of DSPs (Ho, 2015; QCOM, 2019)) only support integer or fixed-point arithmetic for accelerated implementation and cannot deploy models with floating-point operations. However, the drop in performance, such as classification accuracy, caused by quantization errors, restricts wide applications of such methods (Zhu et al., 2016).

To address this challenge, many approaches have been proposed, which can be categorized into *simulated* quantization, *integer-only* quantization, and *fixed-point* quantization (Gholami et al., 2021). Fig. 1 shows a comparison between these implementations. For simulated quantization, previous works propose to use trainable clipping-levels (Choi et al., 2018), together with scaling techniques on activations (Jin et al., 2020b) and/or gradients (Esser et al., 2019), to facilitate training for the quantized models. However, some operations in these works, such as batch normalization (BN),

[*]Work done during an internship at Snap Inc. Code is available at https://github.com/snap-research/F8Net.

Source: https://arxiv.org/pdf/2202.05239
Google Scholar research | 2026-10-08 | Article 13 | Original p. 1 | Orange: discussion; blue: reference



(a) Full-precision. (b) Simulated quant. (c) Integer-only quant. (d) Fixed-point quant.

Figure 1: Inspired by Gholami et al. (2021), we show the comparison of full-precision model (presented in (a)) and different quantizations settings: (b) simulated quantization; (c) integer-only quantization; and (d) fixed-point quantization. Note the combination of last two operations in integer-only quantization is termed as dyadic scaling in literature (Yao et al., 2021).

are conducted with full-precision to stabilize training (Jin et al., 2020b; Esser et al., 2019), limiting the practical application of integer-only hardware. Meanwhile, integer-only quantization, where the model inference can be implemented with integer multiplication, addition, and bit shifting, has shown significant progress in recent studies (Jacob et al., 2018; Yao et al., 2021; Kim et al., 2021). Albeit floating-point operations are removed to enable models running on devices with limited support of operation types, INT32 multiplication is still required for these methods. On the other hand, fixed-point quantization, which also applies low-precision logic for arithmetic, does not require INT32 multiplication or integer division. For example, to replace multiplication by bit shifting, Jain et al. (2019) utilize trainable power-of-2 scale factors to quantize the model.

In this work, we adopt fixed-point quantization. Our work differs from previous efforts (Jain et al., 2019) in three major aspects. First, to determine the minimum error quantization threshold, we conduct statistical analysis on fixed-point numbers. Second, we unify parameterized clipping activation (PACT) and fixed-point arithmetic to achieve high performance and high efficiency. Third, we discuss and propose quantization fine-tuning methods for different models. We dub our method as F8Net, as it consists in only **F**ixed-point **8**-bit multiplication employed for **Net**work quantization. We thoroughly study the problem with fixed-point numbers, where only INT8 multiplication is involved, without any INT32 multiplication, neither floating-point nor fixed-point types. Throughout this paper we focus on 8-bit quantization, the most widely supported case for different devices and is typically sufficient for efficiency and performance requirements. Our contribution can be elaborated as follows.

- We show 8-bit fixed-point number is able to represent a wide range of values with negligible relative error, once the format is properly chosen (see Fig. 3 and Fig. 4). This critical characteristic enables fixed-point numbers a much stronger representative capability than integer values.
- We propose a method to determine the fixed-point format, also known as fractional length, for weights and activations using their variance. This is achieved by analyzing the statistical behaviors of fixed-point values of different formats, especially those quantized from random variables with normal distribution of different variances. The analysis reveals the relationship between relative quantization error and variance, which further helps us build an approximated formula to determine the fractional length from the variance.
- We develop a novel training algorithm for fixed-point models by unifying fixed-point quantization and PACT (Choi et al., 2018). Besides, we show the impact of fractional length sharing for residual blocks, which is also important to obtain good performance for quantized models.
- We validate our approach for various models, including MobileNet V1/V2 and ResNet18/50 on ImageNet for image classification, and demonstrate better performance than existing methods that resort to 32-bit multiplication. We also integrate the recent proposed fine-tuning method to train quantized models from pre-trained full-precision models with ours for further verification.

2 Related Work

Quantization is one of the most widely-used techniques for neural network compression (Courbariaux et al., 2015; Han et al., 2015; Zhu et al., 2016; Zhou et al., 2016; 2017; Mishra et al., 2017; Park et al.,

Source: https://arxiv.org/pdf/2202.05239
Google Scholar research | 2026-10-08 | Article 13 | Original p. 2 | Orange: discussion; blue: reference

2017; Banner et al., 2018), with two types of training strategies: Post-Training Quantization directly quantizes a pre-trained full-precision model (He & Cheng, 2018; Nagel et al., 2019; Fang et al., 2020a;b; Garg et al., 2021); Quantization-Aware Training uses training data to optimize quantized models for better performance (Gysel et al., 2018; Esser et al., 2019; Hubara et al., 2020; Tailor et al., 2020). In this work, we focus on the latter one, which is explored in several directions. One area uses uniform-precision quantization where the model shares the same precision (Zhou et al., 2018; Wang et al., 2018; Choukroun et al., 2019; Gong et al., 2019; Langroudi et al., 2019; Jin et al., 2020a; Bhalgat et al., 2020; Chen et al., 2020; Yang et al., 2020; Darvish Rouhani et al., 2020; Oh et al., 2021). Another direction studies mixed-precision that determines bit-width for each layer through search algorithms, aiming at better accuracy-efficiency trade-off (Dong et al., 2019; Wang et al., 2019; Habi et al., 2020; Fu et al., 2020; 2021; Yang & Jin, 2020; Zhao et al., 2021a;b; Ma et al., 2021b). There is also binarization network, which only applies 1-bit (Rastegari et al., 2016; Hubara et al., 2016; Cai et al., 2017; Bulat et al., 2020; Guo et al., 2021). Despite the fact that quantization helps reduce energy consumption and inference latency, it is usually accompanied by performance degradation. To alleviate this problem, several methods are proposed.

One type of effort focuses on simulated quantization. The strategy is to leave some operations, e.g., BN, in full-precision for the stabilized training of quantized models (Choi et al., 2018; Esser et al., 2019; Jin et al., 2020b). Nevertheless, these methods limit the application of the quantized models on resource-demanding hardware, such as DSP, where full-precision arithmetic is not supported for accelerated computing (QCOM, 2019; Ho, 2015). To completely eliminate floating-point operations from the quantized model, integer-only quantization techniques emulate the full-precision multiplication by 32-bit integer multiplication followed by bit shifting (Jacob et al., 2018; Zhu et al., 2020; Wu et al., 2020; Yao et al., 2021; Kim et al., 2021). However, the calculation of INT32 multiplication in these works requires one more operation, which results in extra energy and higher latency (Gholami et al., 2021). In parallel, recent work (Jain et al., 2019) proposes to restrict all scaling factors as power-of-2 values for all weights and activations, which belongs to fixed-point quantization methods (Lin et al., 2016; Jain et al., 2019; Kim & Kim, 2021; Mitschke et al., 2019; Enderich et al., 2019b; Chen et al., 2017; Enderich et al., 2019a; Zhang et al., 2020; Goyal et al., 2021). This enables the model to only incorporate INT8 or even INT4 multiplications, followed by INT32 bit shifting. However, there still a lack of a thorough study of the benefits of using fixed-point arithmetic. Also, the power-of-2 scaling factors are directly determined from the training data without theoretical analysis and guidance. In this work, we give an extensive analysis, especially on the potential and theoretical principle of using fixed-point values for quantized models, and demonstrate that with proper analysis and design, a model quantized with only INT8 multiplication involved is able to achieve comparable and even better performance to the integer-only methods implemented with INT32 multiplication.

3 Analysis of Fixed-Point Representation

In this section, we first introduce the fixed-point multiplication (Smith et al., 1997; Tan & Jiang, 2018) and analyze the distribution of weight from different layers in a well-trained full-precision model (Sec. 3.1). We then investigate the statistical property of fixed-point numbers, and demonstrate the potential of approximating full-precision values by 8-bit fixed-point numbers with different formats (Sec. 3.2). After that, we study the relationship between standard deviation of random variables and the optimal fixed-point format with the smallest quantization error. Finally, we derive an approximated formula relating the standard deviation and fixed-point format, which is verified empirically and employed in our final algorithms (Sec. 3.3).

3.1 Advantages of Fixed-Point Arithmetic

Fixed-point number is characterized by its format, which includes both the word length indicating the whole bit-width of the number and the fractional length (FL) characterizing the range and resolution of the represented values (Smith et al., 1997). Fixed-point arithmetic—especially fixed-point multiplication—is widely utilized for applications in, e.g., digital signal processing (Smith et al., 1997; Tan & Jiang, 2018). Compared with integer or floating-point multiplication, fixed-point multiplication has two major characteristics: First, multiplying two fixed-point numbers is more efficient than multiplying two floating-point numbers, especially on resource-constrained devices such as DSP. Second, it is more powerful than its integer counterpart due to its versatility and the

Source: https://arxiv.org/pdf/2202.05239

Google Scholar research | 2026-10-08 | Article 13 | Original p. 3 | Orange: discussion; blue: reference

using quantized input yet full-precision weight of the convolution layer, and use the output to update the running statistics of BN. In this way, the effective weight to quantize is available. Note there is no backpropagation for this step. *For the second forward*, we quantize the combined effective weight to get the final output of the two layers of convolution and BN and do the backpropagation.

Updating Fractional Length. Different from existing work that directly trains the fractional length (Jain et al., 2019), we define the fractional length for weight on-the-fly during training by inferring from current value of weight, using (1). For the fractional length of activation, we use a buffer to store and update the value with a momentum of 0.1, similar to how to update BN running statistics. Once the fractional lengths are determined after training, we keep them fixed for inference.

4.3 Relating Scaling Factors between Adjacent Layers

As shown in (4), there are still two extra factors during the quantization operation, which we denote as a fix scaling factor η_{fix}:

$$\eta_{\text{fix}} = \frac{2^{\text{FL}}\alpha}{2^{\text{WL}} - 1}. \tag{5}$$

Now α is a trainable parameter with full-precision, which means the fix scaling factor is also in full-precision. To eliminate undesired extra computation, we absorb it into the above-mentioned effective weights for quantization (Sec. 4.2). However, the fix scaling factor occurs twice, one for rescaling after quantization (η_{fix}) and the other for scaling before quantization ($1/\eta_{\text{fix}}$). To completely absorb it, we need to relate two adjacent layers. In fact, for a mapping that includes convolution, BN, and ReLU (more details are shown in Appx. 7.5), we apply PACT quantization to relate the activation between two adjacent layers as:

$$q_i^{(l+1)} = \text{fix_quant}\left(\underbrace{\sum_{j=1}^{n^{(l)}} \frac{\gamma_i^{(l)}}{\sigma_i^{(l)}} \frac{\eta_{\text{fix}}^{(l)}}{\eta_{\text{fix}}^{(l+1)}} W_{ij}^{(l)} q_j^{(l)}}_{\text{Effective Weight}} + \underbrace{\frac{1}{\eta_{\text{fix}}^{(l+1)}} \left(\beta_i^{(l)} - \frac{\gamma_i^{(l)}}{\sigma_i^{(l)}} \mu_i^{(l)} \right)}_{\text{Effective Bias}} \right), \tag{6}$$

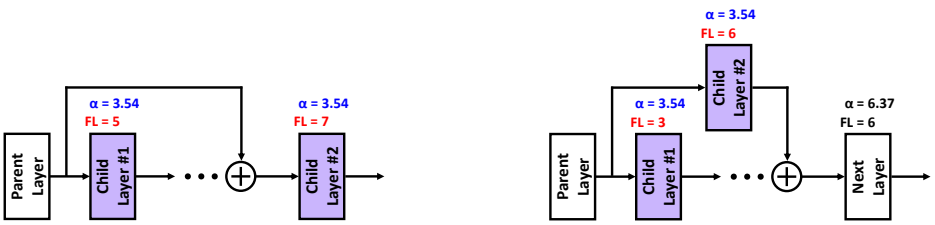
where q is the fixed-point activation, W the full-precision weight of the convolution layer, i and j the spatial indices, n the total number of multiplication, and the superscript (l) indicates the l-th block consisting of convolution and BN. γ, β, σ, μ are the learned weight, bias, running standard deviation, and running mean for the BN layer, respectively. Also, we set $\text{WL} = 8$ for all layers. As can be seen from (6), to obtain the final effective weight for fixed-point quantization, for the l-th Conv-BN block, we need to access the fix scaling factor, or equivalently, the clipping-level α and the activation fractional length FL, from its following $(l+1)$-th block(s). To achieve this, we apply two techniques.

Pre-estimating Fractional Length. As mentioned above, we determine the activation fractional length from its standard deviation. Also, (5) indicates that the fix scaling factor relies on such fractional length for each layer. However, in (6), we need the fix scaling factor from the next layer to determine the effective weight under quantization, which we have not yet updated. Thus, when calculating the effective weights during training, we use the activation fractional length stored in the buffer, instead of the one for quantizing the input of the next layer.

Clipping-Level Sharing. As shown in Fig. 5, for residual blocks, some layers have two following layers (which we also name as child layer). Since we need the fix scaling factor from the child layer to calculate the effective weight for the parent (see (6)), inconsistent fix scaling factors between all children layers will be a problem. To this end, we define one layer as master and force all its siblings to share its clipping-level. In fact, the best way is to share both the clipping-level and the fractional length among siblings, but we find sharing fractional length leads to considerable performance drop, especially for deep models such as MobileNet V2 and ResNet50. This is because the fractional lengths play two roles here: one is for the fix scaling factor, and the other is for the representing region (or equivalently the clipping-level). Using different fractional lengths effectively enables different clipping-levels (although only differ by a factor of power-of-2, see Appx. 7.6), which can be beneficial because the activation scales might vary from layer to layer. Moreover, breaking the constraint of sharing activation fractional length does not introduce much computational cost, as the value only differs in storing format, and typically the values are stored in 32-bit, i.e., the accumulation results are only quantized into 8-bit for multiplication. Note that when computing the effective weight

Source: https://arxiv.org/pdf/2202.05239

Google Scholar research | 2026-10-08 | Article 13 | Original p. 7 | Orange: discussion; blue: reference



(a) ResBlock with direct connection. (b) ResBlock with downsampling.

Figure 5: The illustration of residual connections. For a layer with several layers (named children layers) directly following it, we choose one to be master, and all its sibling layers use the master layer's clipping level. On the other hand, since using different fractional length only cause bit shifting or different fixed-point quantization formats, and the values are stored in 32-bit before quantized into 8-bit, we do not share the fractional formats to allow more degrees of freedom. The two figures show the case of direct residual connection (a) and that with downsampling convolution layer (b).

of the parent layer, we only use the master child's activation fractional length. For effective weight of each child layer and fixed-point quantization on its input, we use its own fractional length.

5 Experiments

In this section, we present our results for various models on ImageNet (Deng et al., 2009) for classification task and compare the results with previous works that focus on quantization-aware training to verify the effectiveness of our method. We show the results for two sets of training. First, we discuss the conventional training method following Jin et al. (2020b). Second, we unify our method with one recent fine-tuning method that quantizes full-precision models with high accuracy (Yao et al., 2021). More detailed experimental settings are described in Appx. 7.1.

Conventional training. We first apply our method using conventional training (Choi et al., 2018; Esser et al., 2019; Jin et al., 2020b; Fu et al., 2021), where the quantized model is trained with the simplest setting as those for full-precision model (more details in Appx. 7.1). To verify the effectiveness of our method, we perform experiments on several models including ResNet18 and MobileNet V1/V2. As shown in Table 1, our method achieves the state-of-the-art results for all models. Additionally, we obtain comparable or even better performance than the full-precision counterparts.

Compared with previous works on simulated quantization (Choi et al., 2018; Park et al., 2018; Esser et al., 2019; Jin et al., 2020b; Fu et al., 2021) that requires full-precision rescaling after INT8 convolution, our approach is not only more efficient but also achieves better performance. On the other hand, compared with previous fixed-point quantization (Jain et al., 2019), our approach gives better results. This might partially due to that our method is based on a more systematic analysis, as explained above in Section 3.3.

Table 1: 8-bit quantization with conventional training for ResNet18 and MobileNet V1/V2b. Following Yao et al. (2021), we abbreviate Integer-Only Quantization as "Int", INT8-Multiplication-Only Quantization as "8-bit", the Baseline Accuracy as "BL", and Top-1 Accuracy as "Top-1". All models are for 8-bit weight and activation quantization. For MobileNet V2, we are using MobileNet V2b version as it is the most typical one.

(a) ResNet18

Method	Int	8-bit	BL	Top-1
Baseline (FP)	✗	✗	70.3	70.3
RVQuant (Park et al., 2018)	✗	✗	69.9	70.0
PACT (Choi et al., 2018)	✗	✗	70.2	69.8
LSQ (Esser et al., 2019)	✗	✗	70.5	71.1
CPT (Fu et al., 2021)	✗	✗	-	69.6
F8Net (ours)	✓	✓	70.3	**71.1**

(b) MobileNet V1

Method	Int	8-bit	BL	Top-1
Baseline (FP)	✗	✗	72.4	72.4
PACT (Choi et al., 2018)	✗	✗	72.1	71.3
TQT (Jain et al., 2019)	✓	✓	71.1	71.1
SAT (Jin et al., 2020b)	✗	✗	71.7	72.6
F8Net (ours)	✓	✓	72.4	**72.8**

(c) MobileNet V2b

Method	Int	8-bit	BL	Top-1
Baseline (FP)	✗	✗	72.7	72.7
PACT (Choi et al., 2018)	✗	✗	72.1	71.7
TQT (Jain et al., 2019)	✓	✓	71.7	71.8
SAT (Jin et al., 2020b)	✗	✗	71.8	72.5
F8Net (ours)	✓	✓	72.7	**72.6**

To further understand the significance of our method, we plot the fractional lengths for weight and activation for each layer. Illustrated in Fig. 2b for MobileNet V2, we find that the fractional lengths for both weight and activation vary from layer to layer. Specifically, for weight quantization, since

Source: https://arxiv.org/pdf/2202.05239
Google Scholar research | 2026-10-08 | Article 13 | Original p. 8 | Orange: discussion; blue: reference

Sambhav R Jain, Albert Gural, Michael Wu, and Chris H Dick. Trained quantization thresholds for accurate and efficient fixed-point inference of deep neural networks. *arXiv preprint arXiv:1903.08066*, 2019.

Qing Jin, Linjie Yang, and Zhenyu Liao. Adabits: Neural network quantization with adaptive bit-widths. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 2146–2156, 2020a.

Qing Jin, Linjie Yang, Zhenyu Liao, and Xiaoning Qian. Neural network quantization with scale-adjusted training. In *BMVC*, 2020b.

Qing Jin, Jian Ren, Oliver J Woodford, Jiazhuo Wang, Geng Yuan, Yanzhi Wang, and Sergey Tulyakov. Teachers do more than teach: Compressing image-to-image models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 13600–13611, 2021.

Sehoon Kim, Amir Gholami, Zhewei Yao, Michael W Mahoney, and Kurt Keutzer. I-bert: Integer-only bert quantization. *arXiv preprint arXiv:2101.01321*, 2021.

Sungrae Kim and Hyun Kim. Zero-centered fixed-point quantization with iterative retraining for deep convolutional neural network-based object detectors. *IEEE Access*, 9:20828–20839, 2021.

Raghuraman Krishnamoorthi. Quantizing deep convolutional networks for efficient inference: A whitepaper. *arXiv preprint arXiv:1806.08342*, 2018.

Hamed F Langroudi, Zachariah Carmichael, David Pastuch, and Dhireesha Kudithipudi. Cheetah: Mixed low-precision hardware & software co-design framework for dnns on the edge. *arXiv preprint arXiv:1908.02386*, 2019.

Zhengang Li, Geng Yuan, Wei Niu, Pu Zhao, Yanyu Li, Yuxuan Cai, Xuan Shen, Zheng Zhan, Zhenglun Kong, Qing Jin, et al. Npas: A compiler-aware framework of unified network pruning and architecture search for beyond real-time mobile acceleration. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 14255–14266, 2021.

Tailin Liang, John Glossner, Lei Wang, Shaobo Shi, and Xiaotong Zhang. Pruning and quantization for deep neural network acceleration: A survey. *Neurocomputing*, 461:370–403, 2021.

Darryl Lin, Sachin Talathi, and Sreekanth Annapureddy. Fixed point quantization of deep convolutional networks. In *International conference on machine learning*, pp. 2849–2858. PMLR, 2016.

Ning Liu, Geng Yuan, Zhengping Che, Xuan Shen, Xiaolong Ma, Qing Jin, Jian Ren, Jian Tang, Sijia Liu, and Yanzhi Wang. Lottery ticket preserves weight correlation: Is it desirable or not? In *International Conference on Machine Learning*, pp. 7011–7020. PMLR, 2021.

Zhuang Liu, Jianguo Li, Zhiqiang Shen, Gao Huang, Shoumeng Yan, and Changshui Zhang. Learning efficient convolutional networks through network slimming. In *Proceedings of the IEEE international conference on computer vision*, pp. 2736–2744, 2017.

Zhuang Liu, Mingjie Sun, Tinghui Zhou, Gao Huang, and Trevor Darrell. Rethinking the value of network pruning. *arXiv preprint arXiv:1810.05270*, 2018.

Ping Luo, Xinjiang Wang, Wenqi Shao, and Zhanglin Peng. Towards understanding regularization in batch normalization. *arXiv preprint arXiv:1809.00846*, 2018.

Xiaolong Ma, Wei Niu, Tianyun Zhang, Sijia Liu, Sheng Lin, Hongjia Li, Wujie Wen, Xiang Chen, Jian Tang, Kaisheng Ma, et al. An image enhancing pattern-based sparsity for real-time inference on mobile devices. In *European Conference on Computer Vision*, pp. 629–645. Springer, 2020.

Xiaolong Ma, Geng Yuan, Xuan Shen, Tianlong Chen, Xuxi Chen, Xiaohan Chen, Ning Liu, Minghai Qin, Sijia Liu, Zhangyang Wang, et al. Sanity checks for lottery tickets: Does your winning ticket really win the jackpot? *Advances in Neural Information Processing Systems*, 34, 2021a.

Yuexiao Ma, Taisong Jin, Xiawu Zheng, Yan Wang, Huixia Li, Guannan Jiang, Wei Zhang, and Rongrong Ji. Ompq: Orthogonal mixed precision quantization. *arXiv preprint arXiv:2109.07865*, 2021b.

Source: https://arxiv.org/pdf/2202.05239

Google Scholar research | 2026-10-08 | Article 13 | Original p. 12 | Orange: discussion; blue: reference

Computers and Electrical Engineering 134 (2026) 111101

Contents lists available at ScienceDirect

Computers and Electrical Engineering

journal homepage: www.elsevier.com/locate/compeleceng

Accelerating configuration of Reconfigurable Intelligent Surfaces through a hardware-enhanced deep learning approach

Rubén Padial-Allué [a,*], Alberto Martín-Martín [a,b], Antonio García [a,c], Encarnación Castillo [a,c], José David Fernández-Rodríguez [d,e], Marcos Baena-Molina [f], Luis Parrilla [a,c]

[a] Department Electronics and Computer Technology, University of Granada, Campus Universitario Fuentenueva, Granada, 18071, Spain
[b] Safran Electronics & Defense, Granada, 18014, Spain
[c] CITIC-UGR, University of Granada, Granada, 18071, Spain
[d] ITIS Software, University of Málaga, c/Arquitecto Francisco Peñalosa 18, Málaga, 29010, Spain
[e] Biomedic Research Institute of Málaga (IBIMA), c/Doctor Miguel Díaz Recio, 28, Málaga, 29010, Spain
[f] Department Signal Theory, Telematics and Communications, CITIC-UGR, University of Granada, Granada, 18071, Spain

ARTICLE INFO

Keywords:
6G
Reconfigurable Intelligent Surfaces
Intelligent reflecting surfaces
Artificial intelligence
Neural networks
Field-Programmable Gate Arrays (FPGA)

ABSTRACT

The dawn of Reconfigurable Intelligent Surfaces (RIS) promises to revolutionize wireless communication by enabling dynamic control of the propagation of electromagnetic waves. However, the practical implementation of RIS demands sophisticated configuration strategies to unlock their full potential. This paper presents a hardware implementation of a Deep Learning (DL) approach to the configuration of RIS, addressing both the complexity and efficiency of the configuration process. A deep learning-based algorithm, designed to optimize the phase shifts of RIS elements and shown to enhance signal quality and system performance, is implemented on two dedicated hardware platforms based on Field-Programmable Gate Arrays (FPGA), which leads to real-time processing and adaptability. This approach leverages the inherent parallelism of FPGAs to accelerate the computationally intensive tasks associated with deep learning inference. As a matter of fact, it is possible to achieve more than 18.000 configurations per second, thus ensuring rapid and efficient RIS configuration with a novel approach within this field.

1. Introduction

The rapid advancement of wireless communication technology necessitates innovative solutions to meet the ever-increasing demands for higher data rates, improved coverage, and energy efficiency. On June 2018, after years of research and development, the initial commercial roll-out of the 5th generation (5G) mobile communication standard was successfully completed. The new generation is already delivering speeds 10 times faster than its predecessor, with up to 20-Gbps peak data rates [1]. According to the Global System for Mobile Communications Association (GSMA), a non-profit organization representing the interest of mobile networks worldwide, 5G is on track to account for 15% of the global mobile industry by 2025 [2].

However, it has become clear during the standardization process of 5G wireless networks that no single enabling technology can meet all the application requirements of 5G [3]. Consequently, researchers have already begun exploring technologies beyond 5G

* Corresponding author.
E-mail address: rubenpadial@ugr.es (R. Padial-Allué).

https://doi.org/10.1016/j.compeleceng.2026.111101
Received 26 November 2025; Received in revised form 2 February 2026; Accepted 4 March 2026

0045-7906/© 2026 The Authors. Published by Elsevier Ltd. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

Source: https://doi.org/10.1016/j.compeleceng.2026.111101
Google Scholar research | 2026-10-08 | Article 14 | Original p. 1 | Orange: discussion; blue: reference

Table 4
Resource usage in the Altera® Arria® 10 SX SoC Development Kit with the Intel® FPGA AI Suite.

		ALM	BlockRAM memory bits	BlockRAM	Variable Precision DSP
Altera® Arria® 10 SX 660	**Total**	251.680	43.642.880	2131	1687
A10 Architecture minArea	**Used**	35.684	5.904.800	360	28
	Usage %	14,18%	13,53%	16,89%	1,66%
A10 Architecture minArea (×5)	**Used**	178.420	29.524.000	1800	140
	Usage %	70,89%	67,65%	84,47%	8,30%
A10 Architecture maxPerformance	**Used**	84.304	19.589.408	1104	1118
	Usage %	33,50%	44,89%	51,81%	66,27%
A10 Architecture Balance	**Used**	47.005	11.995.424	661	166
	Usage %	18,68%	27,49%	31,02%	9,84%

Table 5
Parameters of the AMD DPU IP architectures for RIS accelerator evaluation.

	Precision	*B*	*CPB*
DPU Architecture minArea	INT8	1	32
DPU Architecture maxPerformance	INT8	5	64
DPU Architecture Balanced	INT8	3	32

Finally, the *'A10 Architecture Balance'* takes a balanced approach, with $KVEC = 16$ and $CVEC = 8$. This configuration represents a trade-off between resource usage and performance, making it suitable for applications that require a mix of efficiency and computational power.

It is important to note that the resource usage includes not only the CoreDLA IP but also all necessary peripherals, including a Nios® V [48] instance, which facilitates communications between the CoreDLA IP and the HPS (Hard Processor System) CPU of the Altera® Arria® 10 SoC device. All the architectures run the CoreDLA IP at 100 MHz. Performance results for all test platforms will be discussed in Section 4.

3.2. AMD

Unlike the implementation on the Altera® FPGA, the architecture available for the AMD platform through Vitis™AI only supports INT8 precision. Therefore, for this platform, the implementation has focused on achieving results similar to those obtained with floating-point in the software model, using the different quantization approaches commented in Section 2.4

The PTQ approach was initially tested, as it generally results in minimal accuracy loss. However, in this specific application, PTQ led to unacceptable misclassification of most RIS cells after quantization. This could be due to the high sensitivity of the network to the reduced precision of weights and activations, which caused significant deviations in the learned feature representations. Additionally, the distribution of activation values in certain layers exhibited heavy-tailed behavior, which the uniform quantization scheme used in PTQ could not adequately capture. These factors disrupted the decision boundaries learned during training, leading to severe degradation in the model's ability to distinguish between RIS cell classes accurately.

Consequently, the focus was shifted to the QAT approach. In QAT, the training process is repeated taking into account that the final precision of the weights will be lower. Although the weights are kept in floating-point representation during training, they are quantized after each gradient update [49]. Additionally, the initial weights are not random but are derived from the pre-trained model, ensuring a more stable starting point. For the quantization strategy, a power-of-two scale quantizer with training quantization threshold (TQT) was used [50].

Furthermore, one more change was necessary to achieve an accuracy similar to the original NN. Although the initial model includes the Hard-Swish activation function to facilitate hardware acceleration, as detailed in Section 2.1, retraining achieved better accuracy metrics when the SiLU activation function was used instead. After QAT, the SiLU activation layer was replaced back by the Hard-Swish layer with the VAI Quantizer tool to enable the mentioned hardware acceleration. The retraining process was carried out for 2000 epochs to closely recover the initial accuracy of the various models. Furthermore, the number of epochs was kept consistent across all retraining procedures to ensure a fair comparison of the results among the different models.

Similarly to what was discussed for the Altera® platform, the AMD DPU IP has two different configuration parameters that have a direct impact on performance and resource utilization. The first parameter configures the number of AIE cores used per batch (CPB). The second parameter configures the number of batches (B) that can be processed at the same time, which will determine the number of batch handlers implemented in the FPGA PL.

By adjusting the B and CPB parameters, the amd DPU IP can be tailored into three specific configurations listed in Table 5, each optimized for different priorities such as resource efficiency or performance. The first configuration, referred to as the *'DPU Architecture minArea'*, sets $B = 1$ and $CPB = 32$. This design prioritizes minimal resource usage, enabling the deployment of

Source: https://doi.org/10.1016/j.compeleceng.2026.111101
Google Scholar research | 2026-10-08 | Article 14 | Original p. 9 | Orange: discussion; blue: reference

[44] OpenVINO™ IR format. 2024, https://docs.openvino.ai/2024/documentation/openvino-ir-format.html. [Online; Accessed 05 March 2026].
[45] Intel® FPGA AI Suite IP reference manual. 2024, https://docs.altera.com/r/docs/768974/2023.2/intel-fpga-ai-suite-ip-reference-manual/. [Online; Accessed 05 March 2026].
[46] Quartus® Prime Design Software. 2024, https://www.altera.com/products/development-tools/quartus. [Online; Accessed 05 March 2026].
[47] Vitis™AI Deep-Learning Processor Unit (DPU). 2023, https://docs.amd.com/r/3.0-English/ug1414-vitis-ai/Deep-Learning-Processor-Unit. [Online; Accessed 05 March 2026].
[48] Nios® V Processor for Intel® FPGA. 2026, https://www.altera.com/products/ip/po-3098/nios-v-processors. [Online; Accessed 05 March 2026].
[49] Gholami A, Kim S, Dong Z, Yao Z, Mahoney MW, Keutzer K. A survey of quantization methods for efficient neural network inference. In: Low-power computer vision. Chapman and Hall/CRC; 2022, p. 291–326. http://dx.doi.org/10.1201/9781003162810-13.
[50] Jain S, Gural A, Wu M, Dick C. Trained quantization thresholds for accurate and efficient fixed-point inference of deep neural networks. Proc Mach Learn Syst 2020;2:112–28.

Source: https://doi.org/10.1016/j.compeleceng.2026.111101
Google Scholar research | 2026-10-08 | Article 14 | Original p. 17 | Orange: discussion; blue: reference

FELIX: A Ferroelectric FET Based Low Power Mixed-Signal In-Memory Architecture for DNN Acceleration

TAHA SOLIMAN, Robert Bosch GmbH
NELLIE LALENI, Fraunhofer IPMS, Center Nanoelectronic Technologies
TOBIAS KIRCHNER, Robert Bosch GmbH
FRANZ MÜLLER, ASHISH SHRIVASTAVA, and THOMAS KÄMPFE, Fraunhofer IPMS, Center Nanoelectronic Technologies
ANDRE GUNTORO, Robert Bosch GmbH
NORBERT WEHN, University of Kaiserslautern

Today, a large number of applications depend on deep neural networks (DNN) to process data and perform complicated tasks at restricted power and latency specifications. Therefore, processing-in-memory (PIM) platforms are actively explored as a promising approach to improve the throughput and the energy efficiency of DNN computing systems. Several PIM architectures adopt resistive non-volatile memories as their main unit to build crossbar-based accelerators for DNN inference. However, these structures suffer from several drawbacks such as reliability, low accuracy, large ADCs/DACs power consumption and area, high write energy, and so on. In this article, we present a new mixed-signal in-memory architecture based on the bit-decomposition of the multiply and accumulate (MAC) operations. Our in-memory inference architecture uses a single FeFET as a non-volatile memory cell. Compared to the prior work, this system architecture provides a high level of parallelism while using only 3-bit ADCs. Also, it eliminates the need for any DAC. In addition, we provide flexibility and a very high utilization efficiency even for varying tasks and loads. Simulations demonstrate that we outperform state-of-the-art efficiencies with 36.5 TOPS/W and can pack 2.05 TOPS with 8-bit activation and 4-bit weight precision in an area of 4.9 mm^2 using 22 nm FDSOI technology. Employing binary operation, we obtain 1169 TOPS/W and over 261 TOPS/W/mm^2 on system level.

CCS Concepts: • **Hardware → Analog and mixed-signal circuits**; • **Computer systems organization → Neural networks;**

Additional Key Words and Phrases: FeFET crossbar array, in-memory computation, convolution neural networks, mixed-signal processing, bit-decomposition, current-mode ADC

This work received funding within the ECSEL Joint Undertaking project TEMPO in collaboration with the European Union's H2020 Framework Program (H2020/2014-2020) and National Authorities, under grant agreement number 826655 and was partially funded by the Carl Zeiss Foundation under the grant Sustainable Embedded AI.

Authors' addresses: T. Soliman, T. Kirchner, and A. Guntoro, Robert Bosch GmbH, Renningen 71272, Germany; emails: {taha.soliman, tobias.kirchner, andre.guntoro}@de.bosch.com; N. Laleni, F. Müller, A. Shrivastava, and T. Kämpfe, Fraunhofer IPMS, Center Nanoelectronic Technologies, Dresden, Germany; emails: {nelli.laleni, franz.mueller, ashish.shrivastava, thomas.kaempfe}@ipms.fraunhofer.de; N. Wehn, University of Kaiserslautern, Kaiserslautern 67663, Germany; email: wehn@eit.uni-kl.de.



This work is licensed under a Creative Commons Attribution International 4.0 License.

© 2022 Copyright held by the owner/author(s). Publication rights licensed to ACM.

1539-9087/2022/10-ART84
https://doi.org/10.1145/3529760

Source: https://doi.org/10.1145/3529760
Google Scholar research | 2026-10-08 | Article 15 | Original p. 1 | Orange: discussion; blue: reference

Table 2. Design Parameters for the System Prototype

FELIX System at 500 MHz				
Component	**Params**	**Value**	**Power [mW]**	**Area [mm^2]**
CS	Size num/PE	256 b 256	0.017 uW	17.42 um^2
ADC	Resolution Freq. num/PE	3-bit 1 GSps 16	0.31 uW	8.54 um^2
1 PE Total			0.02468	0.0046
1 Cluster Total	num. PE/Cluster	8	0.341	0.037
Tile Module **1 Tile Total**	num. Cluster/Tile	4	0.089 1.59	0.000289 0.148
Buffers Control unit	Size	1 MB	5.12 0.082	0.131 0.000389
Chip total	num. Tile/Chip	32	**56.1**	**4.9**

as it presents a good balance between the benchmarks' accuracy and system performance. We use a batch size of 1 for the selected benchmarks to represent worst case of parallelization from an input perspective as our architecture targets low power inference applications. We benchmark our system with Squeeznet [16] for Imagenet [12], Resnet18 [15] for Imagenet, Resnet20 [15] for CIFAR10 [21], MobilenetV2 [35] for CIFAR10, Dialated model [45] for Cityscapes [9], and LeNet-5 [23] for MNIST [13]. These benchmarks represent different computational and memory loads as shown in Table 4.

Finally, we built a mapping framework based on Section 5 which is responsible for defining the storage unit for weights of different benchmarks. Also, we built a simulation framework based on the different system estimations to measure system utilization and performance across different benchmarks.

6.2 Benchmarks Accuracy

Since our architecture performance depends on the precision of the inferred networks, we tested the applicability of the usage of 8-bit/4-bit quantization (input/weight) based on the optimizations presented in [17]. We measured the classification accuracy over the test dataset in the case of (CIFAR10 and MNIST), and over the validation dataset when using ImageNet. For the dilated model, the Mean Intersection Over Union is reported as the CNN accuracy over the validation dataset.

As shown in Table 3, compared to the floating-point (Top-1) accuracy the 8-bit/8-bit maintains the accuracy with a maximum 0.6% accuracy loss. In the case of 8-bit/4-bit, we were still able to maintain accuracy with a maximum loss of 2%. Despite using highly optimized quantization (8-bit/4-bit), the system FeFET variations, and the ADC leakage, our architecture maintains networks' accuracy with almost no accuracy loss (<0.2%). In the following section, it can be observed how such quantization optimization can boost the system performance and use the advantage of flexible precision the system is offering.

6.3 Performance Analysis

In this section, we benchmark our architecture across the different presented networks. We focus also on how our system can achieve high average utilization across different computational loads to maximize the performance in comparison to the peak performance.

Source: https://doi.org/10.1145/3529760

Google Scholar research | 2026-10-08 | Article 15 | Original p. 17 | Orange: discussion; blue: reference

and reliability. *IEEE Transactions on Electron Devices* 67, 7 (2020), 2981–2987. DOI: https://doi.org/10.1109/TED.2020.2995781

[2] Stefano Ambrogio, Pritish Narayanan, Hsinyu Tsai, Robert M. Shelby, Irem Boybat, Carmelo di Nolfo, Severin Sidler, Massimo Giordano, Martina Bodini, Nathan C. P. Farinha, B. Killeen, C. Cheng, Y. Jaoudi, and G. W. Burr. 2018. Equivalent-accuracy accelerated neural-network training using analogue memory. *Nature* 558, 7708 (2018), 60–67.

[3] K. Ando, K. Ueyoshi, K. Orimo, H. Yonekawa, S. Sato, H. Nakahara, S. Takamaeda-Yamazaki, M. Ikebe, T. Asai, T. Kuroda, and M. Motomura. 2018. BRein memory: A single-chip binary/ternary reconfigurable in-memory deep neural network accelerator achieving 1.4 TOPS at 0.6 W. *IEEE Journal of Solid-State Circuits* 53, 4 (2018), 983–994.

[4] Shaahin Angizi, Zhezhi He, and Deliang Fan. 2018. DIMA: A depthwise CNN in-memory accelerator. In *Proceedings of the 2018 IEEE/ACM International Conference on Computer-Aided Design*. 1–8. DOI: https://doi.org/10.1145/3240765.3240799

[5] S. Beyer, S. Dünkel, M. Trentzsch, J. Müller, A. Hellmich, D. Utess, J. Paul, D. Kleimaier, J. Pellerin, S. Müller, J. Ocker, A. Benoist, H. Zhou, M. Mennenga, M. Schuster, F. Tassan, M. Noack, A. Pourkeramati, F. Müller, M. Lederer, T. Ali, R. Hoffmann, T. Kämpfe, K. Seidel, H. Mulaosmanovic, E. T. Breyer, T. Mikolajick, and S. Slesazeck. 2020. FeFET: A versatile CMOS compatible device with game-changing potential. In *Proceedings of the 2020 IEEE International Memory Workshop*. IEEE, 1–4.

[6] T. S. Böscke, J. Müller, D. Bräuhaus, U. Schröder, and U. Böttger. 2011. Ferroelectricity in hafnium oxide thin films. *Applied Physics Letters* 99, 10 (2011), 102903.

[7] Y. Cai, T. Tang, L. Xia, B. Li, Y. Wang, and H. Yang. 2020. Low bit-width convolutional neural network on RRAM. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 39, 7 (2020), 1414–1427.

[8] X. Chen, X. Yin, M. Niemier, and X. S. Hu. 2018. Design and optimization of FeFET-based crossbars for binary convolution neural networks. In *Proceedings of the 2018 Design, Automation Test in Europe Conference Exhibition*. 1205–1210.

[9] Marius Cordts, Mohamed Omran, Sebastian Ramos, Timo Rehfeld, Markus Enzweiler, Rodrigo Benenson, Uwe Franke, Stefan Roth, and Bernt Schiele. 2016. *The cityscapes dataset for semantic urban scene understanding*. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*.

[10] S. Cosemans, B. Verhoef, J. Doevenspeck, I. A. Papistas, F. Catthoor, P. Debacker, A. Mallik, and D. Verkest. 2019. Towards 10000TOPS/W DNN inference with analog in-memory computing – a circuit blueprint, device options and requirements. In *Proceedings of the 2019 IEEE International Electron Devices Meeting*. 22.2.1–22.2.4. DOI: https://doi.org/10.1109/IEDM19573.2019.8993599

[11] Cecilia De la Parra, Andre Guntoro, and Akash Kumar. 2020. ProxSim: GPU-based simulation framework for cross-layer approximate DNN optimization. In *Proceedings of the 2020 Design, Automation Test in Europe Conference Exhibition*. 1193–1198. DOI: https://doi.org/10.23919/DATE48585.2020.9116476

[12] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. 2009. ImageNet: A large-scale hierarchical image database. In *Proceedings of the 2009 IEEE Conference on Computer Vision and Pattern Recognition*.

[13] L. Deng. 2012. The MNIST database of handwritten digit images for machine learning research [Best of the Web]. *IEEE Signal Processing Magazine* 29, 6 (2012), 141–142. DOI: https://doi.org/10.1109/MSP.2012.2211477

[14] S. Dünkel, M. Trentzsch, R. Richter, P. Moll, C. Fuchs, O. Gehring, M. Majer, S. Wittek, B. Müller, T. Melde, H. Mulaosmanovic, S. Slesazeck, S. Müller, J. Ocker, M. Noack, D. LÃűhr, P. Polakowski, J. Müller, T. Mikolajick, J. HÃűntschel, B. Rice, J. Pellerin, and S. Beyer. 2017. A FeFET based super-low-power ultra-fast embedded NVM technology for 22nm FDSOI and beyond. In *Proceedings of the 2017 IEEE International Electron Devices Meeting*. IEEE, 19.7.1–19.7.4.

[15] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2016. Deep residual learning for image recognition. In *Proceedings of the 2016 IEEE Conference on Computer Vision and Pattern Recognition*. IEEE.

[16] Forrest N. Iandola, Song Han, Matthew W. Moskewicz, Khalid Ashraf, William J. Dally, and Kurt Keutzer. 2016. SqueezeNet: AlexNet-level accuracy with 50× fewer parameters and <0.5MB model size. arXiv:1602.07360. Retrieved from https://arxiv.org/abs/1602.07360.

[17] Sambhav R. Jain, Albert Gural, Michael Wu, and Chris Dick. 2019. Trained Quantization Thresholds for Accurate and Efficient Fixed-Point Inference of Deep Neural Networks. arXiv:1903.08066. Retrieved from https://arxiv.org/abs/1903.08066.

[18] P. Judd, J. Albericio, and A. Moshovos. 2017. Stripes: Bit-serial deep neural network computing. *IEEE Computer Architecture Letters* 16, 1 (2017), 80–83.

[19] A. Keshavarzi, K. Ni, W. Van Den Hoek, S. Datta, and A. Raychowdhury. 2020. FerroElectronics for edge intelligence. *IEEE Micro* 40, 6 (2020), 33–48. DOI: https://doi.org/10.1109/MM.2020.3026667

[20] H. Kim, Q. Chen, T. Yoo, T. T. Kim, and B. Kim. 2019. A 1-16b precision reconfigurable digital in-memory computing macro featuring column-MAC architecture and bit-serial computation. In *Proceedings of the ESSCIRC 2019 - IEEE 45th European Solid State Circuits Conference*. 345–348.

Source: https://doi.org/10.1145/3529760

Google Scholar research | 2026-10-08 | Article 15 | Original p. 23 | Orange: discussion; blue: reference

arXiv:2007.09952v1 [cs.LG] 20 Jul 2020

HMQ: Hardware Friendly Mixed Precision Quantization Block for CNNs

Hai Victor Habi, Roy H. Jennings, and Arnon Netzer

Sony Semiconductor Israel
{hai.habi, roy.jennings, arnon.netzer}@sony.com

July 21, 2020

Abstract

Recent work in network quantization produced state-of-the-art results using mixed precision quantization. An imperative requirement for many efficient edge device hardware implementations is that their quantizers are uniform and with power-of-two thresholds. In this work, we introduce the Hardware Friendly Mixed Precision Quantization Block (HMQ) in order to meet this requirement. The HMQ is a mixed precision quantization block that repurposes the Gumbel-Softmax estimator into a smooth estimator of a pair of quantization parameters, namely, bit-width and threshold. HMQs use this to search over a finite space of quantization schemes. Empirically, we apply HMQs to quantize classification models trained on CIFAR10 and ImageNet. For ImageNet, we quantize four different architectures and show that, in spite of the added restrictions to our quantization scheme, we achieve competitive and, in some cases, state-of-the-art results.

1 Introduction

In recent years, convolutional neural networks (CNNs) produced state-of-the-art results in many computer vision tasks including image classification [14, 17, 21, 22, 38, 39], object detection [29, 36, 40], semantic segmentation [31, 37], etc. Deploying these models on embedded devices is a challenging task due to limitations on available memory, computational power and power consumption. Many works address these issues using different methods. These include pruning [16, 45, 47], efficient neural architecture design [14, 21, 24, 38], hardware and CNN co-design [14, 20, 43] and quantization [6, 13, 15, 23, 24, 46].

In this work, we focus on quantization, an approach in which the model is compressed by reducing the bit-widths of weights and activations. Besides re-

The code of this work is available in https://github.com/sony-si/ai-research.

Source: https://arxiv.org/pdf/2007.09952

Google Scholar research | 2026-10-07 | Article 16 | Original p. 1 | Orange: discussion; blue: reference

duction in memory requirements, depending on the specific hardware, quantization usually also results in the reduction of both latency and power consumption. The challenge of quantization is to reduce the model size without compromising its performance. For high compression rates, this is usually achieved by *fine-tuning* a pre-trained model for quantization. In addition, recent work in quantization focused on making quantizers more *hardware friendly* (amenable to deployment on embedded devices) by restricting quantization schemes to be: per-tensor, uniform, symmetric and with thresholds that are powers of two [24, 41].

Recently, *mixed-precision* quantization was studied in [12, 41, 42, 44]. In these works, the bit-widths of weights and activations are not equal across the model and are learned during some optimization process. In [42], reinforcement learning is used, which requires the training of an agent that decides the bit-width of each layer. In [44], neural architecture search is used, which implies duplication of nodes in the network and that the size of the model grows proportionally to the size of the search space of bit-widths. Both of these methods limit the bit-width search space because of their computational cost. In [12], the bit-widths are not searched during training, but rather, this method relies on the relationship between the layer's Hessian and its sensitivity to quantization.

An imperative requirement for many efficient edge device hardware implementations is that their quantizers are symmetric, uniform and with power-of-two thresholds (see [24]). This removes the cost of special handling of zero points and real value scale factors. In this work, we introduce a novel quantization block we call the *Hardware Friendly Mixed Precision Quantization Block* (HMQ) that is designed to search over a finite set of quantization schemes that meet this requirement. HMQs utilize the Gumbel-Softmax estimator [25] in order to optimize over a categorical distribution whose samples correspond to quantization scheme parameters.

We propose a method, based on HMQs, in which both the bit-width and the quantizer's threshold are searched simultaneously. We present state-of-the-art results on MobileNetV1, MobileNetV2 and ResNet-50 in most cases, in spite of the hardware friendly restriction applied to the quantization schemes. Additionally, we present the first (that we know of) mixed precision quantization results of EfficientNet-B0. In particular, our contributions are the following:

- We introduce HMQ, a novel, hardware friendly, mixed precision quantization block which enables a simple and efficient search for quantization parameters.
- We present an optimization method, based on HMQs, for mixed precision quantization in which we search simultaneously for both the bit-width and the threshold of each quantizer.
- We present competitive and, in most cases, state-of-the-art results using our method to quantize ResNet-50, EfficientNet-B0, MobileNetV1 and MobileNetV2 classification models on ImageNet.

Source: https://arxiv.org/pdf/2007.09952

Google Scholar research | 2026-10-07 | Article 16 | Original p. 2 | Orange: discussion; blue: reference

2 Related Work

Quantization lies within an active area of research that tries to reduce memory requirements, power consumption and inference latencies of neural networks. These works use techniques such as pruning, efficient network architectures and distillation (see e.g. [7, 14, 15, 18, 19, 21, 30, 34, 35, 38, 39, 48]). Quantization is a key method in this area of research which compresses and accelerates the model by reducing the number of bits used to represent model weights and activations.

Quantization. Quantization techniques can be roughly divided into two families: post-training quantization techniques and quantization-aware training techniques. In post-training quantization techniques, a trained model is quantized without retraining the model (see e.g. [1, 5]). In quantization-aware training techniques, a model undergoes an optimization process during which the model is quantized. A key challenge in this area of research, is to compress the model without significant degradation to its accuracy. Post-training techniques suffer from a higher degradation to accuracy, especially for high compression rates.

Since the gradient of quantization functions is zero almost everywhere, most quantization-aware training techniques use the straight through estimator (STE) [4] for the estimation of the gradients of quantization functions. These techniques mostly differ in their choice of quantizers, the quantizers' parametrization (thresholds, bit-widths, step size, etc.) and their training procedure. During training, the network weights are usually stored in full-precision and are quantized before they are used in feed-forward. The full-precision weights are then updated via back-propagation. Uniform quantizers are an important family of quantizers that have several benefits from a hardware point-of-view (see e.g. [13, 24, 41]). Non-uniform quantizers include clustering, logarithmic quantization and others (see e.g. [3, 33, 46, 49]).

Mixed precision. Recent works on quantization produced state-of-the-art results using mixed precision quantization, that is, quantization in which the bit-widths are not constant across the model (weights and activations). In [42], reinforcement learning is used to determine bit-widths. In [12], second order gradient information is used to determine bit-widths. More precisely, the bit-widths are selected by ordering the network layers using this information. In [41], bit-widths are determined by learnable parameters whose gradients are estimated using STE. This work focuses on the choice of parametrization of the quantizers and shows that the threshold (dynamic range) and step size are preferable over parametrizations that use bit-widths explicitly.

In [44], a mixed precision quantization-aware training technique is proposed where the bit-widths search is converted into a network architecture search (based on [27]). More precisely, in this solution, the search space of all possible quantization schemes is, in fact, a search for a sub-graph in a super-net. The disadvantage of this approach, is that the size of the super net grows substantially with every optional quantization edge/path that is added to the super

Source: https://arxiv.org/pdf/2007.09952

Google Scholar research | 2026-10-07 | Article 16 | Original p. 3 | Orange: discussion; blue: reference

net. In practice, this limits, the architecture search space. Moreover, this work deals with bit-widths and thresholds as two separate problems where thresholds follow the solution in [8].

3 The HMQ Block

The *Hardware Friendly Mixed Precision Quantization Block* (HMQ) is a network block that learns, via standard SGD, a uniform and symmetric quantization scheme. The scheme is parametrized by a pair (t, b) of threshold t and bit-width b. During training, an HMQ searches for (t, b) over a finite space $\mathrm{T}\times\mathrm{B} \subseteq \mathbb{R}^+\times\mathbb{N}$. In this work, we make HMQs "hardware friendly" by also forcing their thresholds to be powers of two. We do this by restricting

$$\mathrm{T} = \{2^M, 2^{M-1}, \ldots, 2^{M-8}\} \tag{1}$$

where $M \in \mathbb{Z}$ is an integer we configure per HMQ (see Section 4).

The step size Δ of a uniform quantization scheme is the (constant) gap between any two adjacent quantization points. Δ is parametrized by (t, b) differently for a signed quantizer, where $\Delta = \frac{2t}{2^b}$, and an unsigned one, where $\Delta = \frac{t}{2^b}$. Note that Δ ties the bit-width and threshold values into a single parameter but Δ is not uniquely defined by them. The definition of the quantizer that we use in this work is similar to the one in [24]. The signed version Q^{s} of a quantizer of an HMQ is defined as follows:

$$Q^{\mathrm{s}}(x, \Delta, t) = \mathrm{clip}\left(\Delta \cdot \left\lceil \frac{\boldsymbol{x}}{\Delta} \right\rfloor, -(t-\Delta), t\right) \tag{2}$$

where $\mathrm{clip}\,(x, a, b) = \min(\max(x, a), b)$ and $\lceil x \rfloor$ is the rounding function. Similarly, the unsigned version Q^{us} is defined as follows:

$$Q^{\mathrm{us}}(x, \Delta, t) = \mathrm{clip}\left(\Delta \cdot \left\lceil \frac{\boldsymbol{x}}{\Delta} \right\rfloor, 0, t-\Delta\right). \tag{3}$$

In the rest of this section we assume that the quantizer Q of an HMQ is signed, but it applies to both signed and unsigned quantizers.

In order to search over a discrete set, the HMQ represents each pair in $T \times B$ as a sample of a categorical random variable of the Gumbel-Softmax estimator (see [25, 32]). This enables the HMQ to search for a pair of threshold and bit-width. The Gumbel-Softmax is a continuous distribution on the simplex that approximates categorical samples. In our case, we use this approximation as a joint discreet probability distribution of thresholds and bit-widths $P_{\mathrm{T,B}}(\mathrm{T}{=}t, \mathrm{B}{=}b|g_{t,b})$ on $\mathrm{T}\times\mathrm{B}$:

$$P_{\mathrm{T,B}}(\mathrm{T}=t, \mathrm{B}=b|g_{t,b}) = \frac{\exp(\frac{\log(\hat{\pi}_{t,b})+g_{t,b}}{\tau})}{\sum_{t'\in\mathrm{T}}\sum_{b'\in\mathrm{B}}\exp(\frac{\log(\hat{\pi}_{t',b'})+g_{t',b'}}{\tau})} \tag{4}$$

Source: https://arxiv.org/pdf/2007.09952

Google Scholar research | 2026-10-07 | Article 16 | Original p. 4 | Orange: discussion; blue: reference

[20] A. Howard, M. Sandler, G. Chu, L.-C. Chen, B. Chen, M. Tan, W. Wang, Y. Zhu, R. Pang, V. Vasudevan, et al. Searching for mobilenetv3. In *Proceedings of the IEEE International Conference on Computer Vision*, pages 1314–1324, 2019.

[21] A. G. Howard, M. Zhu, B. Chen, D. Kalenichenko, W. Wang, T. Weyand, M. Andreetto, and H. Adam. Mobilenets: Efficient convolutional neural networks for mobile vision applications. *arXiv preprint arXiv:1704.04861*, 2017.

[22] J. Hu, L. Shen, and G. Sun. Squeeze-and-excitation networks. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 7132–7141, 2018.

[23] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko. Quantization and training of neural networks for efficient integer-arithmetic-only inference. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 2704–2713, 2018.

[24] S. R. Jain, A. Gural, M. Wu, and C. Dick. Trained quantization thresholds for accurate and efficient fixed-point inference of deep neural networks. *arXiv preprint arXiv:1903.08066*, 2019.

[25] E. Jang, S. Gu, and B. Poole. Categorical reparametrization with gumble-softmax. In *International Conference on Learning Representations (ICLR 2017)*. OpenReview. net, 2017.

[26] A. Krizhevsky, G. Hinton, et al. Learning multiple layers of features from tiny images. Technical report, Citeseer, 2009.

[27] H. Liu, K. Simonyan, and Y. Yang. DARTS: Differentiable architecture search. In *International Conference on Learning Representations*, 2019.

[28] L. Liu, H. Jiang, P. He, W. Chen, X. Liu, J. Gao, and J. Han. On the variance of the adaptive learning rate and beyond. In *International Conference on Learning Representations*, 2020.

[29] W. Liu, D. Anguelov, D. Erhan, C. Szegedy, S. Reed, C.-Y. Fu, and A. C. Berg. Ssd: Single shot multibox detector. In *European conference on computer vision*, pages 21–37. Springer, 2016.

[30] Z. Liu, J. Li, Z. Shen, G. Huang, S. Yan, and C. Zhang. Learning efficient convolutional networks through network slimming. In *Proceedings of the IEEE International Conference on Computer Vision*, pages 2736–2744, 2017.

[31] J. Long, E. Shelhamer, and T. Darrell. Fully convolutional networks for semantic segmentation. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 3431–3440, 2015.

Source: https://arxiv.org/pdf/2007.09952

Google Scholar research | 2026-10-07 | Article 16 | Original p. 17 | Orange: discussion; blue: reference

Up or Down? Adaptive Rounding for Post-Training Quantization

Markus Nagel [*1] **Rana Ali Amjad** [*1] **Mart van Baalen** [1] **Christos Louizos** [1] **Tijmen Blankevoort** [1]

Abstract

When quantizing neural networks, assigning each floating-point weight to its nearest fixed-point value is the predominant approach. We find that, perhaps surprisingly, this is not the best we can do. In this paper, we propose AdaRound, a better weight-rounding mechanism for post-training quantization that adapts to the data and the task loss. AdaRound is fast, does not require fine-tuning of the network, and only uses a small amount of unlabelled data. We start by theoretically analyzing the rounding problem for a pre-trained neural network. By approximating the task loss with a Taylor series expansion, the rounding task is posed as a quadratic unconstrained binary optimization problem. We simplify this to a layer-wise local loss and propose to optimize this loss with a soft relaxation. AdaRound not only outperforms rounding-to-nearest by a significant margin but also establishes a new state-of-the-art for post-training quantization on several networks and tasks. Without fine-tuning, we can quantize the weights of Resnet18 and Resnet50 to 4 bits while staying within an accuracy loss of 1%.

1. Introduction

Deep neural networks are being used in many real-world applications as the standard technique for solving tasks in computer vision, machine translation, voice recognition, ranking, and many other domains. Owing to this success and widespread applicability, making these neural networks efficient has become an important research topic. Improved efficiency translates into reduced cloud-infrastructure costs and makes it possible to run these networks on heterogeneous devices such as smartphones, internet-of-things applications, and even dedicated low-power hardware.

One effective way to optimize neural networks for inference is neural network quantization (Krishnamoorthi, 2018; Guo, 2018). In quantization, neural network weights and activations are kept in a low-bit representation for both memory transfer and calculations in order to reduce power consumption and inference time. The process of quantizing a network generally introduces noise, which results in a loss of performance. Various prior works adapt the quantization procedure to minimize the loss in performance while going as low as possible in the number of bits used.

As Nagel et al. (2019) explained, the practicality of neural network quantization methods is important to take into consideration. Although many methods exist that do quantization-aware training (Jacob et al., 2018; Louizos et al., 2019) and get excellent results, these methods require a user to spend significant time on re-training models and hyperparameter tuning.

On the other hand, much attention has recently been dedicated to *post-training quantization* methods (Nagel et al., 2019; Cai et al., 2020; Choukroun et al., 2019; Banner et al., 2019), which can be more easily applied in practice. These types of methods allow for network quantization to happen on-the-fly when deploying models, without the user of the model spending time and energy on quantization. Our work focuses on this type of network quantization.

Rounding-to-nearest is the predominant approach for all neural network weight quantization work that came out thus far. This means that the weight vector $\mathbf{w}$ is rounded to the nearest representable quantization grid value in a fixed-point grid by

$$\widehat{\mathbf{w}} = \mathrm{s} \cdot clip\left(\left\lfloor \frac{\mathbf{w}}{\mathrm{s}} \right\rfloor, \mathrm{n}, \mathrm{p}\right), \tag{1}$$

where s denotes the quantization scale parameter and, n and p denote the negative and positive integer thresholds for clipping. We could round any weight down by replacing $\lfloor \cdot \rceil$ with $\lfloor \cdot \rfloor$, or up using $\lceil \cdot \rceil$. But, rounding-to-nearest seems the most sensible, as it minimizes the difference per-weight in the weight matrix. Perhaps surprisingly, we show that for post-training quantization, rounding-to-nearest is not optimal.

Our contributions in this work are threefold:

[*]Equal contribution [1]Qualcomm AI Research, an initiative of Qualcomm Technologies, Inc.. Correspondence to: Markus Nagel <markusn@qti.qualcomm.com>, Rana Ali Amjad <ramjad@qti.qualcomm.com>, Tijmen Blankevoort <tijmen@qti.qualcomm.com>.

Proceedings of the 37th International Conference on Machine Learning, Online, PMLR 119, 2020. Copyright 2020 by the author(s).

Source: https://proceedings.mlr.press/v119/nagel20a/nagel20a.pdf

Google Scholar research | 2026-10-07 | Article 17 | Original p. 1 | Orange: discussion; blue: reference

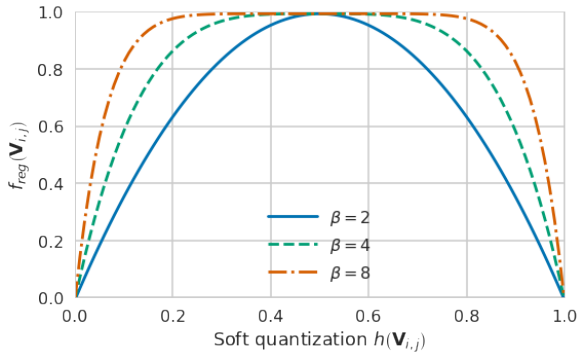


Figure 2. Effect of annealing b on regularization term (24).

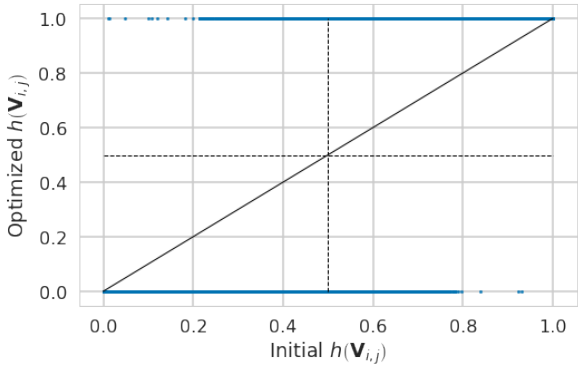


Figure 3. Comparison of $h(\mathbf{V}_{i,j})$ before (x-axis, corresponding to floating point weights) vs after (y-axis) optimizing (21). We see that all $h(\mathbf{V}_{i,j})$ have converged to 0 or 1. Top left and lower right quadrants indicate the weights that have different rounding using (21) vs rounding-to-nearest.

encourage $h(\mathbf{V}_{i,j})$ to move to the extremities. For regularization we use

$$f_{reg}(\mathbf{V}) = \sum_{i,j} 1 - |2h(\mathbf{V}_{i,j}) - 1|^{\beta}, \qquad (24)$$

where we anneal the parameter β. This allows most of the $h(\mathbf{V}_{i,j})$ to adapt freely in the initial phase (higher β) to improve the MSE and encourages it to converge to 0 or 1 in the later phase of the optimization (lower β), to arrive at the binary solution that we are interested in. The effect of annealing β is illustrated in Fig. 2. Fig. 3 shows how this combination of rectified sigmoid and f_{reg} leads to many weights learning a rounding that is different from rounding to the nearest, to improve the performance, while ultimately converging close to 0 or 1.

This method of optimizing (21) is a specific instance of the general family of Hopfield methods used for binary constrained optimization problems. These types of methods are commonly used as an efficient approximation algorithm for large scale combinatorial problems (Hopfield & Tank, 1985; Smith et al.).

To quantize the whole model, we optimize (21) layer-by-layer sequentially. However, this does not account for the quantization error introduced due to the previous layers. In order to avoid the accumulation of quantization error for deeper networks as well as to account for the activation function, we use the following asymmetric reconstruction formulation

$$\arg\min_{\mathbf{V}} \left\| f_a(\mathbf{W}\mathbf{x}) - f_a\left(\widetilde{\mathbf{W}}\hat{\mathbf{x}}\right) \right\|_F^2 + \lambda f_{reg}(\mathbf{V}), \qquad (25)$$

where $\hat{\mathbf{x}}$ is the layer's input with all preceding layers quantized and f_a is the activation function. A similar formulation of the loss has been used previously in (Zhang et al., 2016; He et al., 2017), albeit for different purposes. (25) defines our final objective that we can optimize via stochastic gradient descent. We call this algorithm AdaRound, as it adapts to the statistics of the input data as well as to (an approximation of) the task loss. In section 5 we elaborate on the influence of our design choices as well as the asymmetric reconstruction loss on the performance.

4. Background and related work

In the 1990s, with the resurgence of the field of neural networks, several works designed hardware and optimization methods for running low-bit neural networks on-device. Hammerstrom (1990) created hardware for 8 and 16-bit training of networks, Holi & Hwang (1993) did an empirical analysis on simple neural networks to show that 8 bits are sufficient in most scenarios, and Hoehfeld & Fahlman (1992) developed a stochastic rounding scheme to push neural networks below 8 bits.

More recently, much attention has gone to quantizing neural networks for efficient inference. This is often done by simulating quantization during training, as described in Jacob et al. (2018) and Gupta et al. (2015), and using a straight-through estimator to approximate the gradients. Many methods have since then extended these training frameworks. Choi et al. (2018) learns the activations to obey a certain quantization range, while Esser et al. (2020); Jain et al. (2019) learn the quantization min and max ranges during training so that they do not have to be set manually. Louizos et al. (2019) also learn the grid and formulate a probabilistic version of the quantization training procedure. Uhlich et al. (2020) learn both the quantization grid, and the bit-width per layer, resulting in automatic bit-width selection during training. Works like Kim et al. (2019); Mishra & Marr (2017) exploit student-teacher training to improve quantized model performance during training. Although quantization-aware training is potent and often gives good results, the process is often tedious and time-consuming. Our work seeks to get high accuracy models without this hassle.

Several easy-to-use methods for quantization of networks without quantization-aware training have been proposed as of recent. These methods are often referred to as *post-training quantization* methods. Krishnamoorthi (2018) show several results of network quantization without fine-

Source: https://proceedings.mlr.press/v119/nagel20a/nagel20a.pdf

Google Scholar research | 2026-10-07 | Article 17 | Original p. 5 | Orange: discussion; blue: reference

Hopfield, J. J. and Tank, D. W. "neural" computation of decisions in optimization problems. *Biological Cybernetics*, 52(3):141–152, 1985.

Jacob, B., Kligys, S., Chen, B., Zhu, M., Tang, M., Howard, A., Adam, H., and Kalenichenko, D. Quantization and training of neural networks for efficient integer-arithmetic-only inference. *Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018.

Jain, S. R., Gural, A., Wu, M., and Dick, C. Trained uniform quantization for accurate and efficient neural network inference on fixed-point hardware. *arxiv preprint arxiv:1903.08066*, 2019.

Kim, J., Bhalgat, Y., Lee, J., Patel, C., and Kwak, N. QKD: quantization-aware knowledge distillation. *arxiv preprint arxiv:1911.12491*, 2019.

Kingma, D. P. and Ba, J. Adam: A method for stochastic optimization. *International Conference for Learning Representations (ICLR)*, 2015.

Kochenberger, G., Hao, J.-K., Glover, F., Lewis, M., Lü, Z., Wang, H., and Wang, Y. The unconstrained binary quadratic programming problem: a survey. *Journal of Combinatorial Optimization*, 28(1):58–81, Jul 2014.

Krishnamoorthi, R. Quantizing deep convolutional networks for efficient inference: A whitepaper. *arXiv preprint arXiv:1806.08342*, 2018.

Lin, D. D., Talathi, S. S., and Annapureddy, V. S. Fixed point quantization of deep convolutional networks. In *International Conference on Machine Learning*, 2016.

Louizos, C., Welling, M., and Kingma, D. P. Learning sparse neural networks through l_0 regularization. *International Conference on Learning Representations (ICLR)*, 2018.

Louizos, C., Reisser, M., Blankevoort, T., Gavves, E., and Welling, M. Relaxed quantization for discretized neural networks. In *International Conference on Learning Representations (ICLR)*, 2019.

Mishra, A. K. and Marr, D. Apprentice: Using knowledge distillation techniques to improve low-precision network accuracy. *arXiv preprint arxiv:1711.05852*, 2017.

Nagel, M., van Baalen, M., Blankevoort, T., and Welling, M. Data-free quantization through weight equalization and bias correction. *International Conference on Computer Vision (ICCV)*, 2019.

Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Kopf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., Bai, J., and Chintala, S. Pytorch: An imperative style, high-performance deep learning library. In *Neural Information Processing Systems (NeuRIPS)*. 2019.

Rubinstein, R. The cross-entropy method for combinatorial and continuous optimization. *Methodology And Computing In Applied Probability*, 1(2):127–190, Sep 1999.

Russakovsky, O., Deng, J., Su, H., Krause, J., Satheesh, S., Ma, S., Huang, Z., Karpathy, A., Khosla, A., Bernstein, M., Berg, A. C., and Fei-Fei, L. ImageNet Large Scale Visual Recognition Challenge. *International Journal of Computer Vision (IJCV)*, 115(3):211–252, 2015.

Smith, K. A., Palaniswami, M., and Krishnamoorthy, M. Neural techniques for combinatorial optimization with applications. *IEEE Trans. Neural Networks*, 9(6):1301–1318.

Stock, P., Joulin, A., Gribonval, R., Graham, B., and Jégou, H. And the bit goes down: Revisiting the quantization of neural networks. In *International Conference on Learning Representations*, 2020.

Uhlich, S., Mauch, L., Yoshiyama, K., Cardinaux, F., García, J. A., Tiedemann, S., Kemp, T., and Nakamura, A. Mixed precision dnns: All you need is a good parametrization. *International Conference on Learning Representations (ICLR)*, 2020.

Wang, P., Hu, Q., Zhang, Y., Zhang, C., Liu, Y., and Cheng, J. Two-step quantization for low-bit neural networks. *Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 4376–4384, 2018.

Zhang, X., Zou, J., He, K., and Sun, J. Accelerating very deep convolutional networks for classification and detection. *IEEE Trans. Pattern Anal. Mach. Intell.*, 38(10): 1943–1955, 2016.

Zhao, R., Hu, Y., Dotzel, J., Sa, C. D., and Zhang, Z. Improving neural network quantization without retraining using outlier channel splitting. *International Conference on Machine Learning, ICML*, 2019.

Source: https://proceedings.mlr.press/v119/nagel20a/nagel20a.pdf

Google Scholar research | 2026-10-07 | Article 17 | Original p. 10 | Orange: discussion; blue: reference

arXiv:2106.08295v1 [cs.LG] 15 Jun 2021

A White Paper on Neural Network Quantization

Markus Nagel[*]
Qualcomm AI Research[†]
markusn@qti.qualcomm.com

Marios Fournarakis[*]
Qualcomm AI Research[†]
mfournar@qti.qualcomm.com

Rana Ali Amjad
Qualcomm AI Research[†]
ramjad@qti.qualcomm.com

Yelysei Bondarenko
Qualcomm AI Research[†]
ybodaren@qti.qualcomm.com

Mart van Baalen
Qualcomm AI Research[†]
mart@qti.qualcomm.com

Tijmen Blankevoort
Qualcomm AI Research[†]
tijmen@qti.qualcomm.com

Abstract

While neural networks have advanced the frontiers in many applications, they often come at a high computational cost. Reducing the power and latency of neural network inference is key if we want to integrate modern networks into edge devices with strict power and compute requirements. Neural network quantization is one of the most effective ways of achieving these savings but the additional noise it induces can lead to accuracy degradation.

In this white paper, we introduce state-of-the-art algorithms for mitigating the impact of quantization noise on the network's performance while maintaining low-bit weights and activations. We start with a hardware motivated introduction to quantization and then consider two main classes of algorithms: Post-Training Quantization (PTQ) and Quantization-Aware-Training (QAT). PTQ requires no re-training or labelled data and is thus a lightweight push-button approach to quantization. In most cases, PTQ is sufficient for achieving 8-bit quantization with close to floating-point accuracy. QAT requires fine-tuning and access to labeled training data but enables lower bit quantization with competitive results. For both solutions, we provide tested pipelines based on existing literature and extensive experimentation that lead to state-of-the-art performance for common deep learning models and tasks.

1 Introduction

With the rise in popularity of deep learning as a general-purpose tool to inject intelligence into electronic devices, the necessity for small, low-latency and energy efficient neural networks solutions has increased. Today neural networks can be found in many electronic devices and services, from smartphones, smart glasses and home appliances, to drones, robots and self-driving cars. These devices are typically subject to strict time restrictions on the execution of neural networks or stringent power requirements for long-duration performance.

One of the most impactful ways to decrease the computational time and energy consumption of neural networks is quantization. In neural network quantization, the weights and activation tensors are stored in lower bit precision than the 16 or 32-bit precision they are usually trained in. When

[*]Equal contribution. [†]Qualcomm AI Research is an initiative of Qualcomm Technologies, Inc.

Source: https://arxiv.org/pdf/2106.08295

Google Scholar research | 2026-10-07 | Article 18 | Original p. 1 | Orange: discussion; blue: reference

$\mathbf{x}_i$, is given by:

$$\begin{aligned}
\frac{\partial \widehat{\mathbf{x}}_i}{\partial \mathbf{x}_i} &= \frac{\partial q(\mathbf{x}_i)}{\partial \mathbf{x}_i} \\
&= s \cdot \frac{\partial}{\partial \mathbf{x}_i} \operatorname{clamp}\left(\left\lfloor \frac{\mathbf{x}_i}{s} \right\rceil; n, p\right) + 0 \\
&= \begin{cases} s \cdot \dfrac{\partial \lfloor \mathbf{x}_i/s \rceil}{\partial (\mathbf{x}_i/s)} \dfrac{\partial (\mathbf{x}_i/s)}{\partial \mathbf{x}_i} & \text{if } q_{\min} \leq \mathbf{x}_i \leq q_{\max}, \\ s \cdot \dfrac{\partial n}{\partial \mathbf{x}_i} & \text{if } \mathbf{x}_i < q_{\min}, \\ s \cdot \dfrac{\partial p}{\partial \mathbf{x}_i} & \text{if } \mathbf{x}_i > q_{\max}, \end{cases} \\
&= \begin{cases} 1 & \text{if } q_{\min} \leq \mathbf{x}_i \leq q_{\max}, \\ 0 & \text{otherwise.} \end{cases}
\end{aligned} \tag{37}$$

Using this gradient definition we can now back-propagate through the quantization blocks. Figure 10 shows a simple computational graph for the forward and backward pass used in quantization-aware training. The forward pass is identical to that of figure 4, but in the backward pass we effectively skip the quantizer block due to the STE assumption. In earlier QAT work the quantization ranges

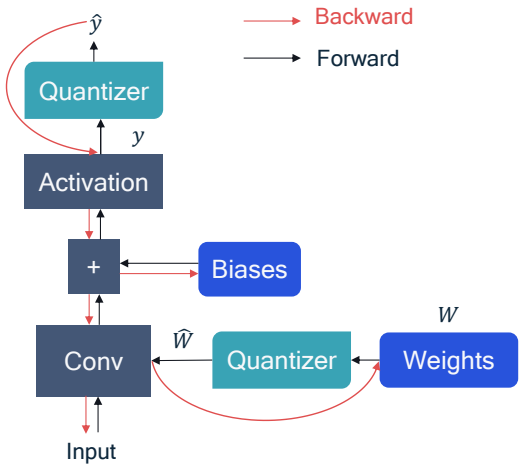


Figure 10: Forward and backward computation graph for quantization aware training with STE assumption.

for weights and activations were updated at each iteration most commonly using the min-max range (Krishnamoorthi, 2018). In later work (Esser et al., 2020; Jain et al., 2019; Bhalgat et al., 2020), the STE is used to calculate the gradient *w.r.t.* the quantization parameters, z and s. Using the chain rule and the STE, we first calculate the gradient *w.r.t.* the scale-factor:

$$\begin{aligned}
\frac{\partial \widehat{\mathbf{x}}_i}{\partial s} &= \frac{\partial}{\partial s}\left[s \cdot \operatorname{clamp}\left(\left\lfloor \frac{\mathbf{x}_i}{s} \right\rceil; n, p\right)\right] \\
&= \begin{cases} -\mathbf{x}_i/s + \lfloor \mathbf{x}_i/s \rceil & \text{if } q_{\min} \leq \mathbf{x}_i \leq q_{\max}, \\ n & \text{if } \mathbf{x}_i < q_{\min}, \\ p & \text{if } \mathbf{x}_i > q_{\max}. \end{cases}
\end{aligned} \tag{38}$$

Originally, we restricted the zero-point to be an integer. To make zero-point learnable we convert into a real number and apply the rounding operator. The modified quantization function is defined as:

$$\widehat{\mathbf{x}} = q(\mathbf{x}; s, z) = s \cdot \left[\operatorname{clamp}\left(\left\lfloor \frac{\mathbf{x}}{s} \right\rceil + \lfloor z \rceil; n, p\right) - \lfloor z \rceil\right] \tag{39}$$

The gradient *w.r.t.* to z is calculated by applying the STE once again to the rounding operator:

$$\frac{\partial \widehat{\mathbf{x}}_i}{\partial z} = \begin{cases} 0 & q_{\min} \leq \mathbf{x}_i \leq q_{\max}, \\ -s & \text{otherwise.} \end{cases} \tag{40}$$

Source: https://arxiv.org/pdf/2106.08295

Google Scholar research | 2026-10-07 | Article 18 | Original p. 20 | Orange: discussion; blue: reference

Model (FP32 accuracy)	ResNet18 (69.68)		MobileNetV2 (71.72)	
	PTQ	QAT	PTQ	QAT
W4A8 w/ min-max weight init	0.12	69.61	0.56	69.96
W4A8 w/ MSE weight init	18.58	**69.76**	12.99	**70.13**
W4A4 w/ min-max act init	7.51	68.23	0.22	**66.55**
W4A4 w/ MSE act init	9.62	**68.41**	0.71	66.29

Table 8: Ablation study for various ways to initialize the quantization grid. The learning rate is individually optimized for each configuration. ImageNet validation accuracy (%) averaged over 3 runs.

this modeling approach is on par or better for per-channel quantization compared to static folding as we can see from the last two rows of table 7.

4.3 Initialization for QAT

In this section, we will explore the effect of initialization for QAT. It is common practice in literature to start from a pre-trained FP32 model (Esser et al., 2020; Krishnamoorthi, 2018; Jacob et al., 2018; Jain et al., 2019). While it is clear that starting from an FP32 model is beneficial, the effect of the quantization initialization on the final QAT result is less studied. Here we explore the effect of using several of our PTQ techniques as an initial step before doing QAT.

Effect of range estimation To assess the effect of the initial range setting (see section 3.1) for weights and activations, we perform two sets of experiments, which are summarized in table 8. In the first experiment, we quantize the weights to 4-bits and keep the activations in 8-bits. We compare the min-max initialization with the MSE based initialization for the weights quantization range. While the MSE initialized model has a significantly higher starting accuracy, the gap closes after training for 20 epochs.

To explore the same effect for activation quantization, we perform a similar experiment, where we now quantize the activation to 4-bits and compare min-max initialization with MSE based initialization. The observations from weight range initialization hold here as well. In figure 11 we show the full training curve of this experiment. In the first few epochs, there is a significant advantage for using MSE initialization, which almost vanishes in the later stage of training. In conclusion, a better initialization can lead to better QAT results, but the gain is usually small and vanishes the longer the training lasts.

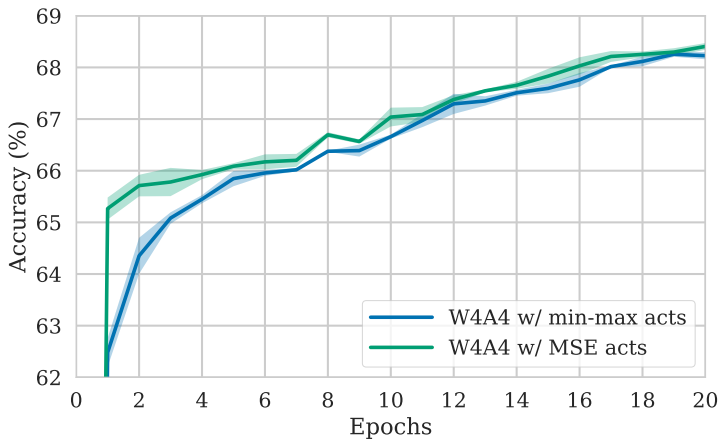


Figure 11: Influence of the initial activation range setting on the QAT training behavior of ResNet18. Average ImageNet validation accuracy (%) after each training epoch over 3 runs (and standard deviation shaded).

Source: https://arxiv.org/pdf/2106.08295

Google Scholar research | 2026-10-07 | Article 18 | Original p. 22 | Orange: discussion; blue: reference

Esser, S. K., McKinstry, J. L., Bablani, D., Appuswamy, R., and Modha, D. S. Learned step size quantization. *International Conference on Learning Representations (ICLR)*, 2020. 20, 22

Finkelstein, A., Almog, U., and Grobman, M. Fighting quantization bias with bias. *arXiv preprint arxiv:1906.03193*, 2019. 13

Gupta, S., Agrawal, A., Gopalakrishnan, K., and Narayanan, P. Deep learning with limited numerical precision. *International Conference on Machine Learning, ICML*, 2015. 14

Horowitz, M. 1.1 computing's energy problem (and what we can do about it). In *2014 IEEE International Solid-State Circuits Conference Digest of Technical Papers (ISSCC)*, pp. 10–14, 2014. doi: 10.1109/ISSCC.2014.6757323. 2, 3

Howard, A. G., Zhu, M., Chen, B., Kalenichenko, D., Wang, W., Weyand, T., Andreetto, M., and Adam, H. Mobilenets: Efficient convolutional neural networks for mobile vision applications. *arXiv preprint arXiv:1704.04861*, 2017. 10

Ioffe, S. and Szegedy, C. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In Bach, F. and Blei, D. (eds.), *Proceedings of the 32nd International Conference on Machine Learning*, volume 37 of *Proceedings of Machine Learning Research*, pp. 448–456, Lille, France, 07–09 Jul 2015. PMLR. URL http://proceedings.mlr.press/v37/ioffe15.html. 6

Jacob, B., Kligys, S., Chen, B., Zhu, M., Tang, M., Howard, A., Adam, H., and Kalenichenko, D. Quantization and training of neural networks for efficient integer-arithmetic-only inference. *Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018. 3, 6, 21, 22

Jain, S. R., Gural, A., Wu, M., and Dick, C. Trained uniform quantization for accurate and efficient neural network inference on fixed-point hardware. *CoRR*, abs/1903.08066, 2019. URL http://arxiv.org/abs/1903.08066. 20, 22

Krishnamoorthi, R. Quantizing deep convolutional networks for efficient inference: A whitepaper. *arXiv preprint arXiv:1806.08342*, 2018. 6, 10, 12, 20, 21, 22

Meller, E., Finkelstein, A., Almog, U., and Grobman, M. Same, same but different: Recovering neural network quantization error through weight factorization. In *Proceedings of the 36th International Conference on Machine Learning, ICML 2019, 9-15 June 2019, Long Beach, California, USA*, pp. 4486–4495, 2019. URL http://proceedings.mlr.press/v97/meller19a.html. 11, 12, 13

Moons, B., Noorzad, P., Skliar, A., Mariani, G., Mehta, D., Lott, C., and Blankevoort, T. Distilling optimal neural networks: Rapid search in diverse spaces. *arXiv preprint arXiv:2012.08859*, 2020. 15

Nagel, M., van Baalen, M., Blankevoort, T., and Welling, M. Data-free quantization through weight equalization and bias correction. *International Conference on Computer Vision (ICCV)*, 2019. 10, 11, 12, 13

Nagel, M., Amjad, R. A., Van Baalen, M., Louizos, C., and Blankevoort, T. Up or down? Adaptive rounding for post-training quantization. In III, H. D. and Singh, A. (eds.), *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pp. 7197–7206. PMLR, 13–18 Jul 2020. URL http://proceedings.mlr.press/v119/nagel20a.html. 13, 14, 15

Nascimento, M. G. d., Fawcett, R., and Prisacariu, V. A. Dsconv: Efficient convolution operator. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, October 2019. 5

Ramachandran, P., Zoph, B., and Le, Q. V. Searching for activation functions. *CoRR*, abs/1710.05941, 2017. URL http://arxiv.org/abs/1710.05941. 7

Rouhani, B., Lo, D., Zhao, R., Liu, M., Fowers, J., Ovtcharov, K., Vinogradsky, A., Massengill, S., Yang, L., Bittner, R., Forin, A., Zhu, H., Na, T., Patel, P., Che, S., Koppaka, L. C., Song, X., Som, S., Das, K., Tiwary, S., Reinhardt, S., Lanka, S., Chung, E., and Burger, D. Pushing the limits of narrow precision inferencing at cloud scale with microsoft floating point. In *Neural Information Processing Systems (NeurIPS 2020)*. ACM, November 2020. 5

Source: https://arxiv.org/pdf/2106.08295

Google Scholar research | 2026-10-07 | Article 18 | Original p. 26 | Orange: discussion; blue: reference

Bringing AI To Edge: From Deep Learning's Perspective

Di Liu, Hao Kong, Xiangzhong Luo, Weichen Liu, Ravi Subramaniam

arXiv:2011.14808v1 [cs.LG] 25 Nov 2020

***Abstract*—Edge computing and artificial intelligence (AI), especially deep learning for nowadays, are gradually intersecting to build a novel system, called *edge intelligence*. However, the development of edge intelligence systems encounters some challenges, and one of these challenges is the *computational gap* between computation-intensive deep learning algorithms and less-capable edge systems. Due to the computational gap, many edge intelligence systems cannot meet the expected performance requirements. To bridge the gap, a plethora of deep learning techniques and optimization methods are proposed in the past years: light-weight deep learning models, network compression, and efficient neural architecture search. Although some reviews or surveys have partially covered this large body of literature, we lack a systematic and comprehensive review to discuss all aspects of these deep learning techniques which are critical for edge intelligence implementation. As various and diverse methods which are applicable to edge systems are proposed intensively, a holistic review would enable edge computing engineers and community to know the state-of-the-art deep learning techniques which are instrumental for edge intelligence and to facilitate the development of edge intelligence systems. This paper surveys the representative and latest deep learning techniques that are useful for edge intelligence systems, including hand-crafted models, model compression, hardware-aware neural architecture search and adaptive deep learning models. Finally, based on observations and simple experiments we conducted, we discuss some future directions.**

***Index Terms*—Deep learning, model optimization, edge computing, neural architecture search**

I. Introduction

In 2012, AlexNet [1] broke the record of ImageNet LSVRC contest [2], improving the prediction accuracy by a large margin of 10%, which marked the milestone for the booming development of deep learning (DL) techniques, or more specifically deep neural network (DNN) [3]. In the subsequent years, as the increasing number of DNN applications [4] emerges, designing DNN-based systems attracts a lot of attention and efforts from academia and industry, spanning from advanced image manipulation and enhancement and convenient voice assistant on mobile phones to Level-4 autonomous driving systems on automotives [5]. AlphaGo [6] is another prominent example which defeated the top professional player in the ancient Chinese game GO, deemed impossible before due to its extremely high complexity.

The successful application of DNN models relies on two stages: **training** and **inference/deployment**. Training indicates the procedure of learning a predictive model from a huge amount of labeled data, while inference denotes the procedure of using the trained model upon new data to make an accurate prediction. The training procedure involves a complex learning process requiring powerful computational units and a huge amount of data and spending considerable time. For instance, training ResNet50 [7] with ImageNet on 8 Nvidia Tesla P100 takes 29 hours [8], where ImageNet dataset [2] has 1.28 M images of 1000 classes for training. Recently, researchers also measure the environmental effect of the training procedure and indicate that training DNN consumes excessive energy and emits substantial CO_2 [9] and environment-sustainable DNN models should be considered.

The trained DNN model is exploited to make prediction on new data, and this procedure is usually called DNN *inference*. Many companies implement their DNN inference on servers to provide convenient services for their customers such as voice-assistant, machine translation, image retrieval, etc [10]. However, inference from cloud suffers from *responsiveness* (e.g., latency) and *privacy* issues which are critical in some scenarios. For instance, DNNs are seen to be a pivotal technique for self-driving cars, in which most of executions are subject to highly rigorous real-time constraints [5], [11], but the high and uncertain communication overhead makes it difficult to satisfy the temporal requirement. In addition, some inference is conducted on confidential data, e.g., manufacturing and production data, and uploading these data to cloud may risk data leakage which will severely hurt their business. To address these concerns, *edge computing* is proposed [12].

Edge computing systems are placed at the proximity of data producer, like sensors, end-users, etc, to rapidly and locally process data. The emergence of edge computing paves the way for pervasive intelligent systems [13]. IEEE Computer Society identifies 'Artificial Intelligence at the edge' as one of the top 12 technology trends to reach adoption in 2020 [14]. Edge systems have a broad spectrum of hardware features, from powerful computation units in communication stations and self-driving cars to small and battery-supplied smart phones and wearable devices. Fig. 1 plots the overview of edge

Di Liu, Hao Kong, and Xiangzhong Luo contribute equally to this work.

This research was conducted in collaboration with HP Inc. and supported by National Research Foundation (NRF) Singapore and the Singapore Government through the Industry Alignment Fund-Industry Collaboration Projects Grant (I1801E0028). This work is also partially supported by NTU NAP M4082282 and SUG M4082087, Singapore.

Di Liu is with HP-NTU Digital Manufacturing Corporate Lab, Nanyang Technological University, Singapore. (Email: liu.di@ntu.edu.sg)

Hao Kong and Weichen Liu are with HP-NTU Digital Manufacturing Corporate Lab, Nanyang Technological University, Singapore, and School of Computer Science and Engineering, Nanyang Technological University, Singapore. (Email: kong.hao@ntu.edu.sg, liu@ntu.edu.sg)

Xiangzhong Luo is with School of Computer Science and Engineering, Nanyang Technological University, Singapore. (Email: xiangzho001@e.ntu.edu.sg)

Ravi Subramaniam is with Innovations and Experiences–Business Personal Systems, HP Inc., USA. (Email: ravi.subramaniam@hp.com)

Source: https://arxiv.org/pdf/2011.14808

Google Scholar research | 2026-10-07 | Article 19 | Original p. 1 | Orange: discussion; blue: reference

criterion for each layer by using a gradient-based method. HRank [99] empirically finds that filters with low-rank is less informative than those with high-rank, and thus uses this observation to prune the unimportant channels.

Wen *et al.* [89] propose SSL to have a training-based pruning method, learning a compact and sparse model from a pre-trained model. Liu *et al.* [100] propose an approach called network slimming, which takes wide and large networks as input models, but during training, insignificant channels are automatically identified and pruned afterwards. Soft filter pruning (SFP) [94] deploy a training-based pruning method to prune a complex network. Huang *et al.* [95] propose the concept of sparsity scaling factor for each filter which is learned during to training, and then filters with scaling factor 0 are removed. Yao *et al.* [92] train an recurrent neural network (RNN) to determine the dropout probability of each filters and prune the network layer by layer according to drop probability.

Most of the above-mentioned *structure pruning* methods are rule-based, i.e., some heuristic algorithms devised according to their own criterion. In contrary, some work employ learning algorithm to automatically prune network model. AMC [96] proposes to use *reinforcement learning* to automatically prune channels of each layer. Anwar *et al.* [101] prune a DNN model at feature level, kernel level, and intra-kernel level (i.e., weight), where they deploy *evolutionary algorithm* to find the best combination of different pruning granularities. However, searching in a discrete space using RL and EA is really costly, so the gradient-based method is recently proposed to find an optimal pruning criterion for each layer [98].

To determine how many channels should be pruned, the above-mentioned works use indirect metrics like FLOPs or compression ratio to prune network. Nevertheless, the reduced FLOPs and compression ratio cannot directly translate to the performance improvement. In addition, a diverse of hardware accelerators have emerged for boosting the execution of DNNs, but these various systems demonstrate different capability to handling network complication. Hence, some pruning studies directly target the direct metric upon a specific hardware, e.g., latency. NetAdapt [47] proposes an automated framework to prune filters in different layers such that the pruned model can be adapted to a target platform. To optimize the latency on a target platform, NetAdapt builds up a look-up-table (LUT) for different operations and layers, so instead of measuring latency on the real platform, it can quickly estimate the latency based on the model architecture and LUT. Yu *et al.* [102] introduce SIMD-aware pruning framework which employs different pruning strategies for different underlying hardware, like weight pruning for low-parallelism CPU and filter pruning for high-parallelism.

Discussion: Table III and IV summarize methods discussed in this section. Pruning is the well-studied topic in model compression and many methods have been proposed to advance the network pruning [103]. These pruning methods show promising results, capable of compressing a DNN model, in some cases more than 40x, while only degrading the prediction accuracy slightly [76]. To effectively adopt model pruning for edge intelligence systems, the existing works suffer from two flaws: 1) the theoretical foundation behind pruning lacks, and thus there is a debate whether we should prune a complex model or directly train a compact model for resource constrained hardware platforms. Some recent research [104] [105] [106] [107] strives to empirically or theoretically find an answer for this question. Since pruning is a time-consuming procedure involving large model training and iterative pruning-retraining procedure, a theoretical foundation or proof may drastically change the way we use model pruning to design edge DNN models; 2) Almost all of pruning methods are *hardware-agnostic* and depend on *hardware-independent* metrics, like MACs and FLOPs. Since different hardware architectures demonstrate different degrees of parallelism, the state-of-the-art methods may unnecessarily prune models without performance improvement (e.g., latency reduction) while reducing the capacity of DNN models which is proven to have a significant impact on model's accuracy [108]. Therefore, we need to devise pruning method with hardware-awareness such that the model can be tailored for various hardware.

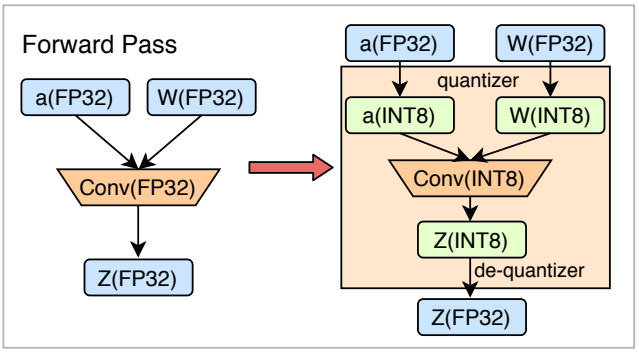


Fig. 6. An example of convolution quantization.

TABLE V
THE CLASSIFICATION OF QUANTIZATION METHODS.

Methodologies	Bits	Weights	Activations	Year
Gupta *et al.* [109]	16-bit	Yes	No	2015
Q-CNN [110]	1-bit	Yes	No	2016
BinaryConnect [111]	1-bit	Yes	No	2016
Ternary [112]	2-bit	Yes	No	2016
DoReFa-Net [113]	Arbitrary	Yes	Yes	2016
XNOR-Net [114]	1-bit	Yes	Yes	2016
INQ [115]	5/4-bit†	Yes	Yes	2017
ABC-Net [116]	1-bit	Yes	Yes	2017
Zhu *et al.* [117]	2-bit	Yes	No	2017
Jacod [118]	8-bit	Yes	Yes	2018
Bi-Real-Net [119]	1-bit	Yes	Yes	2018
LQ-Net [120]	Arbitrary	Yes	Yes	2018
TQT [121]	8-bit	Yes	Yes	2019
HAQ [122]	(1∼8)-bit§	Yes	Yes	2019
HAWQ [123]	2/4-bit†	Yes	Yes	2019
Jung *et al.* [124]	(2∼4)-bit	Yes	Yes	2019
Zhuang *et al.* [125]	2-bit	Yes	Yes	2020
XOR-Net [126]	2-bit	Yes	Yes	2020

a/*b*-bit † denotes the method quantizes the network weights for a bits while operating intermediate activation feature maps for b bits.
($a \sim b$)-bit§ represents the method quantizes the network weights or activations from a bits to b bits.

B. Quantization

Network pruning reduces the complexity of DNN models by removing redundant weights or channels. However, the

Source: https://arxiv.org/pdf/2011.14808
Google Scholar research | 2026-10-07 | Article 19 | Original p. 8 | Orange: discussion; blue: reference

state-of-the-art models have more than billions of parameters and at the same time during inference a model produces a large portion of intermediate results (activation/feature maps) which usually occupy a large memory space. As a result, the huge memory requirement prohibits DNN models from implementing on memory-limited edge devices [127], [128]. For example, ResNet-50 [7] has 26 million parameter weights, generates 16 million activations in one inference, requires around 168 MB memory space, and needs at lease 3GB/s memory bandwidth. It is not difficult to see that it is unlikely to deploy these state-of-the-art models to edge devices, which have limited storage and computational resources.

In this case, *quantization* becomes a promising approach to address the aforementioned issue, which encodes full-precision (FP32) weights and activations with low-precision ones (e.g., FP16, INT8, binary) while preserving even the same level of accuracy. Some early work has shown that using FP16 to train DNN models can reduce the computational cost while retaining accuracy [109]. Quantization significantly benefits DNN models on resource-limited devices, and it is capable of fitting the whole model into on-chip memory of edge devices such that the high overhead occurred by off-chip memory access can be mitigated. In addition, since operations with low-bit representation usually consume less energy and execute faster, quantization reduces energy consumption and latency as well on some hardware platforms [129]. In this section, we discuss some state-of-the-art quantization studies.

Wu *et al.* [110] propose a unified quantization framework, which improves the quantization performance by minimizing the estimation error of each layer's response. On this basis, an error correction training strategy is included. Jain *et al.* [121] propose a trained uniform quantization method for accurate and efficient neural network inference on fixed-point hardware. Jacob *et al.* [118] quantize models into 8-bit integer and propose a quantize-aware training method to eliminate the accuracy loss caused by quantization. Instead of quantizing all weights in a DNN, IQN [115] adopts a group-wise quantization method to gradually quantize weights of a DNN model. The advantage of IQN is that it is able to derive the quantized model without accuracy loss.

The above works uniformly convert all weights and activation into the same low-bit representation, but many emergent hardware and accelerators support mixed precision operations, e.g., Nvidia Turing GPU architecture supports 1-bit, 4-bit, 8-bit and 16-bit arithmetic operations [130]. This provides a more effective and flexible way to quantize weights and activations of a DNN model. Wang *et al.* [122] propose a hardware-aware automated quantization approach, namely HAQ. HAQ exploits reinforcement learning to select different quantization width for each layer upon a target hardware. Additionally, the hardware architecture is involved into the learning loop, so that it can directly reduce the inference latency, energy and storage on the target hardware. HAWQ [123] also considers to quantize a full-precision model into a mix-precision model where the quantized precision is determined for each layer based on Hessian matrix.

Some works use binary or ternary quantization to maximally compress DNN models. Then, binarized or ternarized network can use cheap bit operation to boost the efficiency on dedicated hardware [131]. In [111], Courbariaux *et al.* propose the binaryconnect, which targets to transform the full-precision weights into the binary format. A very straightforward binarization method would be based on the sign function:

$$w_b = \begin{cases} +1 & \text{if } w \geq 0 \\ -1 & \text{otherwise} \end{cases} \tag{1}$$

where w_b is the binary weight and w is the full-precision weight. Due to the fact that this is a deterministic operation, averaging the discretization over the many input weights of a hidden unit could compensate for the loss of information. An alternative that allows a finer and more correct averaging process to take place is to binarize stochatically, which helps to improve the model generalization capability:

$$w_b = \begin{cases} +1 & \text{with probability } p = \delta(w) \\ -1 & \text{with probability } 1 - p \end{cases} \tag{2}$$

where δ is the *hard sigmoid* formula:

$$\delta(x) = \text{clip}(\frac{x+1}{2}, 0, 1) = \max(0, \min(1, \frac{x+1}{2})) \tag{3}$$

Intuitively, applying a binarized method is an easy way to quantize the full precision weights. However, this will be harder for the training process to converge due to the highly discrete parameter space, which drastically degrades the model performance.

Later, Rastegari *et al.* [114] present an efficient approximation strategy, which constrains the full precision weights to $+1$ and -1 instead of directly rounding them. With a scaling factor α, an convolutional operation can be approximated as follows:

$$\mathcal{I} * \mathcal{W} \approx \mathcal{I} * (\alpha \mathcal{B}) = \alpha \mathcal{I} * \mathcal{B} \tag{4}$$

where $\mathcal{I}$ is the input image and $\mathcal{B} \in \{-1, +1\}$ is the binarized weight. Other binarized methods are proposed to improve the degraded accuracy caused by binarization like [113], [116], [119]. XOR-Net [126] takes the implementation of BNNs into account and simplifies the number of instructions used in binary dot product to accelerate BNN inference on edge devices.

Inspired by [114], Li *et al.* [112] propose ternary weight networks (TWNs) with weights constrained to $\{-1, 0, +1\}$, which minimizes the Euclidian distance between full precision weights $\mathcal{W}$ and the ternary weights $\mathcal{W}^t$ along with a scaling factor α. Here the quantized weights are obtained with a threshold-based ternary function:

$$\mathcal{W}_i^t = \begin{cases} +1, & \text{if } \mathcal{W}_i > \Delta \\ +0, & \text{if } |\mathcal{W}_i| \leq \Delta \\ +0, & \text{if } \mathcal{W}_i < -\Delta \end{cases} \tag{5}$$

where Δ is a positive threshold parameter. Thus, the optimization objective can be formulated as follows:

$$\alpha^*, \mathcal{W}^{t*} = \arg\min J(\alpha, \mathcal{W}^t) = ||\mathcal{W} - \alpha \mathcal{W}^t||_2^2 \tag{6}$$

Source: https://arxiv.org/pdf/2011.14808

[96] Y. He, J. Lin, Z. Liu, H. Wang, L.-J. Li, and S. Han, "Amc: Automl for model compression and acceleration on mobile devices," in *The European Conference on Computer Vision (ECCV)*, September 2018.

[97] Z. You, K. Yan, J. Ye, M. Ma, and P. Wang, "Gate decorator: Global filter pruning method for accelerating deep convolutional neural networks," in *Advances in Neural Information Processing Systems*, 2019, pp. 2130–2141.

[98] Y. He, Y. Ding, P. Liu, L. Zhu, H. Zhang, and Y. Yang, "Learning filter pruning criteria for deep convolutional neural networks acceleration," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2020, pp. 2009–2018.

[99] M. Lin, R. Ji, Y. Wang, Y. Zhang, B. Zhang, Y. Tian, and L. Shao, "Hrank: Filter pruning using high-rank feature map," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2020, pp. 1529–1538.

[100] Z. Liu, J. Li, Z. Shen, G. Huang, S. Yan, and C. Zhang, "Learning efficient convolutional networks through network slimming," in *Proceedings of the IEEE International Conference on Computer Vision*, 2017, pp. 2736–2744.

[101] S. Anwar, K. Hwang, and W. Sung, "Structured pruning of deep convolutional neural networks," *ACM Journal on Emerging Technologies in Computing Systems (JETC)*, vol. 13, no. 3, pp. 1–18, 2017.

[102] J. Yu, A. Lukefahr, D. J. Palframan, G. S. Dasika, R. Das, and S. A. Mahlke, "Scalpel: Customizing dnn pruning to the underlying hardware parallelism," *2017 ACM/IEEE 44th Annual International Symposium on Computer Architecture (ISCA)*, pp. 548–560, 2017.

[103] D. Blalock, J. J. G. Ortiz, J. Frankle, and J. Guttag, "What is the state of neural network pruning?" *arXiv preprint arXiv:2003.03033*, 2020.

[104] Z. Liu, M. Sun, T. Zhou, G. Huang, and T. Darrell, "Rethinking the value of network pruning," in *International Conference on Learning Representations*, 2019. [Online]. Available: https://openreview.net/forum?id=rJlnB3C5Ym

[105] J. Frankle and M. Carbin, "The lottery ticket hypothesis: Finding sparse, trainable neural networks," in *International Conference on Learning Representations*, 2019. [Online]. Available: https://openreview.net/forum?id=rJl-b3RcF7

[106] Y. Wang, X. Zhang, L. Xie, J. Zhou, H. Su, B. Zhang, and X. Hu, "Pruning from scratch," *arXiv preprint arXiv:1909.12579*, 2019.

[107] E. Malach, G. Yehudai, S. Shalev-Shwartz, and O. Shamir, "Proving the lottery ticket hypothesis: Pruning is all you need," *arXiv preprint arXiv:2002.00585*, 2020.

[108] S. Zagoruyko and N. Komodakis, "Wide residual networks," *arXiv preprint arXiv:1605.07146*, 2016.

[109] S. Gupta, A. Agrawal, K. Gopalakrishnan, and P. Narayanan, "Deep learning with limited numerical precision," in *International Conference on Machine Learning*, 2015, pp. 1737–1746.

[110] J. Wu, C. Leng, Y. Wang, Q. Hu, and J. Cheng, "Quantized convolutional neural networks for mobile devices," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2016, pp. 4820–4828.

[111] M. Courbariaux, Y. Bengio, and J.-P. David, "Binaryconnect: Training deep neural networks with binary weights during propagations," in *Advances in neural information processing systems*, 2015, pp. 3123–3131.

[112] F. Li, B. Zhang, and B. Liu, "Ternary weight networks," *arXiv preprint arXiv:1605.04711*, 2016.

[113] S. Zhou, Y. Wu, Z. Ni, X. Zhou, H. Wen, and Y. Zou, "Dorefa-net: Training low bitwidth convolutional neural networks with low bitwidth gradients," *arXiv preprint arXiv:1606.06160*, 2016.

[114] M. Rastegari, V. Ordonez, J. Redmon, and A. Farhadi, "Xnor-net: Imagenet classification using binary convolutional neural networks," in *European conference on computer vision*. Springer, 2016, pp. 525–542.

[115] A. Zhou, A. Yao, Y. Guo, L. Xu, and Y. Chen, "Incremental network quantization: Towards lossless cnns with low-precision weights," *arXiv preprint arXiv:1702.03044*, 2017.

[116] X. Lin, C. Zhao, and W. Pan, "Towards accurate binary convolutional neural network," in *Advances in Neural Information Processing Systems*, 2017, pp. 345–353.

[117] C. Zhu, S. Han, H. Mao, and W. J. Dally, "Trained ternary quantization," *arXiv preprint arXiv:1612.01064*, 2016.

[118] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2018, pp. 2704–2713.

[119] Z. Liu, B. Wu, W. Luo, X. Yang, W. Liu, and K.-T. Cheng, "Bi-real net: Enhancing the performance of 1-bit cnns with improved representational capability and advanced training algorithm," in *Proceedings of the European conference on computer vision (ECCV)*, 2018, pp. 722–737.

[120] D. Zhang, J. Yang, D. Ye, and G. Hua, "Lq-nets: Learned quantization for highly accurate and compact deep neural networks," in *Proceedings of the European conference on computer vision (ECCV)*, 2018, pp. 365–382.

[121] S. R. Jain, A. Gural, M. Wu, and C. Dick, "Trained uniform quantization for accurate and efficient neural network inference on fixed-point hardware," *arXiv preprint arXiv:1903.08066*, 2019.

[122] K. Wang, Z. Liu, Y. Lin, J. Lin, and S. Han, "Haq: Hardware-aware automated quantization with mixed precision," in *The IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2019.

[123] Z. Dong, Z. Yao, A. Gholami, M. W. Mahoney, and K. Keutzer, "Hawq: Hessian aware quantization of neural networks with mixed-precision," in *Proceedings of the IEEE International Conference on Computer Vision*, 2019, pp. 293–302.

[124] S. Jung, C. Son, S. Lee, J. Son, J.-J. Han, Y. Kwak, S. J. Hwang, and C. Choi, "Learning to quantize deep networks by optimizing quantization intervals with task loss," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2019, pp. 4350–4359.

[125] B. Zhuang, L. Liu, M. Tan, C. Shen, and I. Reid, "Training quantized neural networks with a full-precision auxiliary module," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2020, pp. 1488–1497.

[126] L. H. K. D. Shien Zhu and W. Liu, "Xor-net: An efficient computation pipeline of binary neural network inference on edge devices," in *The 26th IEEE International Conference on Parallel and Distributed Systems (ICPADS)*. IEEE, 2020.

[127] N. D. Lane, S. Bhattacharya, P. Georgiev, C. Forlivesi, and F. Kawsar, "An early resource characterization of deep learning on wearables, smartphones and internet-of-things devices," in *Proceedings of the 2015 international workshop on internet of things towards applications*, 2015, pp. 7–12.

[128] H. Cai, C. Gan, L. Zhu, and S. Han, "Tiny transfer learning: Towards memory-efficient on-device learning," *arXiv preprint arXiv:2007.11622*, 2020.

[129] W. J. Dally, Y. Turakhia, and S. Han, "Domain-specific hardware accelerators," *Communications of the ACM*, vol. 63, no. 7, pp. 48–57, 2020.

[130] J. Burgess, "Rtx on—the nvidia turing gpu," *IEEE Micro*, vol. 40, no. 2, pp. 36–44, 2020.

[131] Y. Umuroglu, N. J. Fraser, G. Gambardella, M. Blott, P. Leong, M. Jahre, and K. Vissers, "Finn: A framework for fast, scalable binarized neural network inference," in *Proceedings of the 2017 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays*, 2017, pp. 65–74.

[132] A. Paszke, S. Gross, S. Chintala, G. Chanan, E. Yang, Z. DeVito, Z. Lin, A. Desmaison, L. Antiga, and A. Lerer, "Automatic differentiation in pytorch," 2017.

[133] M. Abadi *et al.*, "TensorFlow: Large-scale machine learning on heterogeneous systems," 2015, software available from tensorflow.org. [Online]. Available: http://tensorflow.org/

[134] A. Madry, A. Makelov, L. Schmidt, D. Tsipras, and A. Vladu, "Towards deep learning models resistant to adversarial attacks," *arXiv preprint arXiv:1706.06083*, 2017.

[135] J. Lin, C. Gan, and S. Han, "Defensive quantization: When efficiency meets robustness," *arXiv preprint arXiv:1904.08444*, 2019.

[136] C. Gong, Z. Jiang, D. Wang, Y. Lin, Q. Liu, and D. Z. Pan, "Mixed precision neural architecture search for energy efficient deep learning." in *ICCAD*, 2019, pp. 1–7.

[137] G. Chen, H. Meng, Y. Liang, and K. Huang, "Gpu-accelerated real-time stereo estimation with binary neural network," *IEEE Transactions on Parallel and Distributed Systems*, vol. 31, no. 12, pp. 2896–2907, 2020.

[138] T. Wang, K. Wang, H. Cai, J. Lin, Z. Liu, H. Wang, Y. Lin, and S. Han, "Apq: Joint search for network architecture, pruning and quantization policy," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2020, pp. 2078–2087.

[139] A. Romero, N. Ballas, S. E. Kahou, A. Chassang, C. Gatta, and Y. Bengio, "Fitnets: Hints for thin deep nets," *arXiv preprint arXiv:1412.6550*, 2014.

[140] G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," *arXiv preprint arXiv:1503.02531*, 2015.

Source: https://arxiv.org/pdf/2011.14808

Google Scholar research | 2026-10-07 | Article 19 | Original p. 20 | Orange: discussion; blue: reference

SUGISAKA MASANORI

Journal of Robotics, Networking and Artificial Life
Vol. 9(2); September (2022), pp. 171–176
ISSN (Online):2352-6386; ISSN (Print):2405-9021
https://alife-robotics.org/jrnal.html



Research Article

INT8 Activation Ternary or Binary Weights Networks: Unifying Between INT8 and Lower-bit Width Quantization

Ninnart Fuengfusin[1], Hakaru Tamukoh[2]
[1]*Graduate School of Life Science and Systems Engineering, Kyushu Institute of Technology, 2-4 Hibikino, Wakamatsu-ku, Kitakyushu, Fukuoka, 808-0196, Japan*
[2]*Research Center for Neuromorphic AI Hardware, Graduate School of Life Science and Systems Engineering, Kyushu Institute of Technology, 2-4 Hibikino, Wakamatsu-ku, Kitakyushu, Fukuoka, 808-0196, Japan*

ARTICLE INFO

Article History
Received 15 December 2021
Accepted 01 July 2022

Keywords
Quantization
Image recognition
Model compression

ABSTRACT
This paper proposes ternary or binary weights with 8-bit integer activation convolutional neural networks. Our proposed model serves as the middle ground between 8-bit integer and lower than 8-bit precision quantized models. Our empirical experiments established that the conventional 1-bit or 2-bit only-weight quantization methods (i.e., BinaryConnect and ternary weights network) can be used jointly with the 8-bit integer activation quantization. We evaluate our model with the VGG16-like model to operate with the CIFAR10 and CIFAR100 datasets. Our models show competitive results to the general 32-bit floating point model.

© 2022 *The Author*. Published by Sugisaka Masanori at ALife Robotics Corporation Ltd
This is an open access article distributed under the CC BY-NC 4.0 license
(http://creativecommons.org/licenses/by-nc/4.0/).

1. Introduction

The invention of residual networks [1] has drastically improved the performances of neural networks (NNs); these networks now far exceed human performances, especially in image recognition tasks. However, NNs require numerous parameters to perform well. When the number of parameters increases, more computational operations are required. This might prove to be challenging when deploying these models to edge devices that lack high computational capacities.

These problems have led to the emergence of several research fields. One such field is quantization, which converts the conventional 32-bit floating-point (FP32) datatype into simpler datatypes, such as the fixed-point datatype and integer datatype. Currently, the default quantization technique supported by major deep learning frameworks (i.e., PyTorch [2] and TensorFlow [3]) is the 8-bit integer (INT8) quantization [4]. INT8 quantization has become the standard because INT8 datatypes and operations are widely supported by vendors of central processing units and graphics processing units. Therefore, the INT8 quantized model can be used across devices. In general, INT8 quantization transforms almost all FP32 parameters to INT8 via an affine transformation. This conversion allows the quantized model to operate primarily with INT8 operations, which are faster than FP32 operations. For example, in the NVIDIA Ampere architecture, NVIDIA A100 delivers approximately 32 times more INT8 operations per second compared with FP32 operations [5]. Furthermore, INT8 reduces the NN's overall memory storage by a factor of four compared with FP32.

INT8 quantization also positively affects FPGA implementations. For example, when we constructed FP32 and INT8 accumulation between two variables with Xilinx Vitis HLS 2021.2 using the target board Xilinx

Corresponding author's E-mail:

Source: https://www.jstage.jst.go.jp/article/jrnal/9/2/9_10/_pdf
Google Scholar research | 2026-10-08 | Article 20 | Original p. 1 | Orange: discussion; blue: reference

Zynq UltraScale+ MPSoC ZCU102 at 100 MHz, we found that the FP32 accumulation consumed 2-3 clock cycles. In contrast, the INT8 accumulation was done in less than 1 clock cycle. Furthermore, in terms of hardware utilization, FP32 accumulation consumes 2 digital signal processing, 180 flip-flops, and 249 lookup tables, whereas INT8 accumulation uses only 15 lookup tables. This shows that INT8 operations provide significantly lower latencies and less hardware utilization than FP32 operations.

In some cases, we may consider reducing the bit width to less than 8 bits. Several research studies have shown that 1- or 2-bit quantization is possible [6,7] with some reduction in the model performance. This low bit-width quantization research can be categorized into two categories: (i) only-weights quantization research and (ii) weights and activation quantization research. In [8,9], we proposed mixed precision weight networks (MPWNs) that determine the model with the optimized layer combinations between the floating-point, ternary, and binary weight layers. In this case, our model's ternary and binary weights are categorized as only-weights quantization. When the only-weight quantized model was compared with the weights and activation quantized models, it was found that the only-weight quantized model outperformed the other models by a significant margin. However, from our FPGA implementation from [9], the only-weight quantized models suffered from higher latency and hardware resource from the floating-point feature maps accumulation.

To reduce the latency and hardware requirements of the only-weight quantized models, we proposed an INT8 activation ternary or binary weights network (ITBWN). ITBWN uses the same weight spaces as MPWN. However, ITBWN aims to address the MPWN problems of high latency and hardware resource utilization from floating-point accumulations. ITBWN is based on both the lower bit-width and INT8-based quantization methods. ITBWN uses the INT8 quantization method to quantize its activations and uses lower-bit-width method to quantize its weights. The activation is quantized into INT8, which migrates the floating-point to INT8 accumulations, involving faster and lower hardware utilization. In [10], we proved that both INT8 and lower bit-width quantization methods could be used jointly with competitive performance to the floating-point model using the CIFAR10. In this research, we have extended our work in [10] by showing that these methods also provide competitive performance in the CIFAR100 dataset.

2. Related Works

This section will cover two related research fields: INT8 and lower-bit quantization.

2.1. *INT8 Quantization*

The INT8 quantization research problem is framed as follows: "How do we accurately approximate a floating-point tensor with an INT8 tensor and several other variables?" In [4], a floating-point tensor is approximated using an affine transform with the floating-point scaling factor S, an 8-bit unsigned integer (UINT8) zero-point variable Z, and an INT8 tensor q, as shown in Eq. (1). S and Z can be selected either as a vector with a channel size of r (per-channel quantization) or as a scalar (per-tensor quantization). In general, per-channel quantization should provide better performances with more parameters to approximate r.

$$r = S(q - Z) \tag{1}$$

The optimal S and Z values can be found by tracking the statistical information (i.e., the minimum and maximum values) during either the training or inference. To further reduce the complexity of this INT8 approximation, the trained quantization thresholds (TQT) method [11] proposed the removal of the zero-point variable Z, which transformed Eq. (1) into Eq. (2). In Eq. (2), TQT designs S as a scaler variable with the power-of-two quantization. This replaces the floating-point multiplication with shift operations between S and q.

$$r = S(q) \tag{2}$$

In contrast, instead of using the statistical information, TQT uses the training process with the straight-through estimator [12] to *determine S and the threshold values*. The threshold is used to clip the minimum and maximum values of r before using Eq. (2). There are several TQT implementations and one of them is Xilinx Brevitas [13]. Xilinx Brevitas provides an easy-to-use PyTorch implementation of TQT.

Source: https://www.jstage.jst.go.jp/article/jrnal/9/2/9_10/_pdf
Google Scholar research | 2026-10-08 | Article 20 | Original p. 2 | Orange: discussion; blue: reference

2.2. *Lower-bit Quantization*

BinaryConnect (BC) [6] and ternary weight networks (TWN) [7] are only-weight and lower-bit quantization methods. BC converts the weights w into binary weights w^q using Eq. (3).

$$w^q = sign(w) \tag{3}$$

To make this BC trainable with Eq. (3) which discretizes the gradient to w, Eq. (4) is used to transfer the gradient from the quantized weights to the floating-point weights. Eq. (4) makes Eq. (3) the same as the identity function during back-propagation.

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial w^q} \tag{4}$$

TWN quantizes the weights w into $\{-S, 0, S\}$ with Eq. (5), where S and Δ are both positive floating-point variables. TWN applies Eq. (4) to make the model trainable with Eq. (5), similar to the case for BC.

$$w^q = \begin{cases} S: w > \Delta \\ 0: |w| \le \Delta \\ -S: w < -\Delta \end{cases} \tag{5}$$

Δ is given in Eq. (6). Where $\boldsymbol{E}$ is an expected value or mean-average of $|w|$.

$$\Delta = 0.7 \times \boldsymbol{E}(|w|) \tag{6}$$

S is given in Eq. (7). Eq. (7) is summarized as mean-average of $|w|$ that have values more than Δ.

$$S = \boldsymbol{E}_{i \in \{i||w_i|>\Delta\}}(|w_i|) \tag{7}$$

3. INT8 Activation Ternary or Binary Weights Networks

In only-weights quantized models, such as BC and TWN, the floating-point activations and binary {−1, 1} or ternary weights {−1, 0, 1} can be multiplied using only simple logic operations [9]. However, the feature maps are still accumulated with complex floating-point accumulations. The motivation for proposing the ITBWN model is to further improve the only-weights quantization by reducing the complexity of this floating-point accumulation. This can be done by converting all activation into INT8; therefore, the floating-point accumulations were converted into INT8 accumulations. The less complex INT8 datatype reduces the overall latency from the accumulation. The INT8 methods are based on approximating the floating-point tensors; therefore, a well-behaved floating-point approximation with the INT8 tensor should behave in the same way as the floating-point tensor.

With this motivation, in ITBWN, we use the TQT, BC, and TWN methods. ITBWN applies either TWN or BC to quantize its weights; however, for its activations, ITBWN uses the TQT INT8 quantization. Fig. 1 shows an overview of the datatypes in a block of the ITBWN model.

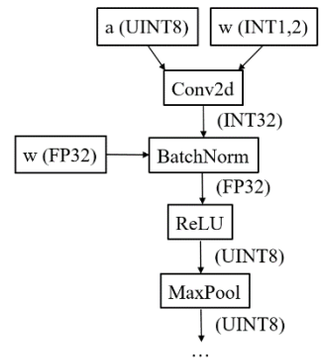


Fig. 1. Overview of the ITBWN datatypes in a basic building block of VGG16. Here, *a* is the activation, *w* are the weights or both weights and biases, and UINT8 represents the 8-bit unsigned integer.

In this study, several modifications are applied to BC, TWN, and TQT. For BC, we did not use the *clip* functions to clip the weights to the [−1, 1] range. For TWN, we did not use any scaling factor *S*, that is, we assigned $S = 1$. For TQT, we used the default implementation provided by Xilinx Brevitas, which has minor differences with the TQT settings [14]. The notable differences are that the Brevitas settings do not use the power-of-two quantized scaling factor, and Brevitas assigns a different initialization method to the scale factor. Finally, we did not use the BC, TWN, or TQT methods in the last layer, because these methods directly affect the model's performance.

4. Experimental Results and Discussion

In this section, we describe the experiments conducted with the CIFAR10 and CIFAR100 datasets. The CIFAR10 dataset consists of 10 classes, and each class

Source: https://www.jstage.jst.go.jp/article/jrnal/9/2/9_10/_pdf
Google Scholar research | 2026-10-08 | Article 20 | Original p. 3 | Orange: discussion; blue: reference

contains 5,000 32 × 32 color training images and 1,000 test images. However, the CIFAR100 dataset contains 100 classes, and each class consists of 500 32 × 32 color training images and 100 test images. With fewer images per class and a higher number of classes than CIFAR10, the CIFAR100 dataset was a more challenging benchmark. In both experiments, we used the VGG16-based model [15], and we modified the VGG16 model to operate with the CIFAR10 and CIFAR100 datasets by removing the first two fully connected layers and adjusting the input and output of the neurons of the last fully connected layer recordings to the dataset. In both the CIFAR10 and CIFAR100 experiments, we set the hyperparameters as shown in Table 1.

Table 1 Hyperparameters for VGG16 in the CIFAR10 and CIFAR100 settings.

Hyper Parameters	Value
Epoch	200
Batch size	256
Weight decay	0.0005
Learning rate	0.1

The images were normalized using the mean and standard deviations of the red, green, and blue channels of images in the training dataset. We performed data augmentation using the following steps. During the training, we zero-padded the image to the image's boundaries and randomly cropped back to the original size. Then, each image was also randomly horizontally flipped. The stochastic gradient descent with the momentum was selected to optimize our models. Finally, the learning rate was scheduled with Cosine annealing [16].

We set the notations as follows: *binary* for BC with TQT, *ternary* for TWN with TQT, *int8* for TQT only, and *float* for all the FP32 floating-point models. The CIFAR10 and CIFAR100 test accuracies over the training epochs are shown in Fig. 2 and Fig. 3, respectively. The best CIFAR10 and CIFAR100 test accuracies are shown in Table 2 and Table 3, respectively.

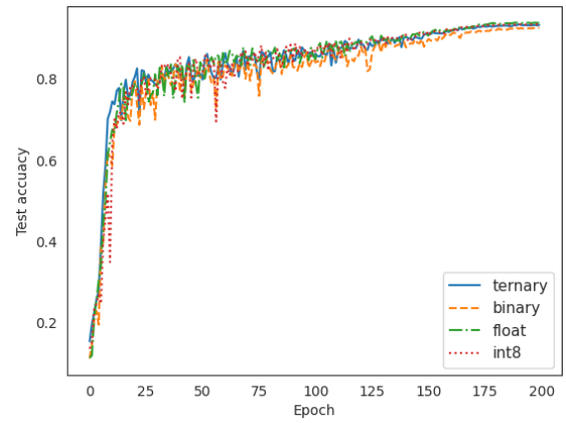


Fig. 2. CIFAR10 test accuracies of different methods as per the training epoch.

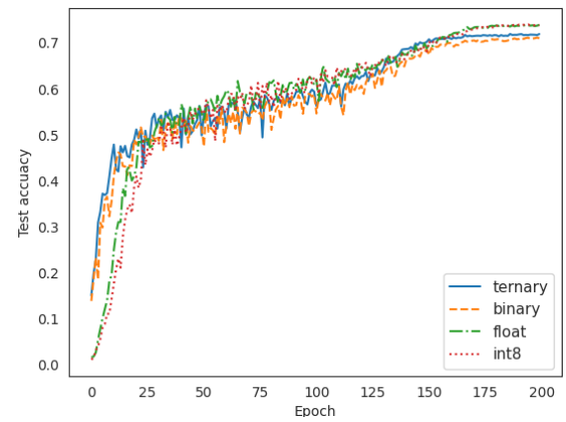


Fig. 3. CIFAR100 test accuracies of different methods as per the training epoch.

Table 2 CIFAR10's best accuracy for the VGG16-like model with and without quantization methods.

Model	Test Accuracy
binary	0.9269
ternary	0.9351
int8	0.9353
float	**0.9389**

Table 3 CIFAR100's best accuracy for the VGG16-like model with and without quantization methods.

Model	Test Accuracy
binary	0.7117
ternary	0.7195
int8	**0.7403**
float	0.7394

We had expected that an increase in the bit precision of the model would improve the test accuracy. Under this

Source: https://www.jstage.jst.go.jp/article/jrnal/9/2/9_10/_pdf
Google Scholar research | 2026-10-08 | Article 20 | Original p. 4 | Orange: discussion; blue: reference

hypothesis, the *float* would provide the best test accuracy, followed by *int8, ternary,* and *binary*, in that order. This hypothesis was confirmed in the CIFAR10 experiment; however, it failed to meet one case in the CIFAR100 experiment, in which *int8* outperformed *float* by a minor margin. We hypothesized that in the CIFAR100 experiment, *int8* quantization might cause a suitable degree of regularization compared with *binary*, *ternary,* and *float. binary* and *ternary* quant ization may have too strong regularization effect, while *float* only has a regularization effect from weight decay. Therefore, with the right degree of regularization from both weight decay and *int8* quantization, the *int8* model outperformed *binary*, *ternary,* and *float*.

5. Conclusion

We proposed ITBWN or a BC or TWN model with INT8 quantized activations. Our experiment shows that ITBWN with ternary weights provides competitive results for both the INT8-quantized and floating-point models. Converting the floating-point activation into INT8 with TQT or BC allowed the only-weight quantized model to deploy without worrying about the hardware or latency costs of floating-point accumulation.

In our future studies, we plan to extend the INT8 quantization-based method to cover quantization methods with lower precisions, such as INT4, INT2, and INT1. This would permit more weight space choices in designing the models. These weight spaces enable finding the best trade-offs between the performance and latency. We would like to apply this quantization technique to the dual steam VGG16 model [17], which can operate in red–green–blue depth images. This allows our model to operate in the robotic domain where depth images may become necessary.

When deploying these models in a robot, which is a hard real-time processing system, we would like to incorporate the concept of models like [18] into our system. This would allow small NNs or sub-NNs to be detachable from the NN. This would allow us to select to perform inference with a faster sub-NN or with the original NN that has better performance. A sub-NN is a part of an NN; therefore, quantizing a sub-NN will affect the original NN. In our future studies, we would like to investigate this dynamic between the sub-NN and original NN using the quantization techniques.

Acknowledgement

This research is based on results obtained from a project, JPNP16007, commissioned by the New Energy and Industrial Technology Development Organization (NEDO).

References

1. K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 770–778, 2016
2. A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, et al., "Pytorch: An imperative style, high-performance deep learning library," *Advances in neural information processing systems,* vol. 32, 2019.
3. M. Abadi, "Tensorflow: learning functions at scale," in *Proceedings of the 21st ACM SIGPLAN International Conference on Functional Programming*, pp. 1–1, 2016.
4. B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *Proceedings of the IEEE conference on computer vision and pattern recognition,* pp. 2704–2713, 2018.
5. NVIDIA A100 Tensor Core GPU Architecture, Accessed May 04, 2022.
6. M. Courbariaux, Y. Bengio, and J.-P. David, "Binaryconnect: Training deep neural networks with binary weights during propagations," *Advances in neural information processing systems,* vol. 28, 2015.
7. F. Li, B. Zhang, and B. Liu, "Ternary weight networks," *arXiv preprint arXiv:1605.04711,* 2016.
8. N. Fuengfusin and H. Tamukoh, "Mixed precision weight networks: Training neural networks with varied precision weights," in *International Conference on Neural Information Processing,* pp. 614–623, Springer, 2018.
9. N. Fuengfusin and H. Tamukoh, "Mixed-precision weights network for field-programmable gate array," *PloS one,* vol. 16, no. 5, p. e0251329, 2021.
10. N. Fuengfusin, and H. Tamukoh, "INT8 Activation Ternary or Binary Weights Networks," Proceedings *of the 2022 International Conference on Artificial Life and Robotics*, OS15-1, 2022.
11. S. Jain, A. Gural, M. Wu, and C. Dick, "Trained quantization thresholds for accurate and efficient fixed-point inference of deep neural networks," *Proceedings of Machine Learning and Systems,* vol. 2, pp. 112–128, 2020.
12. Y. Bengio, N. L eonard, and A. Courville, "Estimating or propagating gradients through stochastic neurons for conditional computation," *arXiv preprint arXiv:1308.3432,* 2013.
13. A. Pappalardo. Xilinx/brevitas. Zenodo, 2021.
14. Cite to quantization techniques in QuantIdentity(bit_width=8) and QuantReLU(bit_width=8).

Source: https://www.jstage.jst.go.jp/article/jrnal/9/2/9_10/_pdf

Google Scholar research | 2026-10-08 | Article 20 | Original p. 5 | Orange: discussion; blue: reference

Efficient SAR Vessel Detection for FPGA-Based On-Satellite Sensing

Colin Laganier*
The Alan Turing Institute
London, United Kingdom

Liam Fletcher
The Alan Turing Institute
London, United Kingdom

Elim Kwan
The Alan Turing Institute
London, United Kingdom

Richard Walters
The Alan Turing Institute
London, United Kingdom

Victoria Nockles
The Alan Turing Institute
London, United Kingdom

Abstract

Rapid analysis of satellite imagery within minutes-to-hours of acquisition is increasingly vital for many remote sensing applications, and is an essential component for developing next-generation autonomous and distributed satellite systems. On-satellite machine learning (ML) has the potential for such rapid analysis, by overcoming latency associated with intermittent satellite connectivity to ground stations or relay satellites, but state-of-the-art models are often too large or power-hungry for on-board deployment. Vessel detection using Synthetic Aperture Radar (SAR) is a critical time-sensitive application in maritime security that exemplifies this challenge. SAR vessel detection has previously been demonstrated only by ML models that either are too large for satellite deployment, have not been developed for sufficiently low-power hardware, or have only been tested on small SAR datasets that do not sufficiently represent the difficulty of the real-world task. Here we systematically explore a suite of architectural adaptations to develop a novel YOLOv8 architecture optimized for this task and FPGA-based processing. We deploy our model on a Kria KV260 MPSoC, and show it can analyze a ~700 megapixel SAR image in less than a minute, within common satellite power constraints (<10W). Our model has detection and classification performance only ~2% and 3% lower than values from state-of-the-art GPU-based models on the largest and most diverse open SAR vessel dataset, xView3-SAR, despite being ~50 and ~2500 times more computationally efficient. This work represents a key contribution towards on-satellite ML for time-critical SAR analysis, and more autonomous, scalable satellites.[1]

*Corresponding author: claganier@turing.ac.uk



This work is licensed under a Creative Commons Attribution-NonCommercial 4.0 International License.

SEC '25, Arlington, VA, USA

© 2025 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2238-7/25/12
https://doi.org/10.1145/3769102.3772713

CCS Concepts

• **Computing methodologies → Computer vision**; • **Hardware → Reconfigurable logic and FPGAs**; • **Applied computing → Earth and atmospheric sciences**.

Keywords

Edge AI, FPGA, Object Detection, On-Satellite Machine Learning, Synthetic Aperture Radar

ACM Reference Format:
Colin Laganier, Liam Fletcher, Elim Kwan, Richard Walters, and Victoria Nockles. 2025. Efficient SAR Vessel Detection for FPGA-Based On-Satellite Sensing. In *The Tenth ACM/IEEE Symposium on Edge Computing (SEC '25), December 3–6, 2025, Arlington, VA, USA.* ACM, New York, NY, USA, 17 pages. https://doi.org/10.1145/3769102.3772713

1 Introduction

Rapid analysis of low earth orbit satellite imagery within hours of data acquisition is vital for a diverse range of remote sensing applications, from wildfire detection, to agricultural monitoring, to disaster response [6, 8, 65], and latency requirements for many applications are moving beyond near real-time (1-3 hours), to quasi real-time (<30 mins), with target latencies of <5 minutes in some cases [39]. Furthermore, these shortest latency requirements will be increasingly important due to growing interest in autonomous and distributed satellite capabilities [4], for example "tip-and-cue" processes [15] (where one satellite uses initial analysis to request further imagery from another satellite). Such autonomous tasking and inter-satellite coordination capabilities have been identified as critical for next-generation Earth observation systems [69], with small test systems already in orbit (e.g. NASA's Starling satellite swarm [22]). But achieving such rapid analysis is challenging, due to 1) ever-increasing dataset sizes of large-area, high-resolution

[1]Code: https://github.com/alan-turing-institute/sar-vessel-detection-fpga

Source: https://doi.org/10.1145/3769102.3772713

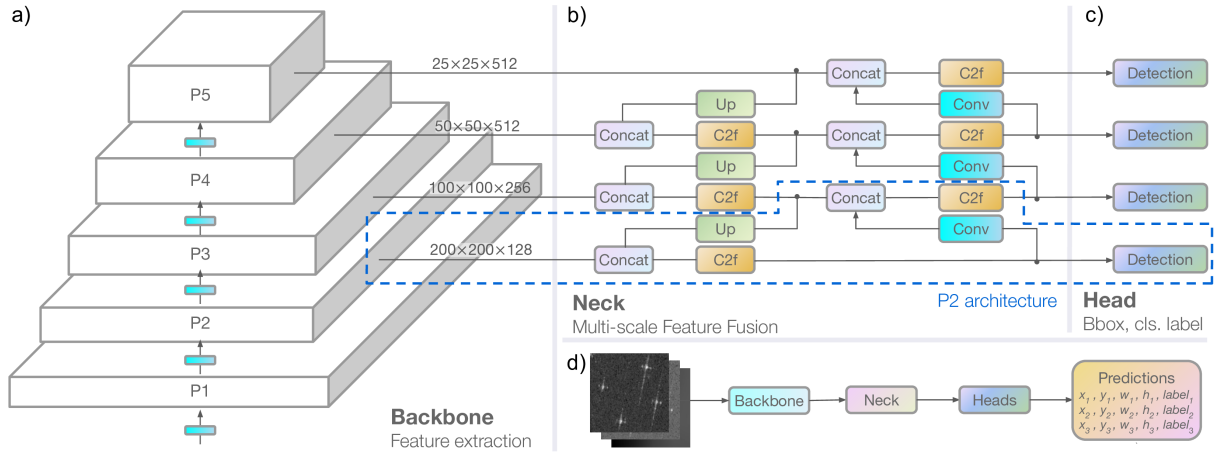


Figure 2: Architecture diagram of the YOLOv8-based object detection model. (a) feature extraction backbone, (b) multi-scale feature fusion neck, (c) detection head, (d) simplified end-to-end detection pipeline from SAR and bathymetry input to predictions. P2 architecture branch is shown in dashed blue box. Diagram modified from [60].

moderate prediction quality. This approach has not previously been used for SAR vessel detection.

3.2 FPGA Deployment

In this study we specifically investigate FPGA deployment, due to their power efficiency and existing widespread use onboard satellites. We focus on a single FPGA device as the one selected fits within the strict power constraints of a representative small satellite platform. We evaluate our proposed solution on the AMD/Xilinx Kria KV260 board, which integrates programmable logic (PL; Table 1) with a processing system (PS) based on a Quad-core Arm Cortex-A53 SoC. For model deployment, we utilized the Vitis AI 3.5 framework and created a custom hardware design targeting the PL of the Kria KV260 and incorporating the Deep Learning Processing Unit (DPU) 4.1 IP using Vivado 2023.2. The DPU is a programmable engine optimized for deep neural networks, which is leveraged by the Vitis AI instruction set to run models in the PL. In our design, the DPU is configured with one B4096 core and a clock frequency of 300 MHz. We quantized the model to INT8 using the Vitis AI Quantizer library, using symmetric quantization with a power-of-two scaling scheme (required configuration for the DPU), allowing for both Post-Training Quantization (PTQ) and Quantization-Aware Training (QAT) using a straight-through estimator [35]. The DPU of the Kria KV260 platform (DPUCZDX8G) does not support the SiLU activation function used in YOLOv8, and so we modified the model to use Hard Swish activations [33] instead. Finally, we deployed the compiled model on the FPGA using the Vitis AI Runtime, and evaluated model performance in terms of speed, power consumption, and accuracy.

Table 1: FPGA resource utilization summary.

LUT	FF	DSP	BRAM	URAM
Kria KV260				
117,129	234,240	1,248	144	64
Vitis AI B4096 DPU				
62,104 (53.0%)	123,486 (53.2%)	710 (56.9%)	109 (75.7%)	40 (62.5%)
Post-Processing Decoding Kernel				
45,671 (39.0%)	32,011 (13.7%)	188 (15.1%)	33 (22.9%)	0 (0.0%)

3.3 Dataset

The xView3-SAR [59] dataset is a large and diverse collection of high-resolution SAR maritime images, including 754 publicly available images with an average size of 29,400 by 24,400 pixels. The scenes are C-band SAR amplitude images acquired by the European Space Agency's (ESA's) Sentinel-1 constellation; specifically the Interferometric Wide Swath mode Level-1 Ground Range Detected product, which includes both vertical-horizontal (VH) and vertical-vertical (VV) polarization bands for each image at a 10-meter pixel spacing. The scenes are supplied having undergone preprocessing, including orbit correction, removal of noise artifacts, radiometric calibration, terrain correction, and geocoding. In addition to the VH and VV SAR images, ancillary data are provided including wind speed, direction, quality, and land/ice mask information from Sentinel-1 Level-2 Ocean product, and bathymetry information from the General Bathymetric Chart of the Oceans (GEBCO, [1]). These

Source: https://doi.org/10.1145/3769102.3772713

the increased performance drop of the smallest model (*nano*) can be attributed to the reduced amount of redundancy with fewer parameters, which leads to a more pronounced impact.

Table 4: Performance of YOLOv8 model variants with INT8 PTQ and QAT on GPU, and QAT on FPGA.

Model	F1 Score			
	Detection	Near-Shore	Vessel	Fishing
*A) YOLOv8n (**PT+FT+TH**)*				
Float	0.650±0.003	0.384±0.003	0.914±0.003	0.731±0.007
PTQ	0.644±0.003	0.327±0.012	0.872±0.016	0.671±0.012
QAT	0.646±0.004	0.377±0.011	0.915±0.002	0.725±0.018
FPGA	0.647	0.386	0.914	0.730
*B) YOLOv8s-Ghost-P2-PIoU2 (**PT+FT+TH**)*				
Float	0.701±0.001	0.442±0.003	0.920±0.018	0.774±0.013
PTQ	0.696±0.005	0.420±0.015	0.903±0.010	0.752±0.013
QAT	0.699±0.001	0.444±0.007	0.932±0.001	0.779±0.001
FPGA	0.697	0.438	0.931	0.785
*C) YOLOv8n-Ghost-P2-PIoU2 (**PT+FT+TH**)*				
Float	0.699±0.004	0.444±0.004	0.923±0.002	0.769±0.006
PTQ	0.697±0.005	0.419±0.009	0.892±0.004	0.740±0.004
QAT	0.701±0.003	0.445±0.001	0.927±0.005	0.771±0.009
FPGA	0.704	0.445	0.921	0.765

4.2.2 Quantization-Aware Training (QAT). To minimize the performance drop observed with PTQ, we modify the second training stage (**FT**) with QAT. We use the same training parameters as the fine-tuning stage detailed in Section 3.5, with the addition of a quantizer parameter learning rate. We study the impact of the quantizer learning rate on the model performance (Table 5) and signal-to-quantization-noise ratio (SQNR; Fig. 5) of the model subgraphs (partitions of the computational graph). The SQNR provides a measure of the fidelity of the quantized representation, where higher values correspond to reduced distortion due to quantization and is defined as SQNR= $10 \cdot \log_{10}(\frac{P_{\text{signal}}}{P_{\text{noise}}})$. We find that higher learning rates (5 and 0.5) produce a broader distribution, with higher SQNR modes (23 and 24 dB) with a longer tail towards lower SQNR values. Lower learning rates (0.05 and 0.005) produce a more concentrated distribution, with a lower SQNR mode (20 dB) and fewer low SQNR subgraphs. However, the detection and classification performance difference is minimal across the different values, and we therefore opt for a learning rate of 0.5 in our solution. QAT implementation introduces minimal performance degradation across most metrics (average -0.4% for detection F1), with our modified architecture models showing slight improvement in classification metrics (e.g. +0.9% for YOLOv8s-Ghost-P2-PIoU2 QAT). The baseline architecture shows the highest average performance drop of 0.7%, with the *nano* modified architectures showing a loss of 0.1% and the *small* a 0.2% improvement overall. Nevertheless, the YOLOv8n-Ghost-P2-PIoU2 model shows the highest detection performance overall, with a 0.701 and 0.445 F1 score for detection and near-shore detection respectively, outperforming the baseline model and the *small* model by 5.5% and 0.2% respectively. These results indicate that with QAT, most of the performance drop observed with PTQ can be recovered, and confirms that our two-stage training approach effectively improves model performance.

Table 5: Impact of Activation and Quantizer Learning Rate on YOLOv8n-Ghost-P2-PIoU2 performance.

Metric	F1 Score			
	Detection	Near-Shore	Vessel	Fishing
*A) Activation Function (**FT**)*				
SiLU	0.691±0.003	0.450±0.003	0.929±0.003	0.767±0.006
HSwish	0.686±0.005	0.446±0.003	0.928±0.003	0.766±0.006
*B) Quantizer Learning Rate (QAT **FT**)*				
5	0.690±0.004	0.443±0.002	0.929±0.002	0.765±0.007
0.5	0.693±0.005	0.443±0.001	0.930±0.001	0.763±0.003
0.05	0.686±0.010	0.439±0.006	0.931±0.003	0.765±0.002
0.005	0.688±0.003	0.444±0.013	0.929±0.008	0.765±0.008

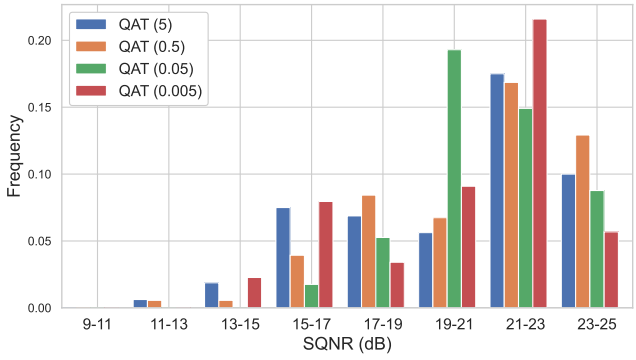


Figure 5: Histogram of per-subgraph SQNR for different QAT learning rates for YOLOv8n-Ghost-P2-PIoU2.

4.3 FPGA Deployment Results

The performance of the deployed compiled QAT models on the Kria KV260 board is shown in Table 4 and matches the expected performance based on the GPU results, with little to no performance degradation observed. For our final model, we also visually illustrate the quality of the results with respect to a state-of-the-art GPU-based model. From

Source: https://doi.org/10.1145/3769102.3772713

[32] Qibin Hou, Daquan Zhou, and Jiashi Feng. 2021. Coordinate Attention for Efficient Mobile Network Design. In *2021 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 13708–13717. doi:10.1109/CVPR46437.2021.01350

[33] Andrew Howard, Mark Sandler, Grace Chu, Liang-Chieh Chen, Bo Chen, Mingxing Tan, Weijun Wang, Yukun Zhu, Ruoming Pang, Vijay Vasudevan, Quoc V. Le, and Hartwig Adam. 2019. Searching for MobileNetV3. arXiv:1905.02244 [cs.CV]

[34] Xueqin Huang, Ke Xu, Ji Chen, Ao Wang, Shengyao Chen, and Hongtao Li. 2024. Real-Time Processing of Ship Detection with SAR Image Based on FPGA. In *IGARSS 2024 - 2024 IEEE International Geoscience and Remote Sensing Symposium*. 8954–8957. doi:10.1109/IGARSS53475.2024.10640725

[35] Sambhav Jain, Albert Gural, Michael Wu, and Chris Dick. 2020. Trained Quantization Thresholds for Accurate and Efficient Fixed-Point Inference of Deep Neural Networks. In *Proceedings of Machine Learning and Systems*, I. Dhillon, D. Papailiopoulos, and V. Sze (Eds.), Vol. 2. 112–128.

[36] JAXA. n.d.. Advanced Land Observing Satellite-4 "Daichi-4" [Technical overview]. Retrieved from https://global.jaxa.jp/projects/sat/alos4/.

[37] Kefeng Ji, Xiangguang Leng, Qingju Fan, Shilin Zhou, and Huanxin Zou. 2016. An land masking algorithm for ship detection in SAR images. In *2016 IEEE International Geoscience and Remote Sensing Symposium (IGARSS)*. 925–928. doi:10.1109/IGARSS.2016.7729234

[38] Ming Kang, Chee-Ming Ting, Fung Fung Ting, and Raphaël C.-W. Phan. 2024. BGF-YOLO: Enhanced YOLOv8 with Multiscale Attentional Feature Fusion for Brain Tumor Detection. In *Proceedings of Medical Image Computing and Computer Assisted Intervention – MICCAI 2024*, Vol. LNCS 15008. Springer Nature Switzerland.

[39] Murray Kerr, Stefania Tonetti, Stefania Cornara, Juan-Ignacio Bravo, Robert Hinz, Antonio Latorre, Francisco Membibre, Chiara Solimini, Stefan Wiehle, Helko Breit, Björn Tings, Otto Koudelka, Franz Teschl, Enrico Magli, Tiziano Bianchi, Andrea Migliorati, Paolo Motto Ros, Michele Caon, Riccardo Freddi, Michele Benetti, Fabio Milani, Guido Curci, Silvia Fraile, Lucia Garcia, Cecilia Marcos, and Aniello Fiengo. 2020. EO-ALERT: A Novel Architecture for the Next Generation of Earth Observation Satellites Supporting Rapid Civil Alert. In *71st International Astronautical Congress (IAC 2020)*.

[40] Boshra Khalili and Andrew W. Smyth. 2024. SOD-YOLOv8—Enhancing YOLOv8 for Small Object Detection in Aerial Imagery and Traffic Scenes. *Sensors* 24, 19 (2024). doi:10.3390/s24196209

[41] Andrey A. Kurekin, Benjamin R. Loveday, Oliver Clements, Graham D. Quartly, Peter I. Miller, George Wiafe, and Kwame Adu Agyekum. 2019. Operational Monitoring of Illegal Fishing in Ghana through Exploitation of Satellite Earth Observation and AIS Data. *Remote Sensing* 11, 3 (2019). doi:10.3390/rs11030293

[42] Haitao Lang, Ruifu Wang, Shaoying Zheng, Siwen Wu, and Jialu Li. 2022. Ship Classification in SAR Imagery by Shallow CNN Pre-Trained on Task-Specific Dataset with Feature Refinement. *Remote Sensing* 14, 23 (2022). doi:10.3390/rs14235986

[43] Phang Kok Leong. 2025. Securing Malaysia's maritime environment using space-derived services. *Contemporary Issues in Air and Space Power* 3, 1 (2025), bp42323989. doi:10.58930/bp42323989

[44] Yonggang Li, Weigang Zhu, and Bakun Zhu. 2021. SAR Image Nearshore Ship Target Detection in Complex Environment. In *2021 IEEE 5th Advanced Information Technology, Electronic and Automation Control Conference (IAEAC)*. 1964–1968. doi:10.1109/IAEAC50856.2021.9391019

[45] Tsung-Yi Lin, Piotr Dollár, Ross Girshick, Kaiming He, Bharath Hariharan, and Serge Belongie. 2017. Feature Pyramid Networks for Object Detection. In *2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 936–944. doi:10.1109/CVPR.2017.106

[46] Can Liu, Kaige Wang, Qing Li, Fazhan Zhao, Kun Zhao, and Hongtu Ma. 2024. Powerful-IoU: More straightforward and faster bounding box regression loss with a nonmonotonic focusing mechanism. *Neural Networks* 170 (2024), 276–284. doi:10.1016/j.neunet.2023.11.041

[47] Shu Liu, Lu Qi, Haifang Qin, Jianping Shi, and Jiaya Jia. 2018. Path Aggregation Network for Instance Segmentation. In *2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 8759–8768. doi:10.1109/CVPR.2018.00913

[48] Yonghang Luo, Ming Li, Guihao Wen, Yunfei Tan, and Chaoshan Shi. 2024. SHIP-YOLO: A Lightweight Synthetic Aperture Radar Ship Detection Model Based on YOLOv8n Algorithm. *IEEE Access* 12 (2024), 37030–37041. doi:10.1109/ACCESS.2024.3373893

[49] Ningning Ma, Xiangyu Zhang, Hai-Tao Zheng, and Jian Sun. 2018. ShuffleNet V2: Practical Guidelines for Efficient CNN Architecture Design. In *Proceedings of the European Conference on Computer Vision (ECCV)*.

[50] Tian J. Ma and Robert J. Anderson. 2023. Remote Sensing Low Signal-to-Noise-Ratio Target Detection Enhancement. *Sensors* 23, 6 (2023). doi:10.3390/s23063314

[51] Abhas Maskey and Mengu Cho. 2020. CubeSatNet: Ultralight Convolutional Neural Network designed for on-orbit binary image classification on a 1U CubeSat. *Engineering Applications of Artificial Intelligence* 96 (2020), 103952. doi:10.1016/j.engappai.2020.103952

[52] Fanlong Meng, Xiangyang Qi, and Huaitao Fan. 2024. LSR-Det: A Lightweight Detector for Ship Detection in SAR Images Based on Oriented Bounding Box. *Remote Sensing* 16, 17 (2024). doi:10.3390/rs16173251

[53] Alexander Montgomerie-Corcoran, Petros Toupas, Zhewen Yu, and Christos-Savvas Bouganis. 2023. SATAY: A Streaming Architecture Toolflow for Accelerating YOLO Models on FPGA Devices. In *2023 International Conference on Field Programmable Technology (ICFPT)*. 179–187. doi:10.1109/ICFPT59805.2023.00025

[54] Alberto Moreira, Pau Prats-Iraola, Marwan Younis, Gerhard Krieger, Irena Hajnsek, and Konstantinos P Papathanassiou. 2013. A tutorial on synthetic aperture radar. *IEEE Geoscience and remote sensing magazine* 1, 1 (2013), 6–43.

[55] NASA-ISRO SAR (NISAR). 2018. *Mission Science Users' Handbook*. Technical Report. NASA Jet Propulsion Laboratory. 261 pages. L: NISAR.

[56] Duy Thanh Nguyen, Hyun Kim, and Hyuk-Jae Lee. 2021. Layer-Specific Optimization for Mixed Data Flow With Mixed Precision in FPGA Design for CNN-Based Object Detectors. *IEEE Transactions on Circuits and Systems for Video Technology* 31, 6 (2021), 2450–2464. doi:10.1109/TCSVT.2020.3020569

[57] OECD. 2019. *The Space Economy in Figures: How Space Contributes to the Global Economy*. OECD Publishing, Paris. doi:10.1787/c5996201-en

[58] Daliang Ouyang, Su He, Guozhong Zhang, Mingzhu Luo, Huaiyong Guo, Jian Zhan, and Zhijie Huang. 2023. Efficient Multi-Scale Attention Module with Cross-Spatial Learning. In *ICASSP 2023 - 2023 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 1–5. doi:10.1109/icassp49357.2023.10096516

[59] Fernando Paolo, Tsu ting Tim Lin, Ritwik Gupta, Bryce Goodman, Nirav Patel, Daniel Kuster, David Kroodsma, and Jared Dunnmon. 2022. xView3-SAR: Detecting Dark Fishing Activity Using Synthetic Aperture Radar Imagery. arXiv:2206.00897 [cs.CV]

[60] RangeKing. 2023. Ultralytics YOLOv8 Architecture Diagram. https://github.com/ultralytics/ultralytics/issues/189. Accessed: 2025-06-13.

[61] Emilio Rapuano, Gabriele Meoni, Tommaso Pacini, Gianmarco Dinelli, Gianluca Furano, Gianluca Giuffrida, and Luca Fanucci. 2021. An FPGA-Based Hardware Accelerator for CNNs Inference on Board Satellites: Benchmarking with Myriad 2-Based Solution for the CloudScout Case Study. *Remote Sensing* 13, 8 (2021). doi:10.3390/rs13081518

Source: https://doi.org/10.1145/3769102.3772713

Google Scholar research | 2026-10-08 | Article 21 | Original p. 16 | Orange: discussion; blue: reference